

CIS 580

# Machine Perception

*Or Geometric Computer Vision*

Instructor: Lingjie Liu

Lec 1: Jan 20, 2026

# Learning Outcomes

- Firm knowledge of fundamentals of geometric computer vision from single and multiple cameras, some image processing, and (if time permits) some deep learning for geometry
- Understanding of challenges: why algorithms work or do not work
- Ability to perform as a vision engineer in vision and robotics companies

**Prerequisites:** mainly undergraduate linear algebra (e.g. vector & matrix products, inverses, determinants, solving systems of linear equations, eigenvectors), and some high school Euclidean geometry

# CIS 580 Machine Perception Spring 2025

- Resources:
  - Canvas
    - Slides & readings
    - (-> Recordings, Ed, Gradescope)

# Instructor Introduction

Lingjie Liu

Assistant Professor, CIS

Penn CG lab and GRASP lab

<https://lingjie0206.github.io/>





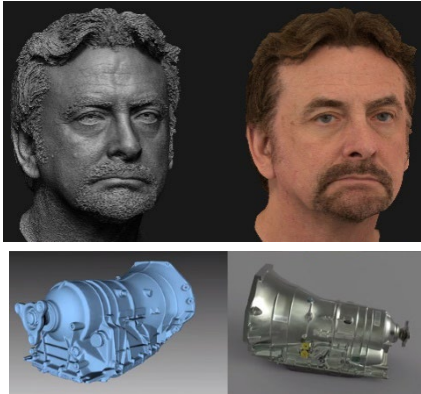
# My Research Interests

- **Reconstruction** of Real-world Dynamic Scenes.

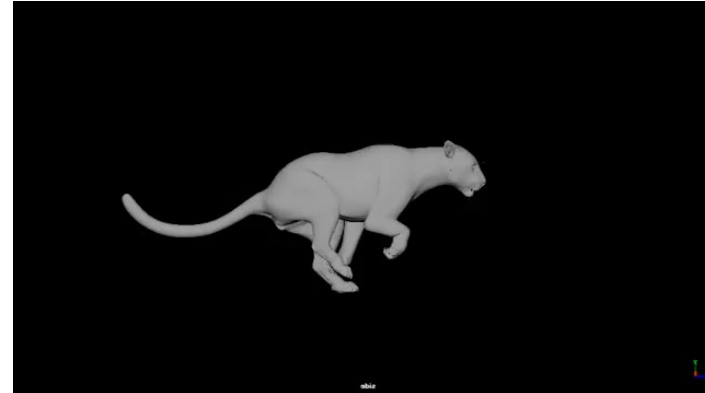


# My Research Interests

- **Reconstruction** of Real-world Dynamic Scenes.



Geometry  
+ Appearance



Motion  
+ Deformation



# My Research Interests

- **Reconstruction of Real-world Dynamic Scenes.**





# My Research Interests

- **Image Synthesis** of Real-world Scenes with 3D Control.



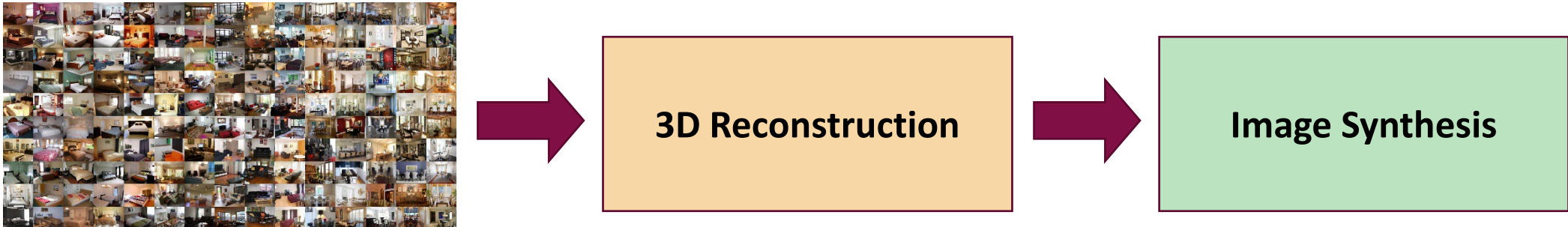
# My Research Interests

- **Image Synthesis** of Real-world Scenes with 3D Control.



# My Research Interests

- **Image Synthesis** of Real-world Scenes with 3D Control.



# My Research Interests

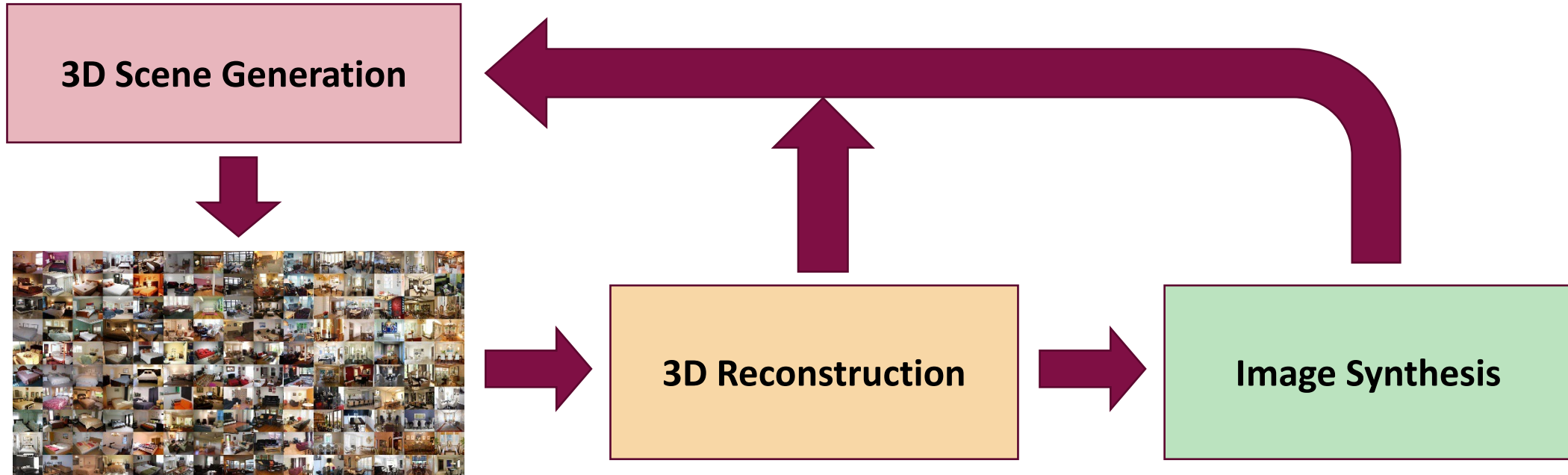
- Large-scale 3D Scene Generation





# My Research Interests

- Large-scale 3D Scene **Generation**





# Grading Scheme

- 4x Homeworks: 36%
- Midterm: 20%
- Final Exam: 20%
- Small Projects: 24%
- Possible bonus for extensive in-class participation (up to 5%)
  - I need your interaction and questions. Do not let the communication become one-way! Interrupt at any time.
- Lenient letter grades. The only way to fail this class is to violate the honor code (cheat, plagiarize)
  - Zero Tolerance. Think before you act: One homework or one question in a midterm is unlikely to affect your final letter grade.

# Spring 2026 Tentative Schedule

| Week | Date*                    | Topics                         |
|------|--------------------------|--------------------------------|
| 1    | Jan 20                   |                                |
| 2    | Jan 27                   |                                |
| 3    | Feb 3                    |                                |
| 4    | Feb 10                   |                                |
| 5    | Feb 17                   |                                |
| 6    | Feb 24                   |                                |
| 7    | Mar 3                    |                                |
| 8    | Mar 10<br>(Spring Break) |                                |
| 9    | Mar 17                   |                                |
| 10   | Mar 24                   | <b>Midterm (20%, date TBD)</b> |
| 11   | Mar 31                   |                                |
| 12   | Apr 7                    |                                |
| 13   | Apr 14                   |                                |
| 14   | Apr 21                   |                                |
| 15   | Apr 28                   |                                |
| 16   | Btw May 4-12             | <b>Final Exam (20%)</b>        |

HW1 (Feb 3 – Feb 17)

10%

HW2 (Feb 17 – Mar 3)

10%

HW3 (March 3 – March 24)

8%

HW4 (March 31– Apr 14)

8%

two small projects(Apr 14 – May 14)

24%

HW will be released  
on Thu of that week  
and will be due two  
weeks later, on Thu.

HW plans might evolve

# Waitlisty

- There is typically a 2–3 day delay for the system to reflect my approval on your end after I process your request. Please be patient.
- Currently, the course is at maximum capacity, constrained by our classroom size. So even if I approve your request on my side, you still won't be able to register. Sorry about that.

# Course Team Spring' 26



Dhyey Shah

[dhyey28@seas.upenn.edu](mailto:dhyey28@seas.upenn.edu)



Ravi Raghavan

[rr1133@seas.upenn.edu](mailto:rr1133@seas.upenn.edu)



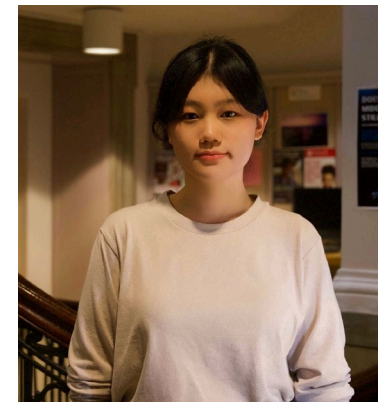
Haoran Yang

[haorany8@seas.upenn.edu](mailto:haorany8@seas.upenn.edu)



Nandagopal Vidhu

[nvidhu@seas.upenn.edu](mailto:nvidhu@seas.upenn.edu)



Yiming Huang

[ymhuang9@seas.upenn.edu](mailto:ymhuang9@seas.upenn.edu)



Prakriti Prasad

[prasadpr@seas.upenn.edu](mailto:prasadpr@seas.upenn.edu)



Aanal Sonara

[sonara29@seas.upenn.edu](mailto:sonara29@seas.upenn.edu)



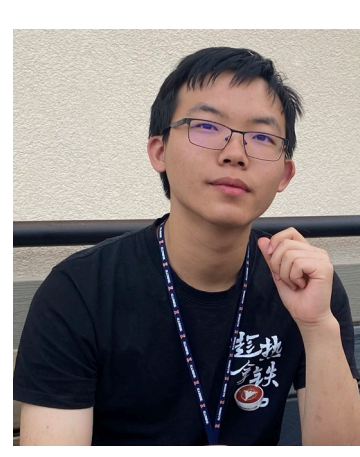
Horace Qihao Yun

[qhyun01@seas.upenn.edu](mailto:qhyun01@seas.upenn.edu)



Siyu Li

[siyul@seas.upenn.edu](mailto:siyul@seas.upenn.edu)



Tony Wang

[tonyw3@seas.upenn.edu](mailto:tonyw3@seas.upenn.edu)



Abhimanyu Suthar

[jingxhou@seas.upenn.edu](mailto:jingxhou@seas.upenn.edu)



Yongzhe Wang

[yongzhe@sas.upenn.edu](mailto:yongzhe@sas.upenn.edu)



# How to Speak With The Course Team

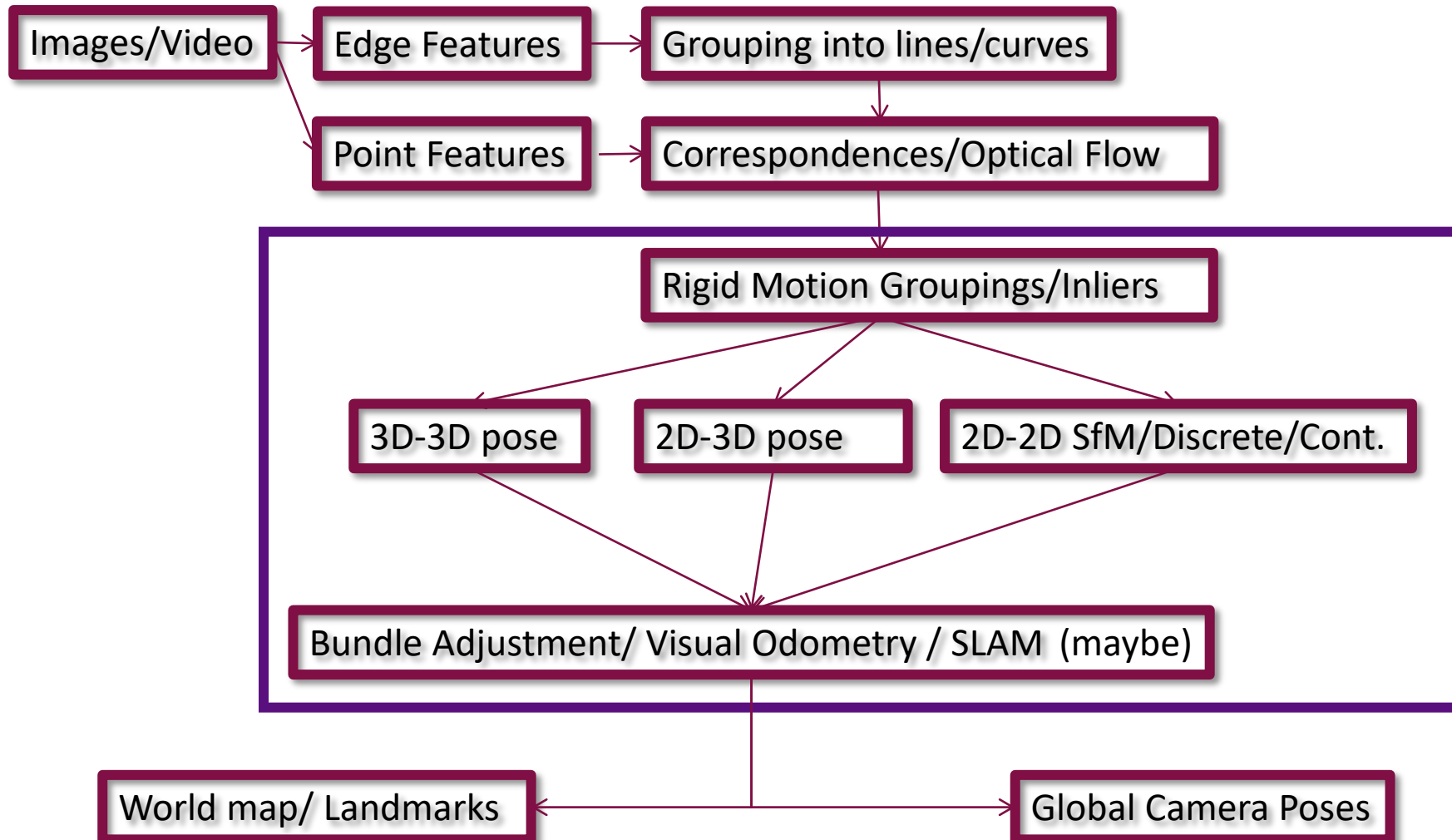
Go to office hours!

- Each of the TAs will have 1 hour of OH each week.
- I will also have 1 hour of OH each week.
- Starting from next week (the week of Jan 26)

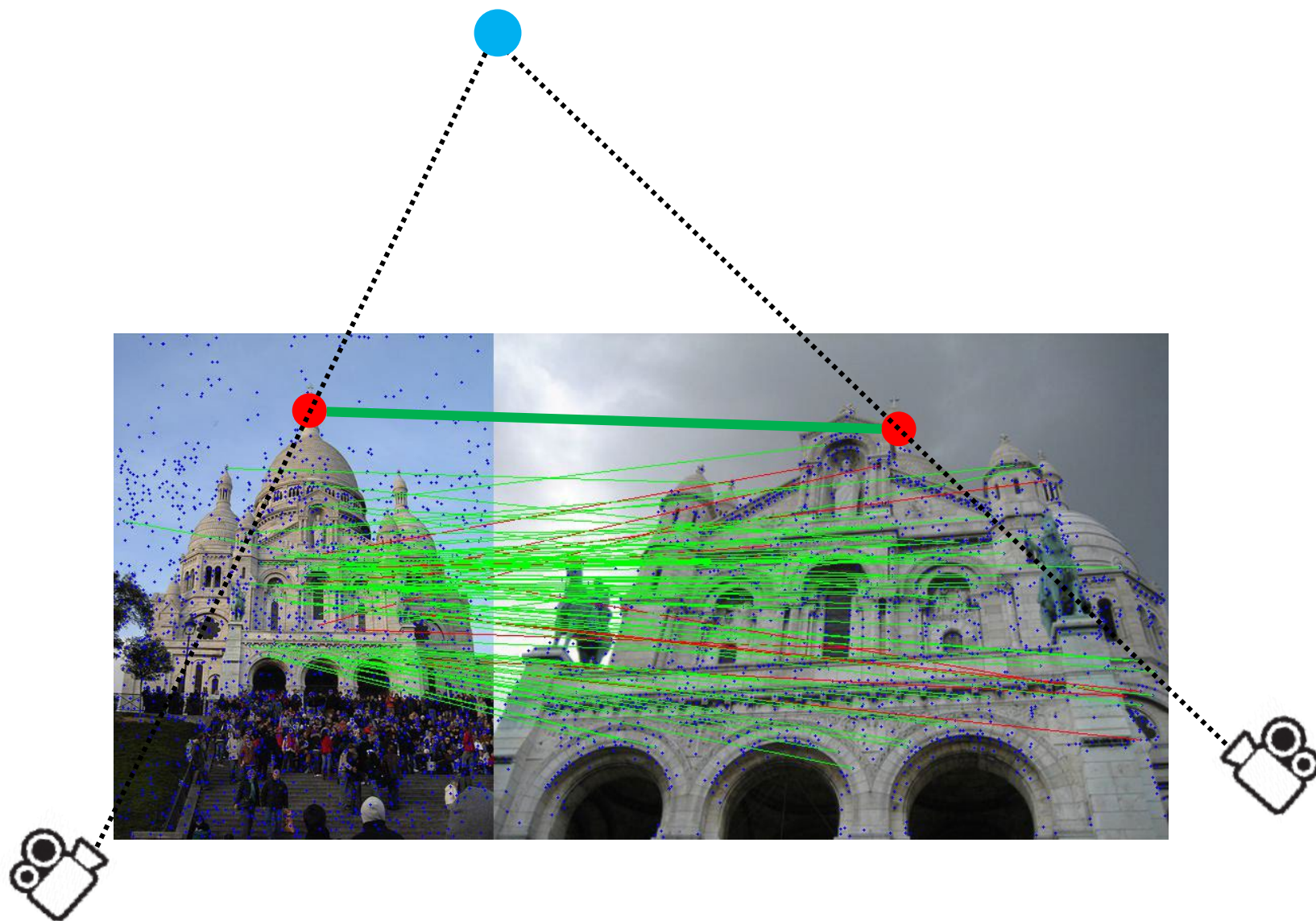


# Lec 1: Perspective Projection

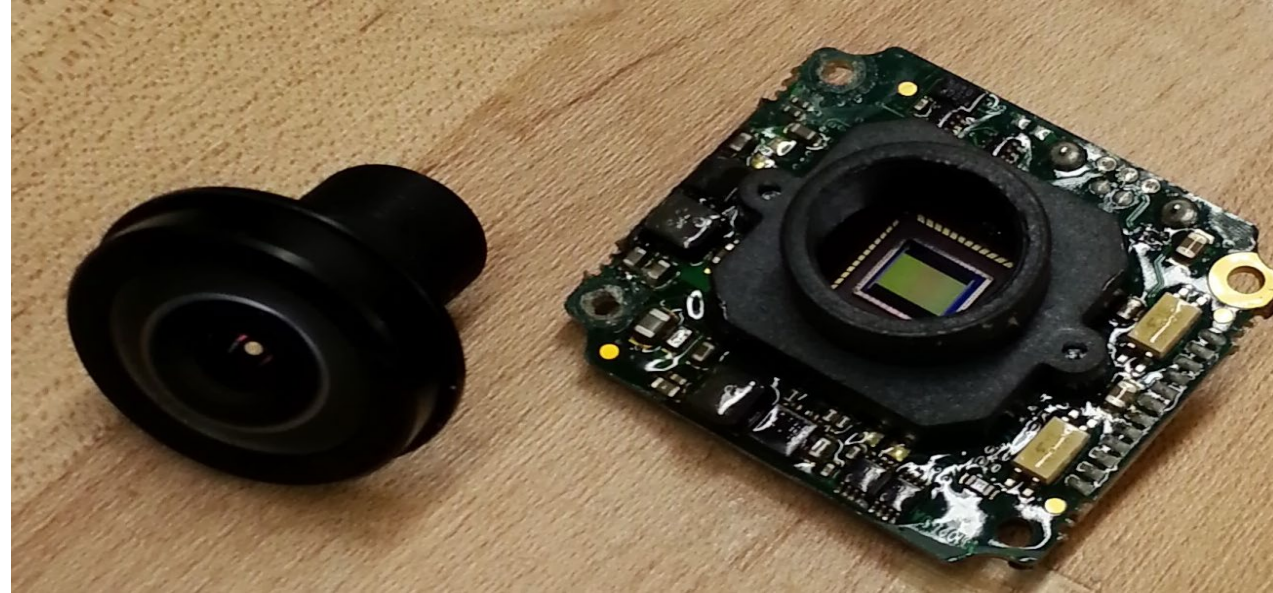
# Geometric Perception Pipeline







# What is a camera?

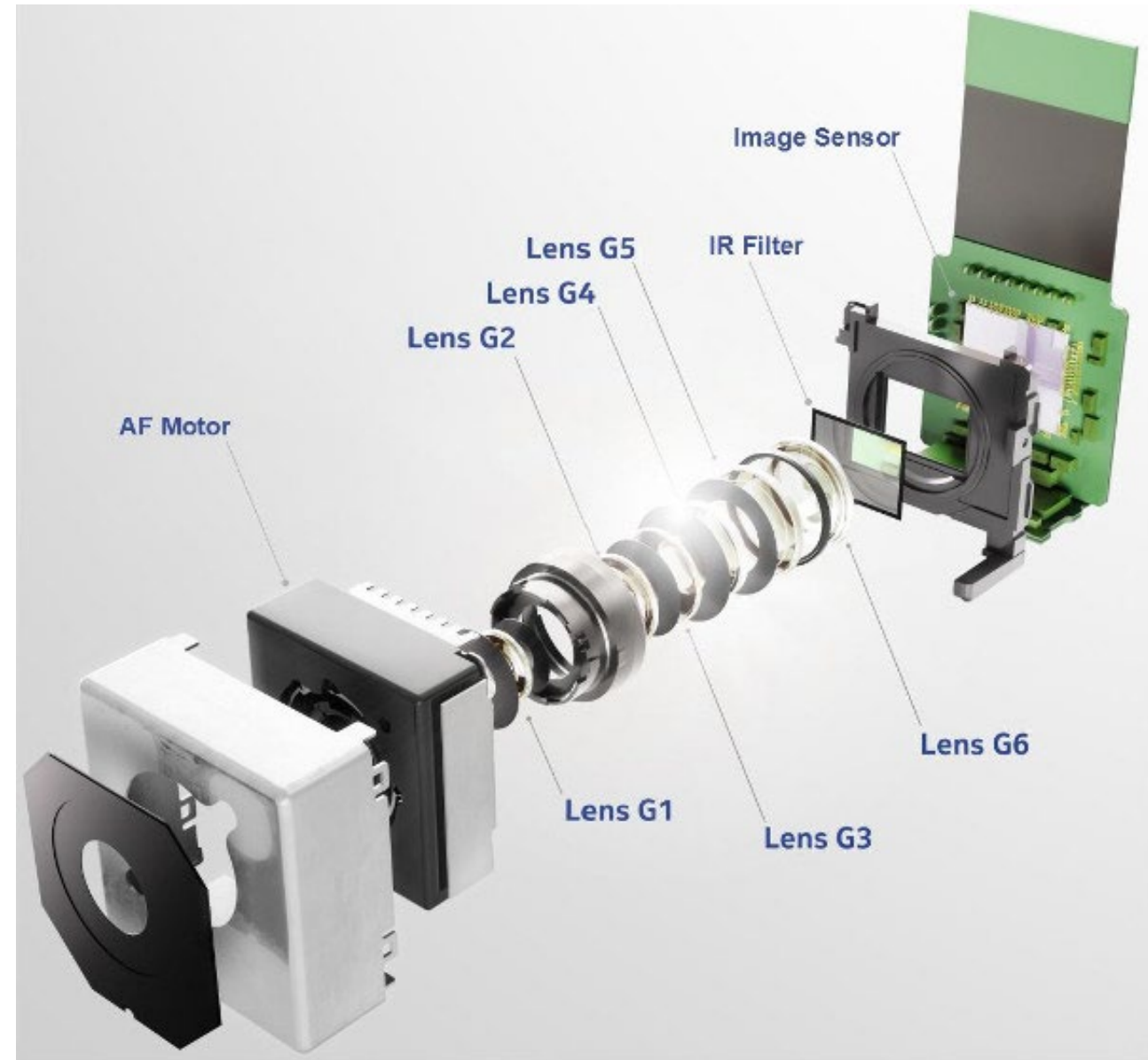


Most basically, an imaging sensor, and a lens

# But real cameras hide many complex details!

- Compound lenses containing many plastic / glass components
- Moving lenses for autofocus
- Lens distortions etc.

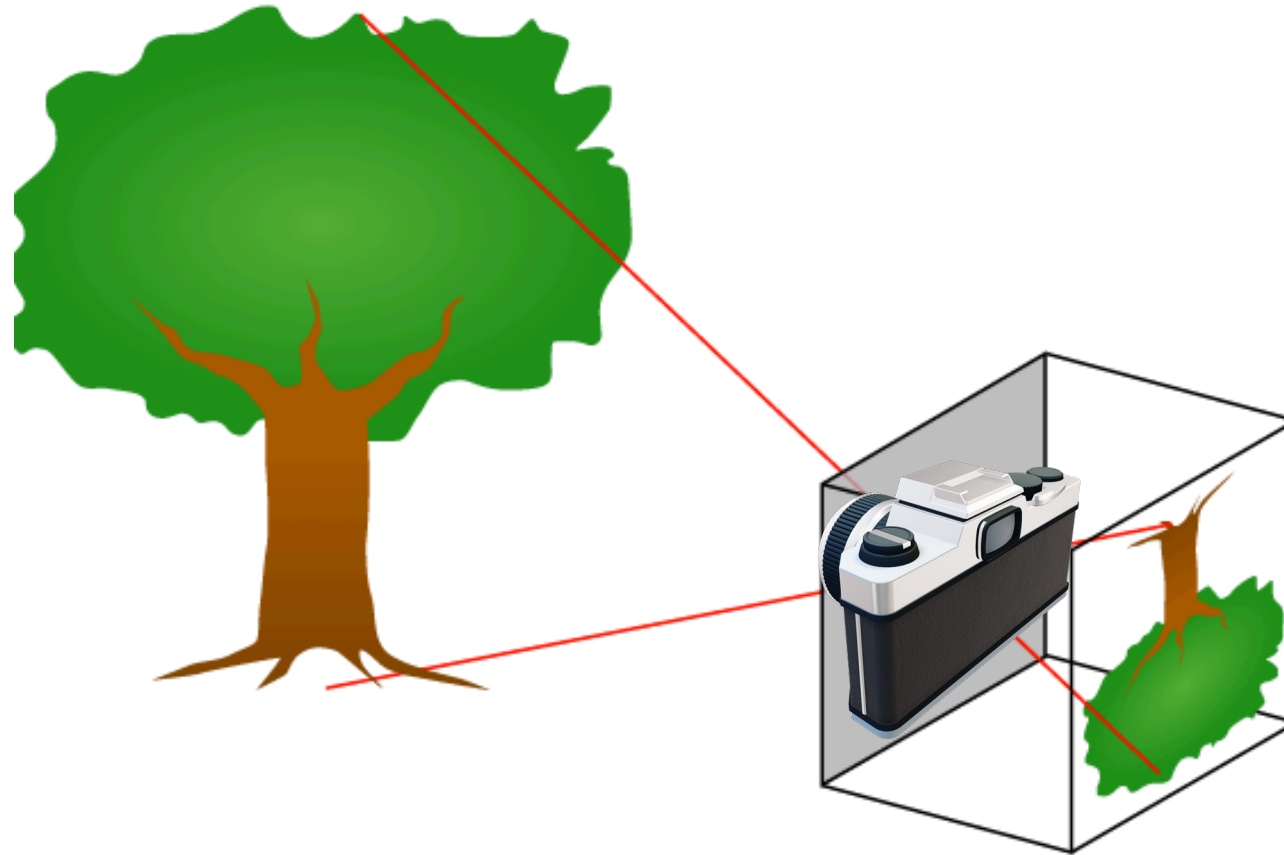
For the most part, we can ignore these complex optics when studying geometric vision, and then deal with them through minor corrections afterwards.



# Introduction to Perspective Projection



# Perspective Projection => Pinhole Camera Model

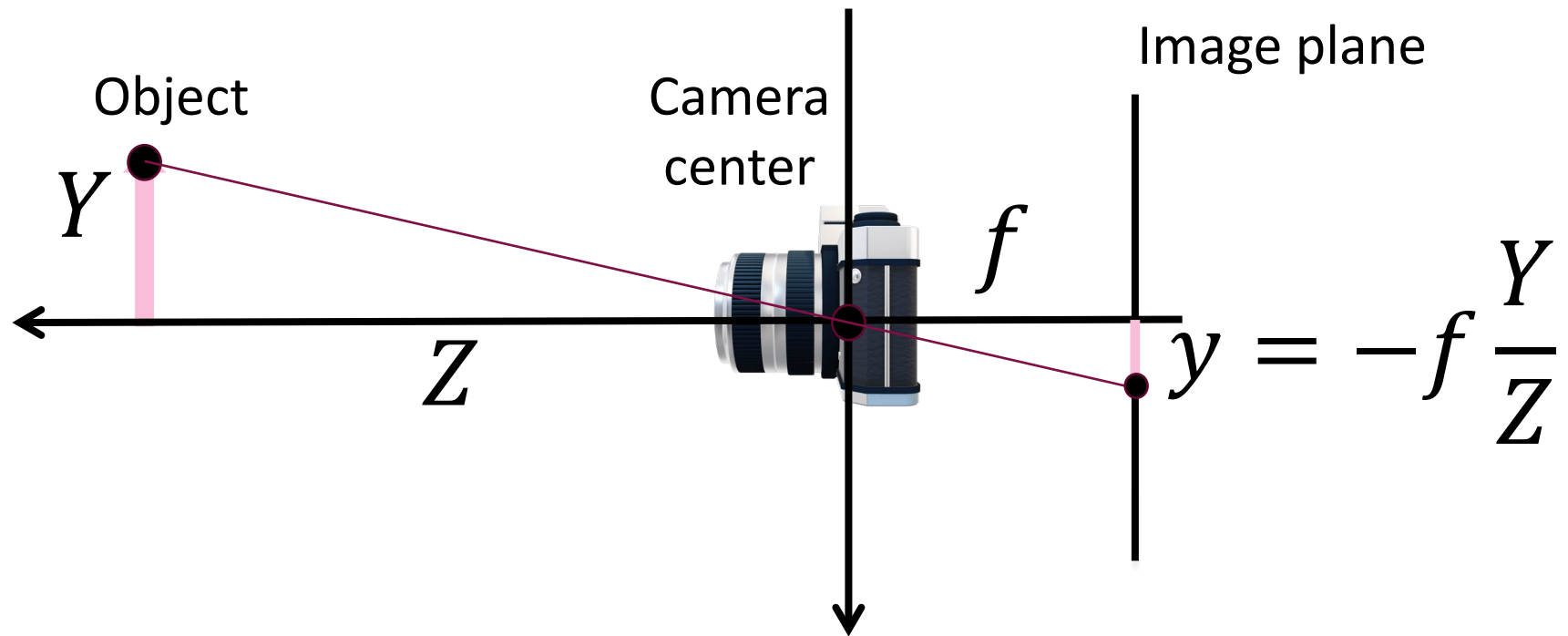


This is a good model for a camera with a lens, as long as the “aperture” is small.



# Pinhole camera model (visualized in 2D)

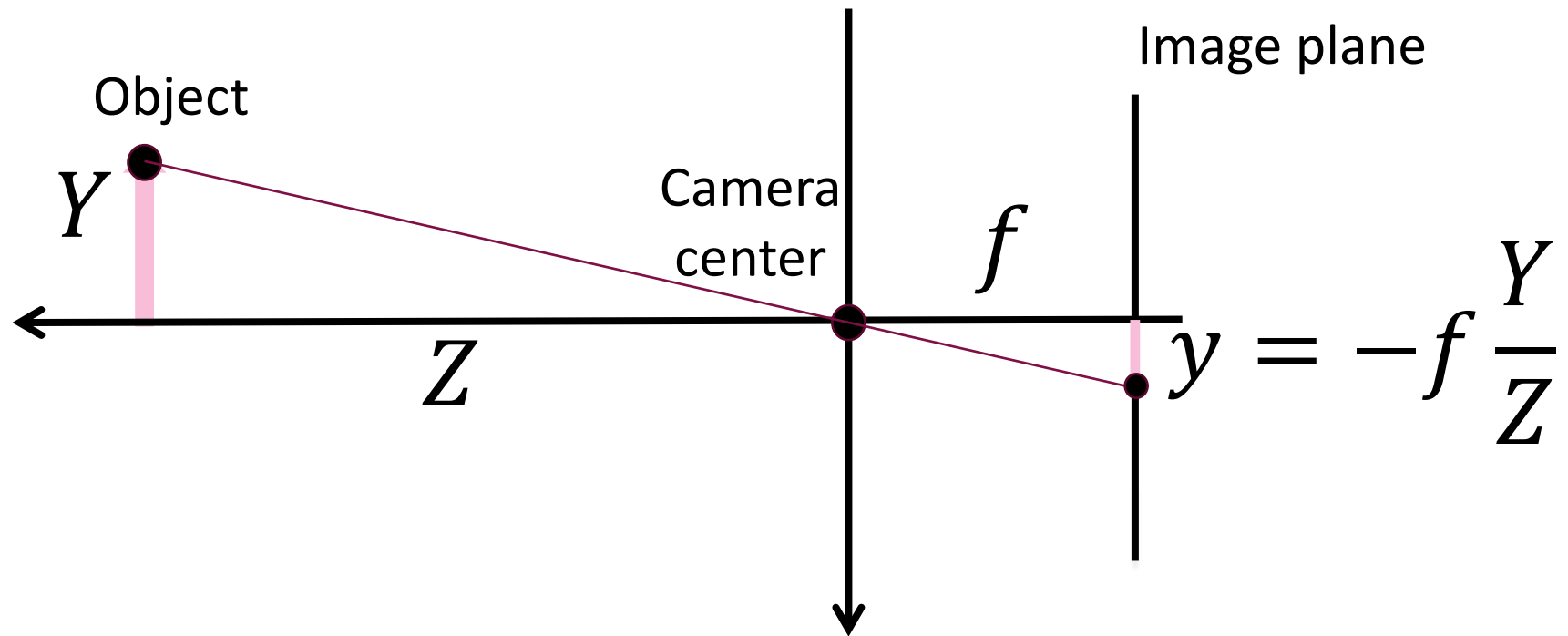
$f$  = distance of camera center from image plane



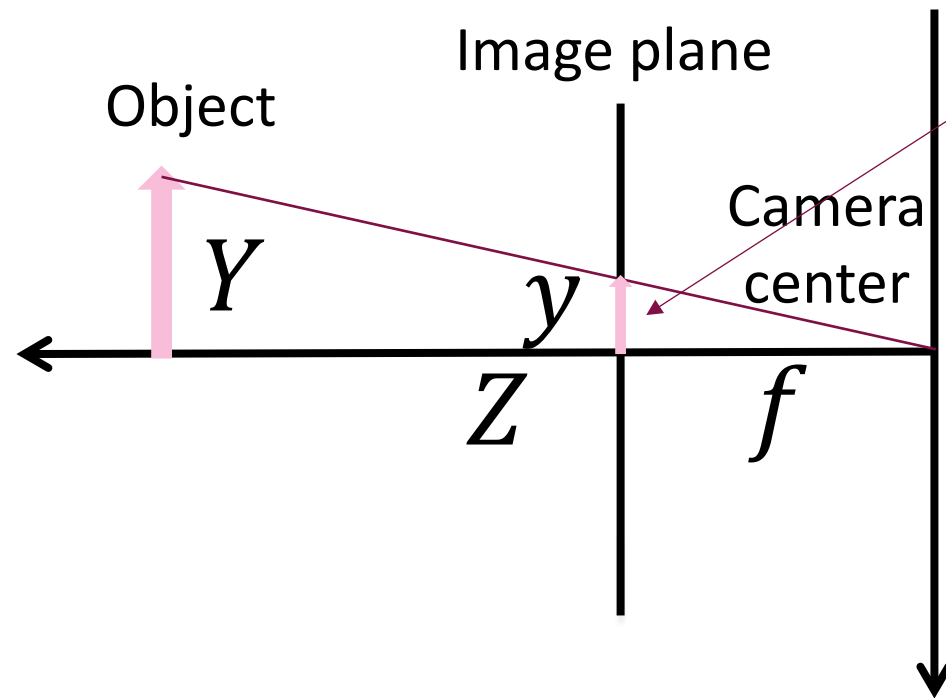
Note how we are expressing all positions like  $Y, Z, y$  in coordinates tied to the camera. We will revisit this choice soon.

# Pinhole camera model (visualized in 2D)

$f$  = distance of camera center from image plane



# Pinhole camera model with the image plane in front

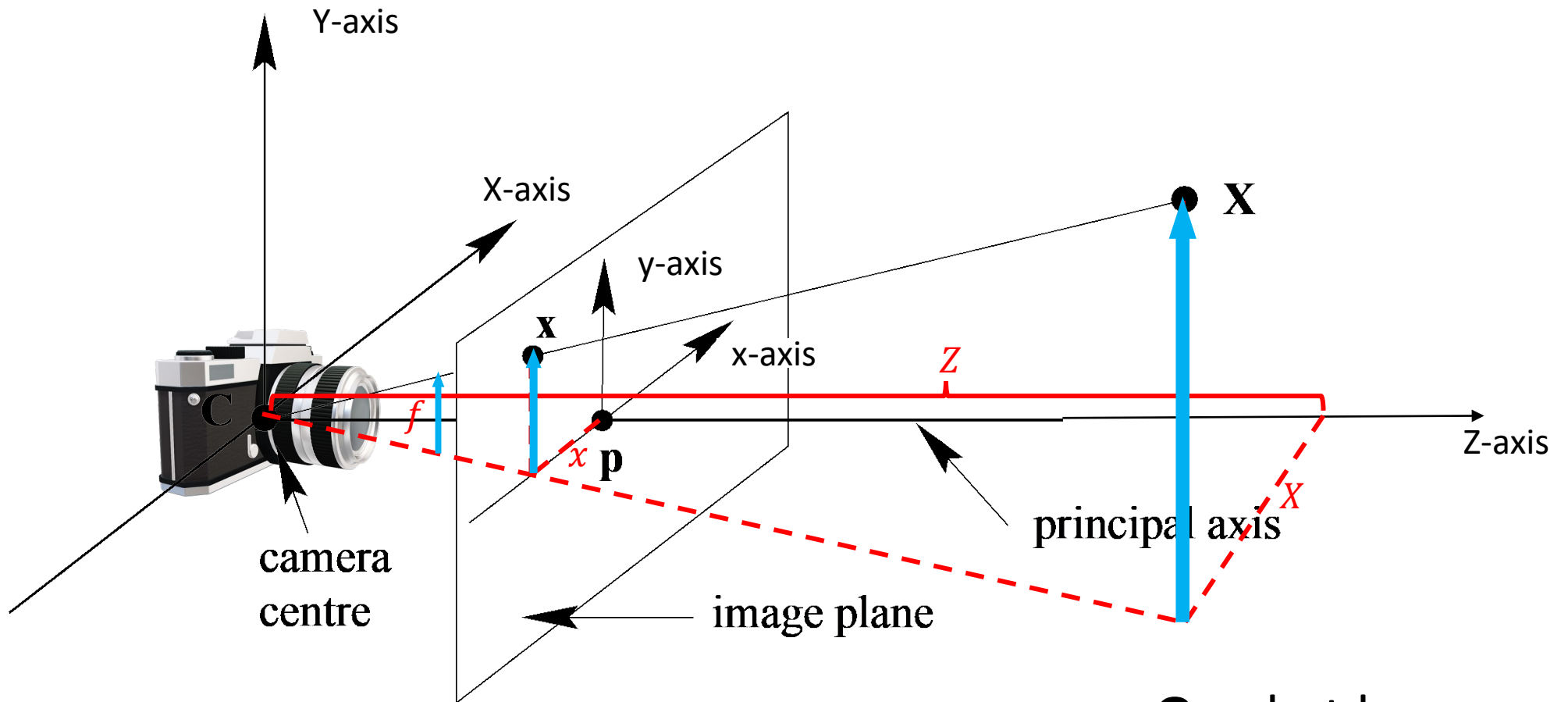


$$y = f \frac{Y}{Z}$$

For convenience move the image plane in front of camera so that objects appear upright.

Can get rid of the pesky negative sign.

# Basic Perspective Projection Equations, now in 3D



$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$

**Q:** what happens to sizes of objects as you move the image plane close or far?

# Perspective Projection in Renaissance Art

Albert Durer (1471-1528)

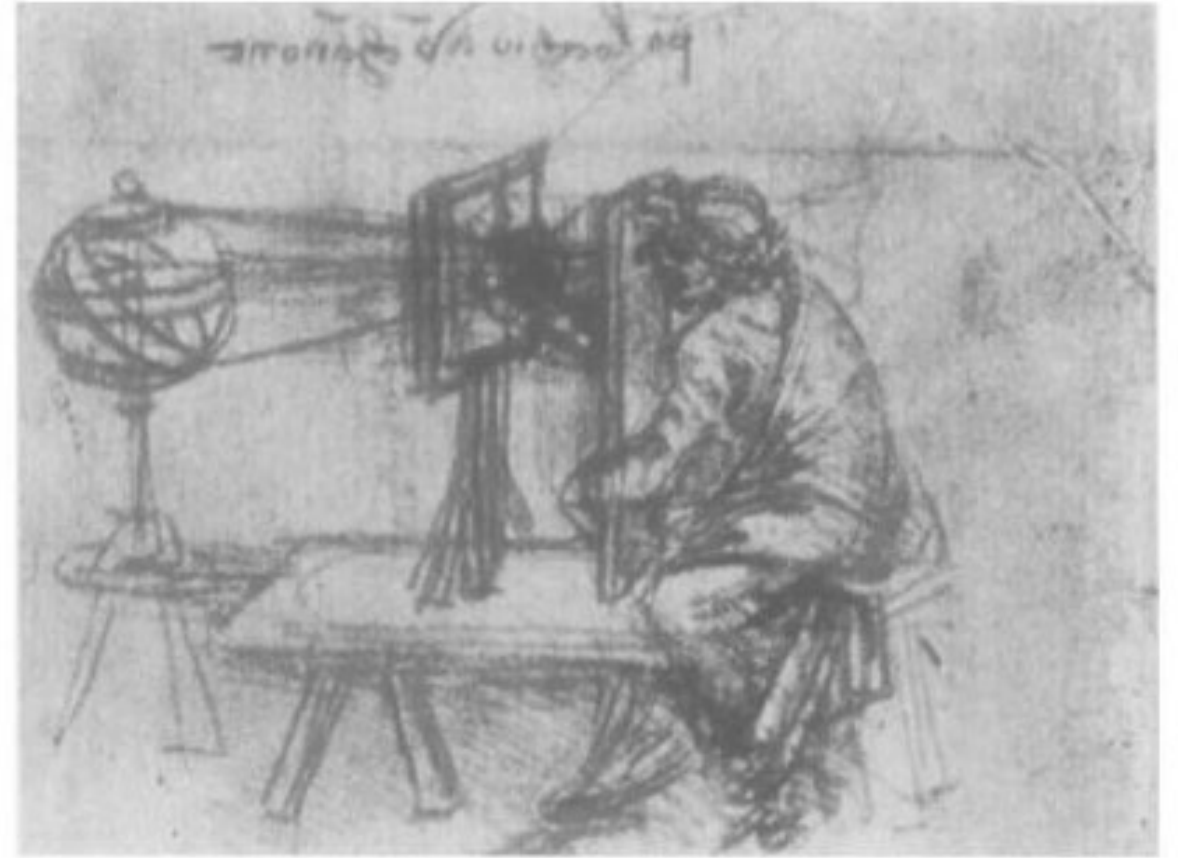
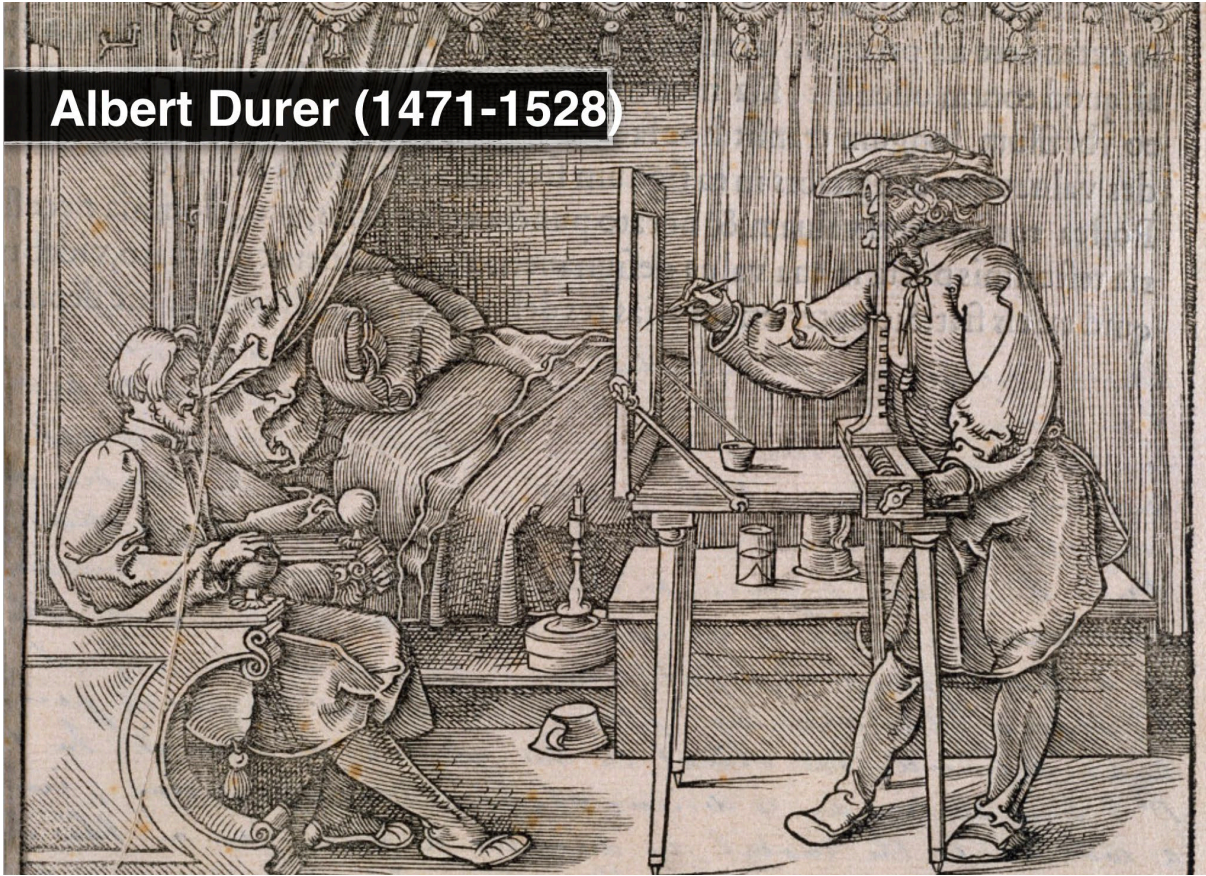


Figure 2.1. Leonardo's technique for making a perspectival drawing of the sphere of the macrocosm (CA 1 ra bis).







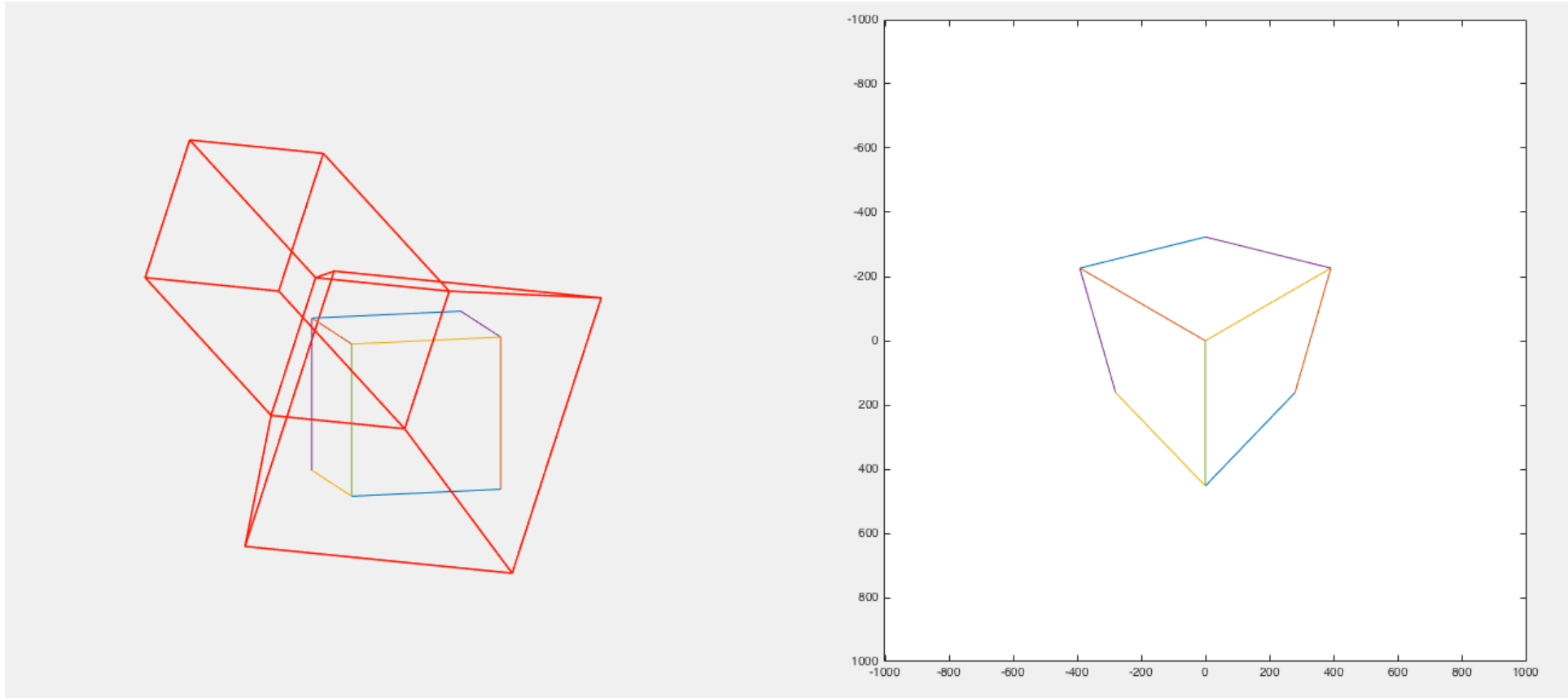
# What properties are preserved by perspective projections?

- Shape?
  - e.g. square, circle, etc.?
- Parallelness / Angles?
- Lengths?
- Ratio of lengths?

**None of the above!**

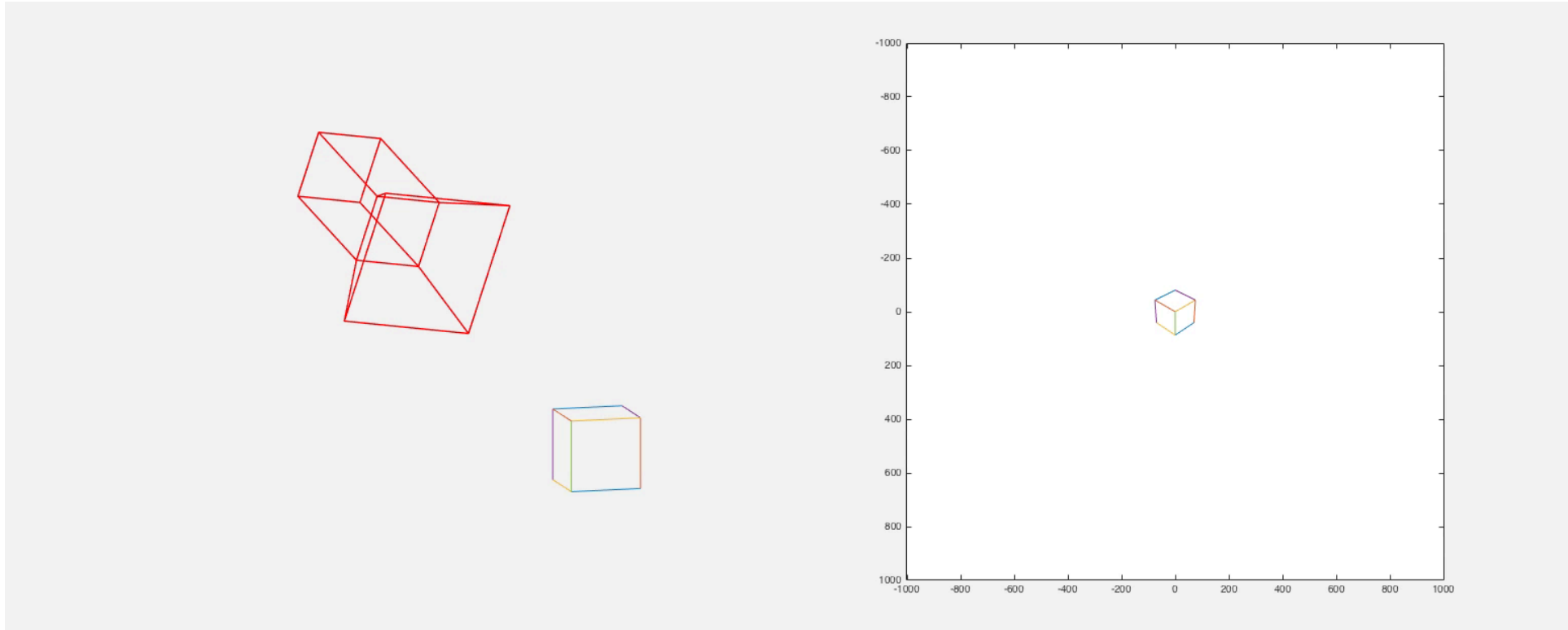
A projection is only required to preserve “straightness”/“collinearity”.

# Perspective effects



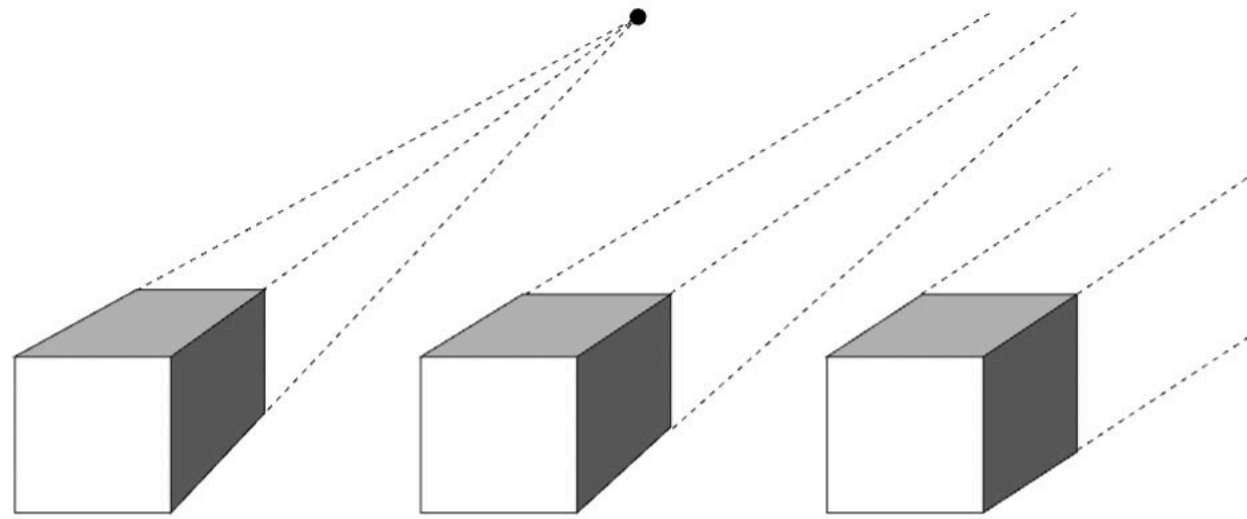
Parallel lines do not remain parallel !

# Perspective effects



Objects decrease in size with distance



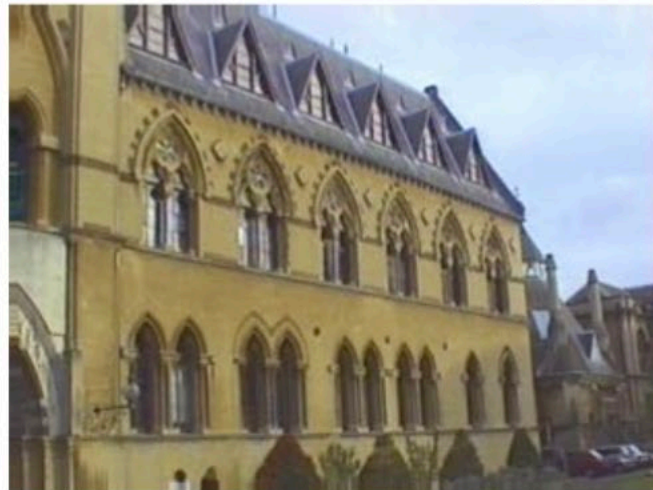


**perspective**

**weak perspective**

————— increasing focal length —————→

————— increasing distance from camera —————→

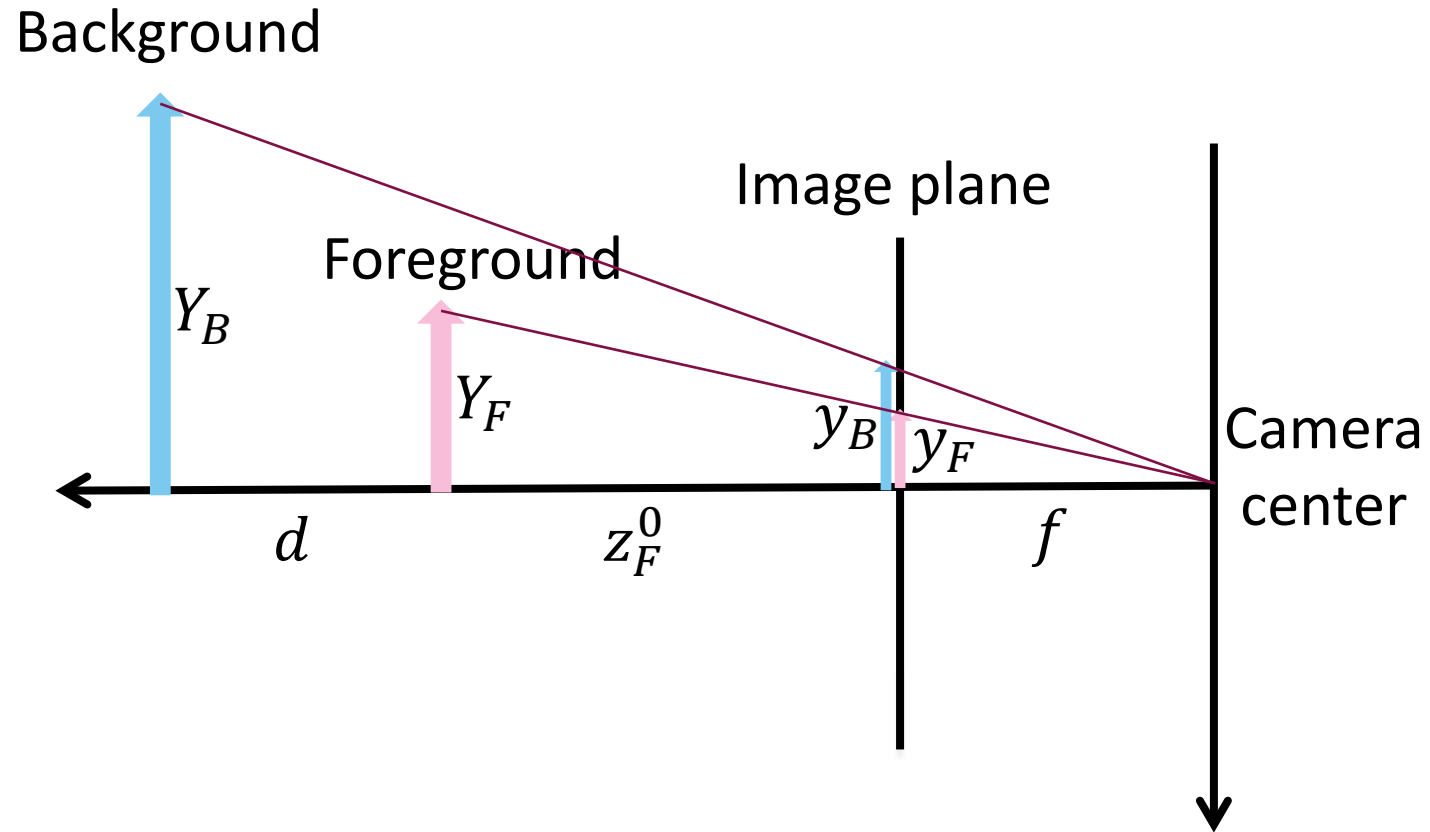


## Dolly Zoom or “Vertigo” Effect: z-motion and Zoom



Goodfellas, Martin Scorsese, 1990

# Dolly Zoom or “Vertigo” Effect: z-motion and Zoom

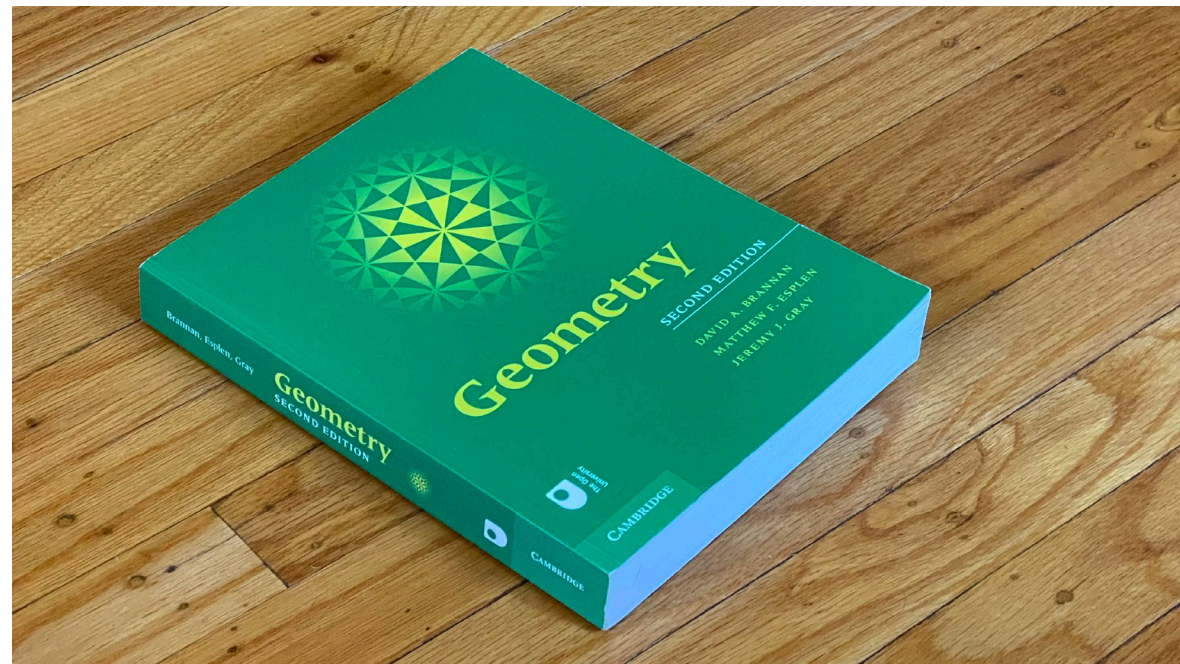
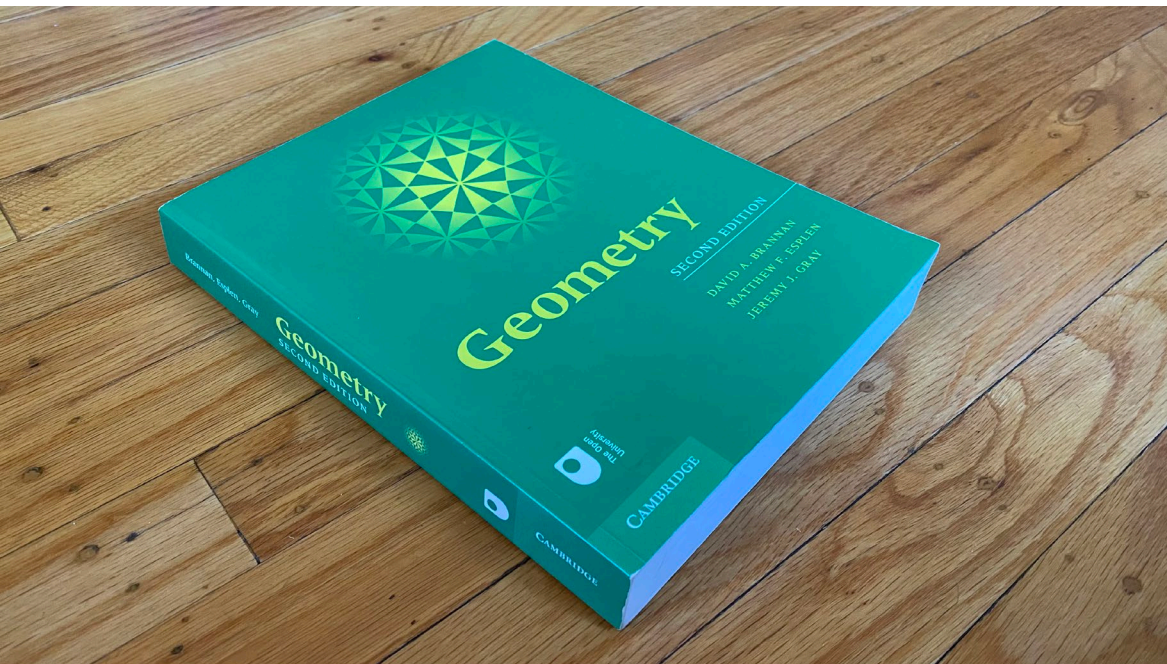


$$y_F = f \frac{Y_F}{Z_F}$$

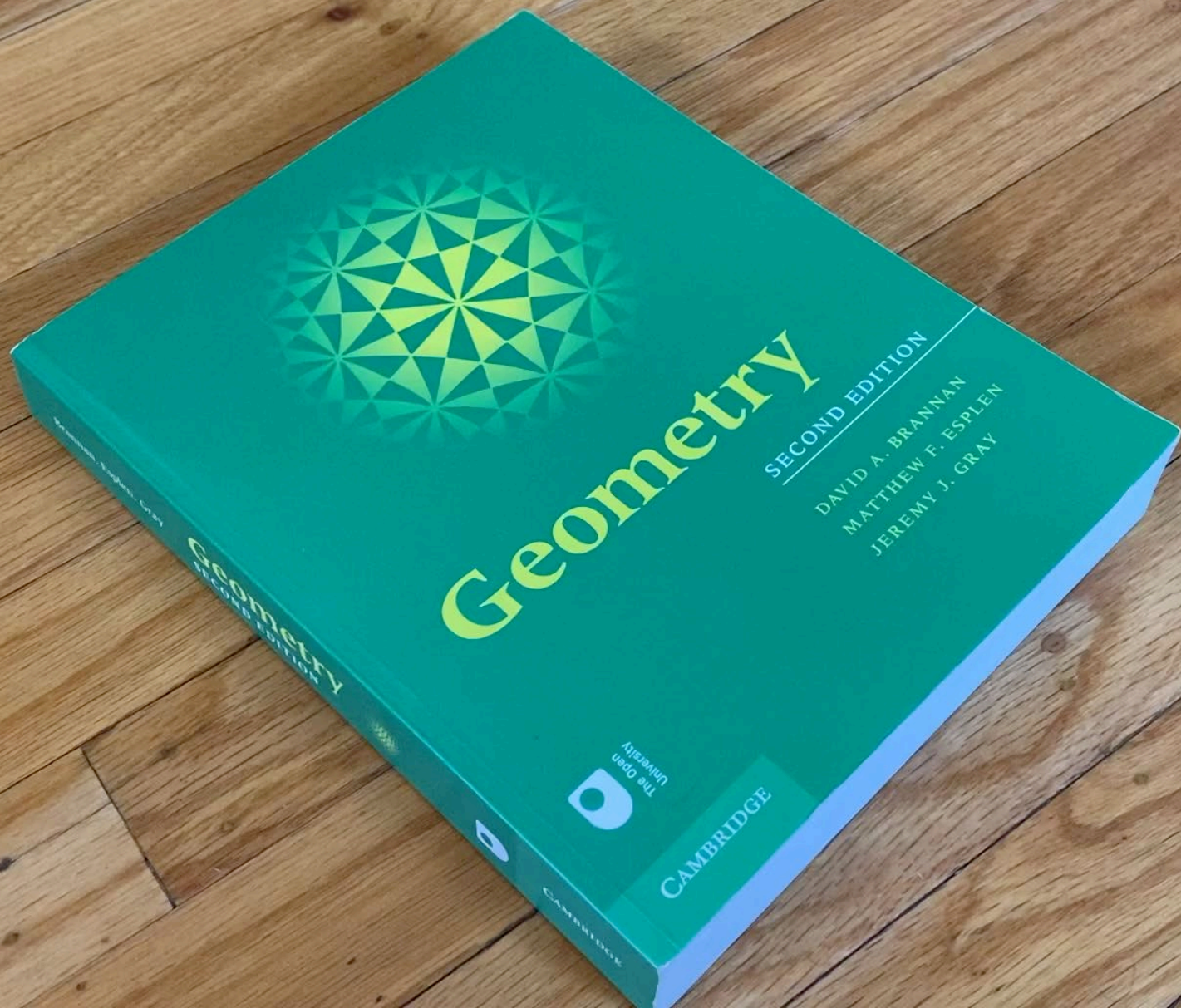
$$y_B = f \frac{Y_B}{Z_B}$$



# Zoom vs distance...







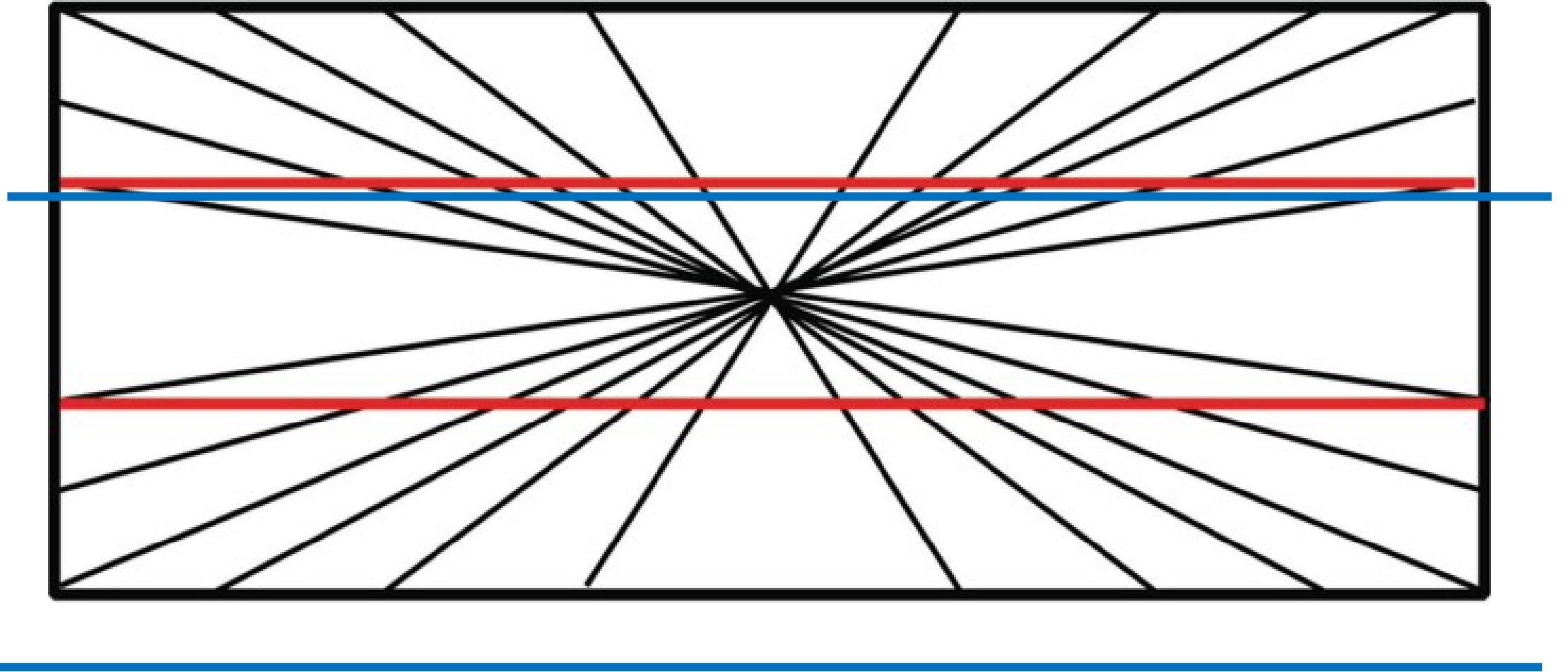


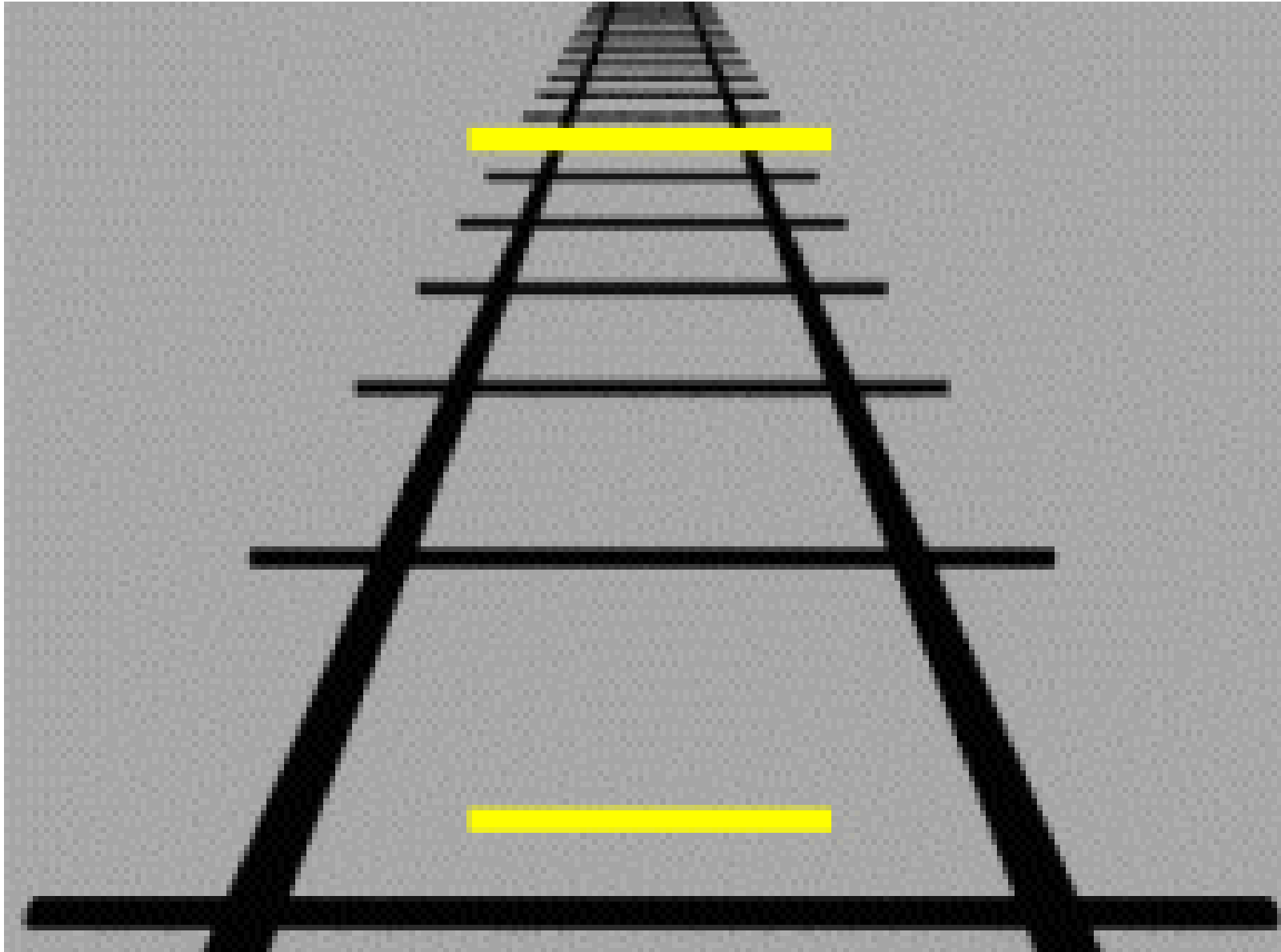


Human visual system is aware of perspective effects!



# Visual Illusion



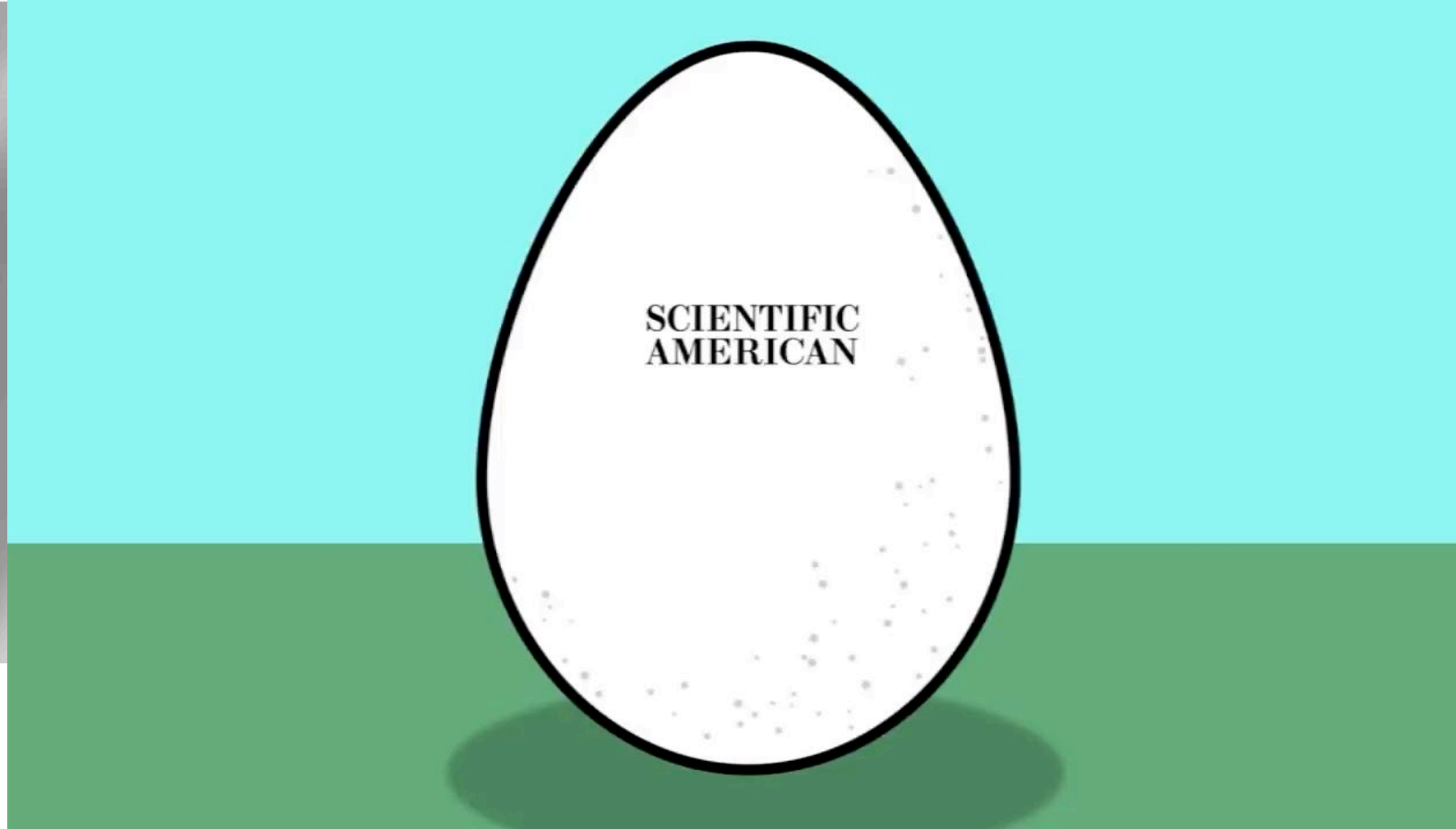
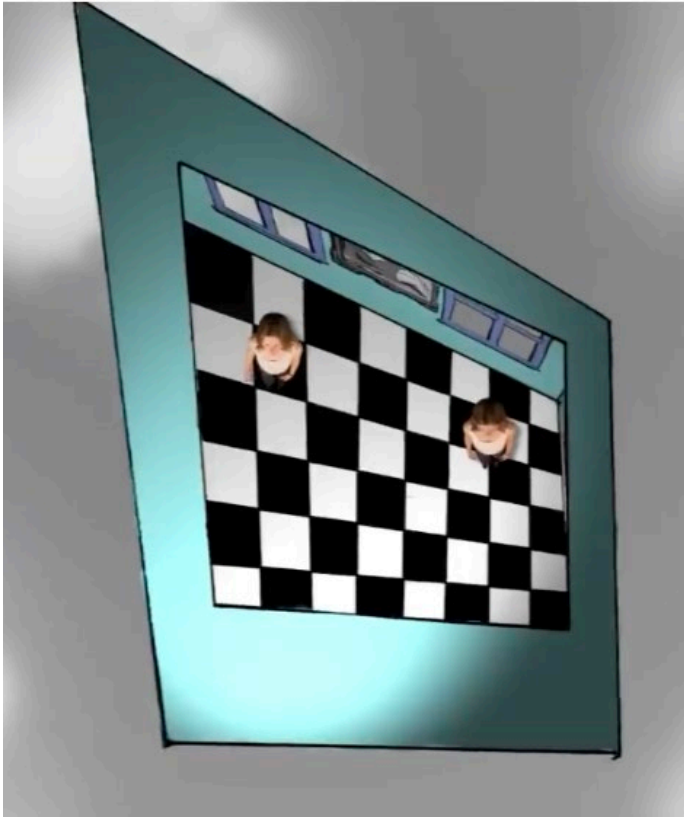




SCIENTIFIC  
AMERICAN



# Ames Room Illusion



<https://www.scientificamerican.com/video/instant-egghead-what-is-the-ames2012-10-09/>

# Ames Room Illusion



<https://www.scientificamerican.com/video/instant-egghead-what-is-the-ames2012-10-09/>



# Vector Algebra Recap: Dot Products

- $\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^T \mathbf{y} = \langle \mathbf{x}, \mathbf{y} \rangle = \sum_i x_i y_i = ||\mathbf{x}|| ||\mathbf{y}|| \cos \theta_{xy}$ 
  - Output is a scalar
  - Measures angles if applied to unit vectors.
  - Measures projection of one vector onto another:  $||\mathbf{y}|| \cos \theta_{xy}$  is projection of  $\mathbf{y}$  onto the direction of  $\mathbf{x}$
- Orthogonality: Two vectors are orthogonal iff  $\mathbf{x}^T \mathbf{y} = \mathbf{0}$ 
  - The equation of a plane in 3D is  $\boldsymbol{\pi}^T \mathbf{x} = \mathbf{0}$ 
    - This means the vector  $\boldsymbol{\pi}$  is normal to the plane
    - (same reasoning also also applies for a 2D line  $\mathbf{l}$ )
- $\mathbf{x} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{x}$



# Vector Algebra Recap: Matrix Products

- $M\mathbf{x} = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}_{2 \times 2} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_{2 \times 1} = \begin{bmatrix} m_{11}x_1 + m_{12}x_2 \\ m_{21}x_1 + m_{22}x_2 \end{bmatrix}_{2 \times 1}$

- Can be reframed in terms of dot products.

- $\mathbf{m}_1 = [m_{11}, m_{12}]$

- $\mathbf{m}_2 = [m_{21}, m_{22}]$

- Then  $M\mathbf{x} = \begin{bmatrix} \mathbf{m}_1 \cdot \mathbf{x} \\ \mathbf{m}_2 \cdot \mathbf{x} \end{bmatrix}_{2 \times 1}$



CIS 580

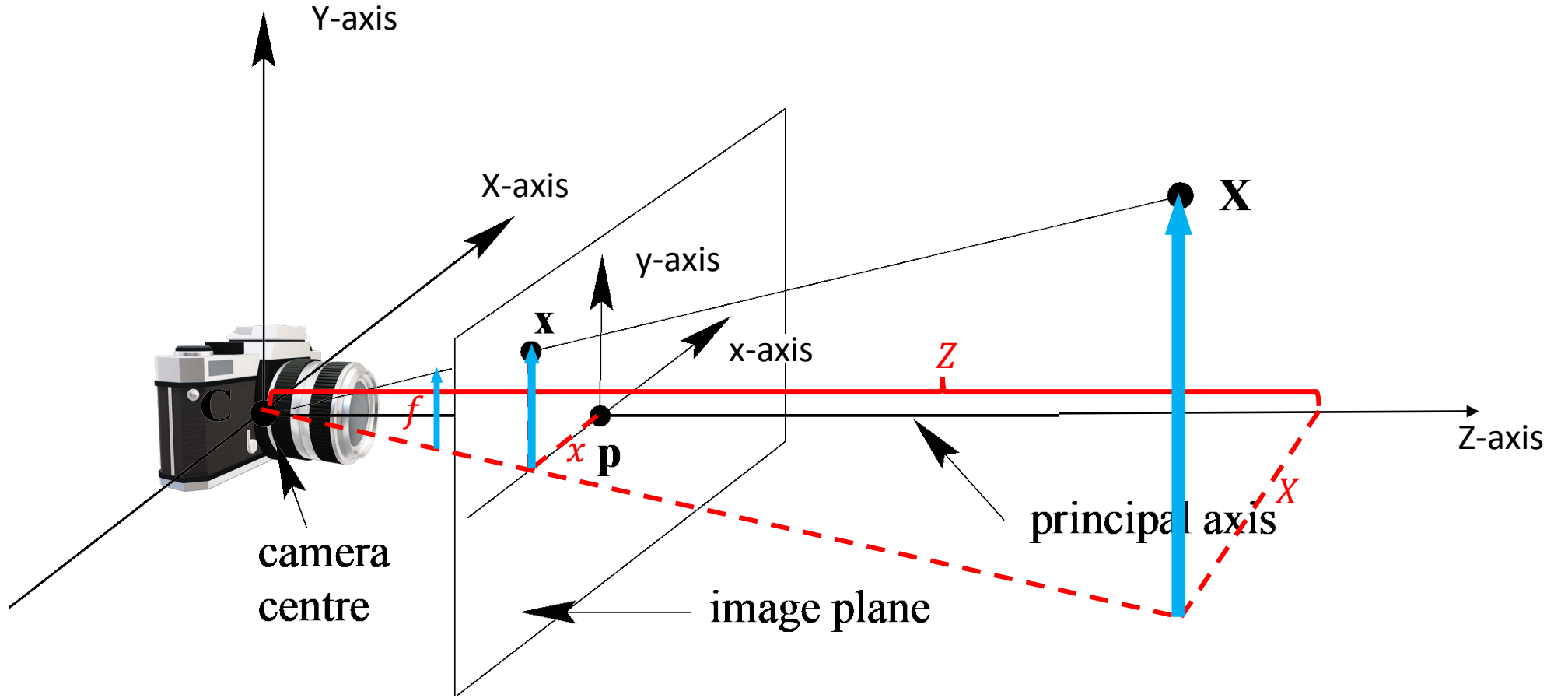
# Machine Perception

*Or Geometric Computer Vision*

Instructor: Lingjie Liu

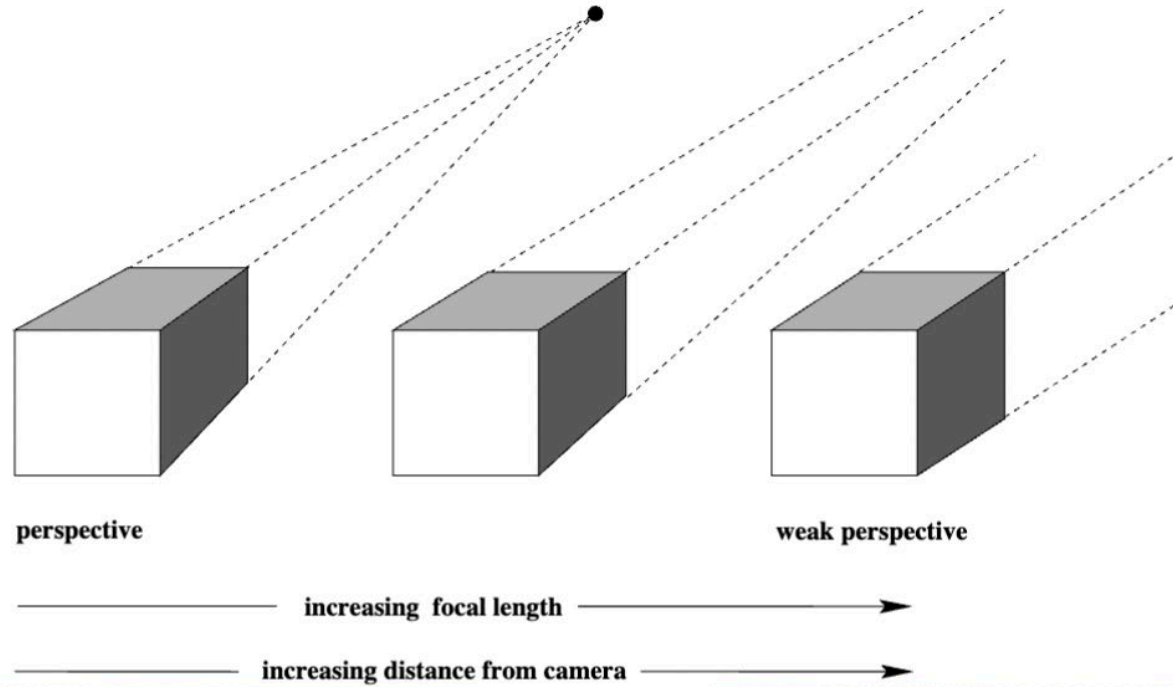
Lec 2: Jan 22, 2026

# Recap: Basic Perspective Projection Equations



$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$

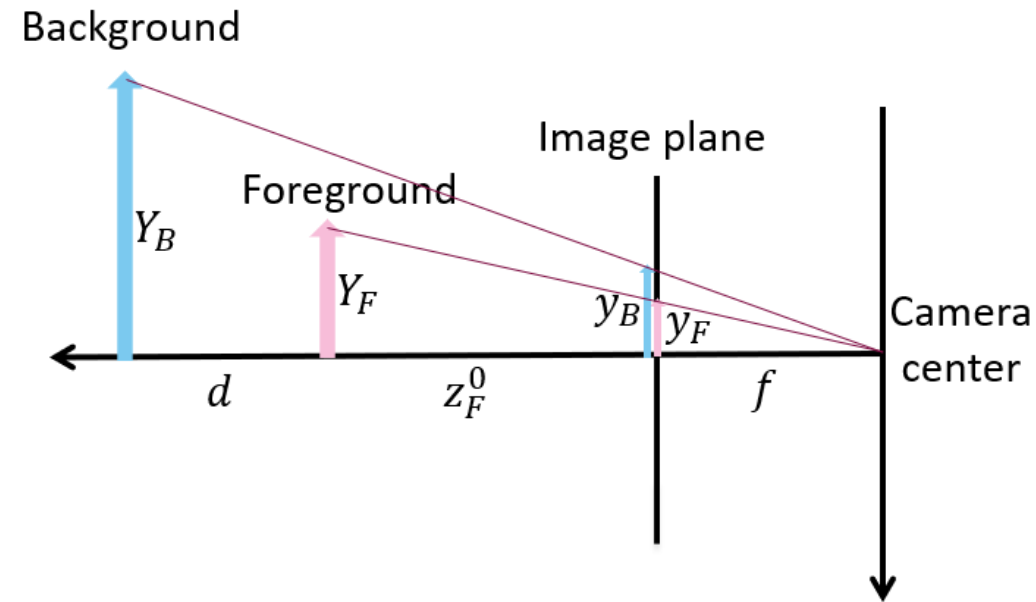
# Recap: Perspective Effects





# Recap: Perspective Effects

## Dolly Zoom or “Vertigo” Effect: z-motion and Zoom

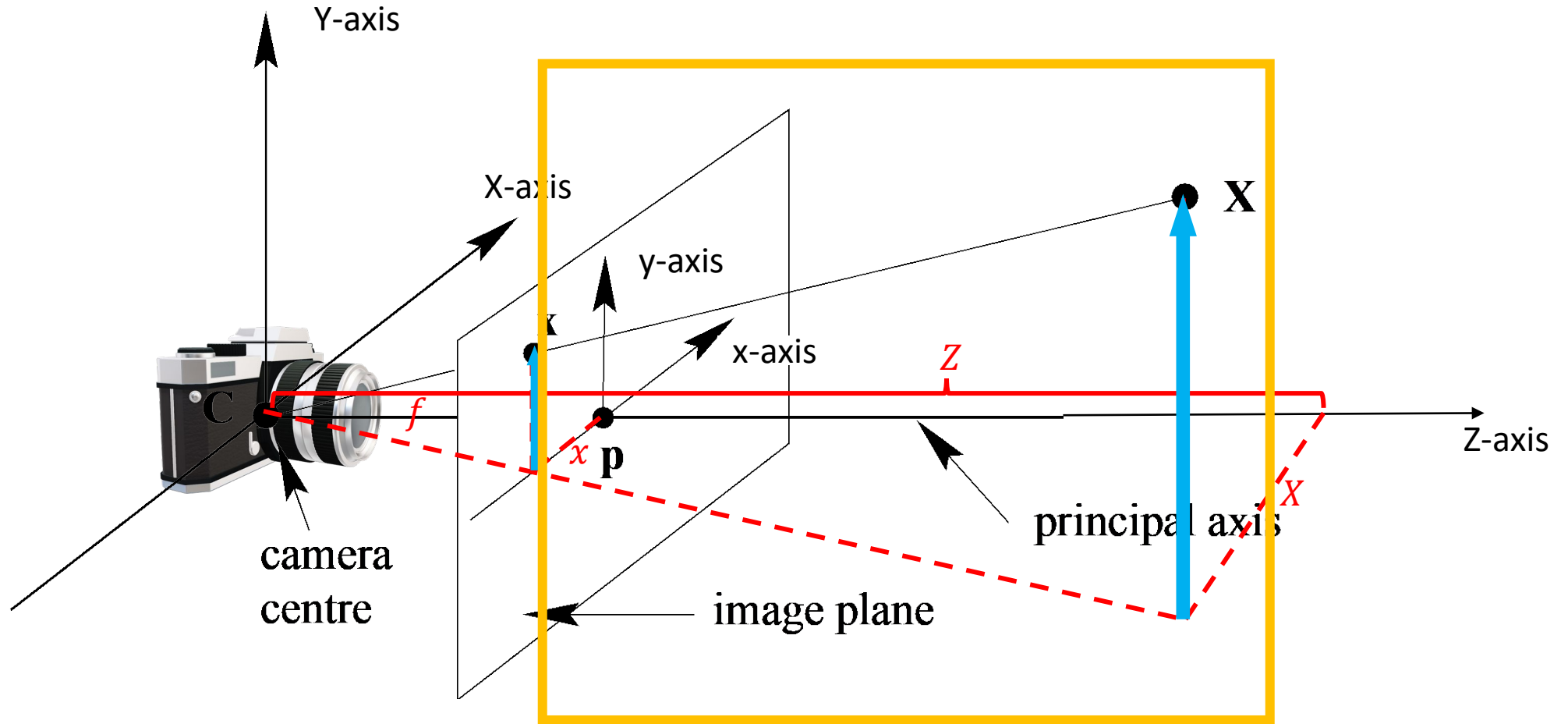


$$y_F = f \frac{Y_F}{Z_F}$$

$$y_B = f \frac{Y_B}{Z_B}$$

# Review: (Euclidean) Geometric Concepts Through Matrix-Vector Algebra

# Recap: Basic Perspective Projection Equations



$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$

# Points and Lines in Euclidean 2D (shortcut $\mathbb{R}^2$ )

- Point in 2D space:  $\mathbf{x} = (x, y)^T \in \mathbb{R}^2$ 
  - Projection onto coordinate axes starting from an origin point (0,0).
  - “Cartesian coordinate system”: requires origin and coordinate frame.
- Line in 2D space:
  - Collection of all points  $(x, y)^T$  that satisfy an equation  $ax + by + c = 0$
  - Can be indicated by the 3-D column vector:  $\mathbf{l} = [a, b, c]^T$ .
  - The equation of a line can then be written as  $\mathbf{l}^T \begin{pmatrix} x \\ y \\ \mathbf{1} \end{pmatrix} = 0$ . (dot product)

# Points and Planes in Euclidean 3D (shortcut $\mathbb{R}^3$ )

- Point in 3D space:  $\mathbf{x} = (x, y, z)^T \in \mathbb{R}^3$
- Plane in 3D space:
  - Collection of all points  $(x, y, z)^T$  that satisfy an equation  $ax + by + cz + d = 0$
  - Can be indicated by the 4-D column vector:  $\boldsymbol{\pi} = [a, b, c, d]^T$ .
  - The equation of a plane can then be written as  $\boldsymbol{\pi}^T \begin{pmatrix} x \\ y \\ z \\ \mathbf{1} \end{pmatrix} = 0$ .



# Lines in $\mathbb{R}^3$

- Line in 3D space:
  - Interpolate between two points  $(x_1, y_1, z_1)^T$  and  $(x_2, y_2, z_2)^T$ .
  - The set of all points that satisfy  $(x, y, z)^T = (x_1, y_1, z_1)^T + \lambda(x_2 - x_1, y_2 - y_1, z_2 - z_1)^T$  for some value of  $\lambda$
  - As a special case of interest: lines through origin:  $(x, y, z)^T = \lambda(x_2, y_2, z_2)^T$

# Lines in $\mathbb{R}^3$

- Alternative definition: intersection of two planes in 3D, so collection of all points that satisfy two equations:

- $\boldsymbol{\pi}_1^T \begin{pmatrix} x \\ y \\ z \\ \mathbf{1} \end{pmatrix} = 0$  and  $\boldsymbol{\pi}_2^T \begin{pmatrix} x \\ y \\ z \\ \mathbf{1} \end{pmatrix} = 0$

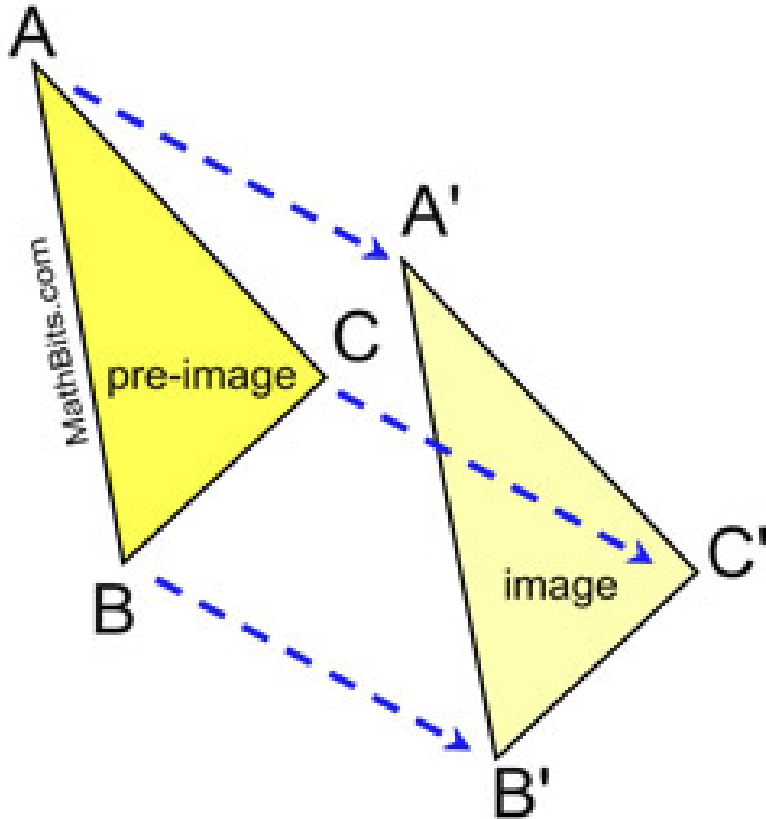
- Can rewrite in matrix form as  $\begin{pmatrix} \boldsymbol{\pi}_1^T \\ \boldsymbol{\pi}_2^T \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

(Line = 1DOF. Needs two linear equality constraints to take away 2DOF. Then 3DOF (because 3D) – 2DOF = 1DOF.)



# Transformations in $\mathbb{R}^3$

- **Shift (Translation):**
- 3 degrees of freedom (DOF)
- Effect on points:  $(x, y, z)^T \rightarrow (x, y, z)^T + \mathbf{t} = (x, y, z)^T + (\mathbf{x}_0, \mathbf{y}_0, \mathbf{z}_0)^T$ 
  - From this, can derive equations for how it affects planes, lines etc.



$$(x, y, z)^T = (x_1, y_1, z_1)^T + \lambda(x_2 - x_1, y_2 - y_1, z_2 - z_1)^T$$

# Transformations in $\mathbb{R}^3$

- **Rotation:**

- 3DOF

- Effect on points:  $(x, y, z)^T \rightarrow \mathbf{R}_{3 \times 3}(x, y, z)^T$

- Where  $\mathbf{R}_{3 \times 3} = [\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3]$  is an *orthonormal* matrix, satisfying:

- $\mathbf{R}^T \mathbf{R} = \mathbf{I}$

- Or equivalently,  $\mathbf{r}_i^T \mathbf{r}_i = \|\mathbf{r}_i\|_2 = 1$  and  $\mathbf{r}_i^T \mathbf{r}_j = 0$  for all  $i \neq j$

- Determinant:  $\text{Det}(\mathbf{R}_{3 \times 3}) = 1$

- 9 entries in R matrix, but only 3 actual DOF.

- Can be expressed in terms of geometric angular rotations in various formalisms.

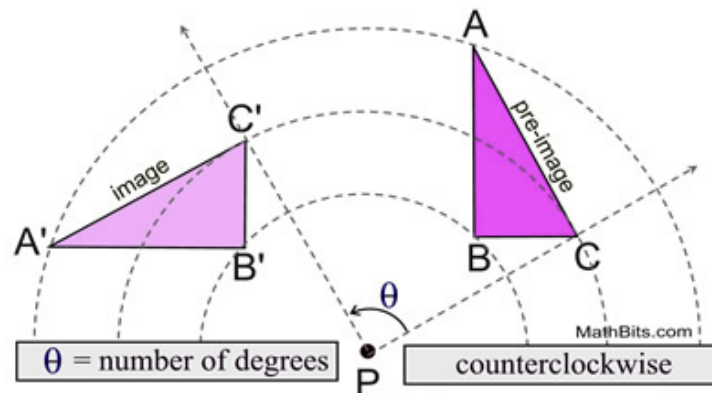
(See: [https://en.wikipedia.org/wiki/Rotation\\_formalisms\\_in\\_three\\_dimensions](https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions))

The determinant of a  $2 \times 2$  matrix is

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc,$$

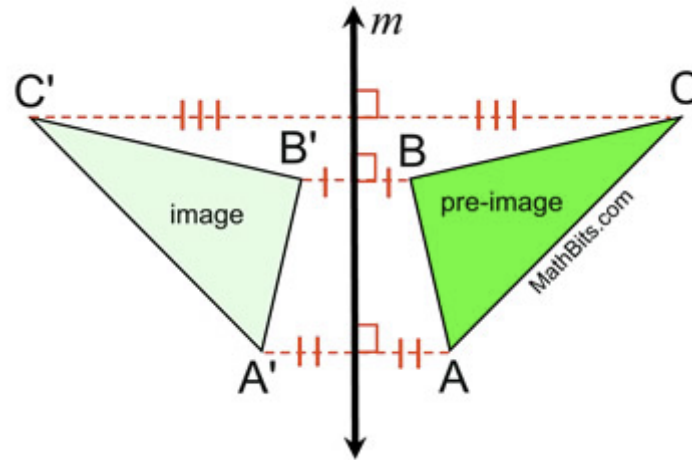
and the determinant of a  $3 \times 3$  matrix is

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = aei + bfg + cdh - ceg - bdi - afh.$$



# Transformations in $\mathbb{R}^3$

- **Reflection**
- Determinant:  $\text{Det}(R_{3 \times 3}) = -1$



How many DOF does a reflection transformation in  $\mathbb{R}^3$  have?



# Transformations in $\mathbb{R}^3$

- “**Euclidean Transformations**” – Rigid Transformation
- The rigid transformations include [rotations](#), [translations](#), [reflections](#), or any sequence of these. **Reflections are sometimes excluded from the definition of a rigid transformation** by requiring that the transformation also preserve the [handedness](#) of objects in the Euclidean space. (A reflection would not preserve handedness; for instance, it would transform a left hand into a right hand.) To avoid ambiguity, a transformation that preserves handedness is known as a **rigid motion**, a **Euclidean motion**, or a **proper rigid transformation**.
- arbitrary compositions of rotations and shifts. 6 DOF
  - Special because they preserve all lengths (“Isometry”), angles, perimeter, and area.

# Transformations in $\mathbb{R}^3$

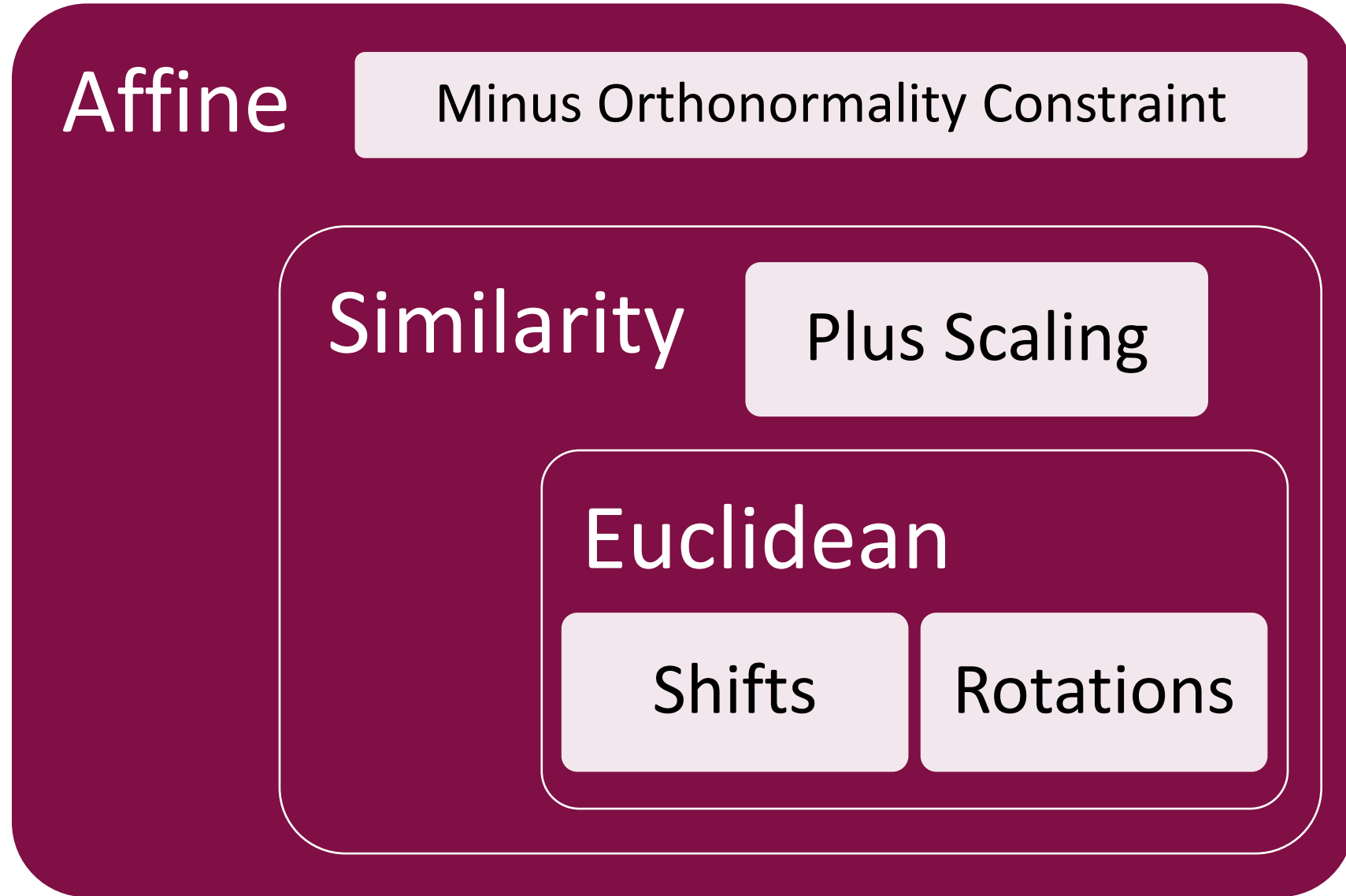
- **Scaling- 1DOF**
  - Effect on points:  $(x, y, z)^T \rightarrow \lambda(x, y, z)^T$
- **“Similarity Transformations”** = combinations of rotations, shifts, scaling.
- 3+3+1=7DOF. Preserves lengths up to a scaling factor
- **“Affine Transformations”**:
  - Remove the orthonormality constraint on  $R$ . It is no longer meaningfully treatable as a “rotation”. So we have a new, arbitrary 3x3 matrix  $A$ 
    - $(x, y, z)^T \rightarrow A_{3 \times 3}(x, y, z)^T + \mathbf{t}$
    - 12 DOF = 9 DOF for  $A$  + 3 DOF for translation
    - Note: does it make sense to do “scaling” any more?
      - Hint: it is equivalently just absorbed into  $A_{3 \times 3}$ .
  - Still preserves parallel lines

# Transformations in $\mathbb{R}^3$

- “Affine Transformations”:  $(x, y, z)^T \rightarrow A_{3 \times 3}(x, y, z)^T + \mathbf{t}$ 
  - preserves parallel lines

Proof?

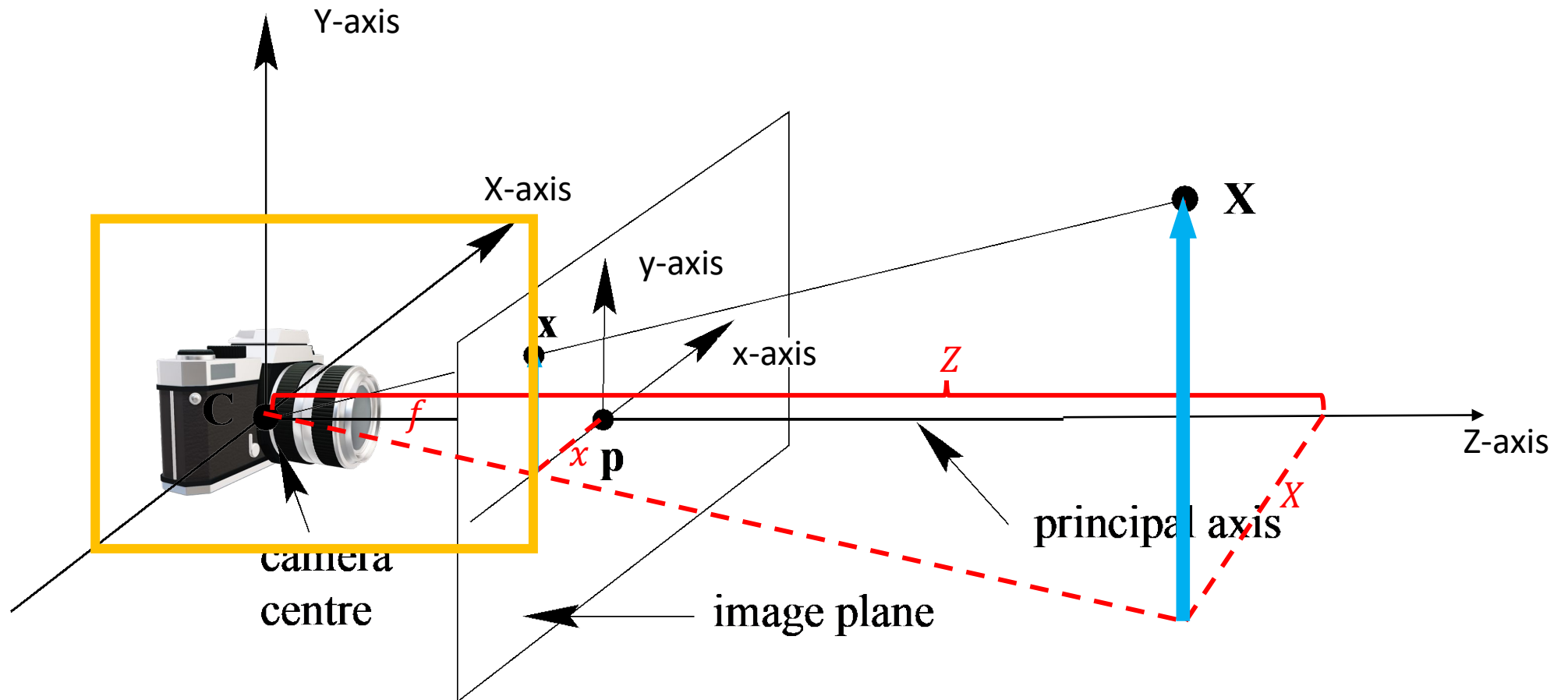
# Hierarchy of Transformations



Soon an even more general transformation (“projective”)!



# Recap: Basic Perspective Projection Equations



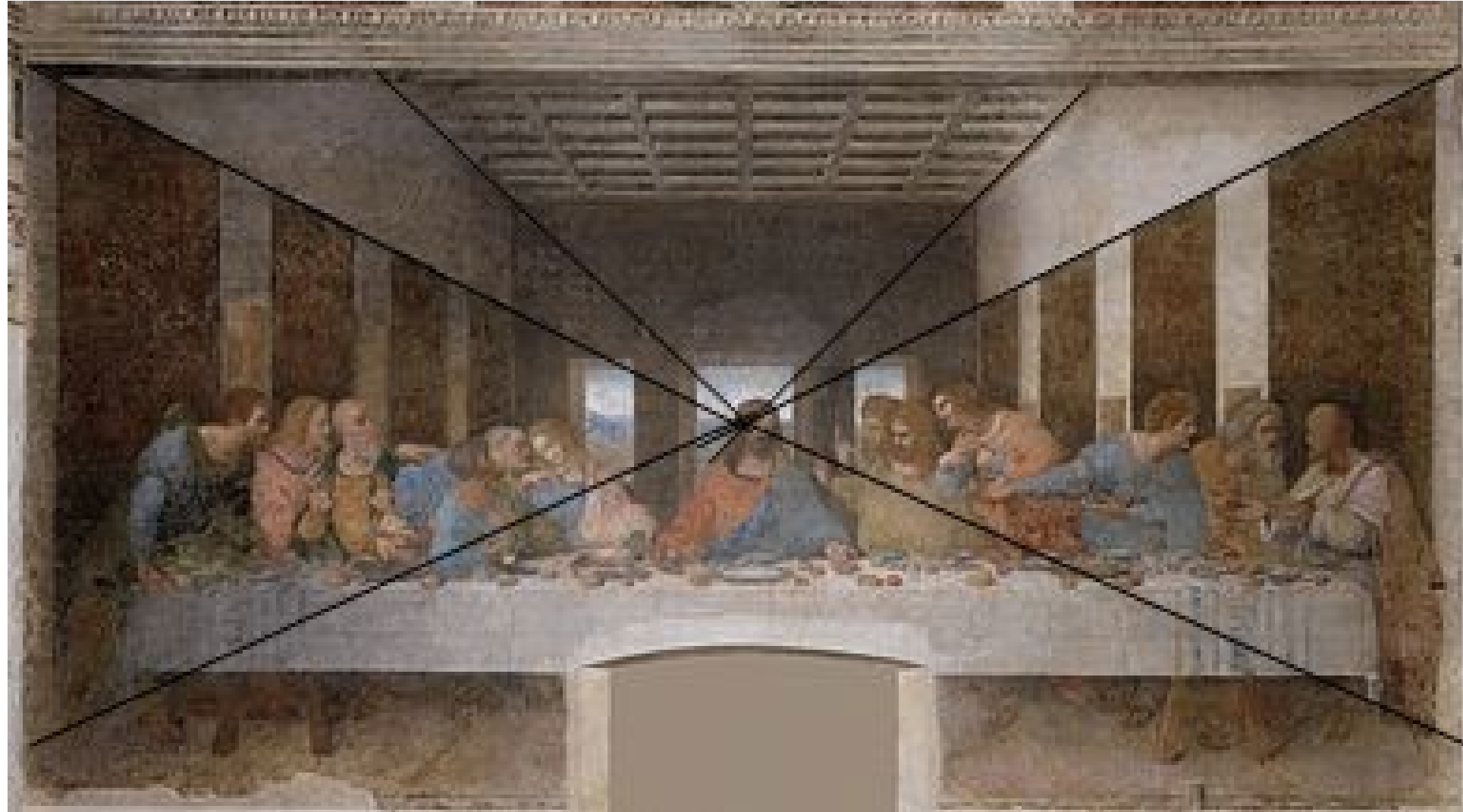
$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$

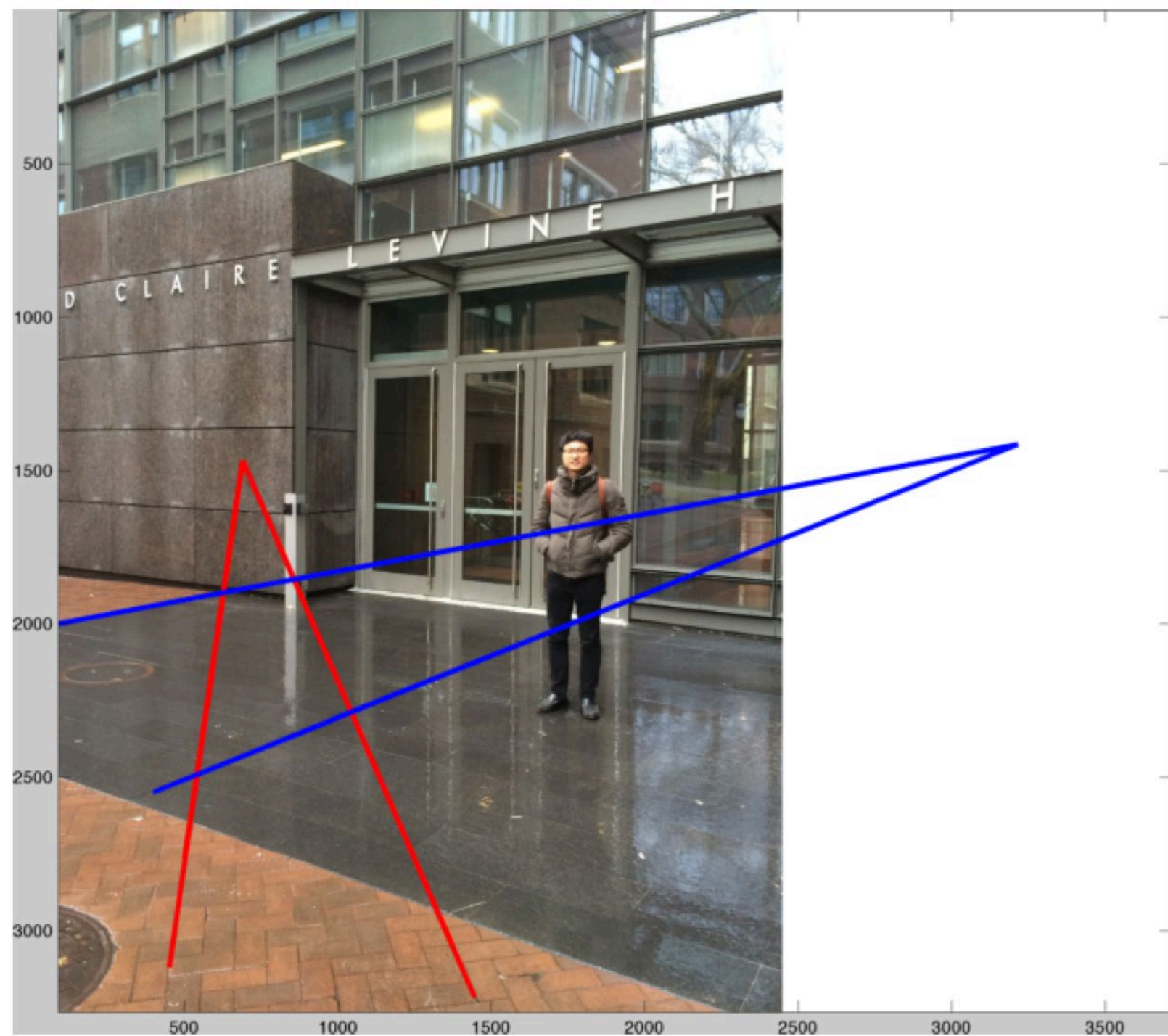


# Projective Geometry

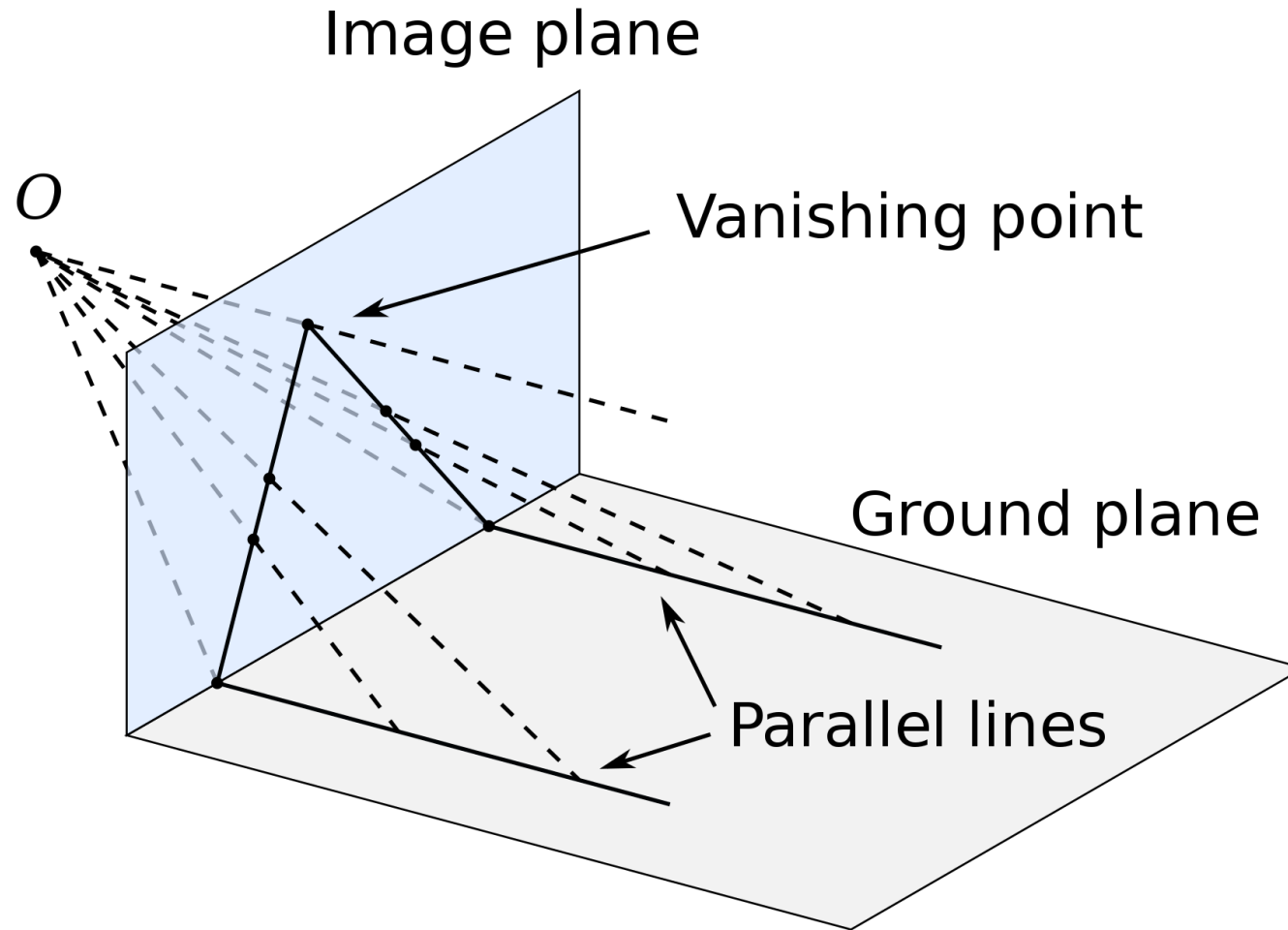
# Vanishing points

A vanishing point is a point on the image plane of a perspective rendering where the two-dimensional perspective projections of mutually parallel lines in three-dimensional space appear to converge.





# Where vanishing points come from



Images of points “at infinity” are often finite!  
In fact, they will turn out to be surprisingly important.  
Need ways to deal with them conveniently and well!

Enter Projective Geometry

We will return to deal with vanishing points and much more  
once we know some projective geometry!





# Projective Geometry

- Extension of Euclidean geometry that deals with points at infinity
  - Other than that, Projective Geometry keeps many of the same features as Euclidean geometry.
- A key property is that two lines always meet in a point
  - (sometimes a point at infinity)

# “Homogeneous coordinates”: Euclidean $\mathbb{R}^n \rightarrow$ Projective $\mathbb{P}^n$

- A point in n-D Euclidean space  $\mathbb{R}^n$  can be injected into n-D projective space  $\mathbb{P}^n$  through the “homogeneous coordinates” notation: simply add one more vector element, and set it to 1.

## ***Homogeneous coordinates***

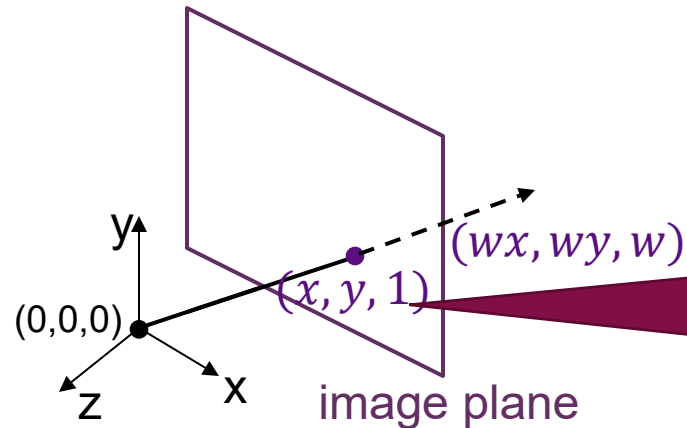
- represent coordinates in  $\mathbb{P}^2$  with a 3-vector

$$\begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{\text{homogeneous coords}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

← We will normally use  $w$  to denote this homogeneous vector element

# Projective $\mathbb{P}^n \rightarrow$ Euclidean $\mathbb{R}^n$

- Just drop the 1? Not so fast!
  - Points in  $\mathbb{P}^2$  do not all have  $w = 1$  e.g.  $(x, y, w = 2)^T$
- Key property:  $(x, y, 1) = (2x, 2y, 2) = (wx, wy, w)$ 
  - So  $(x, y, 2) = (x/2, y/2, 1)$  in  $\mathbb{P}^2 = (x/2, y/2) \in \mathbb{R}^2$
- Geometric intuition matching to perspective projection:
  - a point in the image is a ray in projective space



Reduce any  $(wx, wy, w) \in \mathbb{P}^2$   
to  $w = 1$  to get image plane  
Euclidean  $\mathbb{R}^2$  coordinates  $(x, y)$

- Each *point*  $(x, y)$  on the plane is represented by a *ray through origin*  $(wx, wy, w)$ 
  - all points on the ray are equivalent:  $(x, y, 1) \approx (wx, wy, w)$

# What about when $w = 0$ ?

- If the homogeneous coordinate is 0,  $(x, y, w = 0)$  in projective coordinates corresponds to what points, what rays?
  - Points at infinity! (Think  $x/0, y/0$ )
  - Equivalently, rays through origin (camera center), parallel to the projective image plane
- This property of projective geometry allows us to handle points at infinity through simple vector operations!

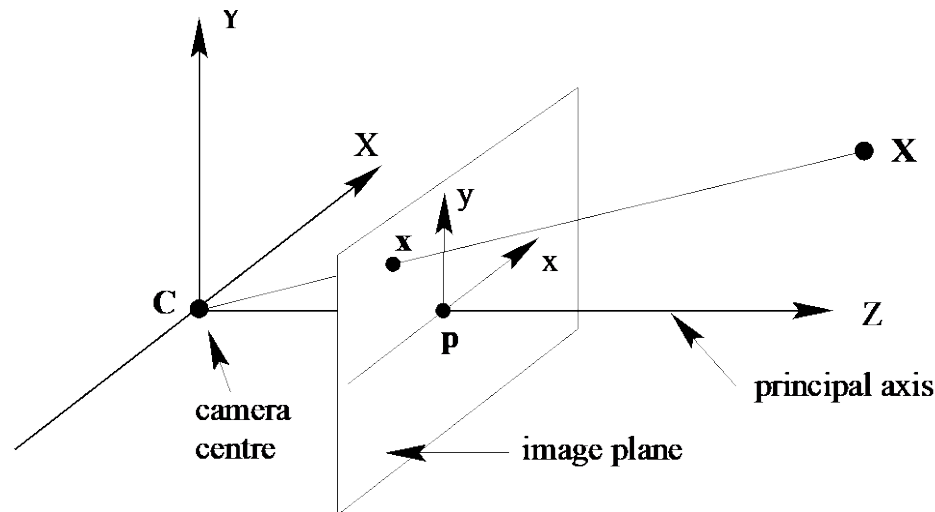
Note:  $\mathbb{P}^2$  does not contain  $(0,0,0)$ .

Projective geometry is ubiquitous in geometric vision, which models image formation as a map from 3-D space into 2-D projective space.

# Perspective Projections are Linear in $\mathbb{P}$

Recall:

$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$



$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix}$$

This encounter with the camera projection equation will not be our last.  
More general versions soon!

# The projective plane in computer vision

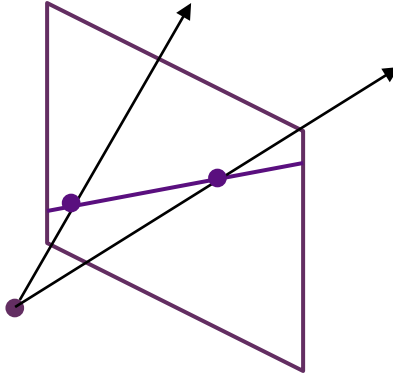
The projective image plane  $\mathbb{P}^2$  consists of all rays through the origin  $\lambda \begin{bmatrix} u \\ v \\ w \end{bmatrix}$ ,  
which correspond either to points in the image plane  $\begin{bmatrix} x = u/w \\ y = v/w \\ 1 \end{bmatrix}$ ,  
or to points at infinity  $\begin{bmatrix} u \\ v \\ 0 \end{bmatrix}$ .





# Projective lines

- What does a line in the image correspond to in projective space?

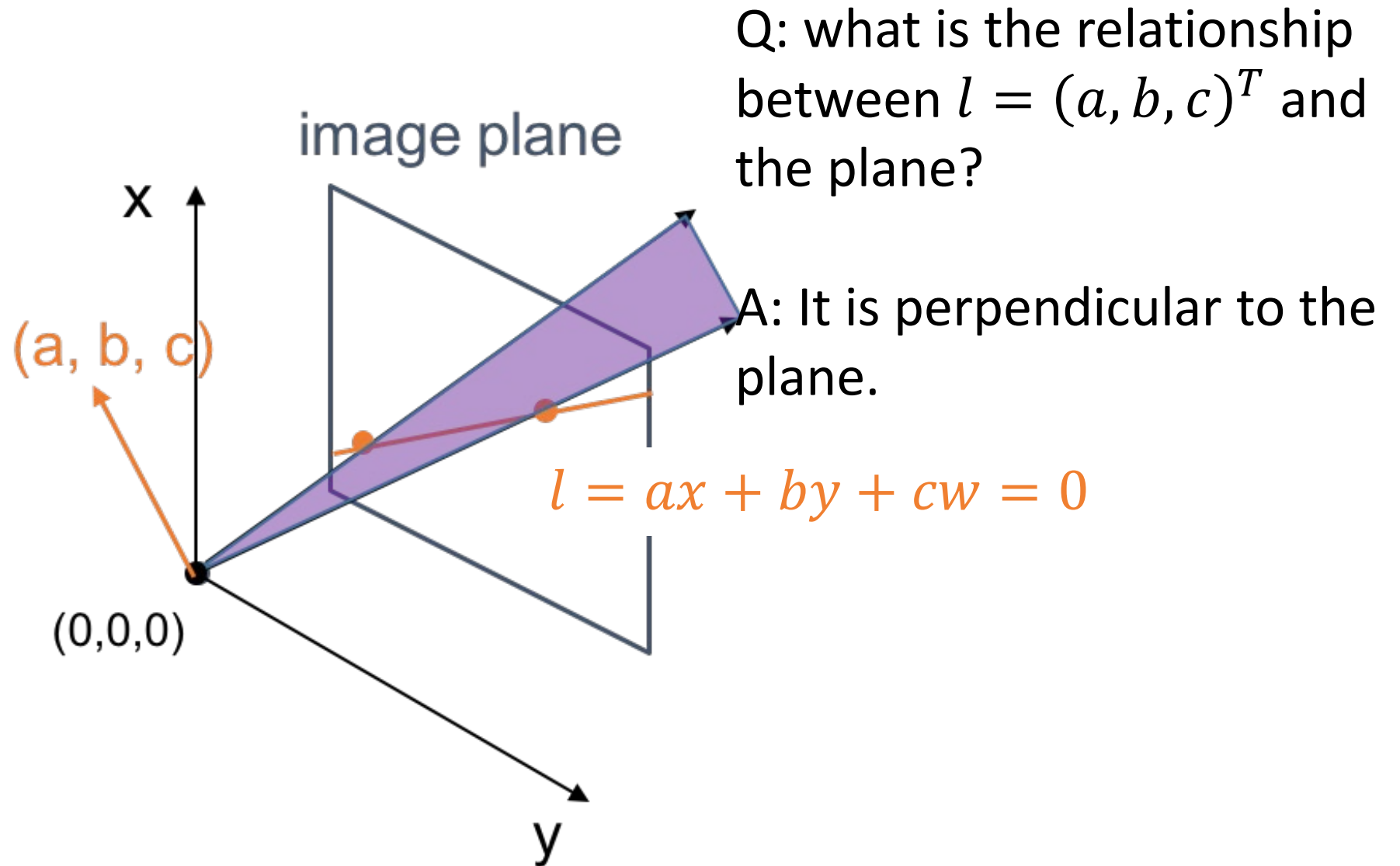


- A line is a *plane* of rays through origin
  - all rays  $(x, y, w)$  satisfying:  $ax + by + cw = 0$

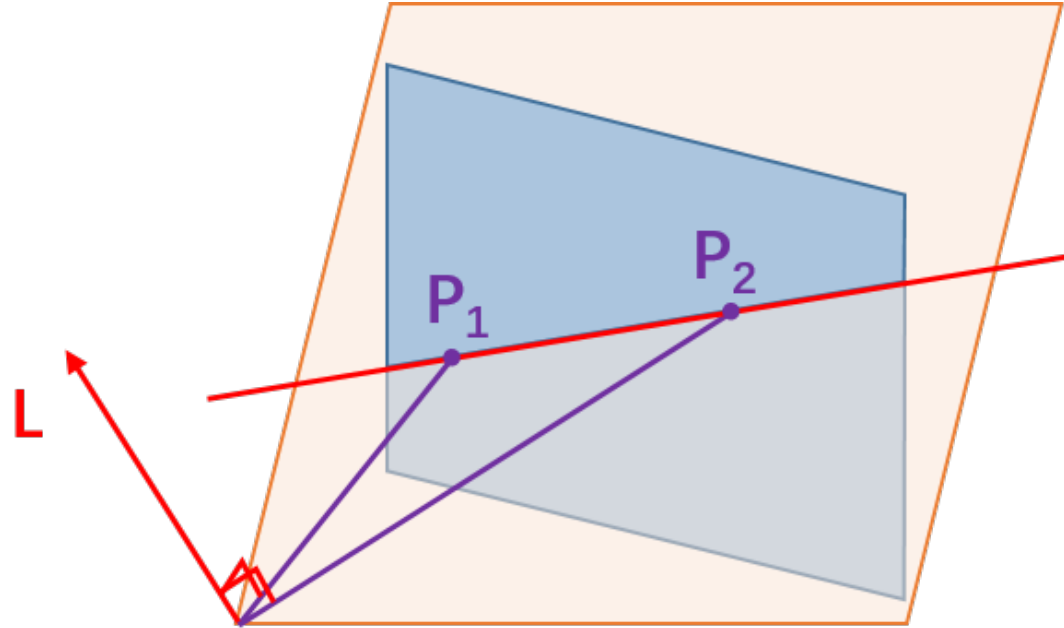
$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} = 0$$

$$l^T x = 0$$

# Projective Lines



# Line passing through two points in $\mathbb{P}^2$



# Line passing through two points in $\mathbb{P}^2$

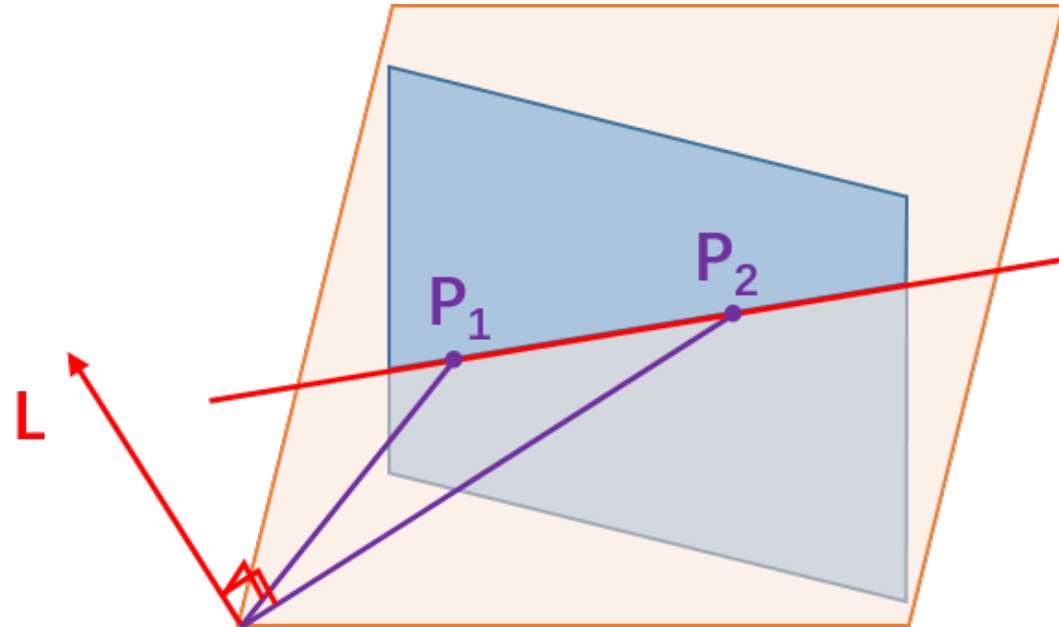
Two points:  $\mathbf{p}_1$  and  $\mathbf{p}_2$

Define a line

$$\mathbf{l} = \mathbf{p}_2 \times \mathbf{p}_1$$

Cross product

Claim:  $\mathbf{l}$  is the line passing through the two points



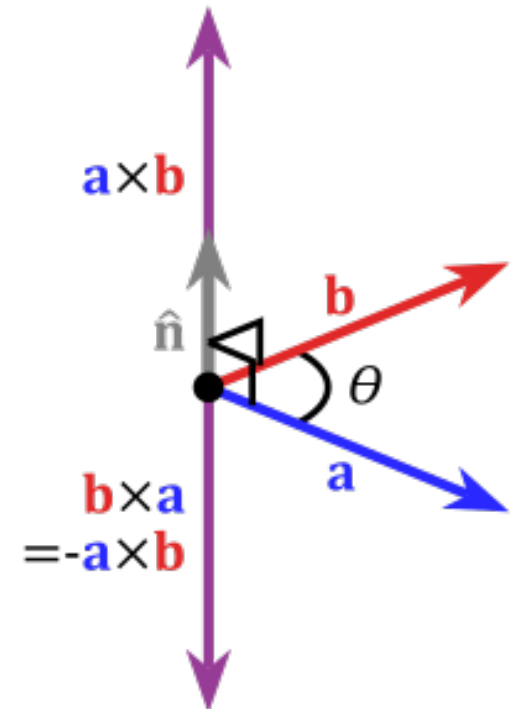
# Vector Algebra Recap: Cross Products

- $\mathbf{a} \times \mathbf{b}$

- Output is a vector with magnitude =  $||\mathbf{a}|| ||\mathbf{b}|| \sin \theta_{ab}$  and direction perpendicular to both vectors.
- Right-hand thumb rule to compute direction.
- Area of the parallelogram with  $\mathbf{a}$  and  $\mathbf{b}$  as sides.

- Computed as the determinant 
$$\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

- $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$



# Line passing through two points in $\mathbb{P}^2$

Two points:  $p_1$  and  $p_2$

Define a line  $l = p_1 \times p_2$

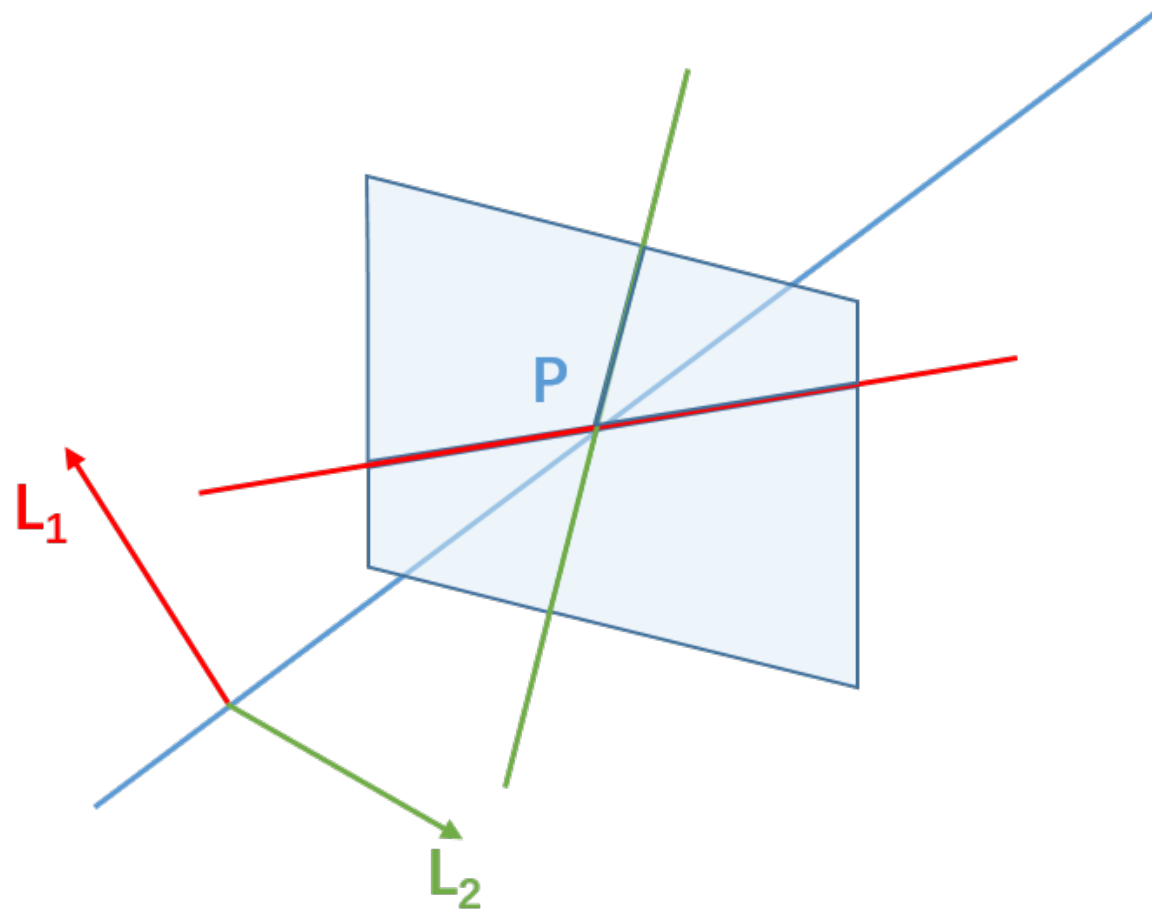
Claim:  $l$  is the line passing through the two points

Proof: The scalar triple product of the vectors  $\mathbf{a}$ ,  $\mathbf{b}$ , and  $\mathbf{c}$ :  $\vec{a} \cdot (\vec{b} \times \vec{c}) = (\vec{a} \times \vec{b}) \cdot \vec{c}$

$$p_1 \cdot l = p_1 \cdot (p_1 \times p_2) = 0 \text{ (property of cross-product)}$$

$$p_2 \cdot l = p_2 \cdot (p_1 \times p_2) = 0$$

The other way: Intersection point of two lines in  $\mathbb{P}^2$





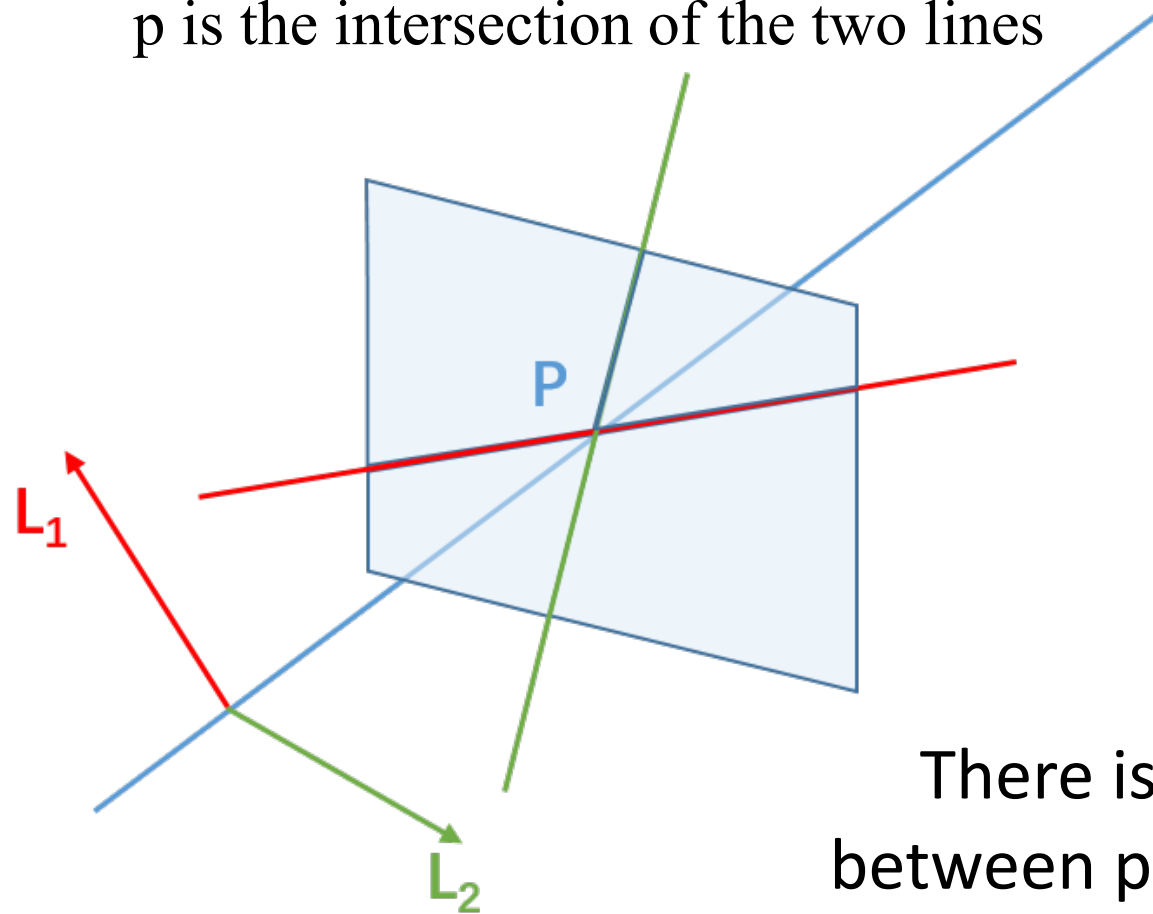
# Intersection point of two lines in $\mathbb{P}^2$

Given two lines:  $l_1$  and  $l_2$

Define a point  $p = l_1 \times l_2$

$p$  is the intersection of the two lines

**Q: Why? Hint: solve for the intersection by finding points that satisfy both line equations.**

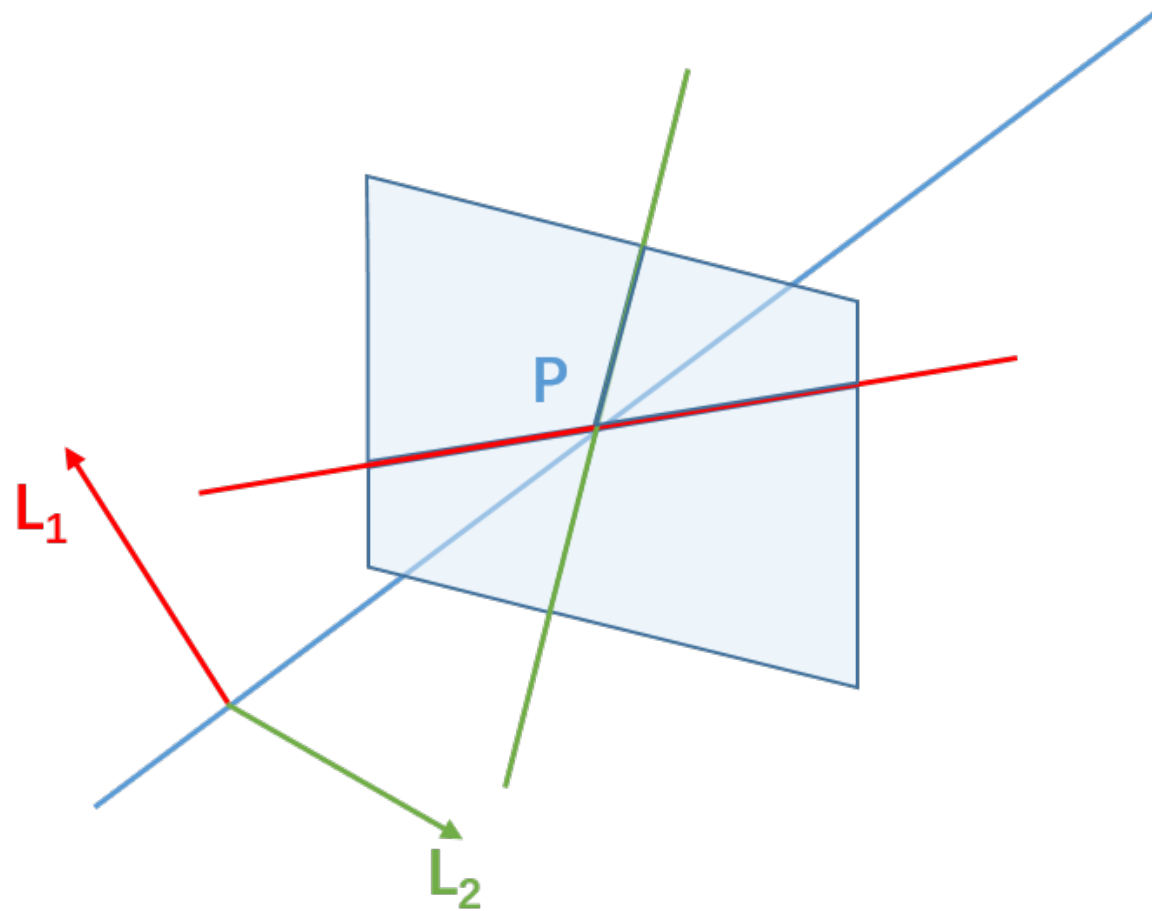


There is a close symmetry between points and lines in  $\mathbb{P}^2$ !

# Exercise

- Where do the lines  $x = 1$  and  $x = 2$  on image plane intersect?

# Intersection point of two lines in $\mathbb{P}^2$



When does  $P$  have the form  $(x,y,0)$ ?



CIS 580

# Machine Perception

*Or Geometric Computer Vision*

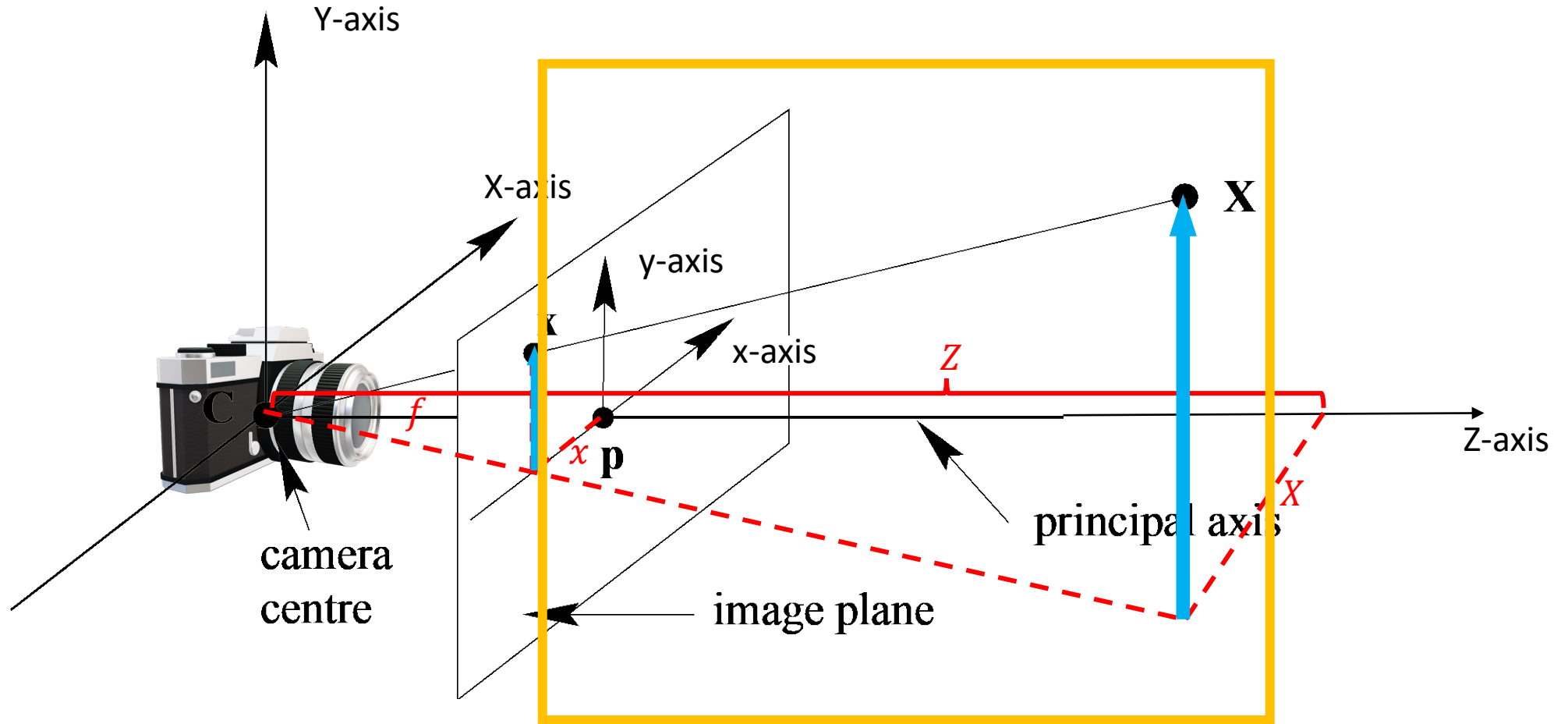
Instructor: Lingjie Liu

Lec 3: Jan 29, 2026

# We will start OH this week on Zoom

|                         | Time                                                                                       | Space                                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Dhyey Shah              | Monday 3pm-5pm                                                                             | Levine Hall Room 612                                                                                                                           |
| Siyu Li                 | Sunday 12-2pm                                                                              | <a href="#">Levine Hall, Moore 103</a>                                                                                                         |
| Prakriti Prasad         | Monday 12-1 pm                                                                             | Levine GRW 5th floor bump space                                                                                                                |
| Abhimanyu Suthar        | Friday 4pm - 6 pm                                                                          | Levine GRW 3rd floor bump space                                                                                                                |
| Yiming Huang            | Wednesday 10 AM - 12 PM                                                                    | <a href="https://upenn.zoom.us/j/8013153196">https://upenn.zoom.us/j/8013153196</a> (Levine GRW 5th floor bump space once returned to campus)) |
| Ravi Raghavan           | Thursday 4 - 6 PM                                                                          | Levine Hall Rm 512                                                                                                                             |
| Nandagopal Vidhu        | Thursday 11 AM - 12 PM (additional OH on Thursday 10 AM - 11 AM during HW submission week) | Levine Rm 612                                                                                                                                  |
| Jie Wang                | Tuesday 11-12 AM                                                                           | Levine Hall Room 459                                                                                                                           |
| Haoran Yang             | Monday 1-3 pm                                                                              | Levine 601 Bump space                                                                                                                          |
| Qihao Yun               | Monday 7pm - 8pm                                                                           | Levine 5th floor Bump Space                                                                                                                    |
| Aanal Maheshbhai Sonara | Monday 5pm - 6pm                                                                           | Levine 501 Bump Space                                                                                                                          |
| Yongzhe Wang            | Friday 12:30 to 1:30                                                                       | Levine; CIS Exclusive; Levine 5 Bump Space                                                                                                     |
| Lingjie Liu             | Friday 11am - 12pm                                                                         | Levine 570                                                                                                                                     |

# Recap: Basic Perspective Projection Equations



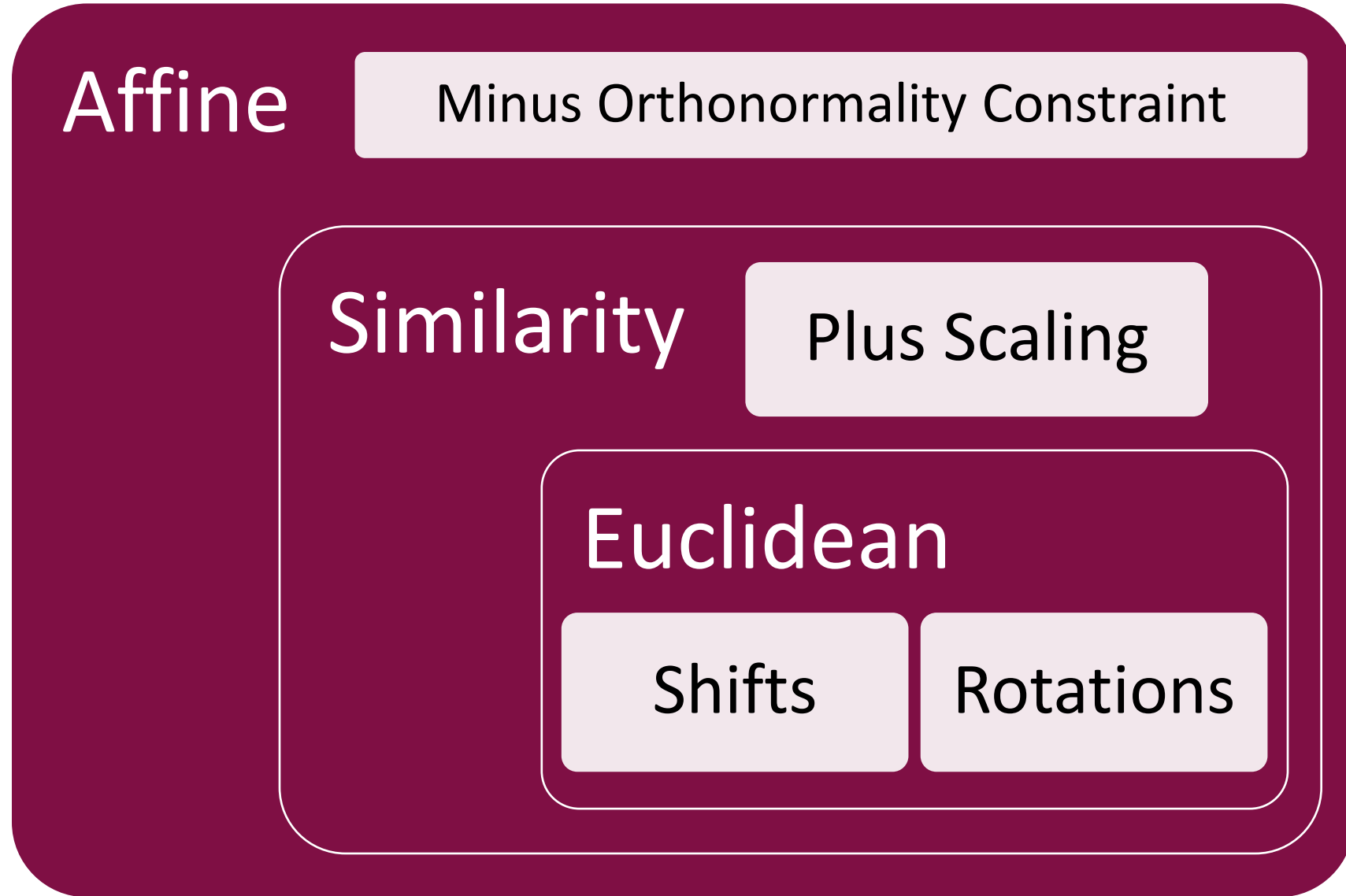
$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$



# Recap: Points and Planes in Euclidean 3D (shortcut $\mathbb{R}^3$ )

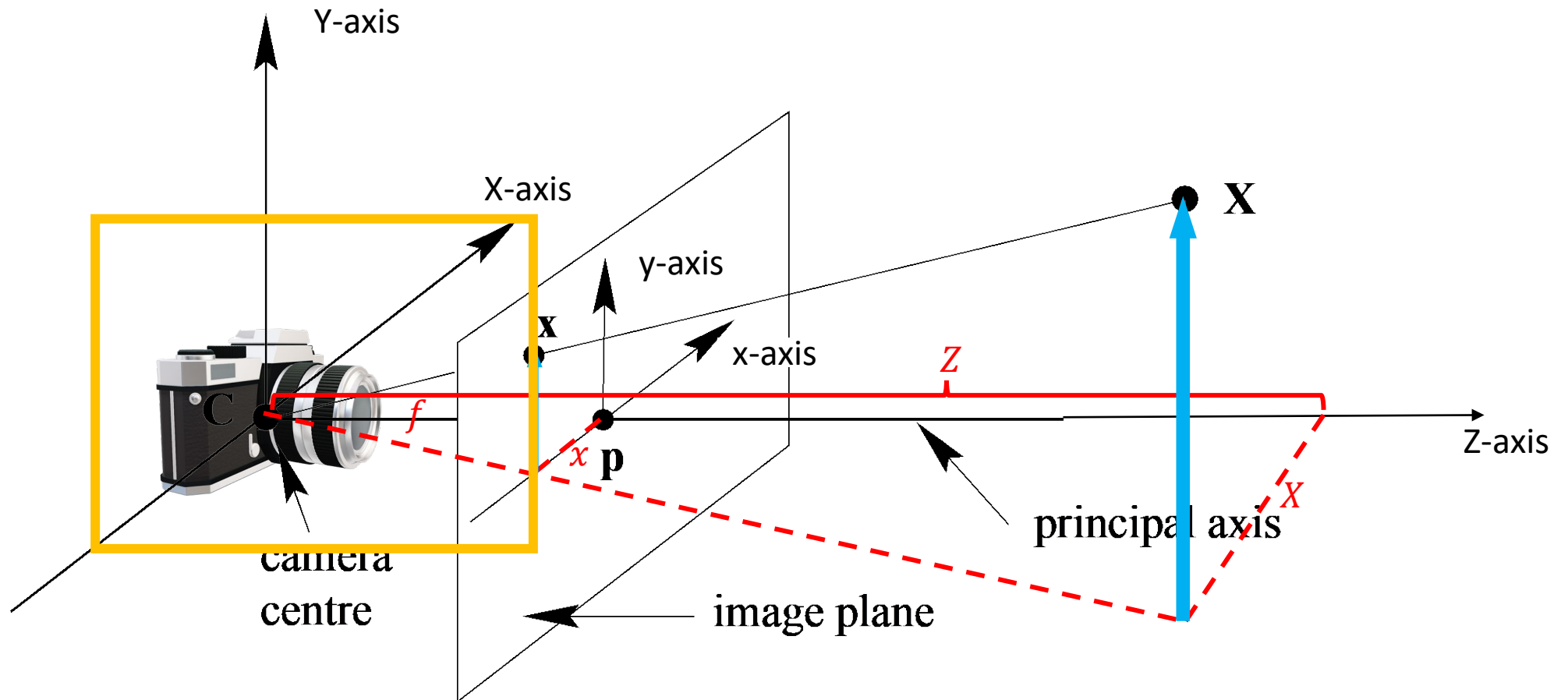
- Point in 3D space:  $\mathbf{x} = (x, y, z)^T \in \mathbb{R}^3$
- Plane in 3D space:
  - $ax + by + cz + d = 0$
  - $\boldsymbol{\pi}^T \begin{pmatrix} x \\ y \\ z \\ \mathbf{1} \end{pmatrix} = 0$ , where  $\boldsymbol{\pi} = [a, b, c, d]^T$
- Line in 3D space:
  - $(x, y, z)^T = (x_1, y_1, z_1)^T + \lambda(x_2 - x_1, y_2 - y_1, z_2 - z_1)^T$  for some value of  $\lambda$
  - $\begin{pmatrix} \boldsymbol{\pi}_1^T \\ \boldsymbol{\pi}_2^T \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

# Recap: Hierarchy of Transformations



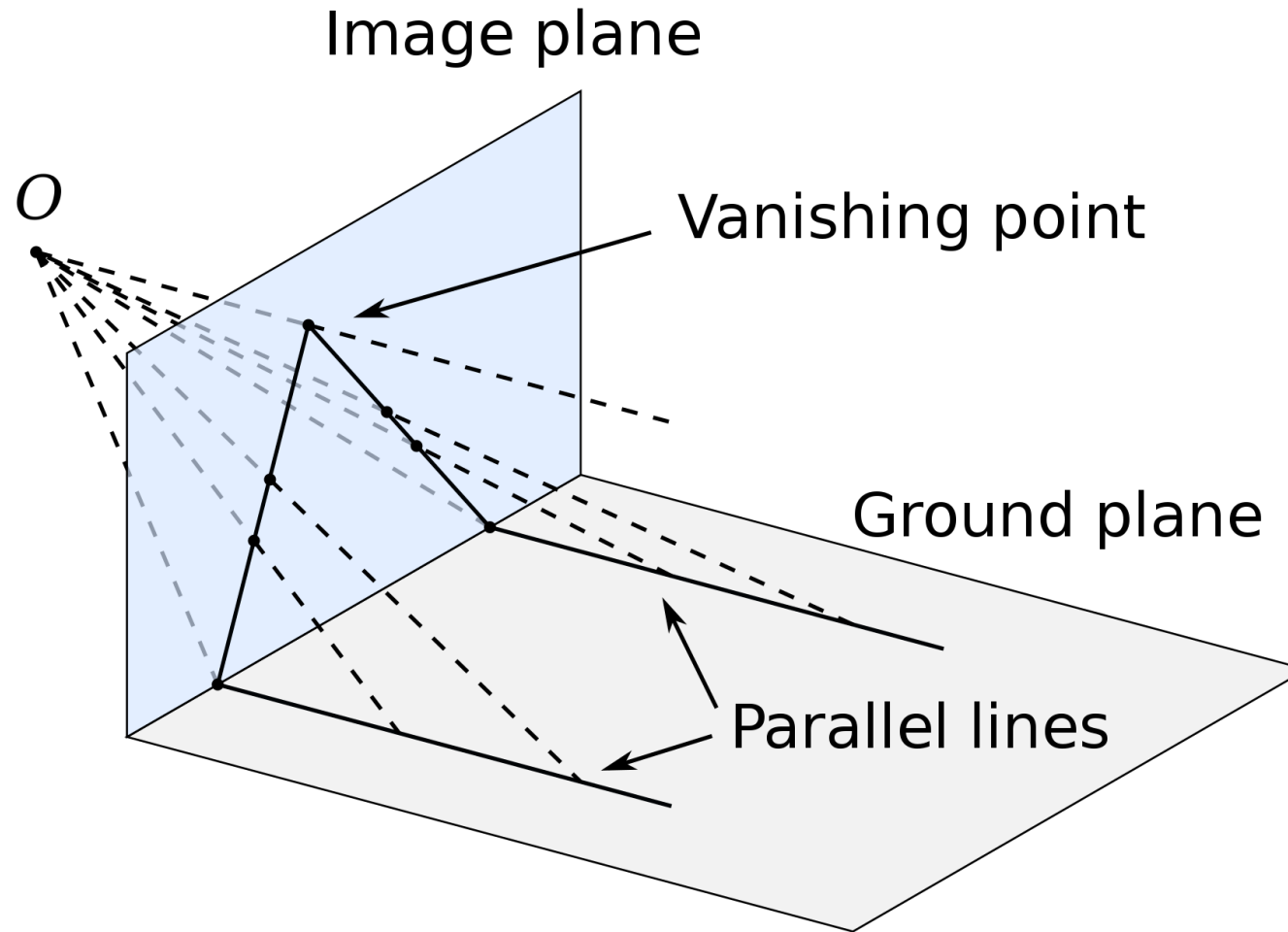
Soon an even more general transformation (“projective”)!

# Recap: Basic Perspective Projection Equations



$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$

# Recap: Where vanishing points come from



# Recap: Projective Geometry

- Extension of Euclidean geometry that deals with points at infinity
  - Other than that, Projective Geometry keeps many of the same features as Euclidean geometry.
- A key property is that two lines always meet in a point
  - (sometimes a point at infinity)

# Recap: “Homogeneous coordinates”: Euclidean $\mathbb{R}^n \rightarrow$ Projective $\mathbb{P}^n$

## ***Homogeneous coordinates***

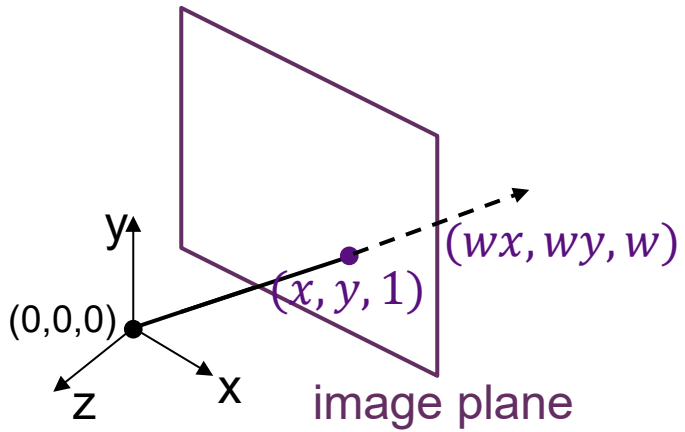
- represent coordinates in  $\mathbb{P}^2$  with a 3-vector

$$\begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{\text{homogeneous coords}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

← We will normally use  $w$  to denote this homogeneous vector element

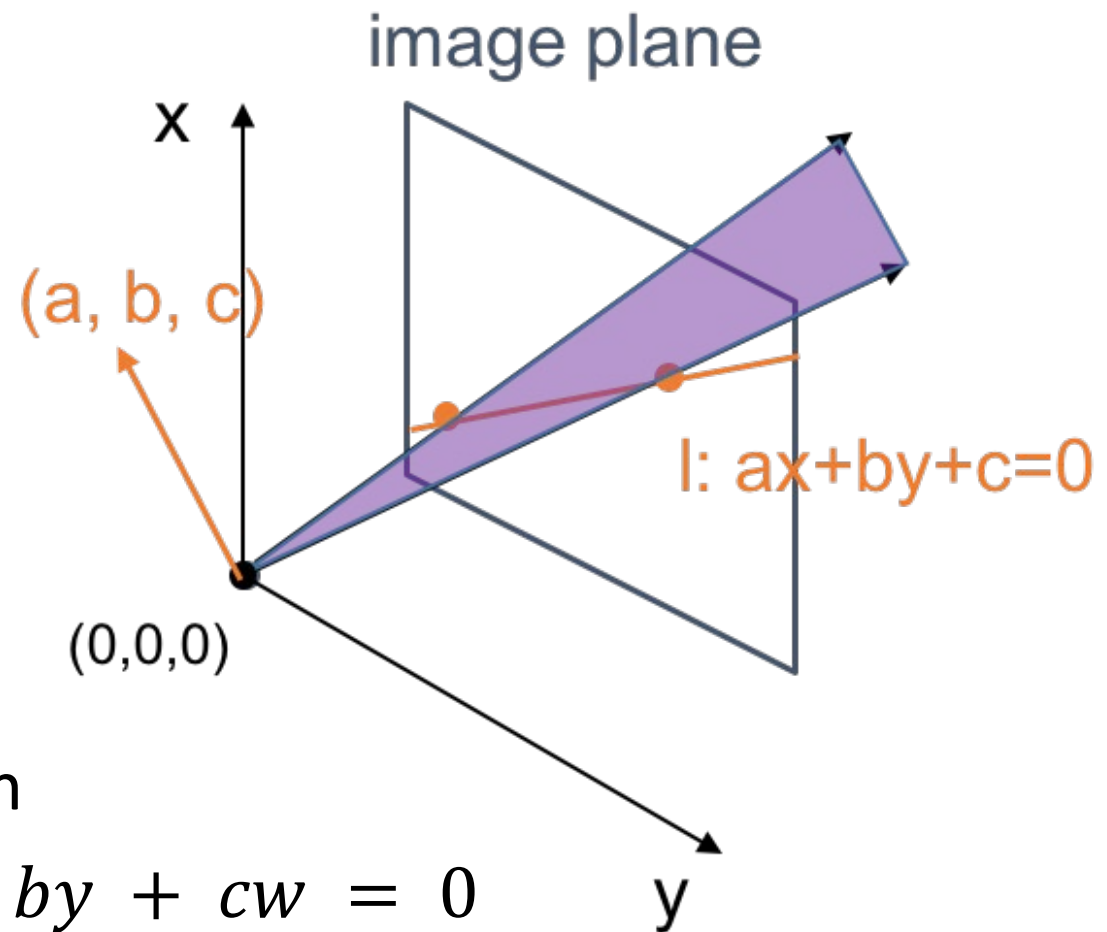
# Recap: Projective $\mathbb{P}^n \rightarrow$ Euclidean $\mathbb{R}^n$

- $(x, y, w) = \left(\frac{x}{w}, \frac{y}{w}, 1\right)$ , if  $w \neq 0$
- Geometric intuition



- $(x, y, 0)$ : an infinity point  
Note:  $\mathbb{P}^2$  does not contain  $(0,0,0)$ .

# Recap: Projective lines



- A line is a *plane* of rays through origin
  - all rays  $(x, y, w)$  satisfying:  $ax + by + cw = 0$

$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} = 0$$

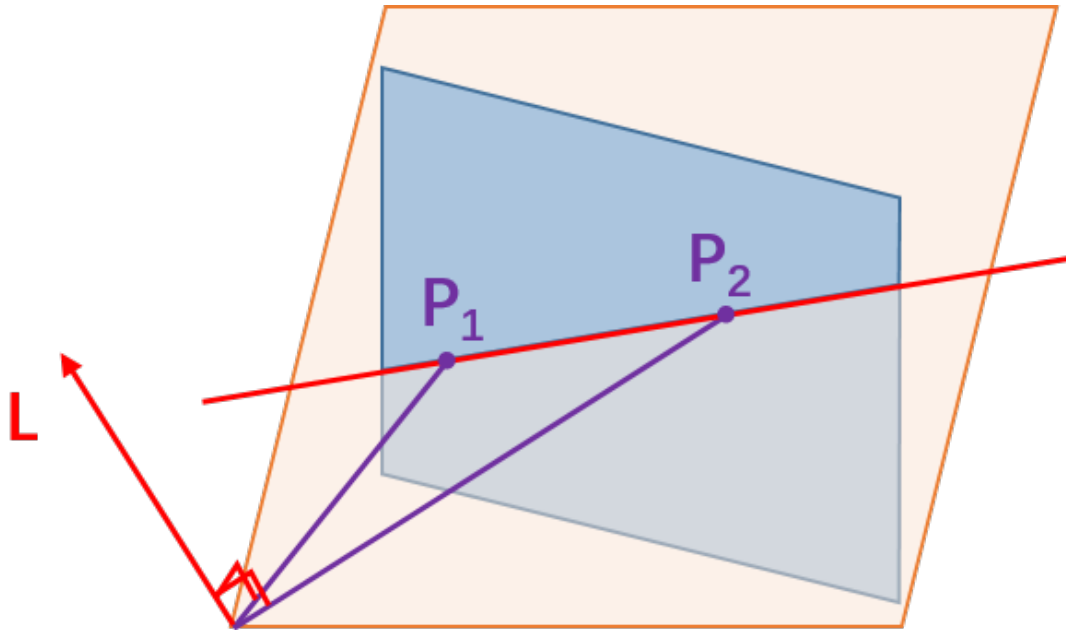
$$l^T x = 0$$



# Recap: Relationship btw Line and Point Representations in $\mathbb{P}^2$

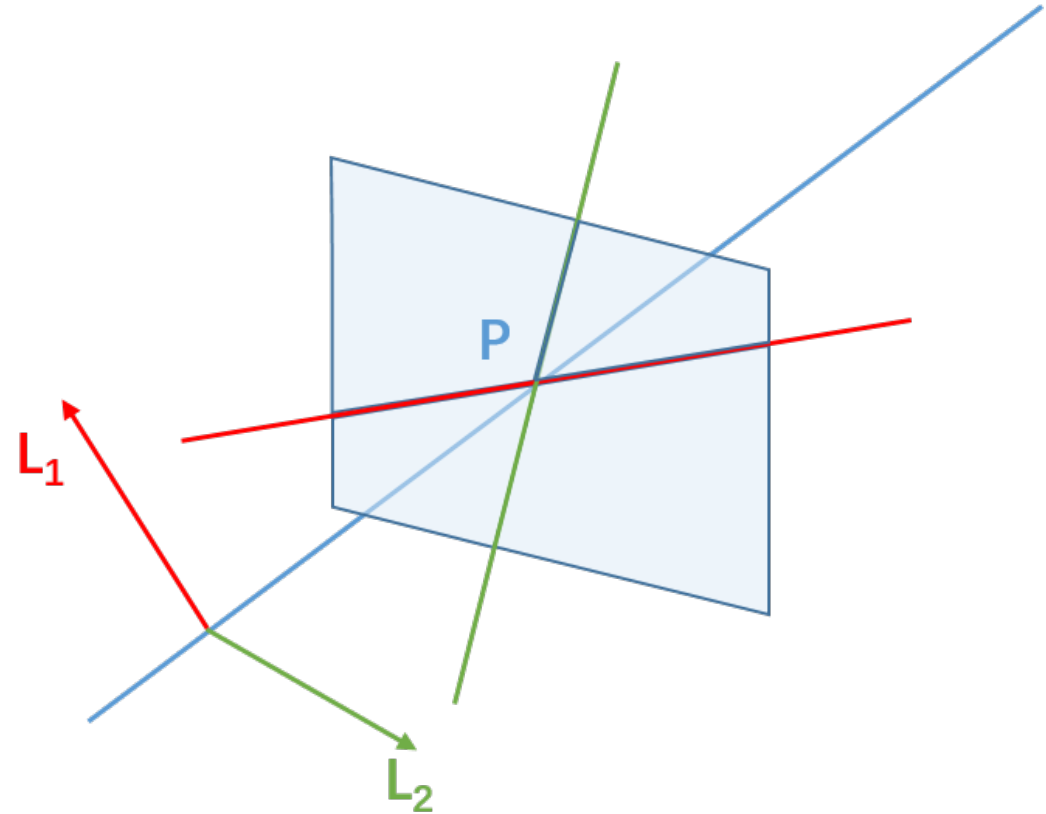
Given two points:  $\mathbf{p}_1$  and  $\mathbf{p}_2$

Define a line  $\mathbf{l} = \mathbf{p}_1 \times \mathbf{p}_2$



Given two lines:  $\mathbf{l}_1$  and  $\mathbf{l}_2$

Define a point  $\mathbf{p} = \mathbf{l}_1 \times \mathbf{l}_2$



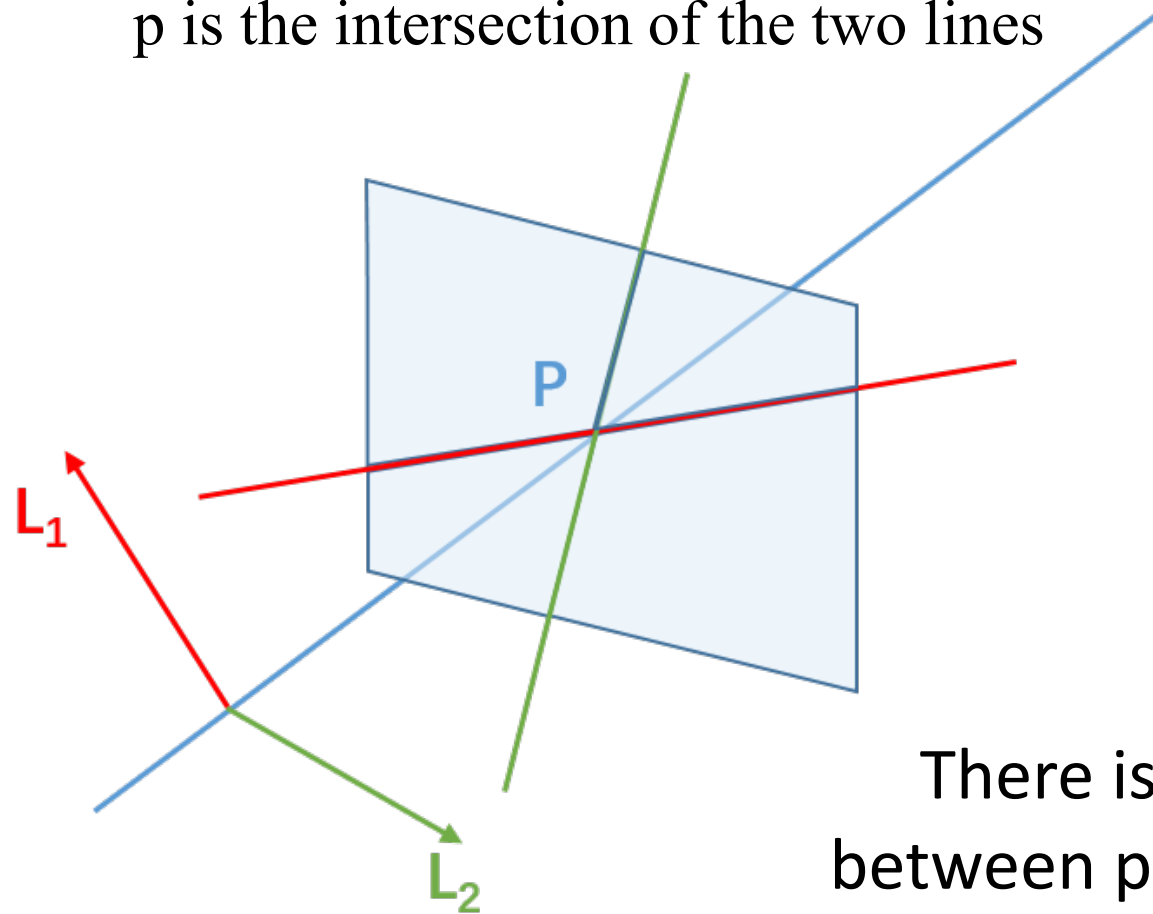
# Intersection point of two lines in $\mathbb{P}^2$

Given two lines:  $l_1$  and  $l_2$

Define a point  $p = l_1 \times l_2$

$p$  is the intersection of the two lines

**Q: Why? Hint: solve for the intersection by finding points that satisfy both line equations.**



There is a close symmetry between points and lines in  $\mathbb{P}^2$ !

# Exercise

- Where do the lines  $x = 1$  and  $x = 2$  on image plane intersect?

# Extending to Any Parallel Lines

$$l = (a, b, c) \qquad l' = (a, b, c')$$

Intersection:

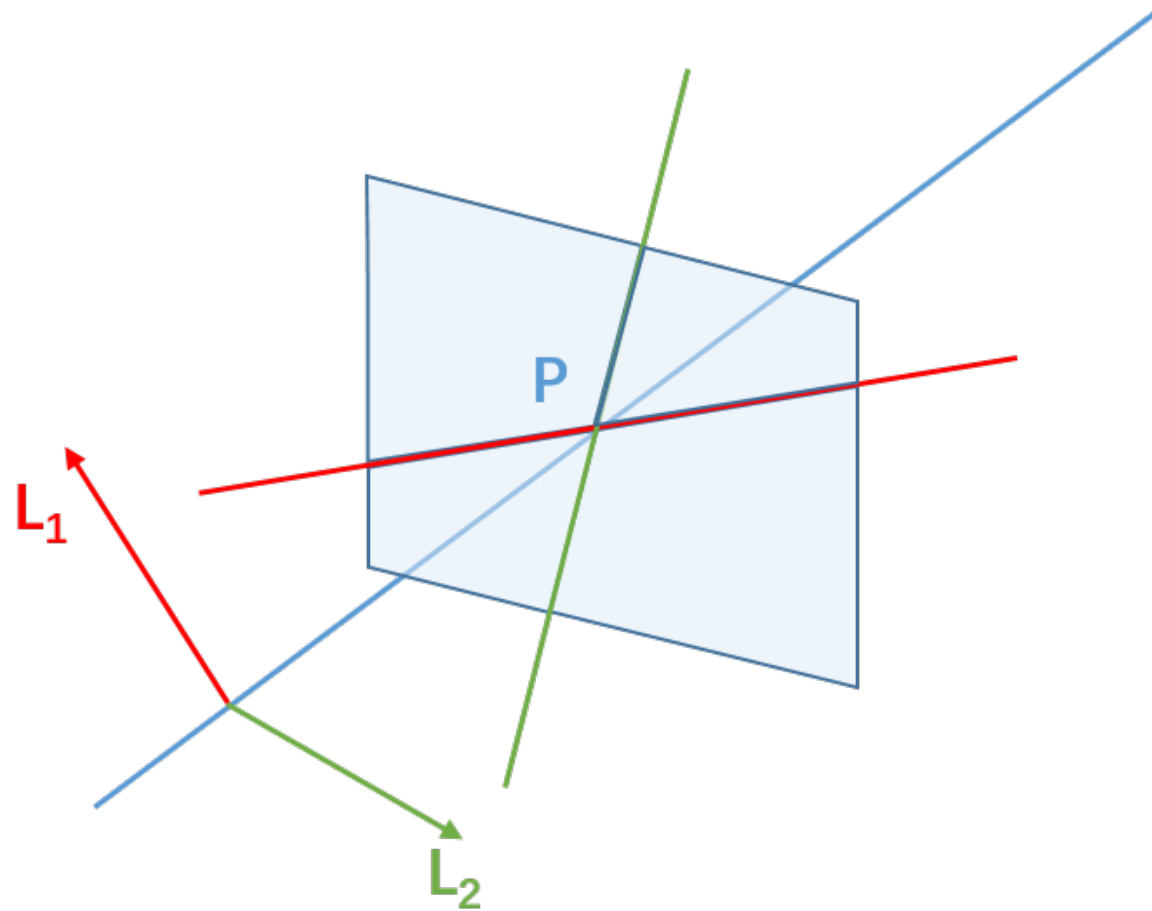
$$\begin{aligned} l \times l' &= l \times l' \\ &= \begin{vmatrix} i & j & k \\ a & b & c \\ a & b & c' \end{vmatrix} \\ &= (bc' - bc, ca - c'a, ab - ab)^T \\ &= (c' - c)(b, -a, 0)^T \end{aligned}$$

Any point  $(x_1, x_2, 0)$  is intersection of parallel lines

# Extending to Any Parallel Lines

- Under projective geometry,
  - All parallel lines intersect at a point at infinity
  - One point at infinity  $\Leftrightarrow$  one parallel line direction  
line  $l = (a, b, c)^T$  intersects at  $(b, -a, 0)^T$

# Intersection point of two lines in $\mathbb{P}^2$

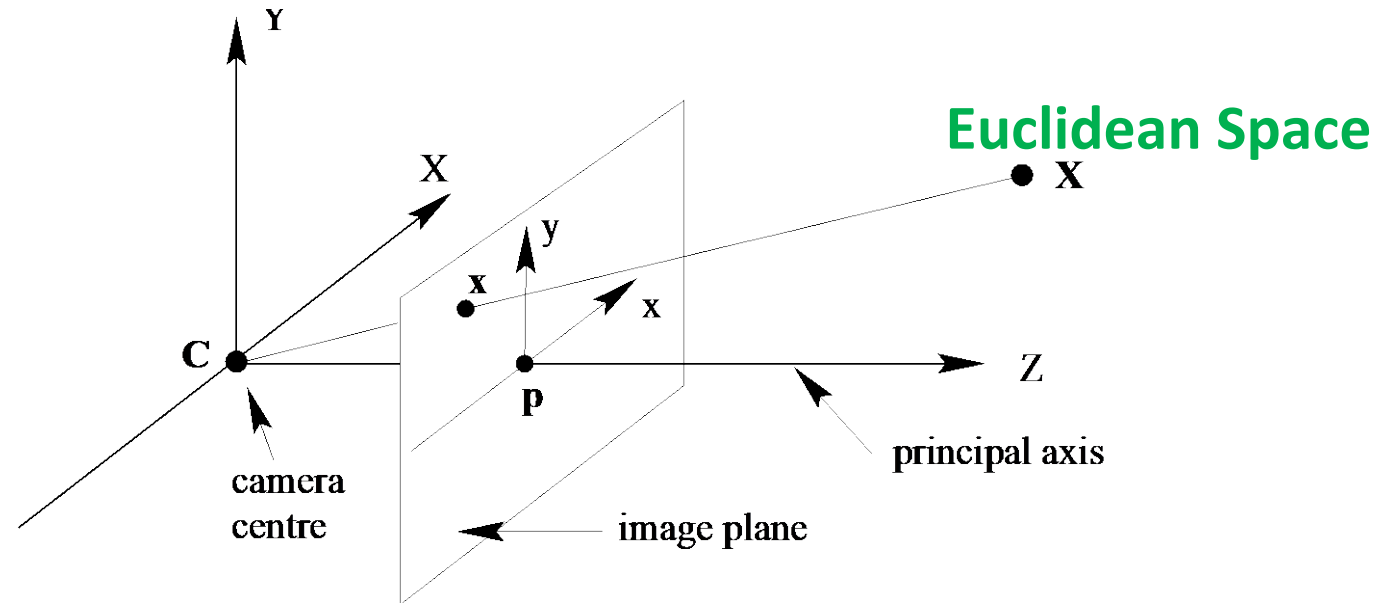
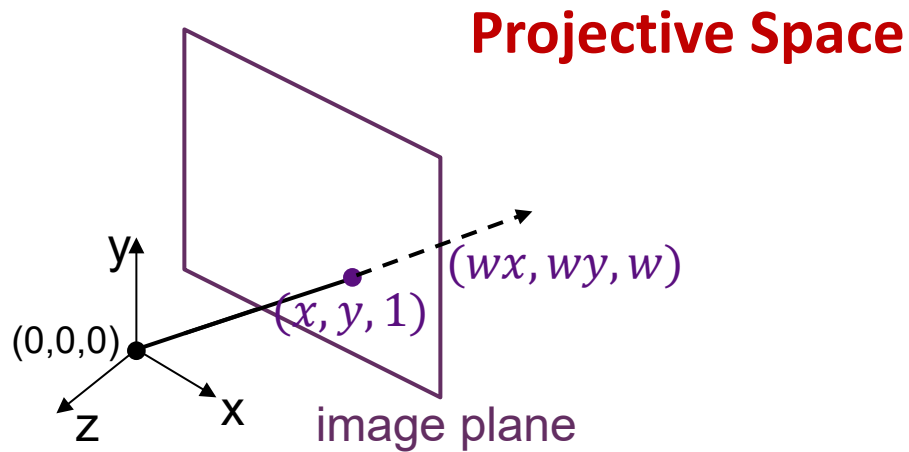


When does  $P$  have the form  $(x,y,0)$ ?

# Projective geometry $\leftrightarrow$ Euclidean interpretation

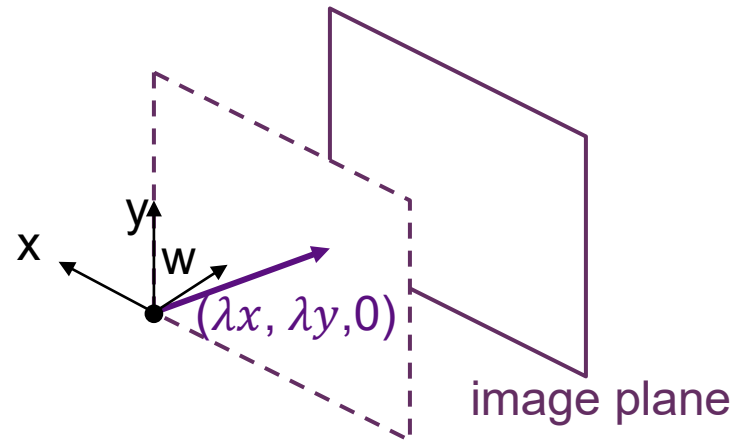
In the Euclidean interpretation, we treat  $w$  as the third spatial coordinate.

- The  $w$  axis is a scaled version of the principal axis  $Z$  (in camera-centric coordinates).
- The image plane is  $w = 1$ , same as  $Z = f$
- $w = 0$  is the same as  $Z = 0$ . Parallel to image plane, passing through camera center.



# Point at infinity / “ideal” points

- Ideal point (“point at infinity”)
  - $p \cong (x, y, 0)$  – rays through camera center parallel to image plane
  - It has infinite image coordinates



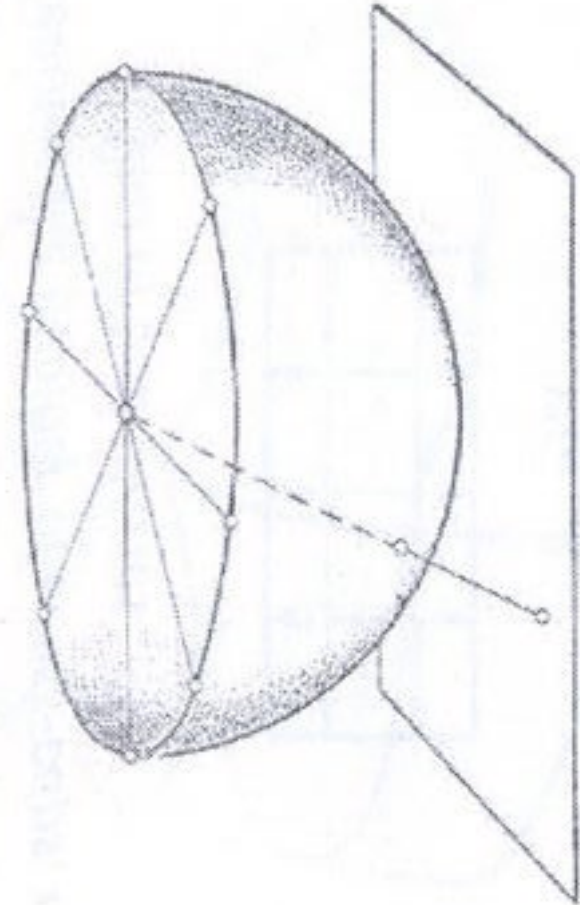


# Point at infinity / “ideal” points

$(x_1, x_2, 0)$

Looking-at direction

“Ideal” points



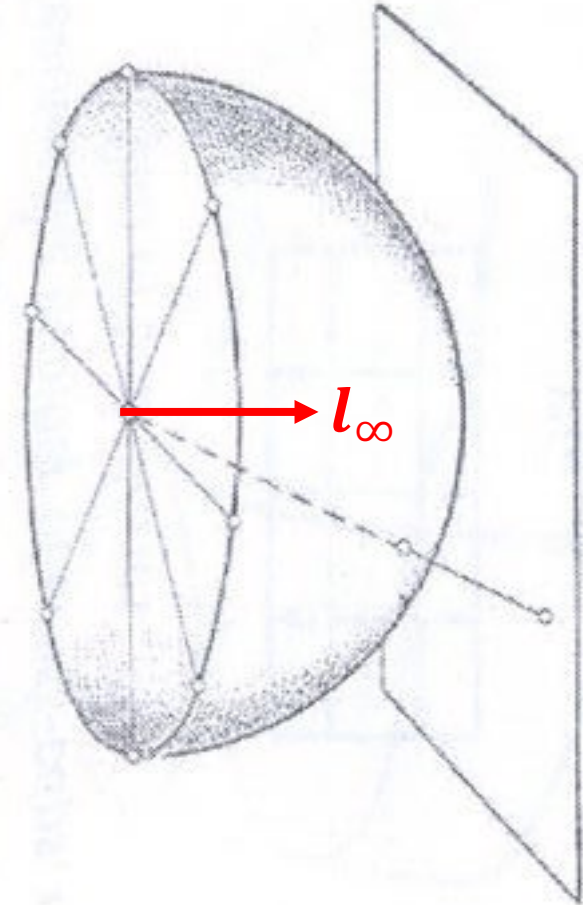
# “Line at infinity”

- A line passing through all ideal points i.e. point

$$l_{\infty} = (0, 0, 1)^T$$

- Because :

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 0 \end{bmatrix} = 0$$

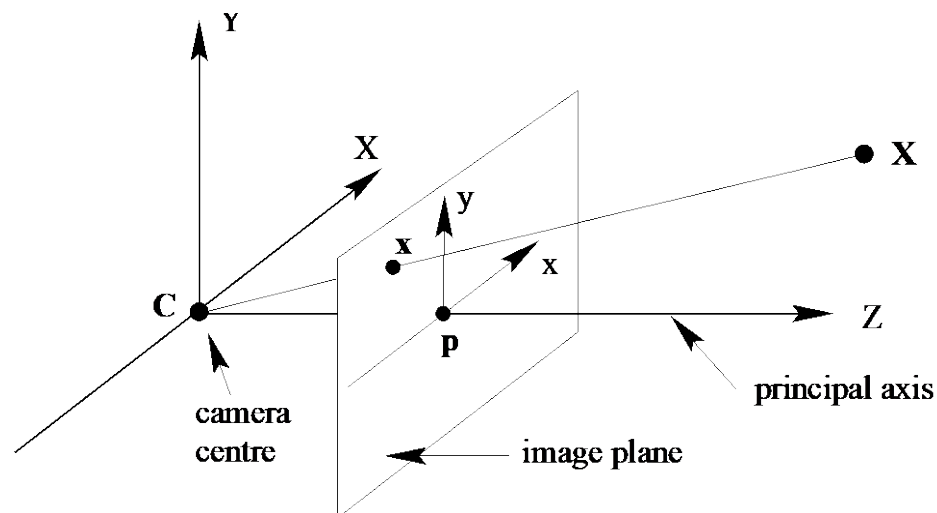




# Perspective Projections are Linear in $\mathbb{P}$

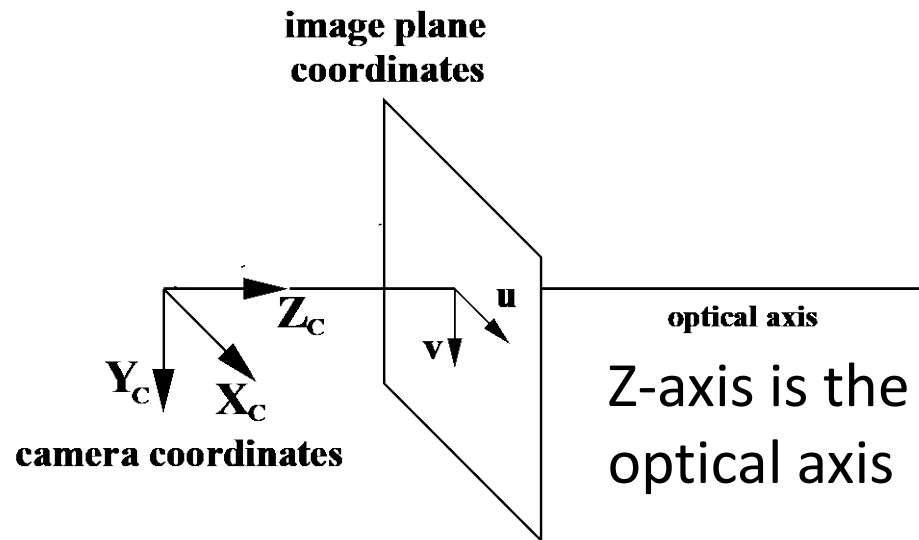
Recall:

$$x = f \frac{X}{Z}, y = f \frac{Y}{Z}$$



$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix}$$

# Camera Coordinate System + Principal Point Offset



The image plane  $(u, v)$  is perpendicular to the optical axis.

Intersection of the image plane with the optical axis is the *image center*  $(u_o, v_o)$

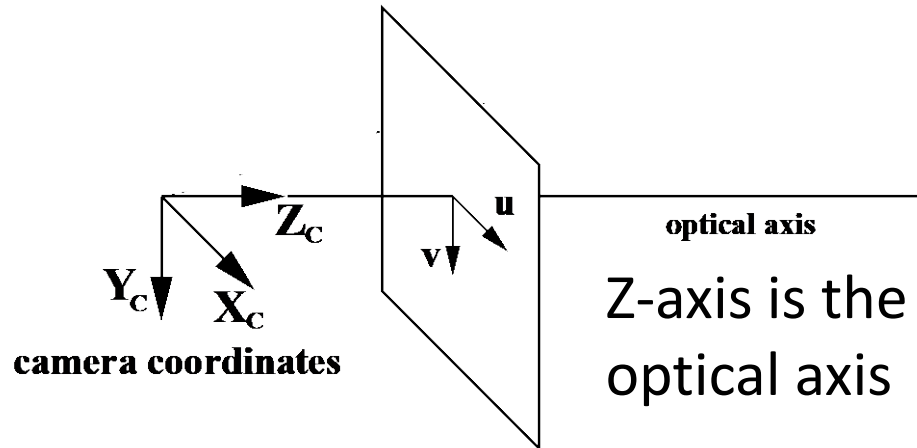
Projection  
in pixels

$$u = f \frac{X_c}{Z_c} + u_0, \quad v = f \frac{Y_c}{Z_c} + v_0$$

# Projection equation with image origin $\neq$ principal point

$$u = f \frac{X_c}{Z_c} + u_0, \quad v = f \frac{Y_c}{Z_c} + v_0$$

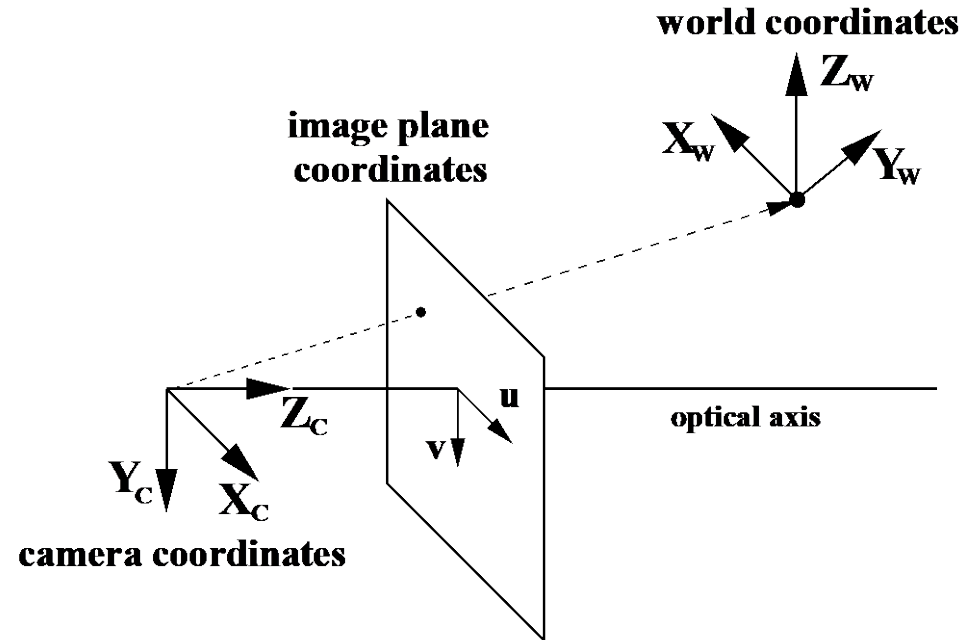
image plane  
coordinates



Bold face to indicate this is the full vector in  $\mathbb{P}^3$

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & & u_0 \\ & f & v_0 \\ & & 1 \end{bmatrix} \begin{bmatrix} 1 & & 0 \\ & 1 & 0 \\ & & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = K[I|0] \mathbf{X}_c$$

# From world to camera: Euclidean transformation



**What does  $R$  mean exactly?**

Think “change of coordinates”.

- The camera has its own axes  $(X_c, Y_c, Z_c)$
- The world has its axes  $(X_w, Y_w, Z_w)$

$R$  tells you how to express world vectors in the camera basis.

**That is,  $R$  denotes the rotation of the world axes w.r.t. camera axes**

= *inverse* rotation of the camera axes w.r.t. world axes.

What about  $t$ ?

**Let  $C$  be the camera center in world coordinates.**

That means:  $C$  is the world point that maps to the camera origin  $(0, 0, 0)$ .

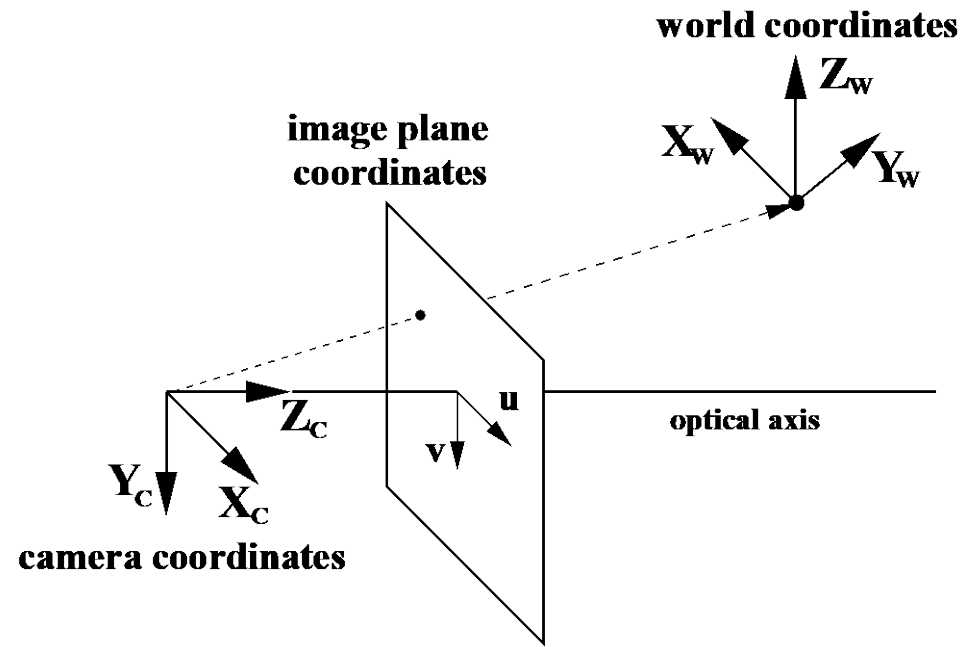
Plug  $X_w = C$  into the equation:

$$0 = R C + t \Rightarrow t = -R C.$$

**$t$  here is actually  $R$  times the translation of world origin from camera origin.**

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R_{3 \times 3} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + t = \begin{bmatrix} R_{3 \times 3} & t \\ 0 & 1 \end{bmatrix} X_w$$

# Putting the pieces together: Projection matrix P



camera 3D coords to pixels      Convert world to camera coordinates

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & & u_0 \\ & f & v_0 \\ & & 1 \end{bmatrix} \begin{bmatrix} R_{3 \times 3} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$



# Summarizing

- Permitting principal point offset (i.e. shifted origin for image plane)

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} fX + Zu_0 \\ fY + Zv_0 \\ Z \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f & u_0 & 0 \\ & f & v_0 & 0 \\ & & 1 & 0 \end{bmatrix}}_{\text{"Intrinsics" } K} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = K[I|\mathbf{0}]\mathbf{X}_c$$

- The above assumes  $X, Y, Z$  in camera-centric coordinates. Permitting different “world” and “camera” coordinate frames:

$$\mathbf{X}_c = \underbrace{R_{3 \times 3}}_{\text{Rotate}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} + \underbrace{\mathbf{t}}_{\text{Translate}} = [R|\mathbf{t}]\mathbf{X}_w,$$

$$\mathbf{x} \sim K[R|\mathbf{t}]\mathbf{X}_w$$

First convert to camera coordinates

Then project as before.

# Perspective Projection in homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} = P_{3 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\underbrace{P_{3 \times 4}}_{\substack{\text{"Camera} \\ \text{projection} \\ \text{matrix"}}} = \underbrace{K_{3 \times 3}}_{\text{"Intrinsics"}} \underbrace{[R_{3 \times 3} | \mathbf{t}_{3 \times 1}]}_{\text{"Extrinsics"}}$$

Q: How many degrees of freedom does  $P$  have given the type of intrinsics we have assumed?



CIS 5800

# Machine Perception

*Or Geometric Computer Vision*

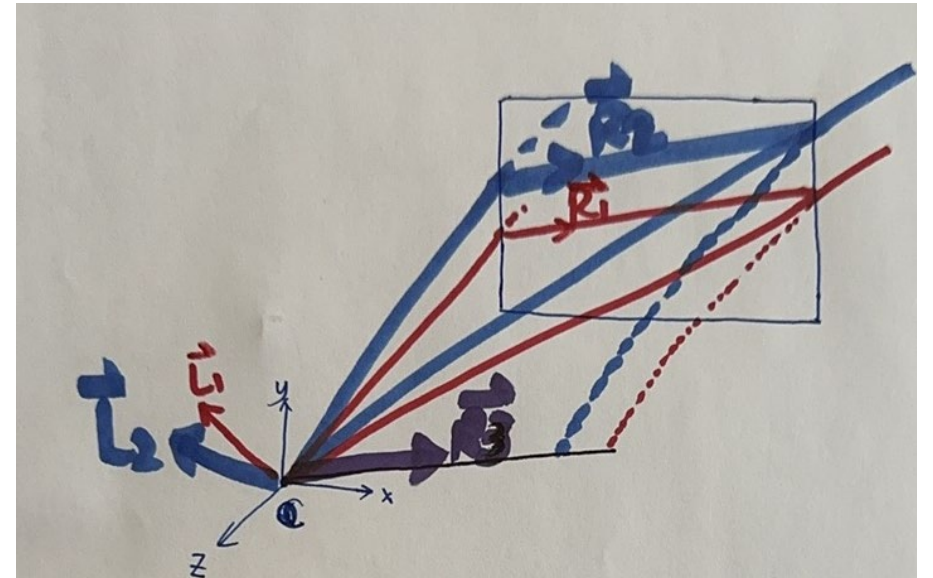
Instructor: Lingjie Liu

Lec 4: Feb 3, 2026

# Recap: Extending to Any Parallel Lines

- Under projective geometry,
  - All parallel lines intersect at a point at infinity
  - One point at infinity  $\Leftrightarrow$  one parallel line direction

line  $l = (a, b, c)^T$  intersects at  $(b, -a, 0)^T$

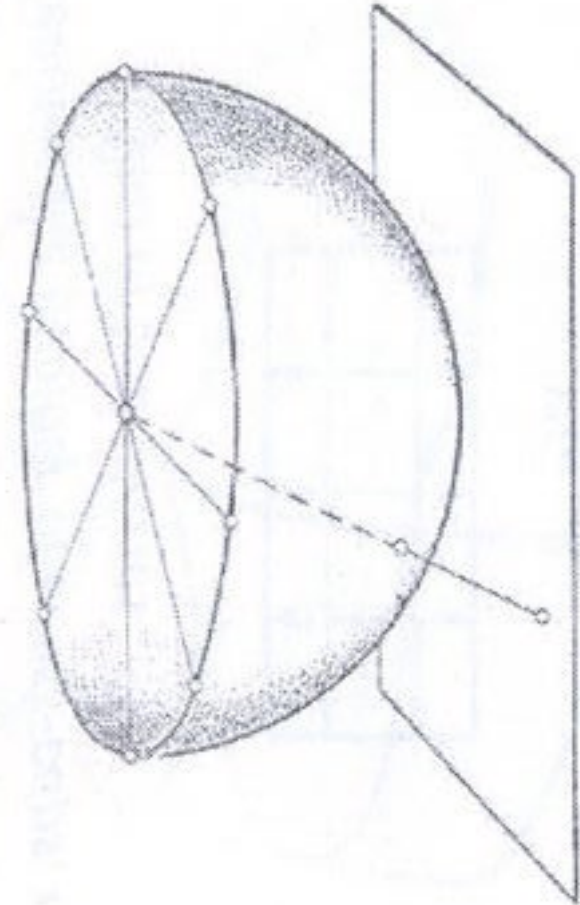


# Recap: Point at infinity / “ideal” points

$(x_1, x_2, 0)$

Looking-at direction

“Ideal” points



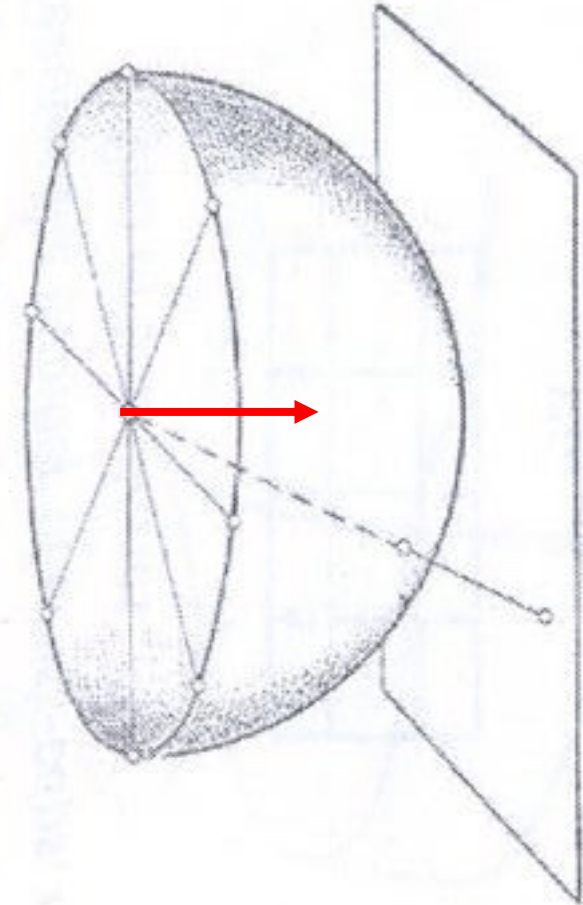
# Recap: “Line at infinity”

- A line passing through all ideal points i.e. point

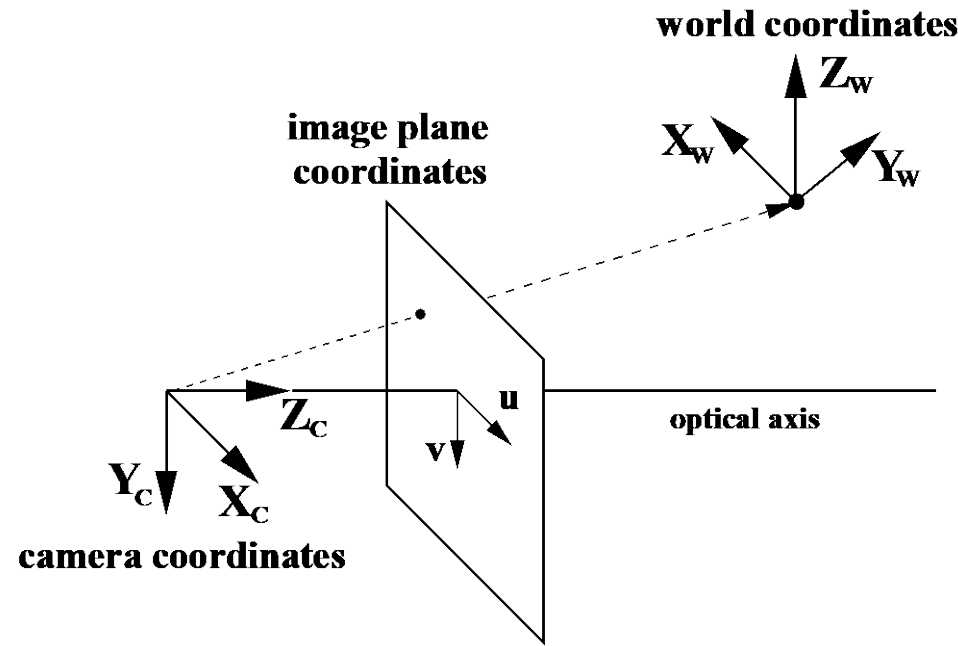
$$l_{\infty} = (0, 0, 1)^T$$

- Because :

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 0 \end{bmatrix} = 0$$



# Recap: Putting the pieces together: Projection matrix P



camera 3D coords to pixels      Convert world to camera coordinates

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & & u_0 \\ & f & v_0 \\ & & 1 \end{bmatrix} \begin{bmatrix} R_{3 \times 3} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$



# Recap: Perspective Projection in homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} = P_{3 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\underbrace{P_{3 \times 4}}_{\substack{\text{"Camera} \\ \text{projection} \\ \text{matrix"}}} = \underbrace{K_{3 \times 3}}_{\text{"Intrinsics"}} \underbrace{[R_{3 \times 3} | \mathbf{t}_{3 \times 1}]}_{\text{"Extrinsics"}}$$

Q: How many degrees of freedom does  $P$  have given the type of intrinsics we have assumed?



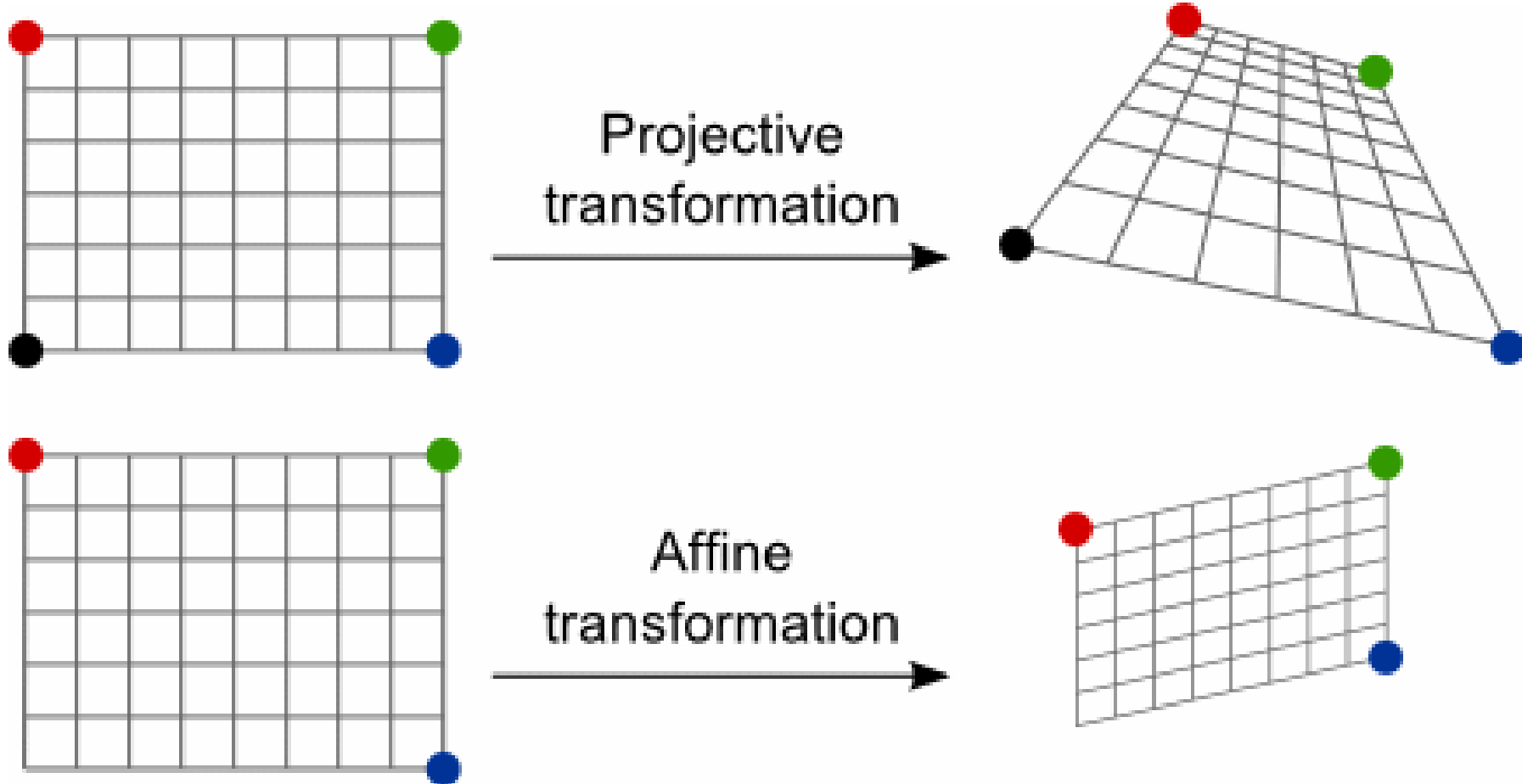
# Projective Transformations

aka Collineations  
aka Homographies  
aka Projectivity

# Example of Projective Transformation

Common notations:  $H$

(Note that some books use  $A$ ; however, we will avoid using  $A$  in this course, as  $A$  is commonly associated with Affine Transformations.)



# Example of Projective Transformation

- A 2D point before H is represented as  $(X, Y)$  , after Projective transformation is  $(u, v)$  :

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \sim H \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \quad \text{or}$$

$$u = \frac{H_{11}X + H_{12}Y + H_{13}}{H_{31}X + H_{32}Y + H_{33}}$$
$$v = \frac{H_{21}X + H_{22}Y + H_{23}}{H_{31}X + H_{32}Y + H_{33}}$$

# Projective Transformation = Homography = Collineation=Projectivity

## Definition

A **projective transformation** is any invertible matrix transformation  $\mathbb{P}^2 \rightarrow \mathbb{P}^2$ .

# Projective Transformation = Homography = Collineation=Projectivity

## Definition

A **projective transformation** is any invertible matrix transformation  $\mathbb{P}^2 \rightarrow \mathbb{P}^2$ .

A projective transformation  $H$  maps  $p$  to  $p' \sim Hp$

Invertibility means that  $\det(H) \neq 0$  and that there exists  $\lambda \neq 0$  such that  $\lambda p' = Hp$

Observe that we will write either  $p' \sim Hp$  or  $\lambda p' = Hp$

# How many unknowns are in a projective transformation $H$ ?

$$(\mathbb{P}^2 \rightarrow \mathbb{P}^2)$$

A projective transformation  $\mu H$  is the same as  $H$  since they map to projectively equivalent points:

$$\mu \lambda p' = \mu H p$$



## How many unknowns are in a projective transformation $H$ ?

$$(\mathbb{P}^2 \rightarrow \mathbb{P}^2)$$

A projective transformation  $\mu H$  is the same as  $H$  since they map to projectively equivalent points:

$$\mu \lambda p' = \mu H p$$

We will be able to determine a projective transformation only up to a scale factor. Hence the 3x3 invertible matrix  $H$  will have only EIGHT independent unknowns.

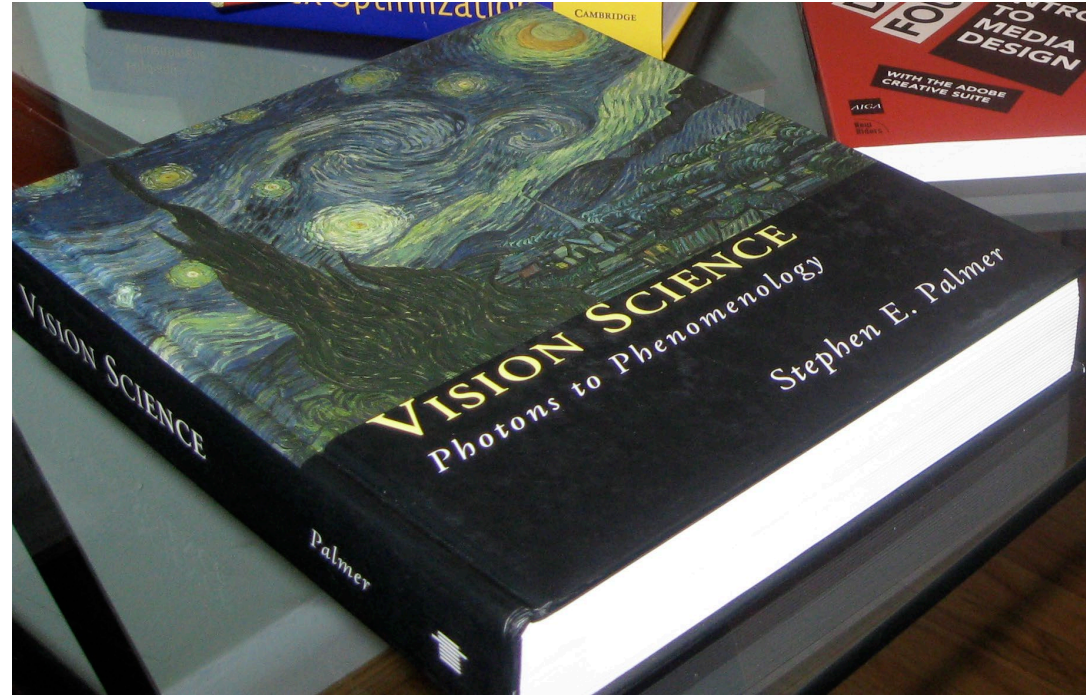
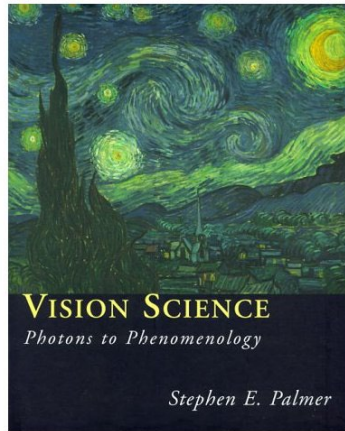
Q: How many unknowns in a projective transformation in  $\mathbb{P}^3$ ?

# Perspective Projection v.s. Projective Transformation

|                      | Perspective Projection                                     | Projective Transformation                                          |
|----------------------|------------------------------------------------------------|--------------------------------------------------------------------|
| Definition           | A mapping from 3D space to a 2D plane (e.g., camera image) | A general mapping between projective space (e.g., $P^2$ to $P^2$ ) |
| Mathematical Formula | $p' = K[R T]P$                                             | $p' = H \cdot p$                                                   |
| Input Space          | $R^3$ (can also be $P^3$ )                                 | $P^n$ (typically $P^2$ in this class)                              |
| Output Space         | $R^2$ (can also be $P^2$ )                                 | $P^n$ (typically $P^2$ in this class)                              |
| Applications         | Image formulation, 3D rendering                            | Image registration, planar transformation, texture mapping         |

# When Perspective Projection -> Projective Transformation?

A perspective camera projection of a plane (i.e., a camera image) is a projective transformation in  $\mathbb{P}^2$

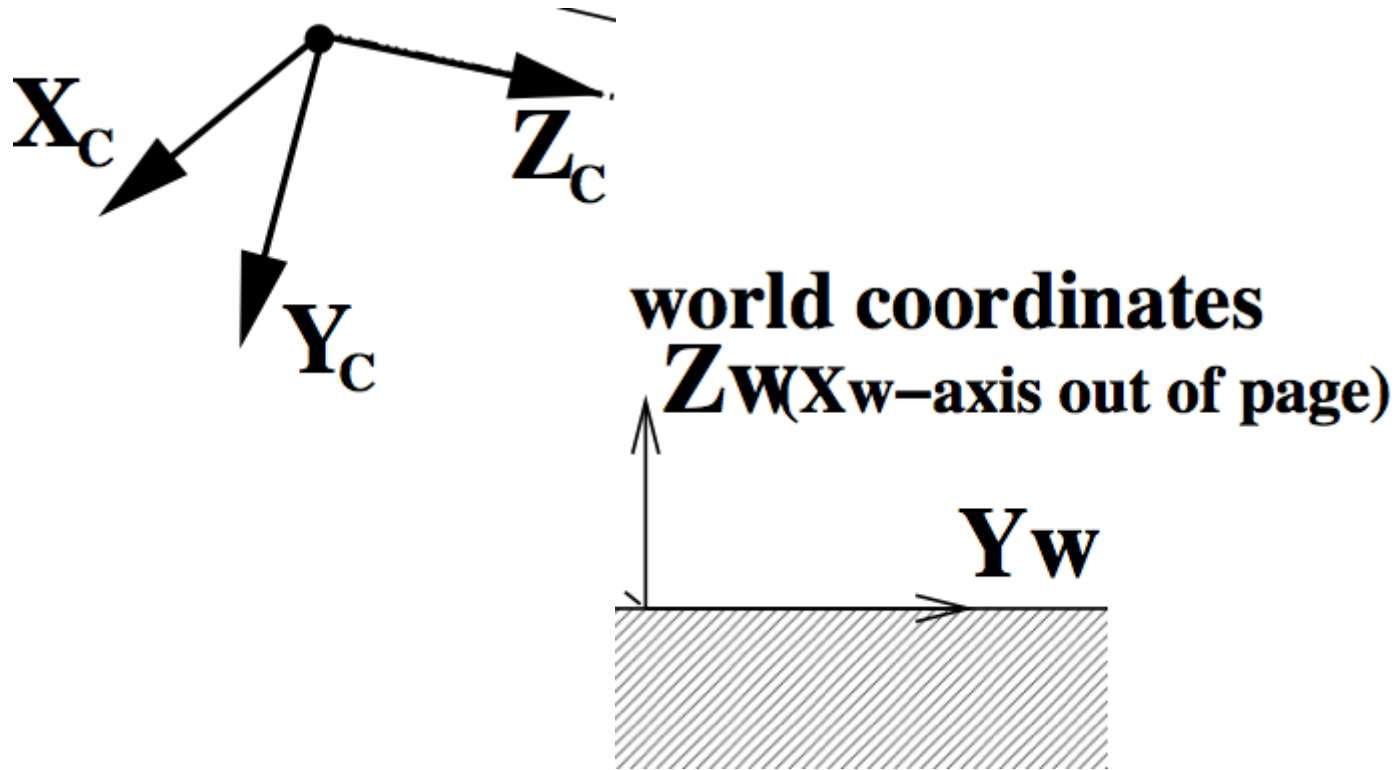


# When Perspective Projection $\rightarrow$ Projective Transformation?

- Can we show that the perspective camera projection from  $\mathbb{P}^3 \rightarrow \mathbb{P}^2$  of a plane in the world is in fact a homography in  $\mathbb{P}^2$  (i.e., projective transformation from  $\mathbb{P}^2 \rightarrow \mathbb{P}^2$ ) when the world plane coordinates are expressed in  $\mathbb{P}^2$ ?
- Remember:

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \sim K_{3 \times 3} [R_{3 \times 3} | \mathbf{t}_{3 \times 1}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Assume world plane  $Z_w = 0$



# When Perspective Projection -> Projective Transformation?

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane  $Z = 0$ .

# Pose From Homography

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane  $Z = 0$ .

Then the transformation reads

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim \underbrace{K \begin{pmatrix} r_1 & r_2 & T \end{pmatrix}}_{\text{Homography}} \begin{pmatrix} X \\ Y \\ W \end{pmatrix}$$

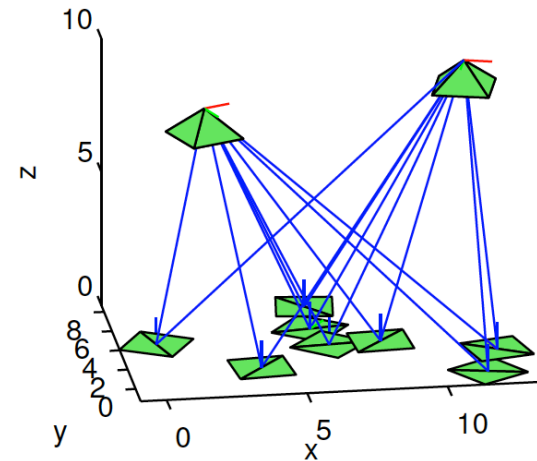
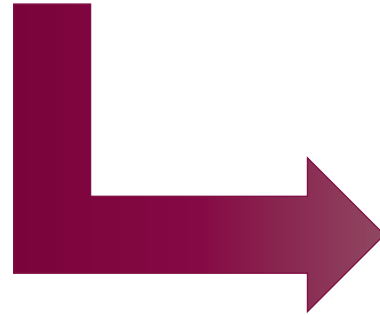
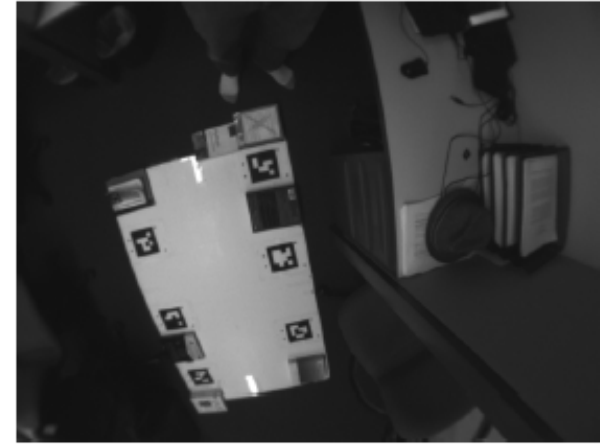
The planar homography  
 $H: \mathbb{P}^2 \rightarrow \mathbb{P}^2$

Computing the homography can tell us how the camera (and therefore, e.g. a robot attached to the camera) is oriented w.r.t. to a world plane! (assuming known K)

Q: Where do you get  $r_3$  from though?

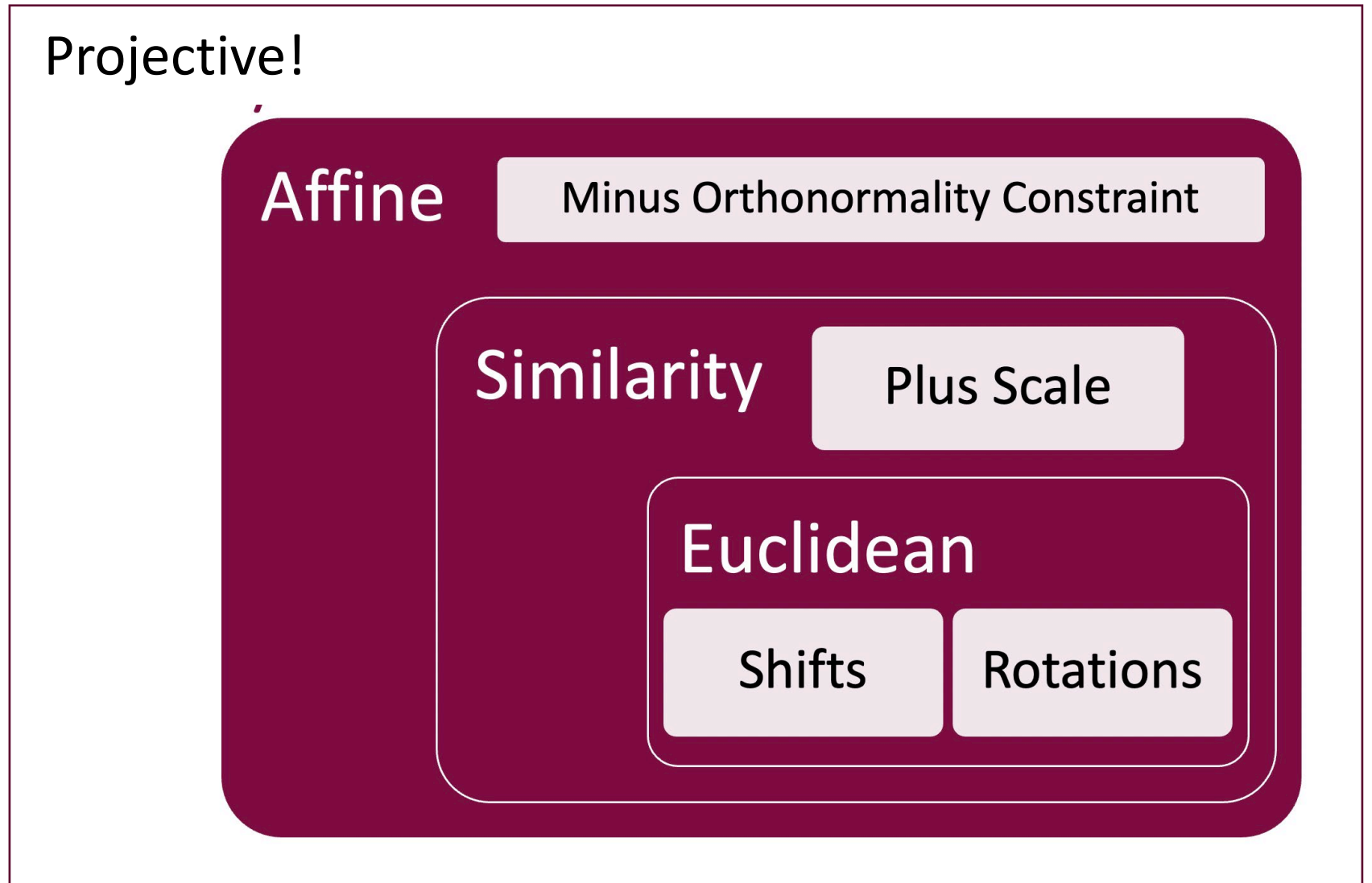
A:  $r_3 = r_1 \times r_2$

# Localization w.r.t. known planes using homographies



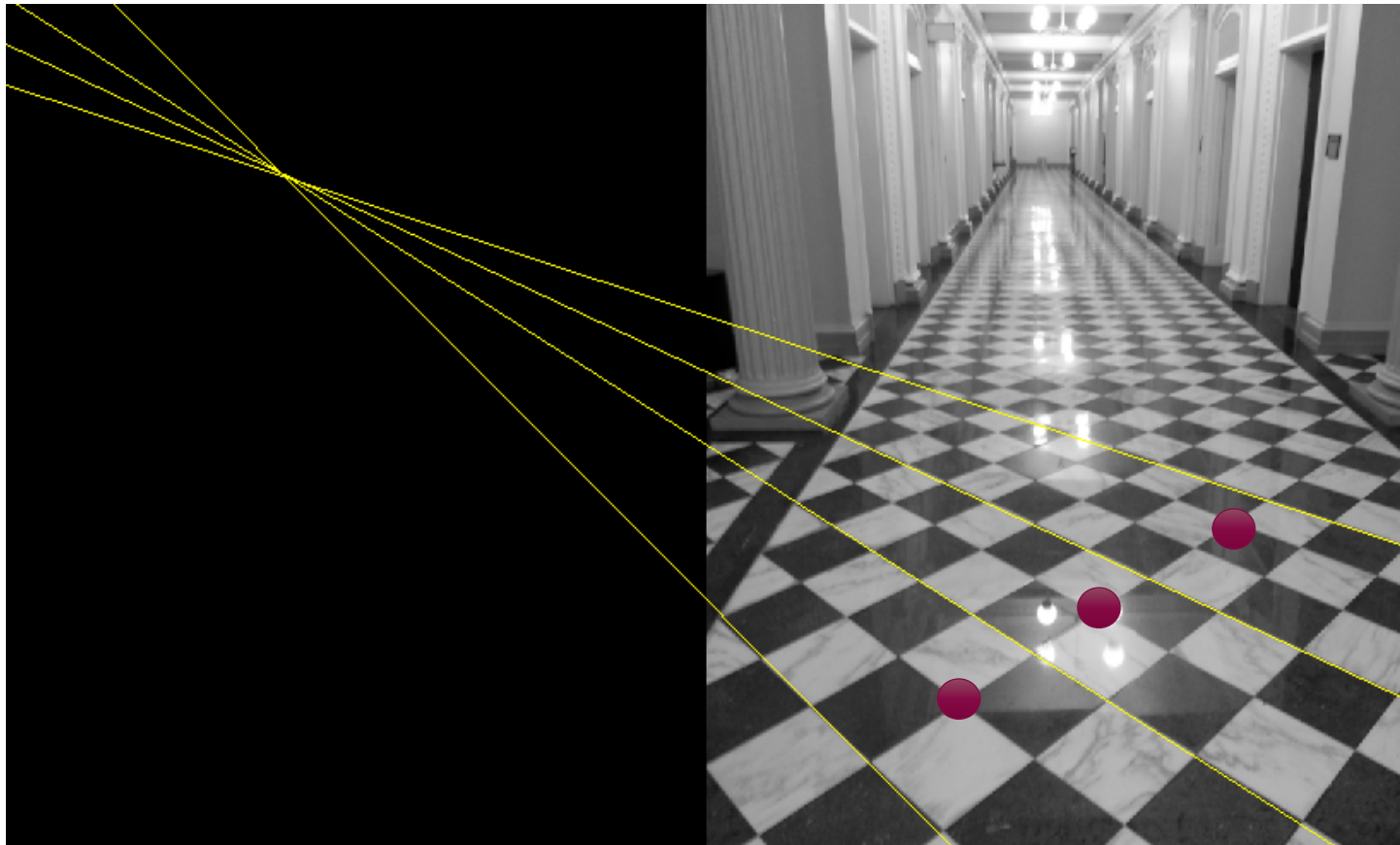


# Place in the Hierarchy of Transformations

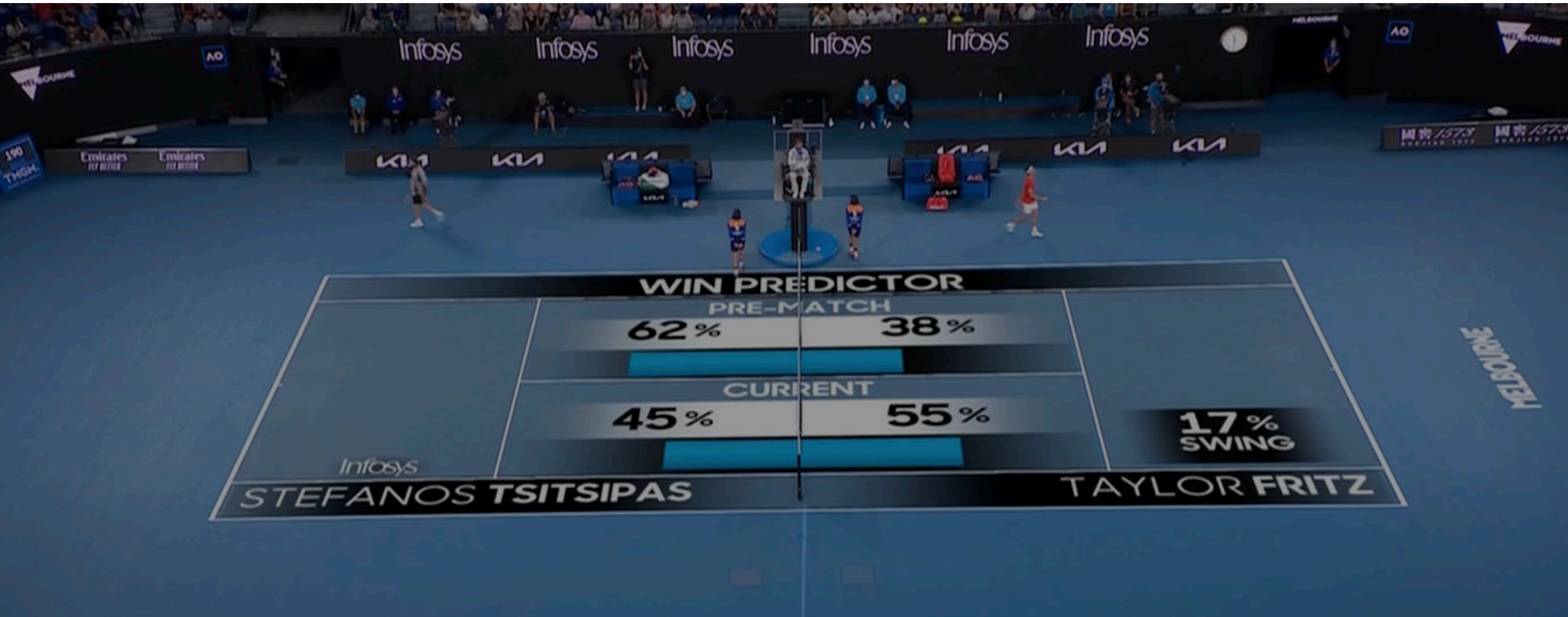


A projective transformation preserves incidence:

- Three collinear points are mapped to three collinear points.
- and three concurrent lines are mapped to three concurrent lines.



# Application Example: Virtual Billboards

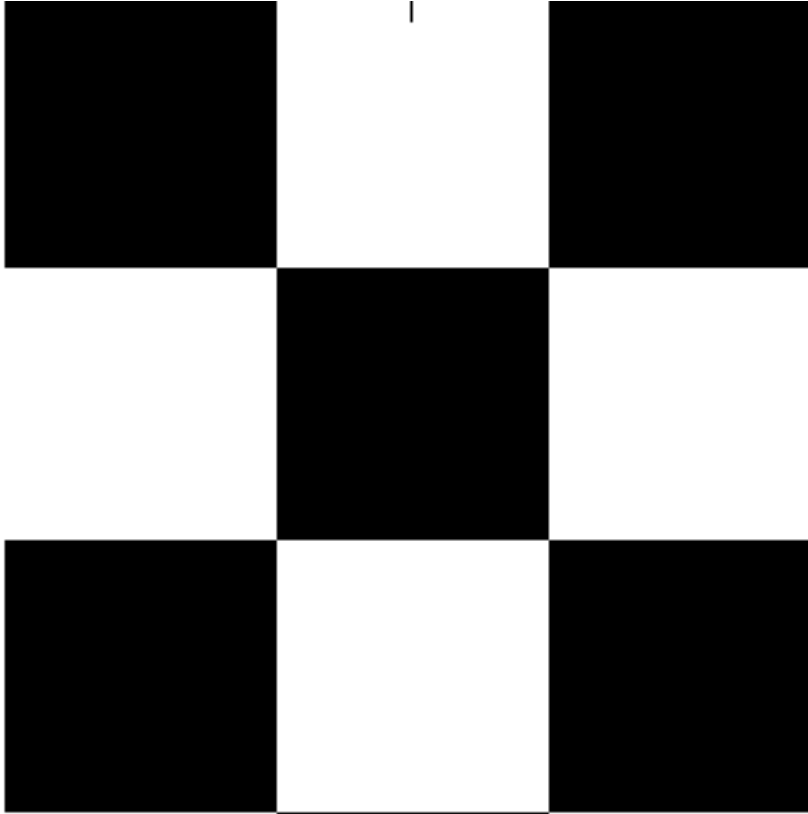




# Computing Homographies From 4 Point Correspondences

“4-point collineation”

# How can we compute the projective transformation between a known pattern and its projection?



Floor tiles measured in [m]

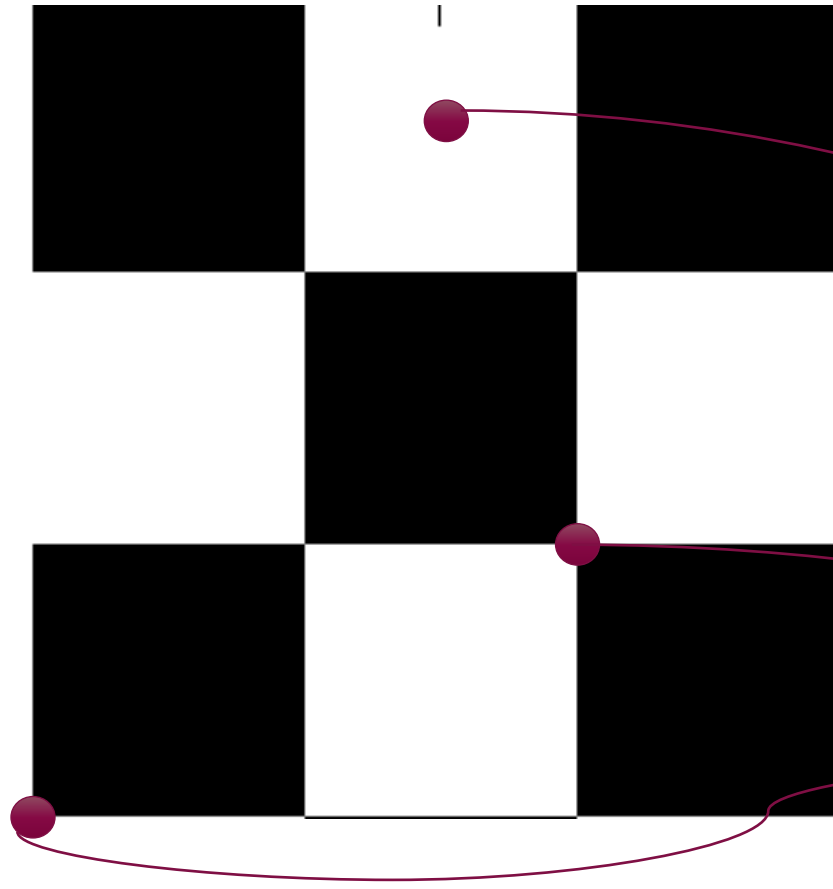
and



Points in pixel coordinates

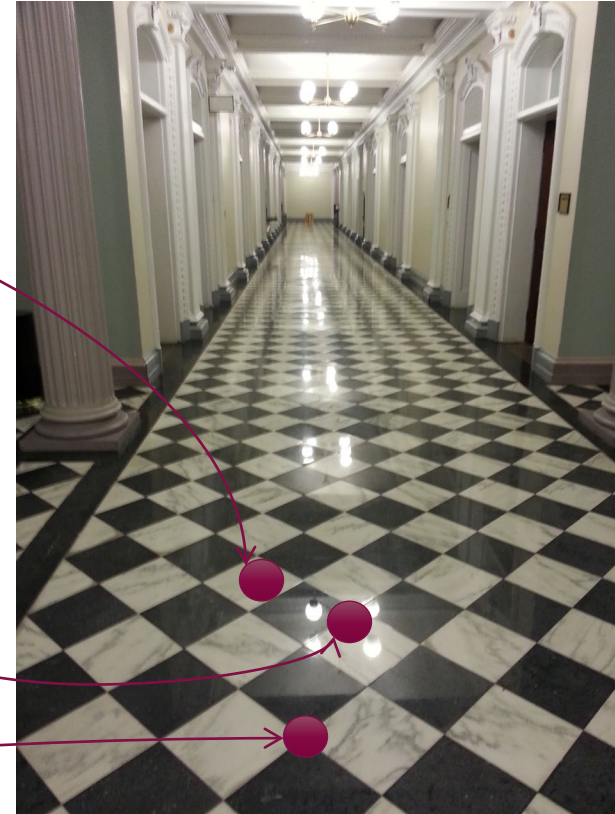
The result of such a transformation would map any point in one plane to the corresponding point in the other

“correspondences”



Floor tiles measured in [m]

and



Points in pixel coordinates



Recap: **How many unknowns are in a projective transformation  $H$  ?**

$$(\mathbb{P}^2 \rightarrow \mathbb{P}^2)$$

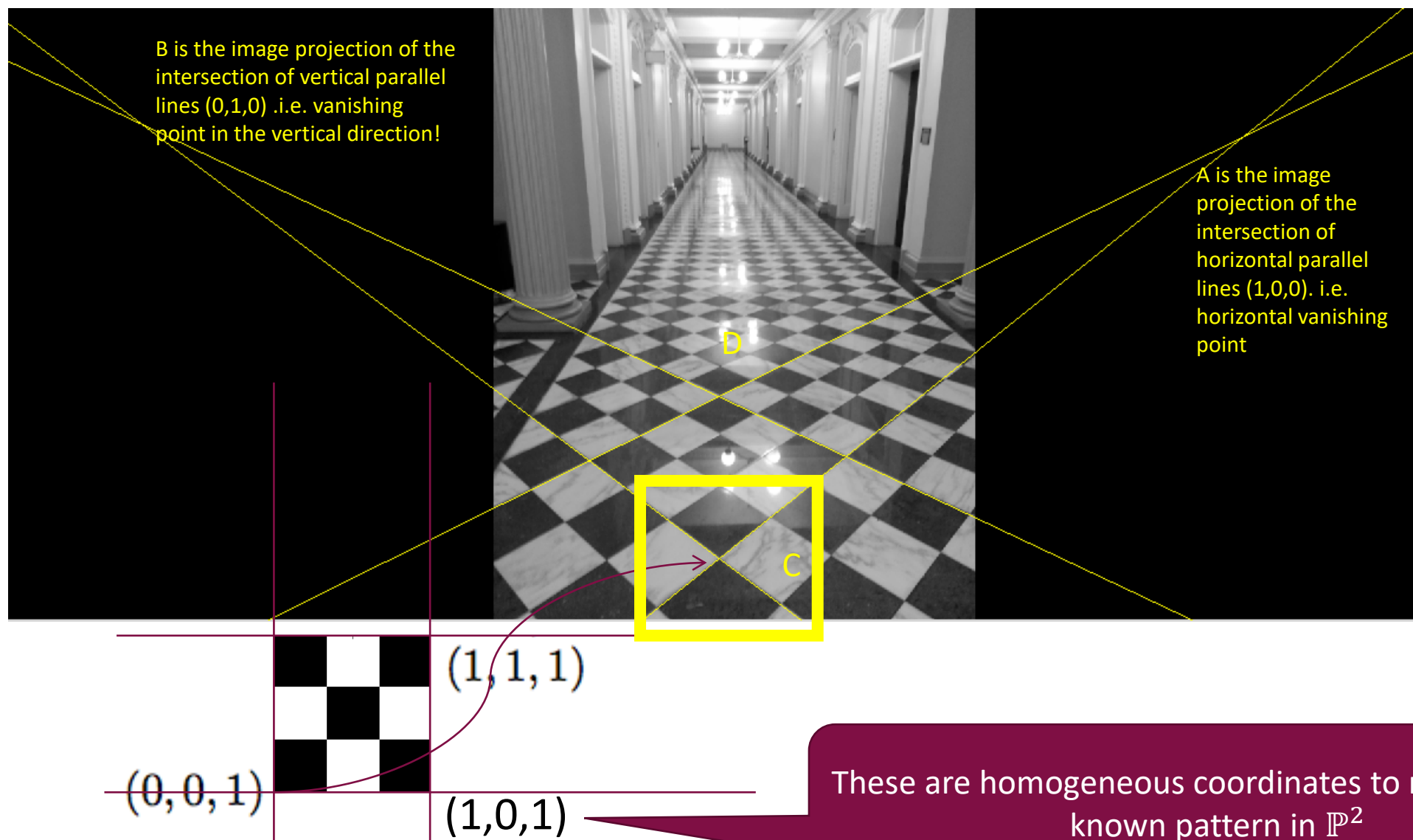
A projective transformation  $\mu H$  is the same as  $H$  since they map to projectively equivalent points:

$$\mu \lambda p' = \mu H p$$

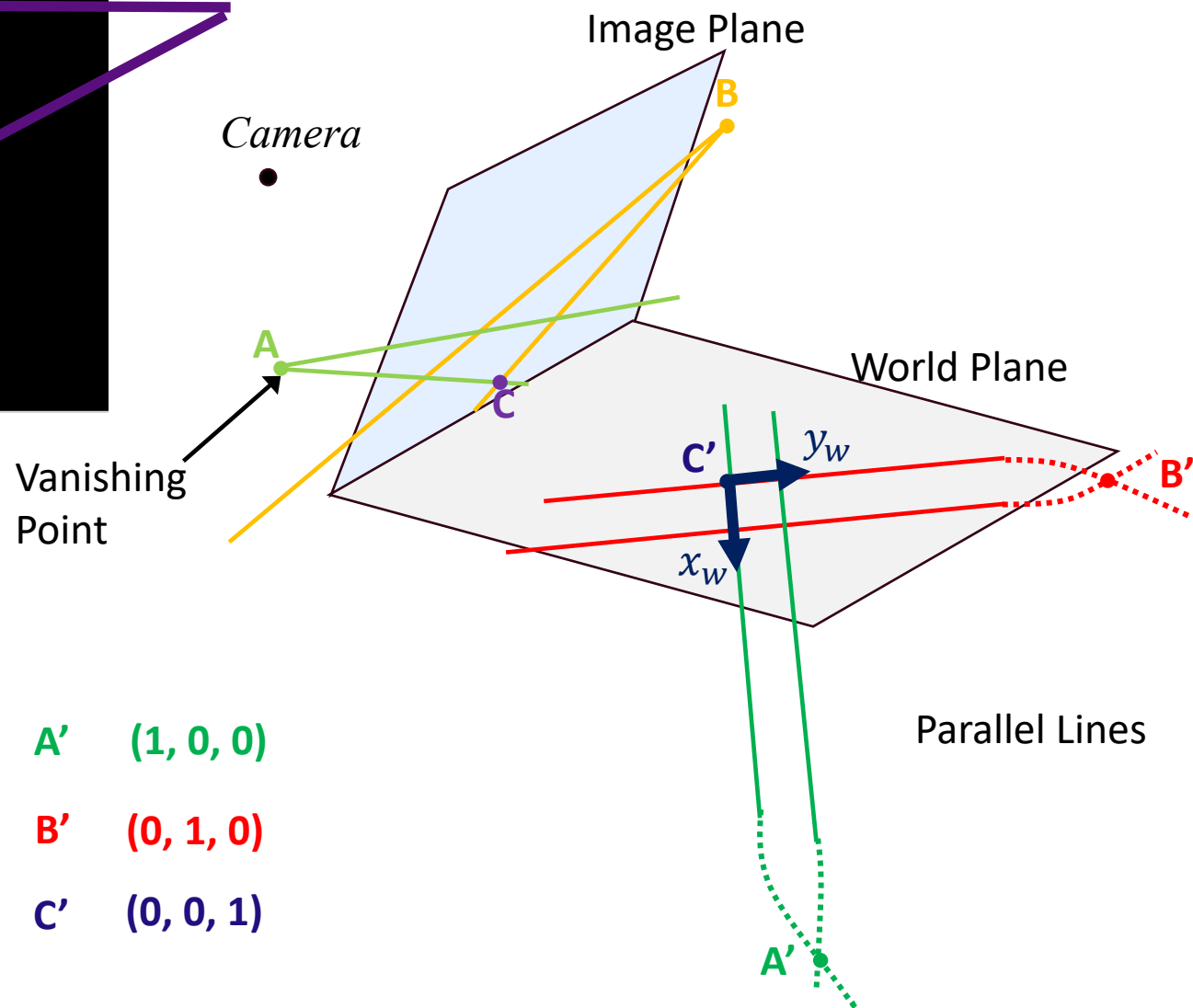
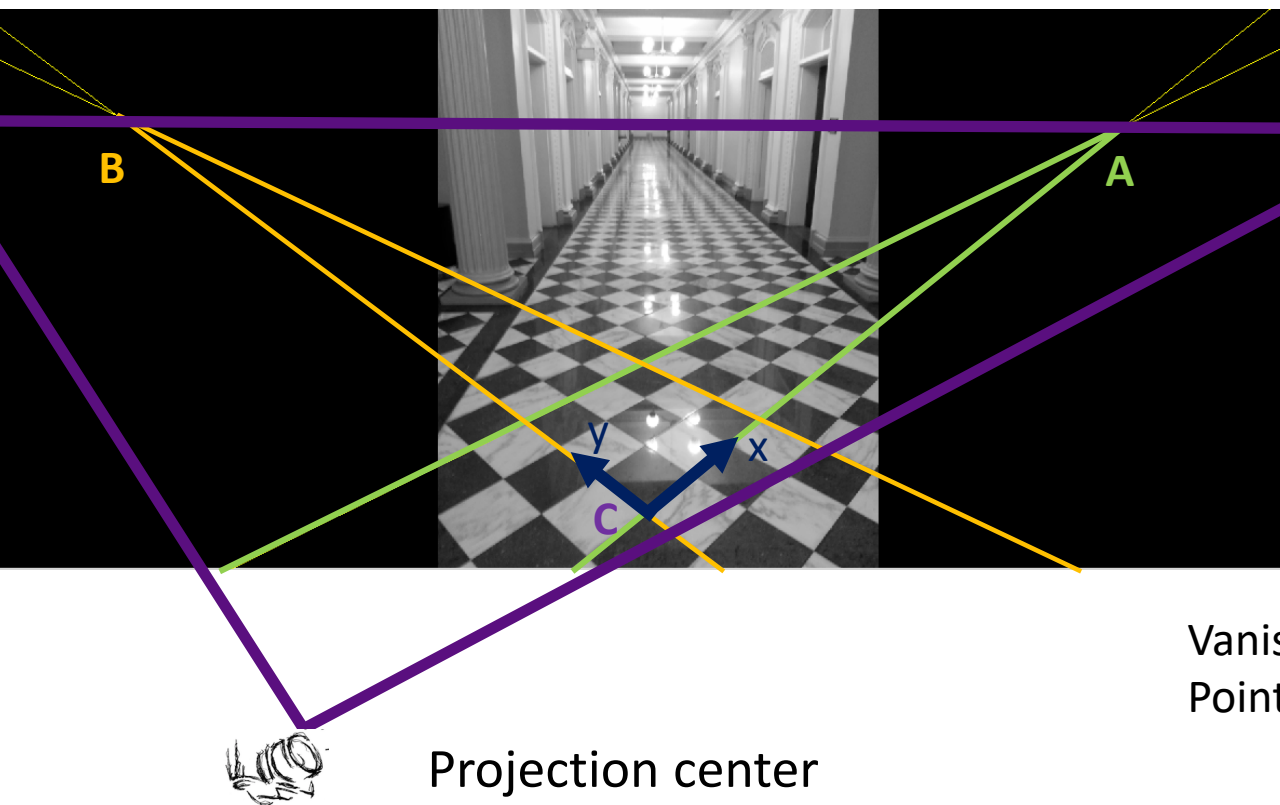
We will be able to determine a projective transformation only up to a scale factor. Hence the 3x3 invertible matrix  $H$  will have only EIGHT independent unknowns.



# How can we compute the projective transformation between a known pattern and its projection?



# Cont.



Correspondences:  
( $P^2 \rightarrow P^2$ )

|   |   |                       |    |           |
|---|---|-----------------------|----|-----------|
| A | a | $\longleftrightarrow$ | A' | (1, 0, 0) |
| B | b | $\longleftrightarrow$ | B' | (0, 1, 0) |
| C | c | $\longleftrightarrow$ | C' | (0, 0, 1) |

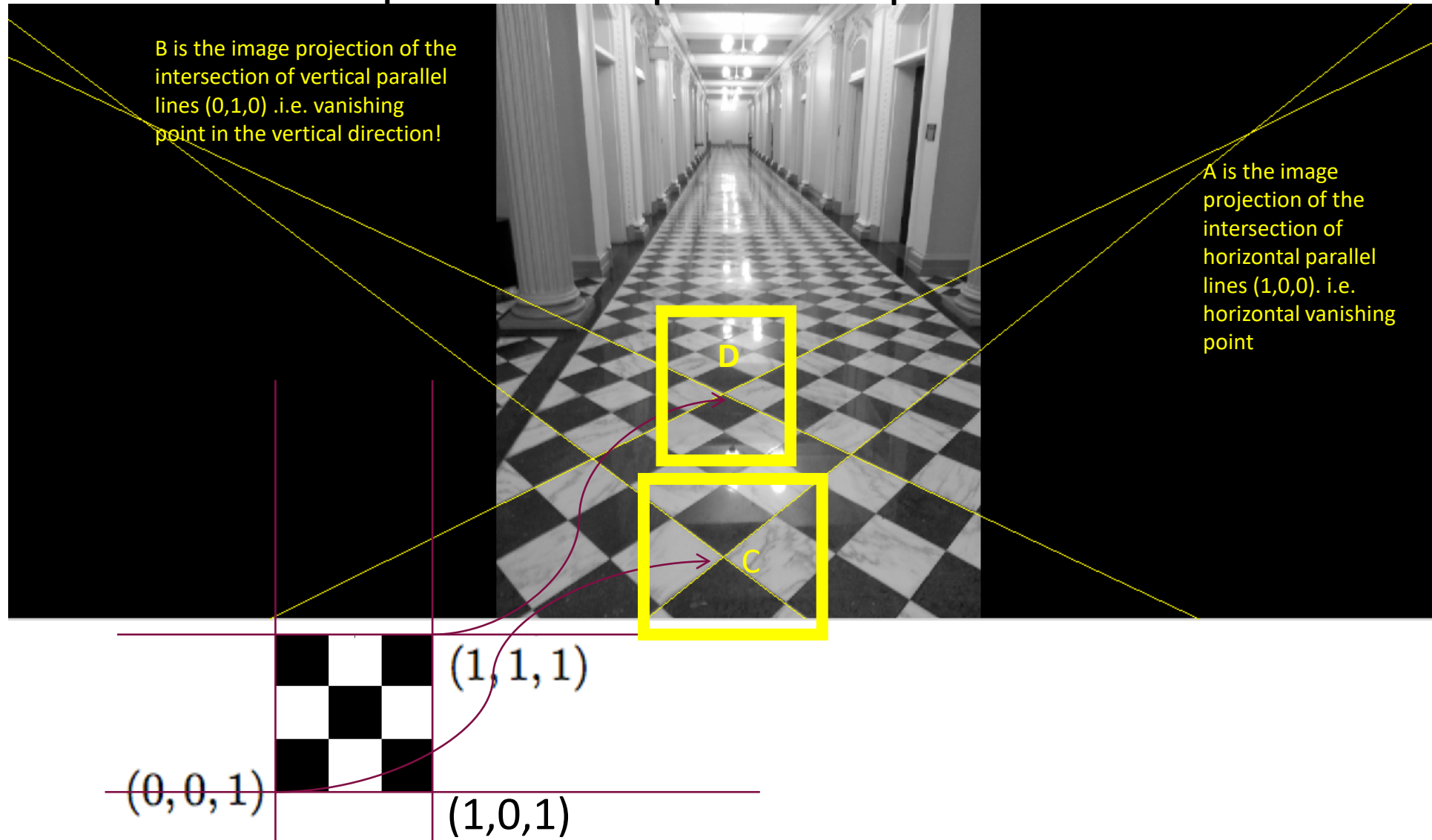
Assume that a mapping  $H$  maps the three points  $(1, 0, 0)$ ,  $(0, 1, 0)$ , and  $(0, 0, 1)$  to the non-collinear points  $A, B, C$

with coordinate vectors  $a, b$  and  $c \in \mathbb{P}^2$ . Then the following is a possible projective transformation:

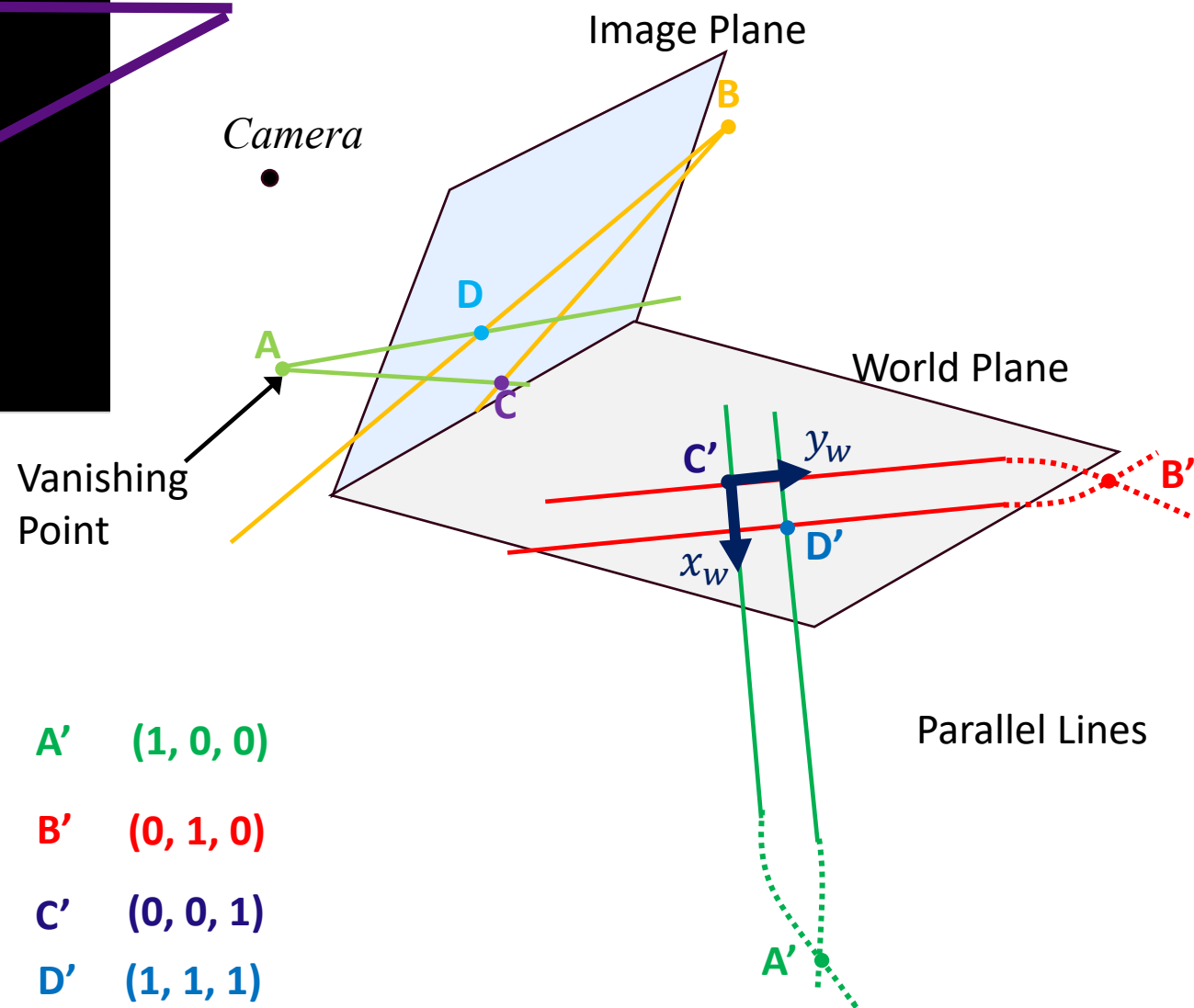
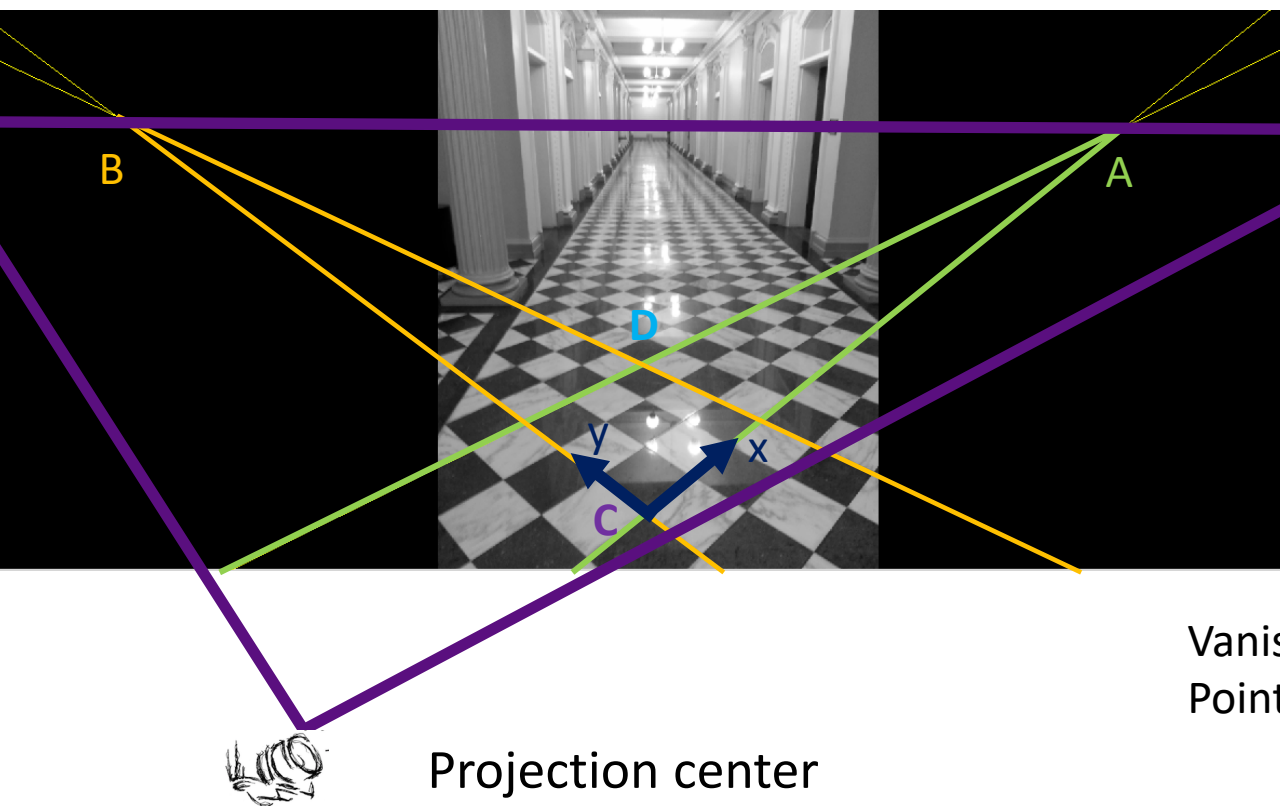
$$\begin{array}{c} \textcolor{green}{A} \quad \textcolor{blue}{B} \quad \textcolor{red}{C} \\ \left( \begin{array}{|c|} \hline \textcolor{green}{a} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{blue}{b} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{red}{c} \\ \hline \end{array} \right) \sim \quad \textcolor{red}{H}_{3 \times 3} \quad \left( \begin{array}{|c|} \hline \textcolor{green}{1} \\ \hline \textcolor{red}{0} \\ \hline \textcolor{blue}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{red}{0} \\ \hline \textcolor{blue}{1} \\ \hline \textcolor{green}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{blue}{0} \\ \hline \textcolor{green}{0} \\ \hline \textcolor{red}{1} \\ \hline \end{array} \right)$$

# Solution: Introduce a 4<sup>th</sup> point correspondence D

Note: makes sense, because after all,  $H$  has 8 degrees of freedom, and each 2D point correspondence pins down 2DOF.



# Cont.



Correspondences:  
( $P^2 \rightarrow P^2$ )

|   |   |                       |    |           |
|---|---|-----------------------|----|-----------|
| A | a | $\longleftrightarrow$ | A' | (1, 0, 0) |
| B | b | $\longleftrightarrow$ | B' | (0, 1, 0) |
| C | c | $\longleftrightarrow$ | C' | (0, 0, 1) |
| D | d | $\longleftrightarrow$ | D' | (1, 1, 1) |

Let us assume that the same  $H$  maps  $(1, 1, 1)$  to the point  $d$ . Then, the following should hold:

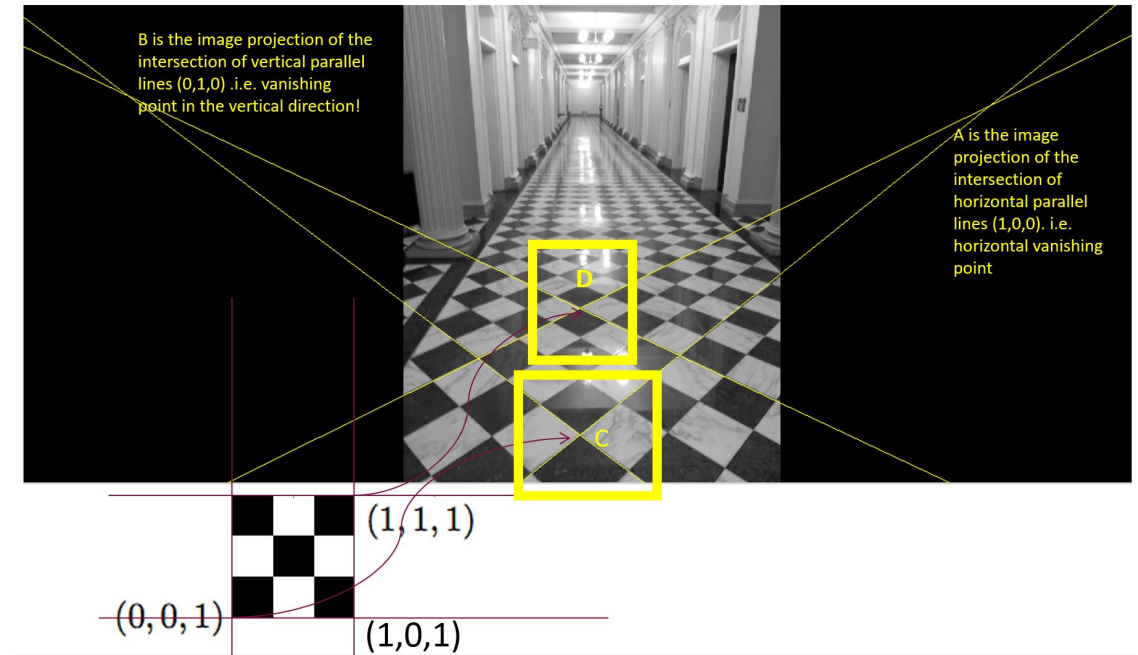
$$d \sim \underset{H_{3 \times 3}}{\begin{pmatrix} \alpha a & \beta b & \gamma c \end{pmatrix}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix},$$

hence

$$d \sim \alpha a + \beta b + \gamma c. \quad \text{or} \quad \lambda d = \alpha a + \beta b + \gamma c$$

Because  $a, b, c$  are not collinear, there exist unique  $\alpha/\lambda, \beta/\lambda, \gamma/\lambda$  for writing this linear combination.

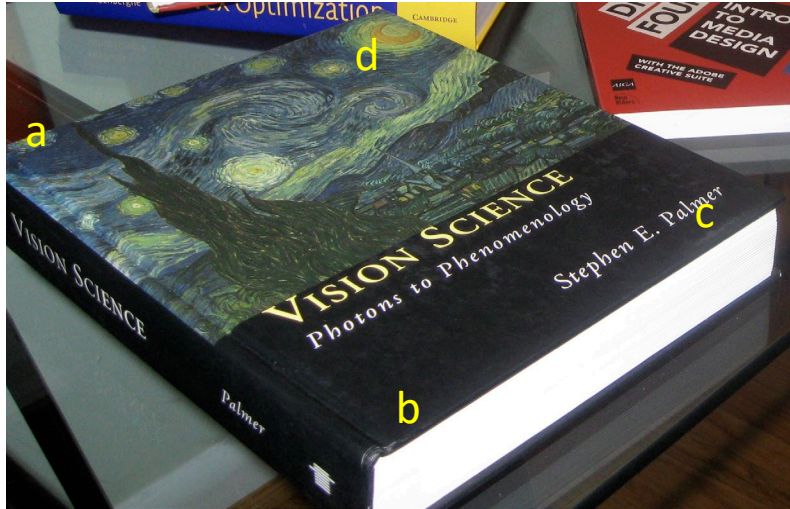
Four points, no three of them collinear, suffice to unambiguously recover a homography



Choosing the points to be the horizontal and vertical vanishing points  $(1,0,0)$ ,  $(0,1,0)$  plus origin  $(0,0,1)$  and the diagonal  $(1,1,1)$  is particularly “nice” especially if you have a square to start from, but really, any four non-collinear points will do.  
(coming up next)



What happens when the original set of points is not a square?



Find projective transformation mapping  $(a, b, c, d) \rightarrow (a', b', c', d')$ :



To determine this mapping we go through the four canonical points.

We find the mapping from  $(1, 0, 0), etc$  to  $(a, b, c, d)$  and we call it  $T$ :

$$a \sim T(1, 0, 0)^T, etc$$

We find the mapping from  $(1, 0, 0), etc$  to  $(a', b', c', d')$  and we call it  $T'$ :

$$a' \sim T'(1, 0, 0)^T, etc$$

Then, back-substituting  $(1, 0, 0)^T \sim T^{-1}a, etc$  we obtain that

$$a' = T'T^{-1}a, etc$$

To determine this mapping we go through the four canonical points.

We find the mapping from  $(1, 0, 0), etc$  to  $(a, b, c, d)$  and we call it  $T$ :

$$a \sim T(1, 0, 0)^T, etc$$

We find the mapping from  $(1, 0, 0), etc$  to  $(a', b', c', d')$  and we call it  $T'$ :

$$a' \sim T'(1, 0, 0)^T, etc$$

Then, back-substituting  $(1, 0, 0)^T \sim T^{-1}a, etc$  we obtain that

$$a' = T'T^{-1}a, etc$$

To determine this mapping we go through the four canonical points.

We find the mapping from  $(1, 0, 0)$ , *etc* to  $(a, b, c, d)$  and we call it  $T$ :

$$a \sim T(1, 0, 0)^T, \text{etc}$$

We find the mapping from  $(1, 0, 0)$ , *etc* to  $(a', b', c', d')$  and we call it  $T'$ :

$$a' \sim T'(1, 0, 0)^T, \text{etc}$$

Then, back-substituting  $(1, 0, 0)^T \sim T^{-1}a$ , *etc* we obtain that

$$a' = T'T^{-1}a, \text{etc}$$

This amounts to computing homographies from a made-up square, then inverting one of them and composing!

# Homography -> Virtual Billboards

- For virtual billboards, we just treat the desired billboard pattern as the world pattern.
- Recall that the homography is a mapping from world to image pixel coordinates. So, just the homography can directly find the image pixel coordinate corresponding to every image in the world pattern.

(Q: is this actually what we need for virtual billboards? A: not quite. )



# Microsoft Office Lens App



## Office Lens

Microsoft Corporation Productivity

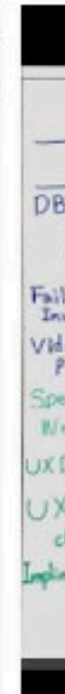
Everyone

This app is compatible with all of your devices.

Top Developer

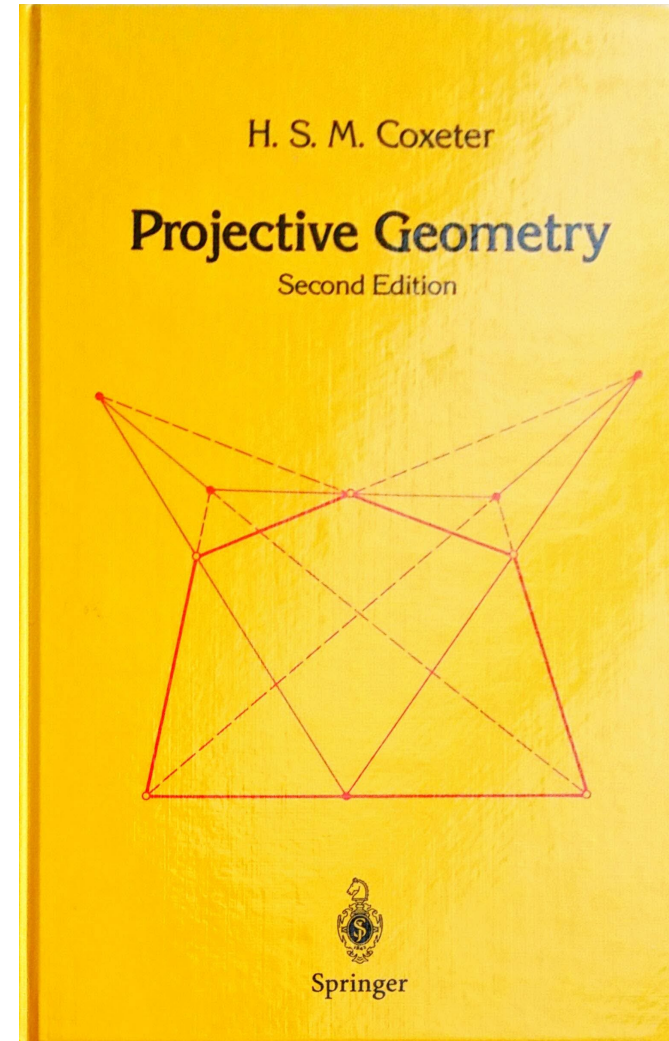
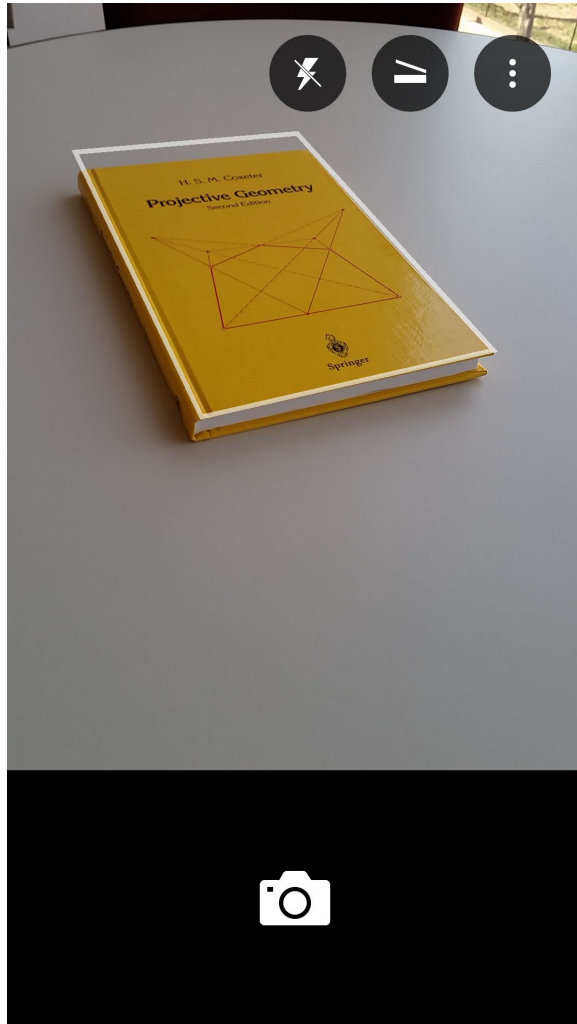
★★★★★ 91,419

Installed





# Office Lens





# Computing homographies with more correspondences

$$\mathbf{x}' \sim H\mathbf{x}$$

$$\lambda \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\lambda x' = h_{11}x + h_{12}y + h_{13}$$

$$\lambda y' = h_{21}x + h_{22}y + h_{23}$$

$$\lambda = h_{31}x + h_{32}y + h_{33}$$



# Converting to a linear system of equations

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$



$$\begin{aligned} -h_{11}x - h_{12}y - h_{13} + h_{31}xx' + h_{32}yx' + h_{33}x' &= 0 \\ -h_{21}x - h_{22}y - h_{23} + h_{31}xy' + h_{32}yy' + h_{33}y' &= 0 \end{aligned}$$



Two linear equations for  
each point correspondence!

$$\begin{pmatrix} a_x \\ a_y \end{pmatrix} h = 0$$

$$a_x = \begin{pmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \end{pmatrix}$$

$$a_y = \begin{pmatrix} 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{pmatrix}$$

$$h = \begin{pmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} & h_{33} \end{pmatrix}^T$$

# Computing homographies with more correspondences

Our matrix  $H$  has 8 degrees of freedom, and so, as each point gives 2 sets of equations, we will need 4 points to solve for  $h$  uniquely. So, given four points (such as the corners provided for this assignment), we can generate vectors  $a_x$  and  $a_y$  for each, and concatenate them together:

$$A = \begin{pmatrix} a_{x,1} \\ a_{y,1} \\ \vdots \\ a_{x,n} \\ a_{y,n} \end{pmatrix} \quad 2n \times 9$$

$$Ah = 0$$

When  $n > 4$ , due to noise in measurements, there may not be an  $h \neq 0$  that produces exactly  $Ah = 0$ .

Instead, we find the vector s.t.  $\|Ah\|_2$  is smallest.

This involves computing the SVD (singular value decomposition) of  $A$ , and setting  $h$  to be the “smallest” right singular vector, with smallest singular value.  
(more info on SVD etc. soon)



CIS 5800

# Machine Perception

*Or Geometric Computer Vision*

Instructor: Lingjie Liu

Lec 5: Feb 5, 2026

# HW1

- HW 1: We plan to release it tomorrow. The deadline will be two weeks after the release date.

# Recap: Projective Transformation = Homography = Collineation=Projectivity

## Definition

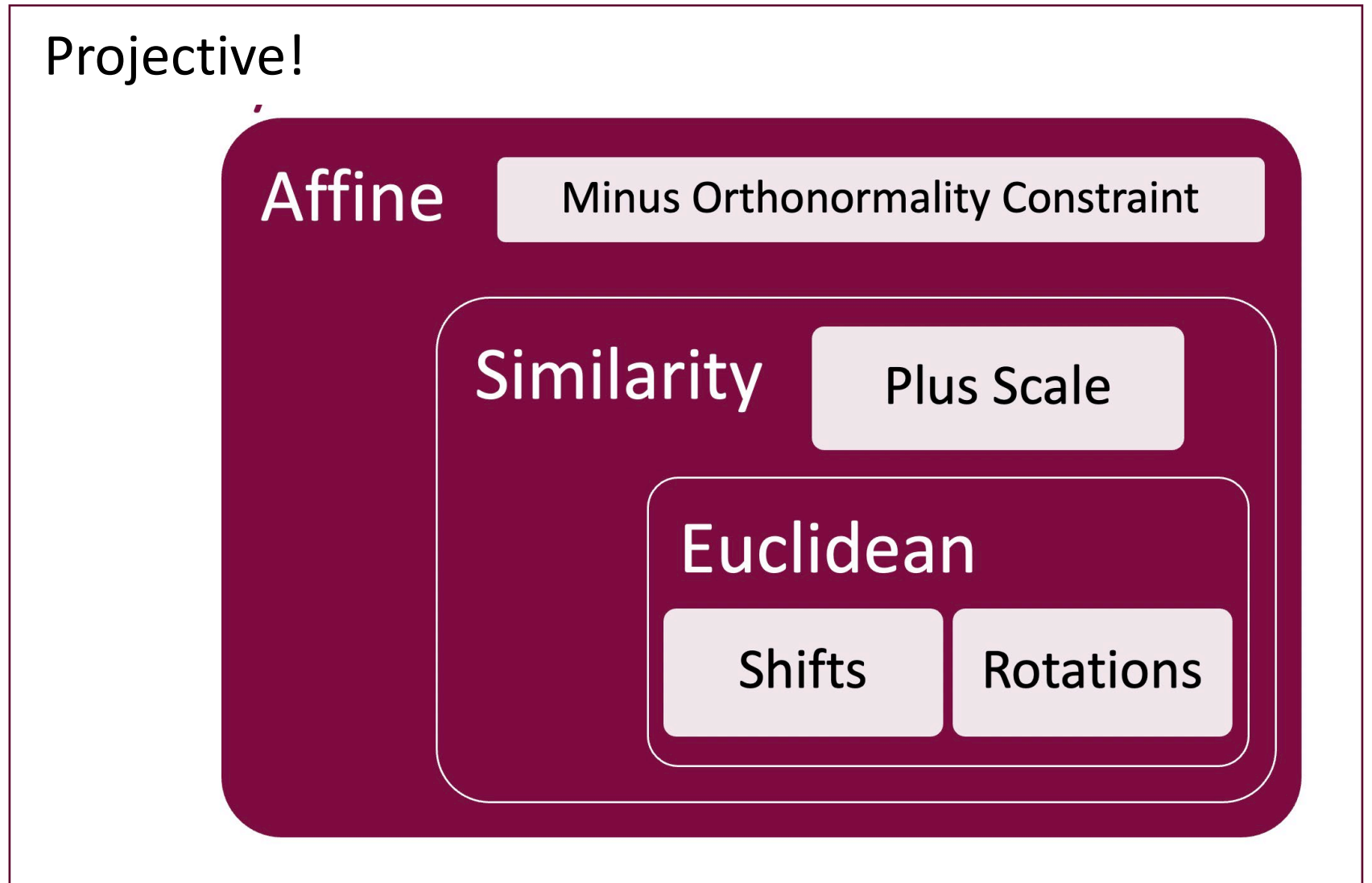
A **projective transformation** is any invertible matrix transformation  $\mathbb{P}^2 \rightarrow \mathbb{P}^2$ .

A projective transformation  $H$  maps  $p$  to  $p' \sim Hp$

Invertibility means that  $\det(H) \neq 0$  and that there exists  $\lambda \neq 0$  such that  $\lambda p' = Hp$

Observe that we will write either  $p' \sim Hp$  or  $\lambda p' = Hp$

# Recap: Place in the Hierarchy of Transformations



# Recap: Perspective Projection v.s. Projective Transformation

|                      | Perspective Projection                                     | Projective Transformation                                          |
|----------------------|------------------------------------------------------------|--------------------------------------------------------------------|
| Definition           | A mapping from 3D space to a 2D plane (e.g., camera image) | A general mapping between projective space (e.g., $P^2$ to $P^2$ ) |
| Mathematical Formula | $p' = K[R T]P$                                             | $p' = H \cdot p$                                                   |
| Input Space          | $R^3$ (can also be $P^3$ )                                 | $P^n$ (typically $P^2$ in this class)                              |
| Output Space         | $R^2$ (can also be $P^2$ )                                 | $P^n$ (typically $P^2$ in this class)                              |
| Applications         | Image formulation, 3D rendering                            | Image registration, planar transformation, texture mapping         |



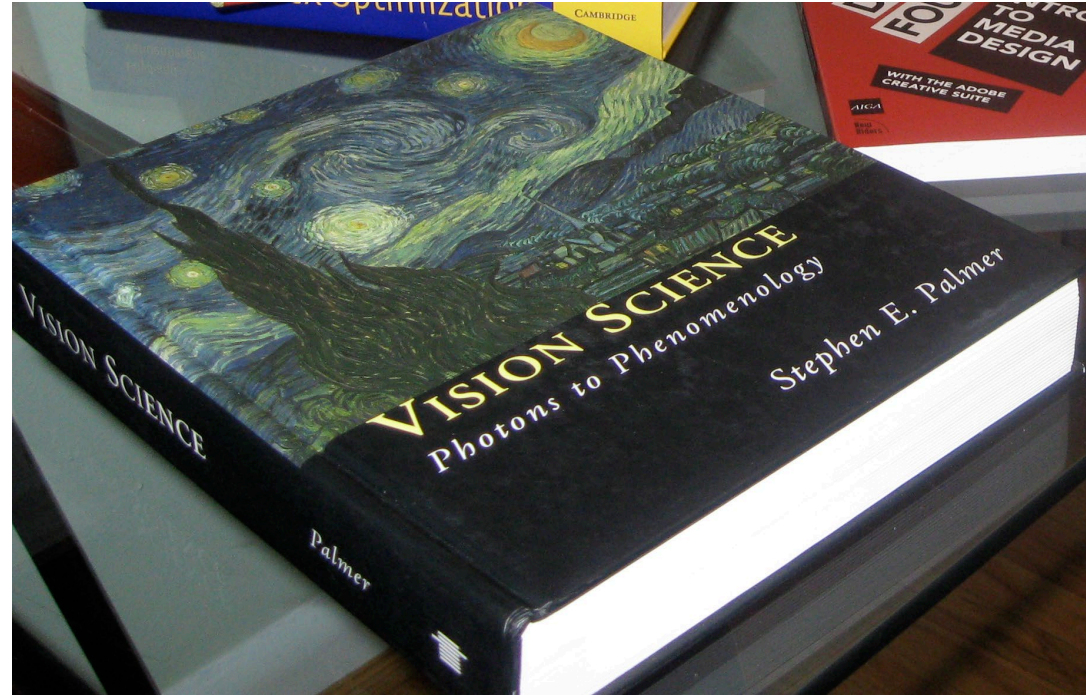
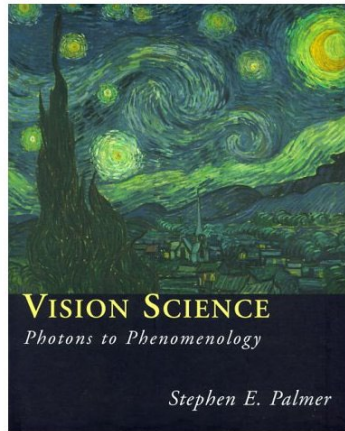
# Recap: Perspective Projection -> Homography

- Can we show that the perspective camera projection from  $\mathbb{P}^3 \rightarrow \mathbb{P}^2$  of a plane in the world is in fact a homography in  $\mathbb{P}^2$  (i.e., projective transformation from  $\mathbb{P}^2 \rightarrow \mathbb{P}^2$ ) when the world plane coordinates are expressed in  $\mathbb{P}^2$ ?
- Remember:

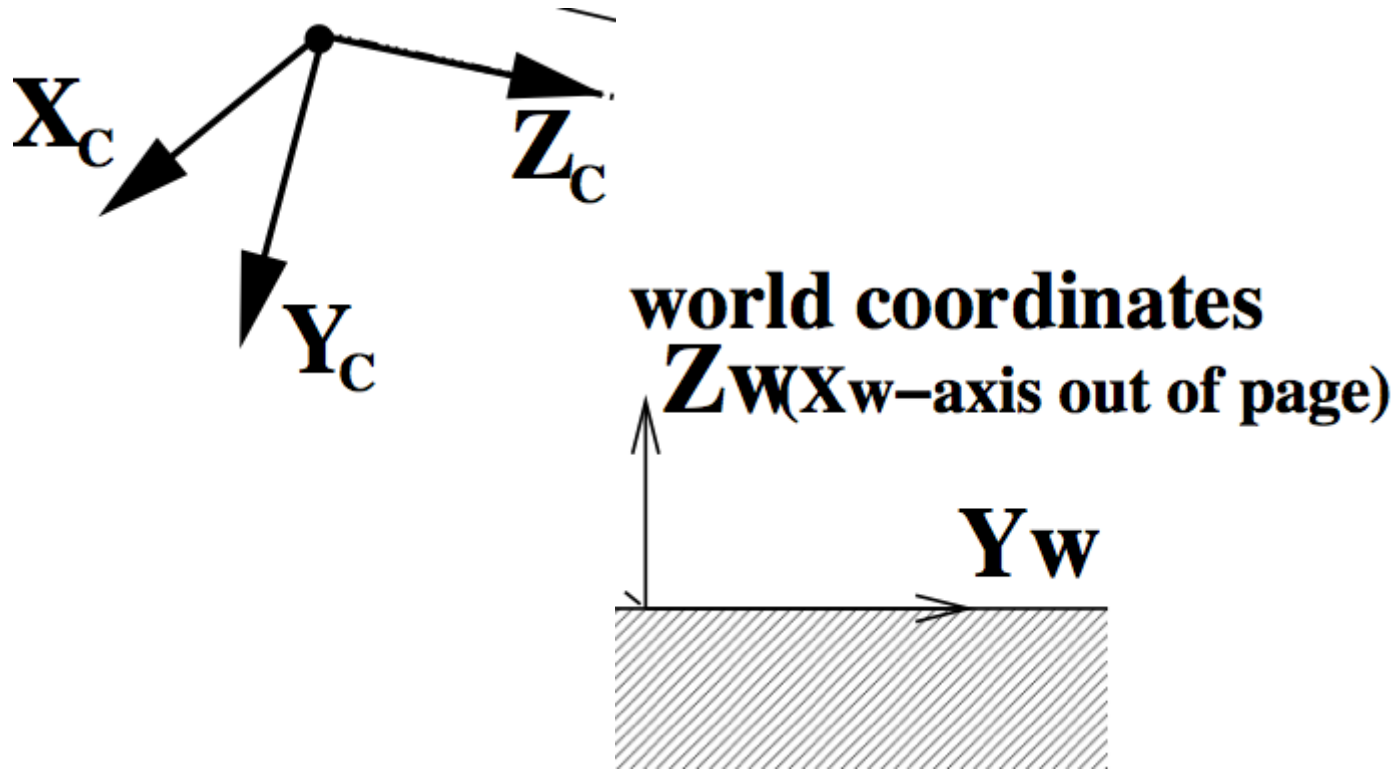
$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \sim K_{3 \times 3} [R_{3 \times 3} | \mathbf{t}_{3 \times 1}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

# Recap: Perspective Projection -> Homography

A perspective camera projection of a plane (i.e., a camera image) is a projective transformation in  $\mathbb{P}^2$



Recap: Assume world plane  $Z_w = 0$



# Recap: Pose From Homography

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane  $Z = 0$ .

Then the transformation reads

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim \underbrace{K \begin{pmatrix} r_1 & r_2 & T \end{pmatrix}} \begin{pmatrix} X \\ Y \\ W \end{pmatrix}$$

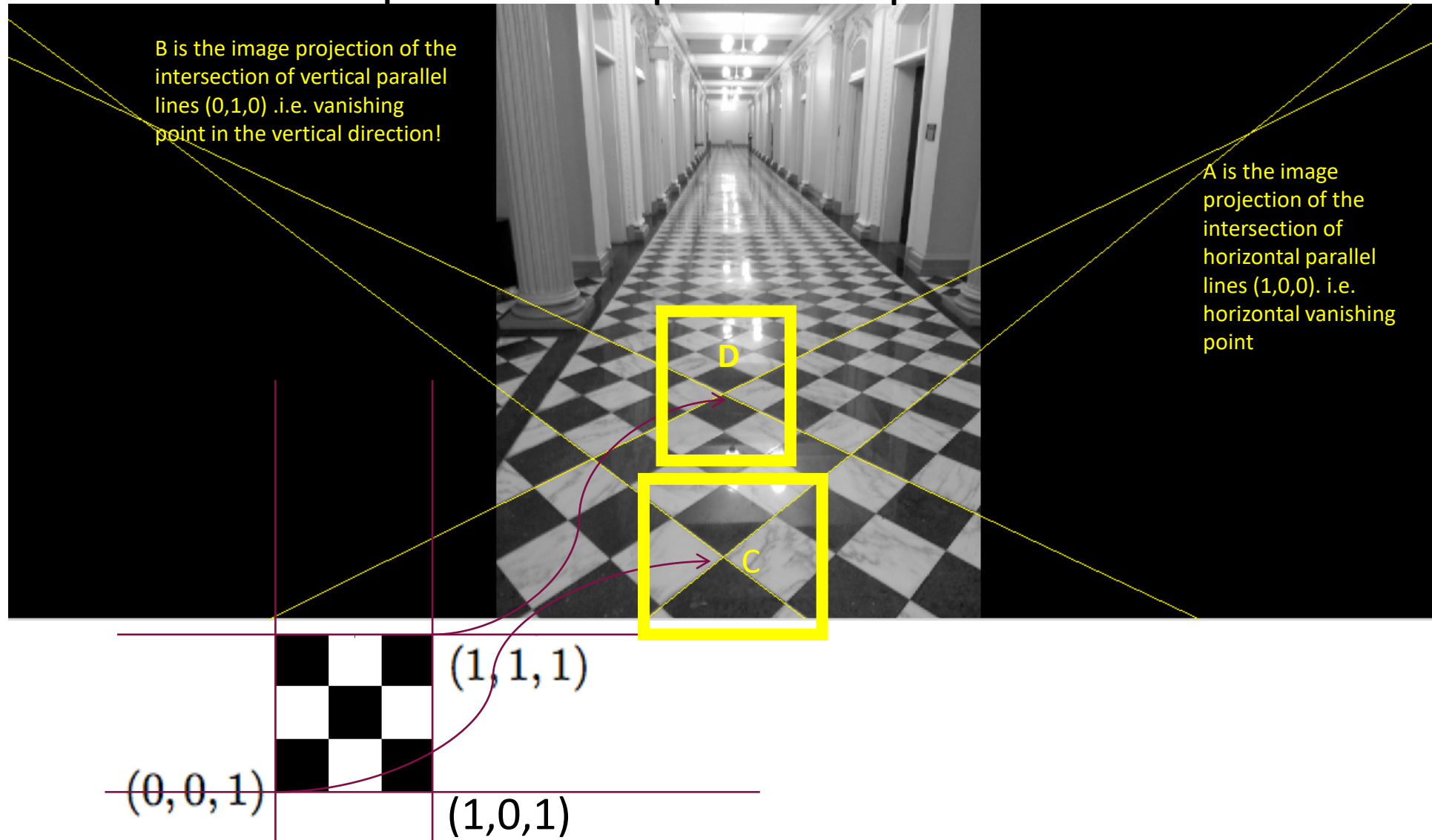
The planar homography  
 $H: \mathbb{P}^2 \rightarrow \mathbb{P}^2$

Computing the homography can tell us how the camera (and therefore, e.g. a robot attached to the camera) is oriented w.r.t. to a world plane! (assuming known  $K$ )

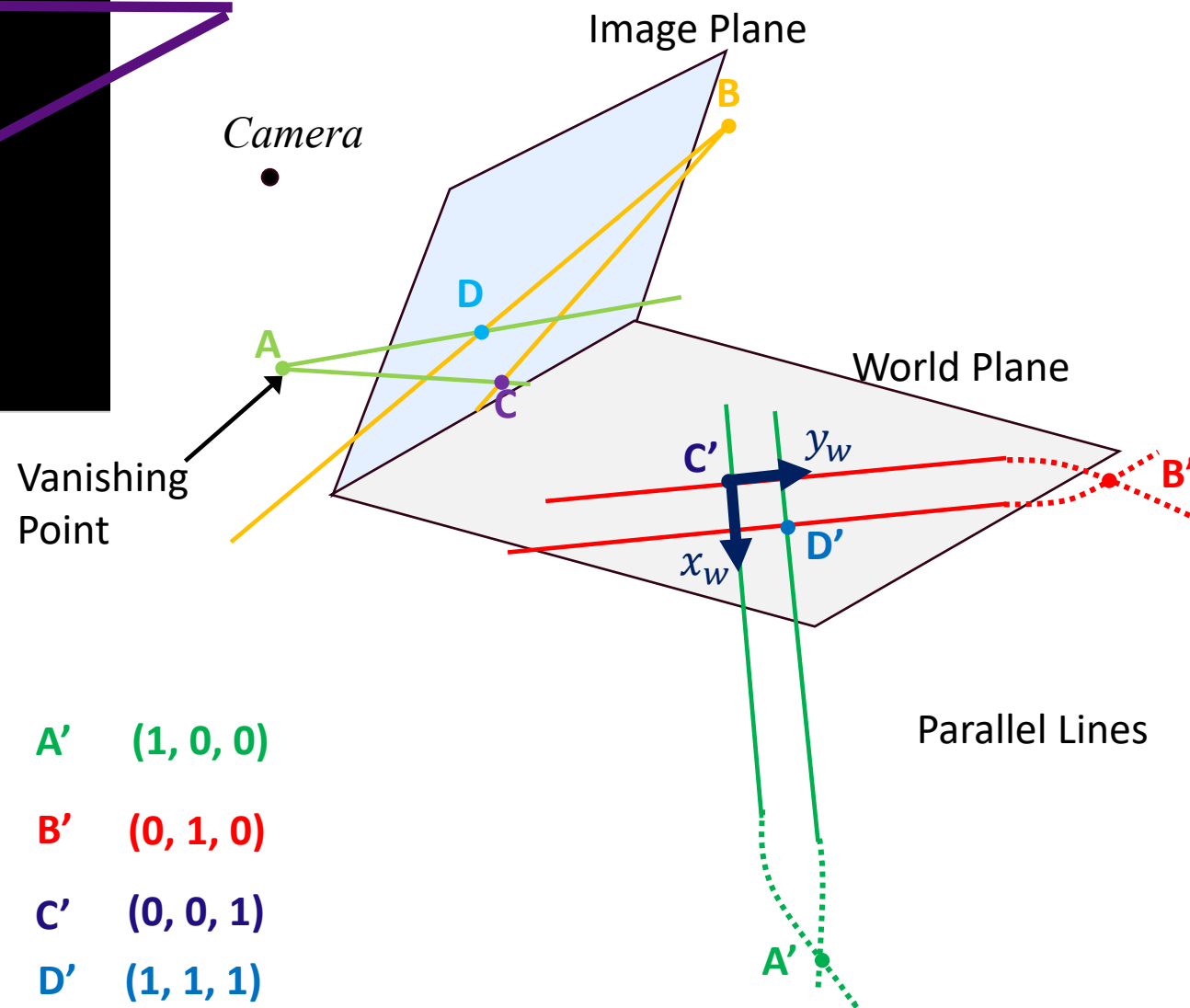
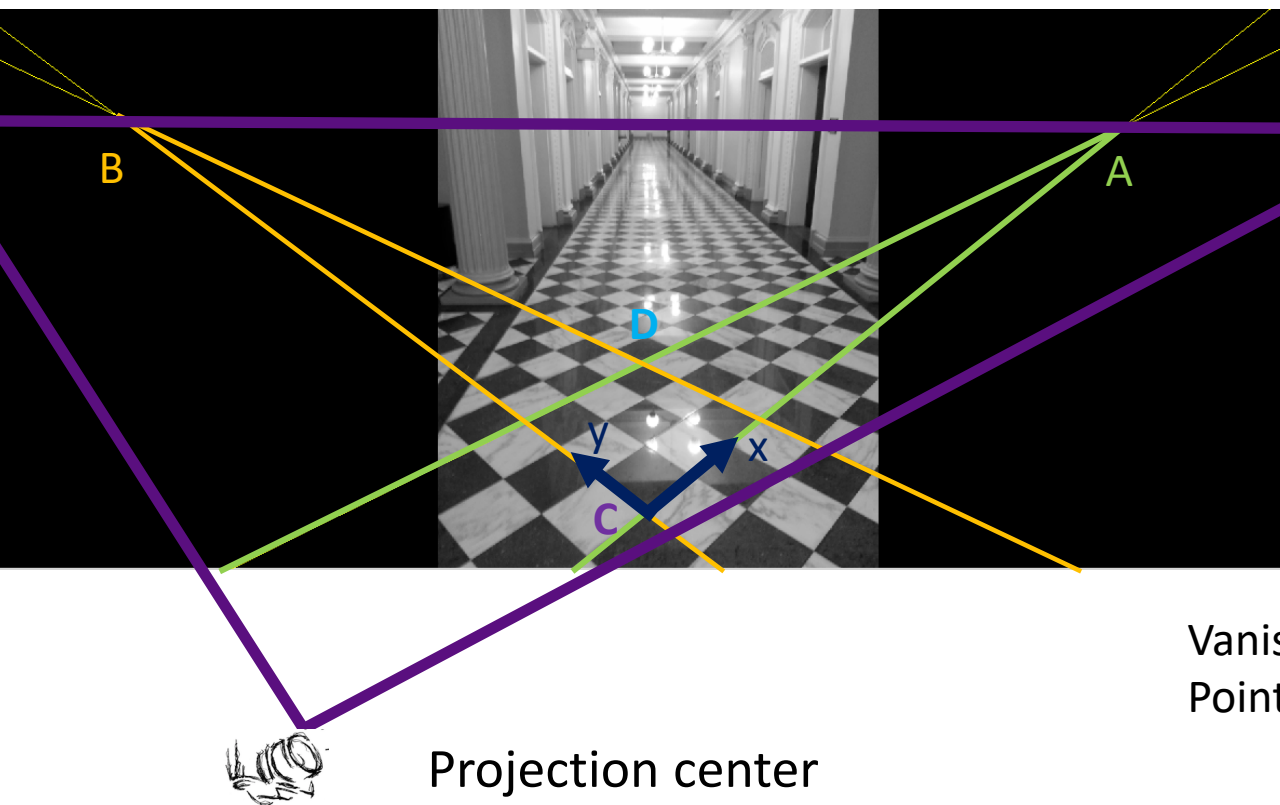
Q: Where do you get  $r_3$  from though? **A:  $r_3 = r_1 \times r_2$**

# Recap: Computing Homography from 4 Point Correspondences

Note: makes sense, because after all,  $H$  has 8 degrees of freedom, and each 2D point correspondence pins down 2DOF.



# Recap: Computing Homography from 4 Point Correspondences

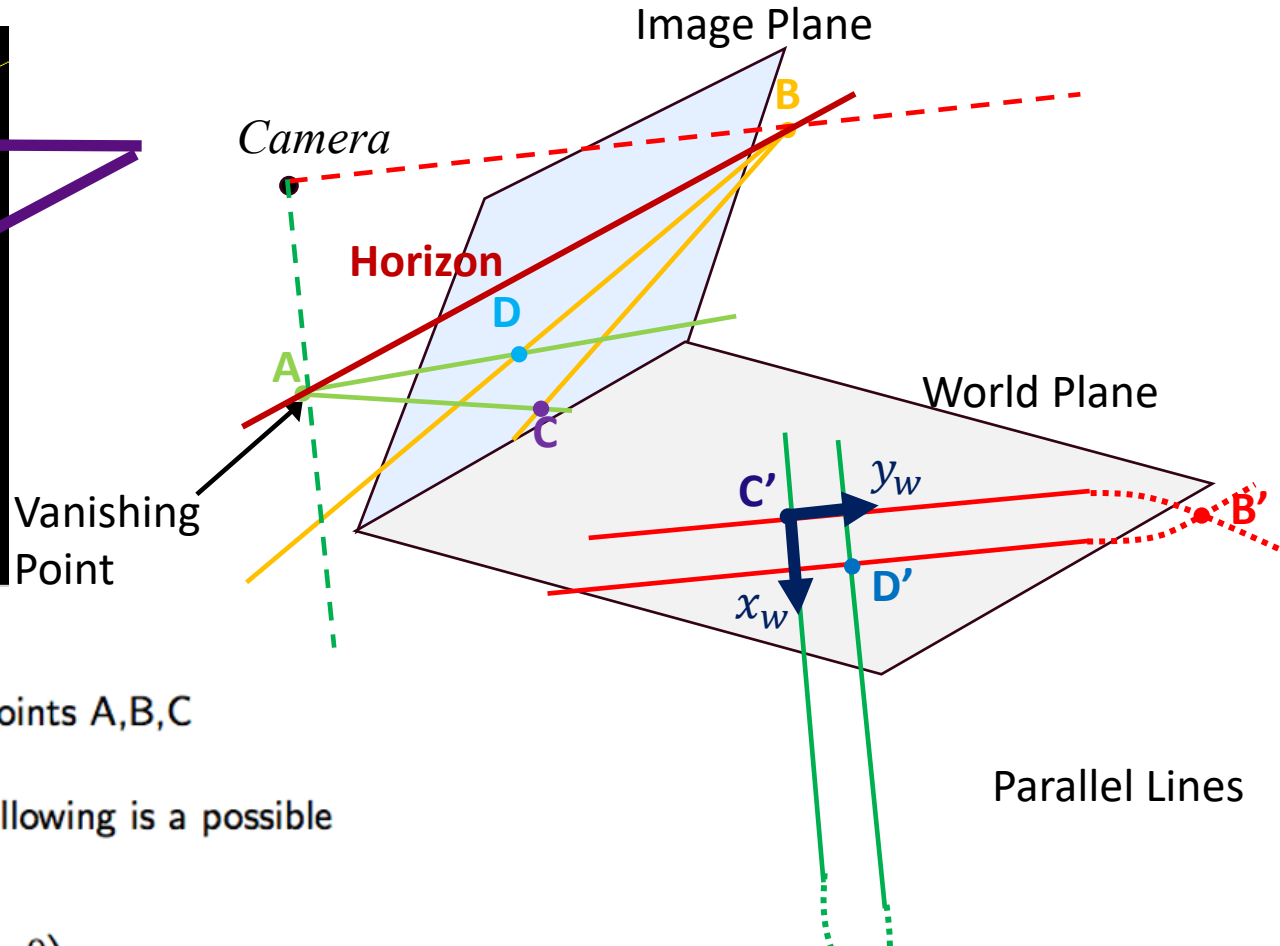
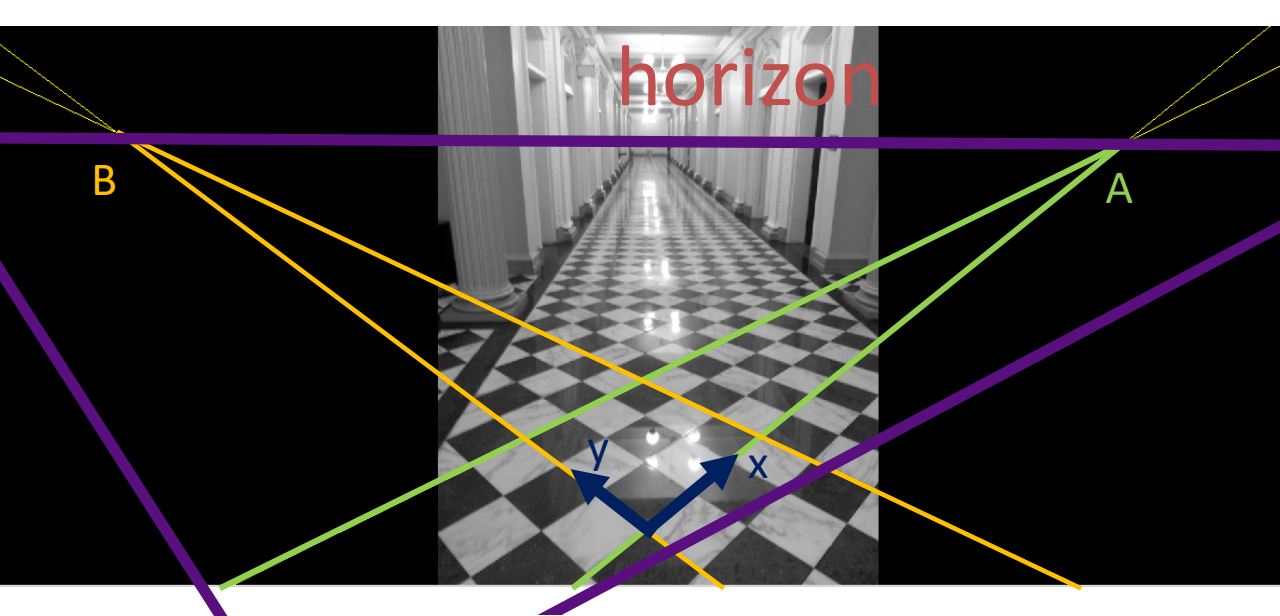


Correspondences:  
( $P^2 \rightarrow P^2$ )

|   |   |                       |    |           |
|---|---|-----------------------|----|-----------|
| A | a | $\longleftrightarrow$ | A' | (1, 0, 0) |
| B | b | $\longleftrightarrow$ | B' | (0, 1, 0) |
| C | c | $\longleftrightarrow$ | C' | (0, 0, 1) |
| D | d | $\longleftrightarrow$ | D' | (1, 1, 1) |



# Recap: Computing Homography from 4 Point Correspondences



Assume that a mapping  $A$  maps the three points  $(1, 0, 0)$ ,  $(0, 1, 0)$ , and  $(0, 0, 1)$  to the non-collinear points  $A, B, C$

with coordinate vectors  $a, b$  and  $c \in \mathbb{P}^2$ . Then the following is a possible projective transformation:

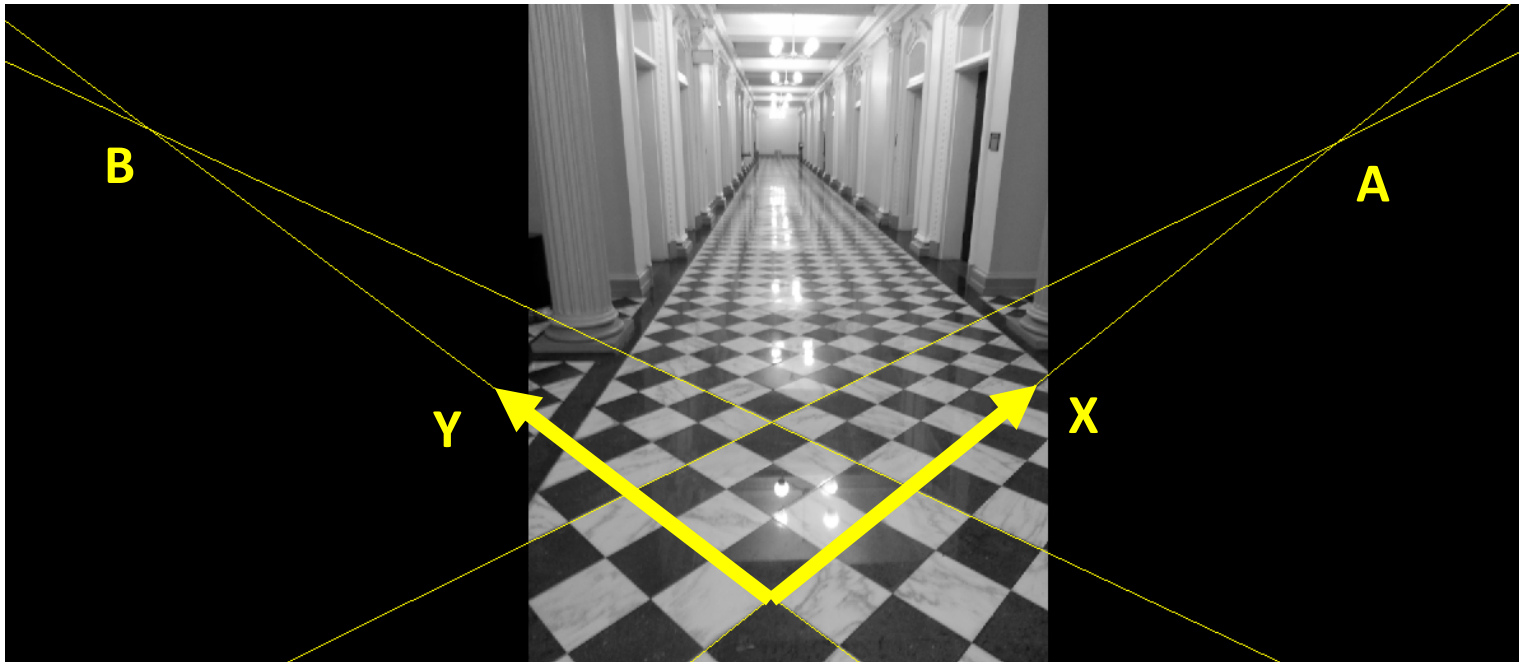
$$(a \quad b \quad c) = \underbrace{\begin{pmatrix} \alpha a & \beta b & \gamma c \end{pmatrix}}_{H_{3 \times 3}} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

If  $H = (h_1 \quad h_2 \quad h_3)$  then  $h_1 \sim a$  and  $h_2 \sim b$

The first two columns of Homography are two orthogonal vanishing points

# Recall: homography columns are vanishing points

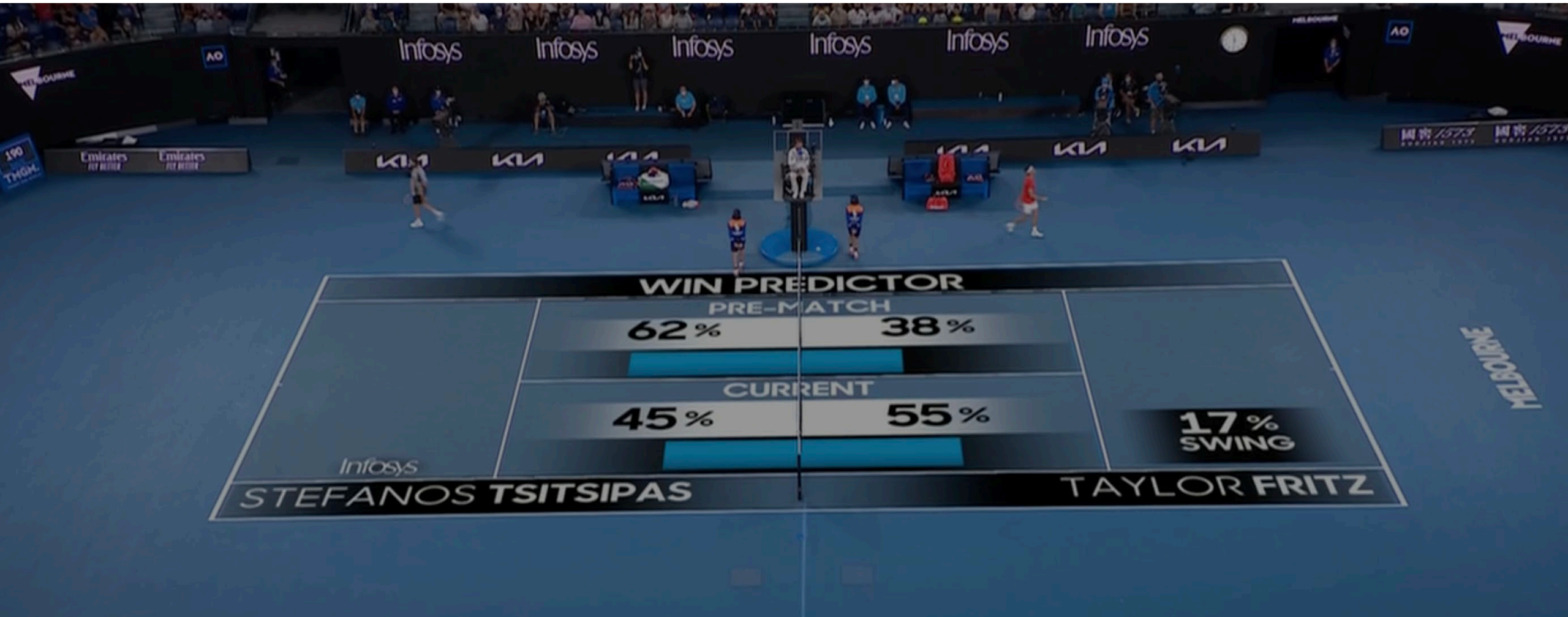
If  $H = (h_1 \ h_2 \ h_3)$  then  $h_1 \sim A$  and  $h_2 \sim B$ .



So the first two columns are two Orthogonal vanishing points



# Application Example: Virtual Billboards



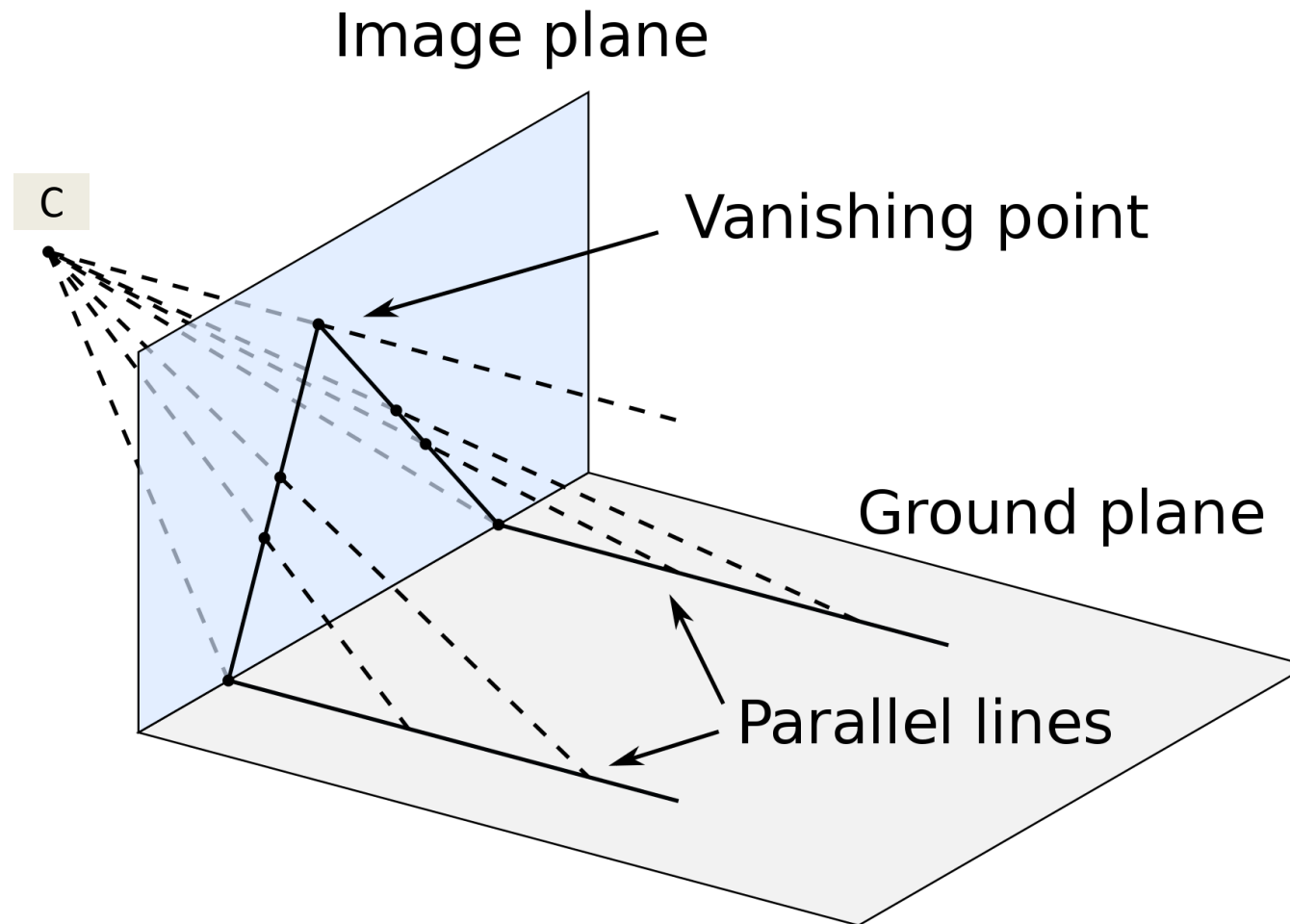
# Application Example: Using homographies to measure lengths!

- Homographies (i.e. World Plane  $\rightarrow$  Image homographies) allow us to measure lengths in the world plane of interest.

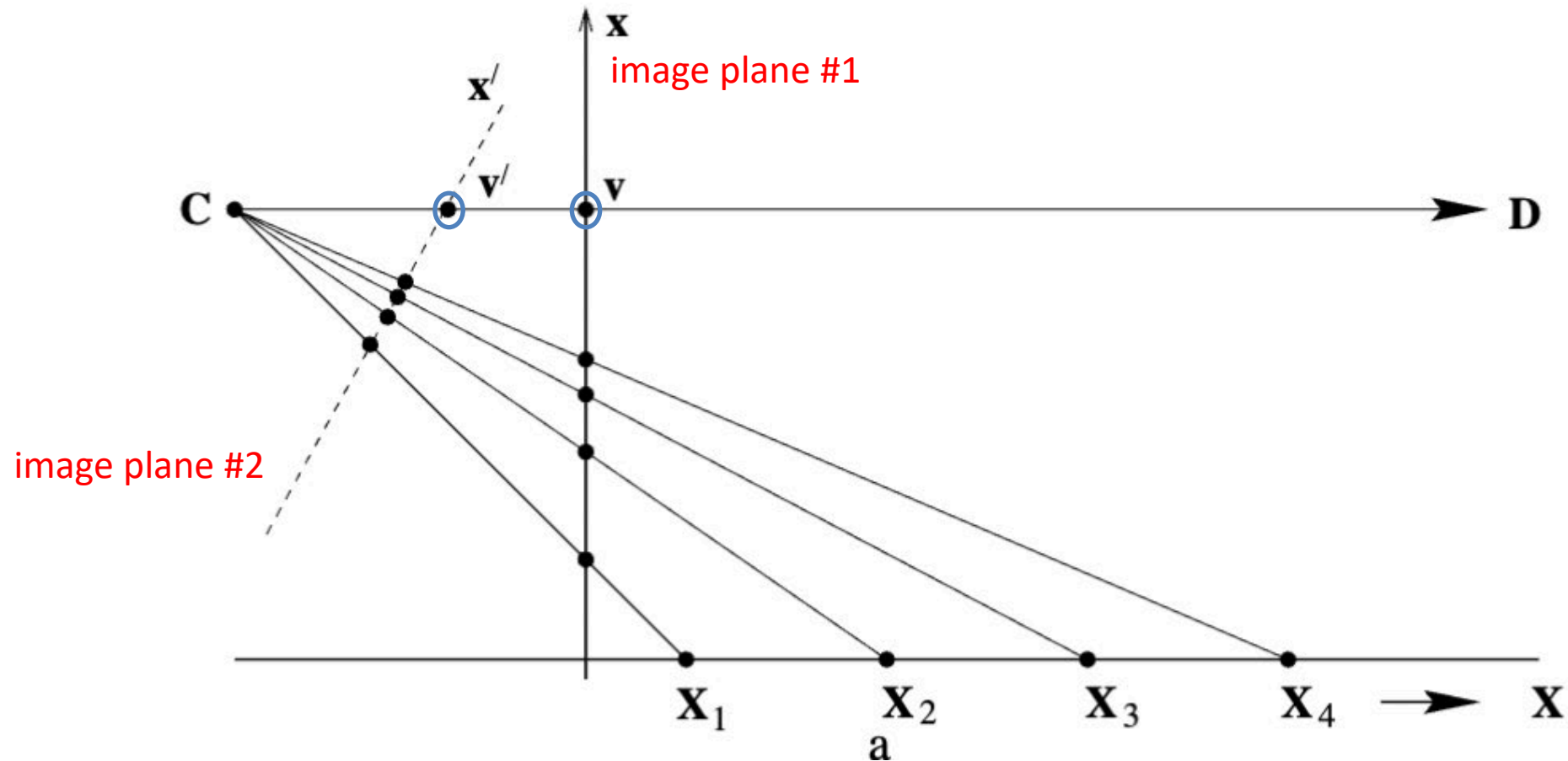


- We will discuss this in detail in the next class.

# Recall: Vanishing Point



The line connecting the camera origin and the vanishing point is parallel to all lines that share the same direction and converge at the vanishing point.



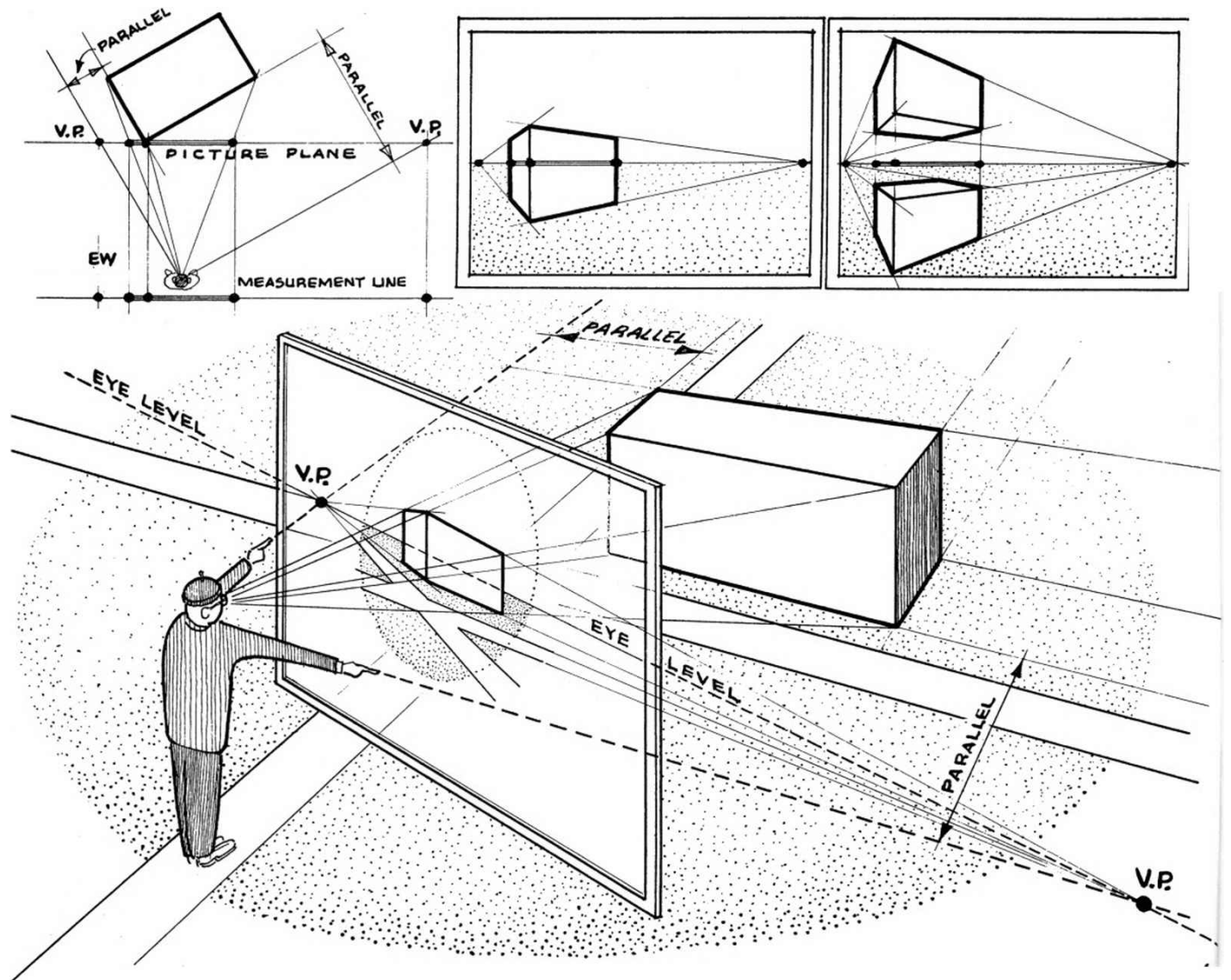


# How Artists Find Vanishing Points

Find VP of a world line by:

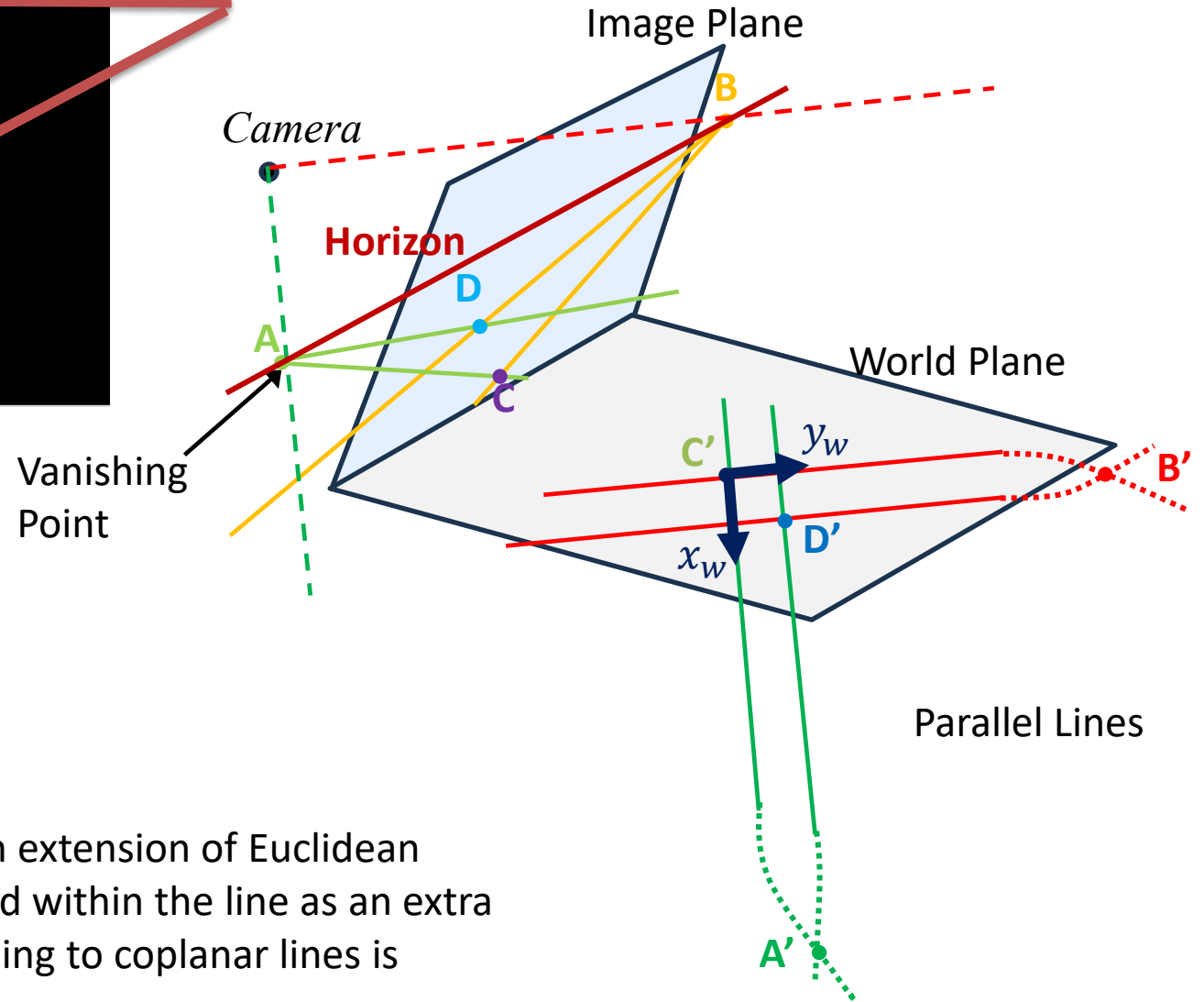
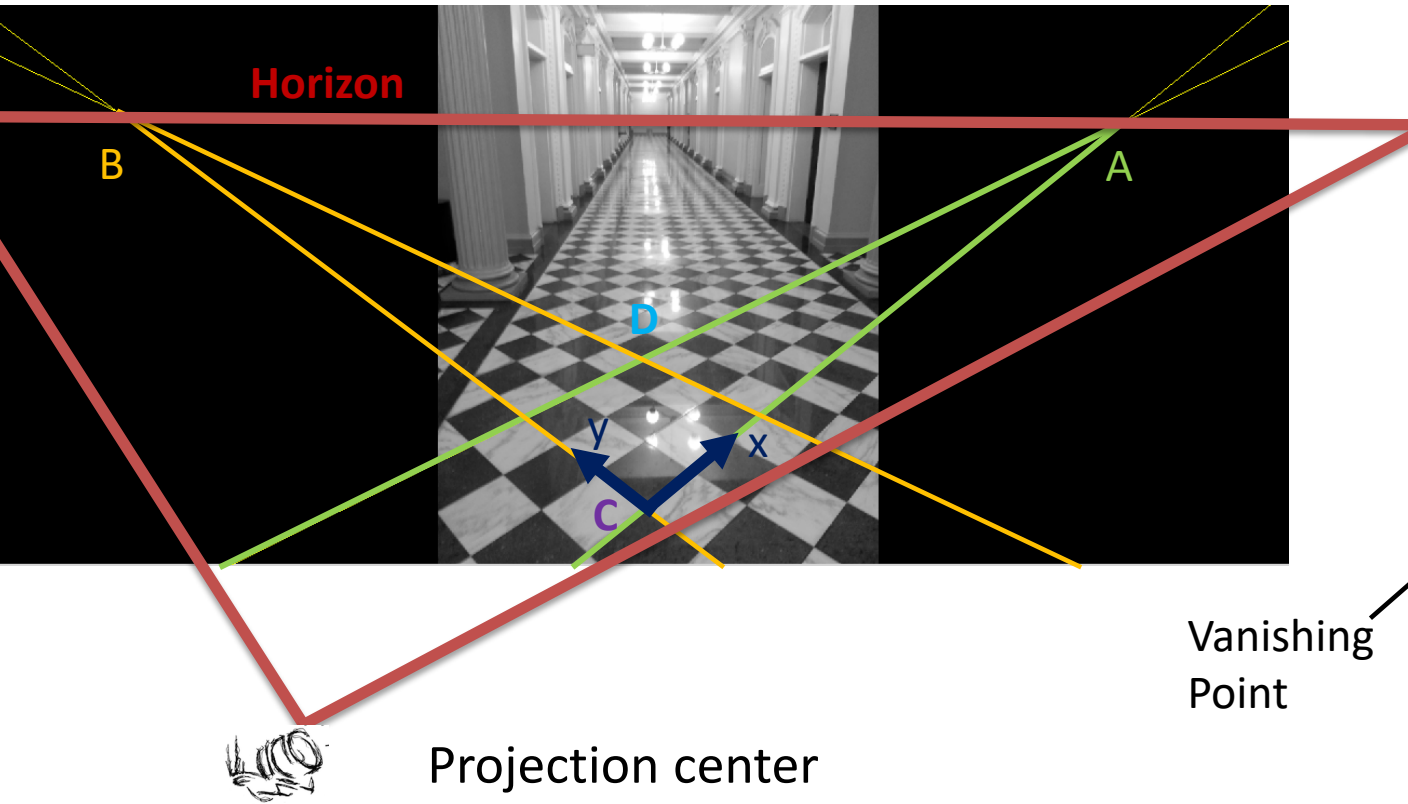
- Standing at “camera center”.
- Holding arm out parallel to the world line.
- Noting its intersection with the “canvas” or image plane. i.e. the arm represents the light ray.

“Vanishing rays of a world line” (camera rays through the VP) are just rays parallel to that line, passing through the camera center.



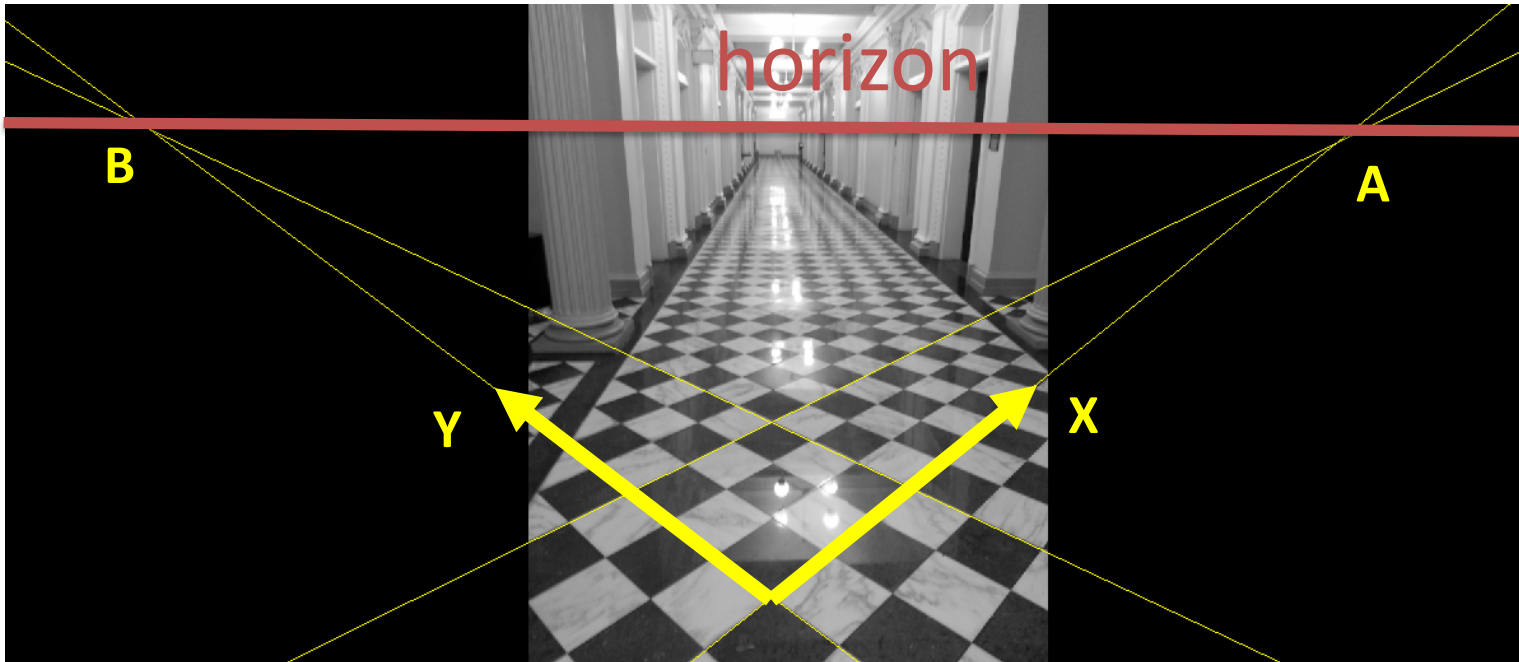


# Horizon



In essence, a projective geometry may be thought of as an extension of Euclidean geometry in which the "direction" of each line is subsumed within the line as an extra "point", and in which a "horizon" of directions corresponding to coplanar lines is regarded as a "line"

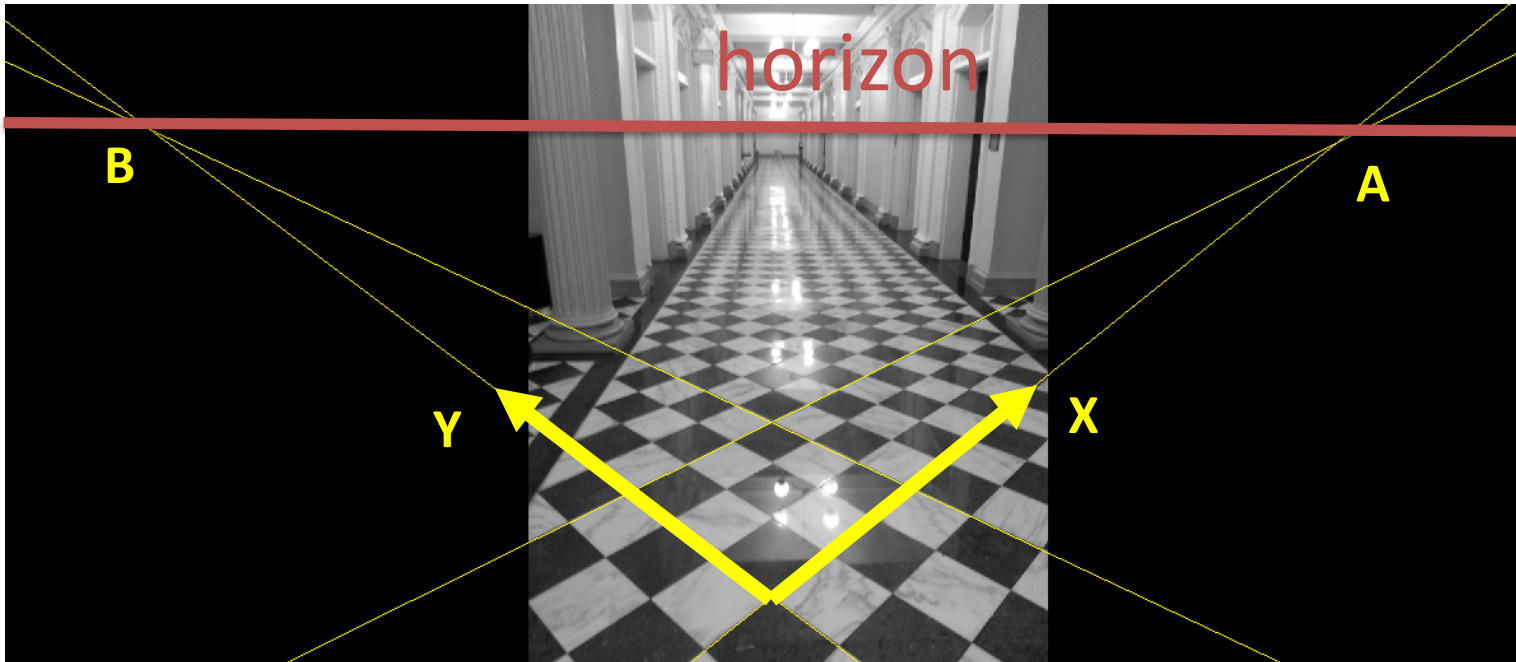
If we connect two vanishing points, we obtain the “horizon”!



Side note: with our assumption of the world plane as being the “XY” plane, and following the common convention that xy plane is horizontal, and z is vertical, this indeed maps to our normal notions of a “horizon”.



Equation of horizon:  $(h_1 \times h_2)^T \begin{pmatrix} x \\ y \\ z \end{pmatrix} = 0$



We will encounter another way to derive this equation of the horizon very soon.

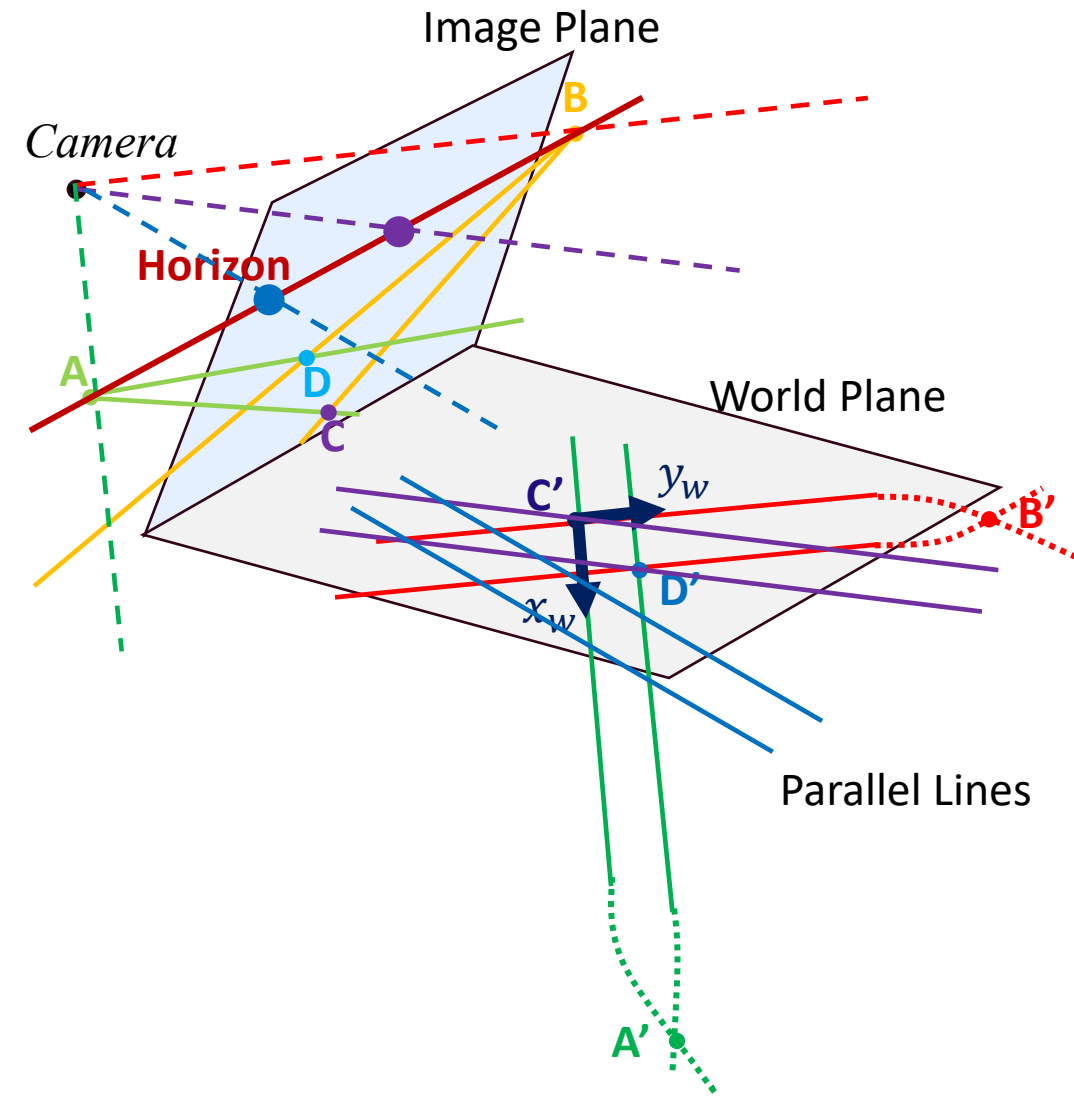


# Projecting the line at infinity to compute the horizon

Points at infinity in the world plane look like  $(X, Y, W = 0)^T$

The “line” connecting them is  $W = 0$ , the “line at infinity”. The image of this line is the horizon, which contains all vanishing points. Expressed in world plane  $\mathbb{P}^2$ , this line’s coefficients are  $(0,0,1)^T$ .

So if we could find the projection of this line, we could find the horizon



# Deriving the equation of a horizon in another way

Q: We know planar projections  $H$  transform points  $p \in \mathbb{P}^2$  as  $p \rightarrow Hp$ . How do they transform lines in  $\mathbb{P}^2$ ?

# Projective Transformation of Lines

If  $H$  maps a point to  $Hp$ , then where does a line  $l$  map to?

Line equation in original plane

$$l^T p = 0$$

Line equation in image plane where any point  $p' = Hp$

$$l^T H^{-1} p' = 0$$

Implies that  $l' = H^{-T} l$

# Projecting the line at infinity to compute the horizon

Points at infinity in the world plane look like  $(X, Y, W = 0)^T$

The “line” connecting them is  $W = 0$ , the “line at infinity”. The image of this line is the horizon, which contains all vanishing points. Expressed in world plane  $\mathbb{P}^2$ , this line’s coefficients are  $(0,0,1)^T$ .

So if we could project this line, we could find the horizon

We have just seen projections of lines are  $H^{-T}l$ , so the horizon is  $H^{-T}(0,0,1)^T$ .

If  $H = [h_1 \ h_2 \ h_3]$  then  $H^{-T}$  is  $[h_2 \times h_3 \ h_3 \times h_1 \ h_1 \times h_2]$ , so the horizon line  $H^{-T}(0,0,1)^T = h_1 \times h_2$ .

This is consistent: the horizon connects the two vanishing points  $h_1$  and  $h_2$ .

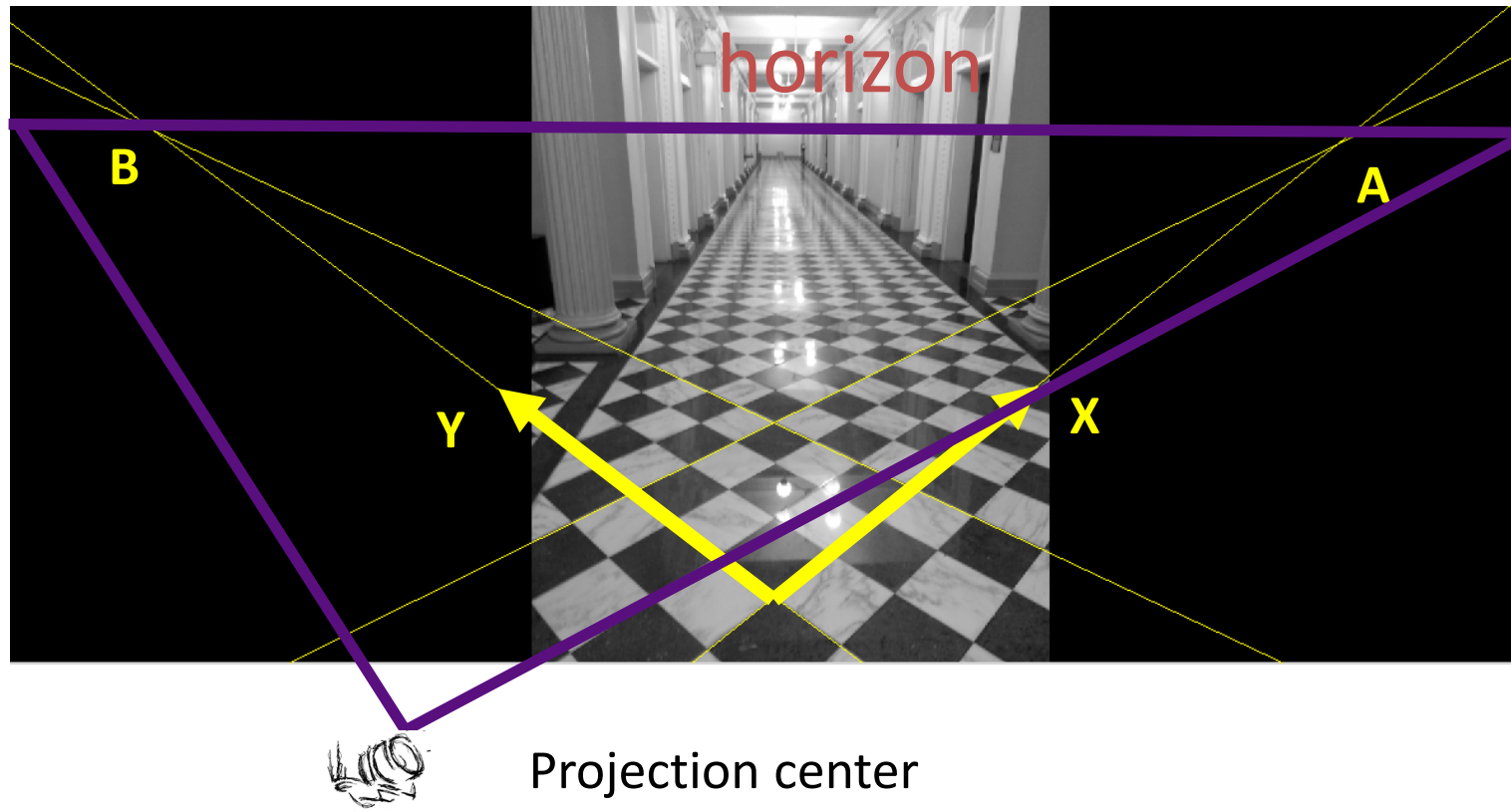
## Summary

Equation of horizon is  $h_1 \times h_2$

=

Normal to the viewing plane through horizon is  $h_1 \times h_2$

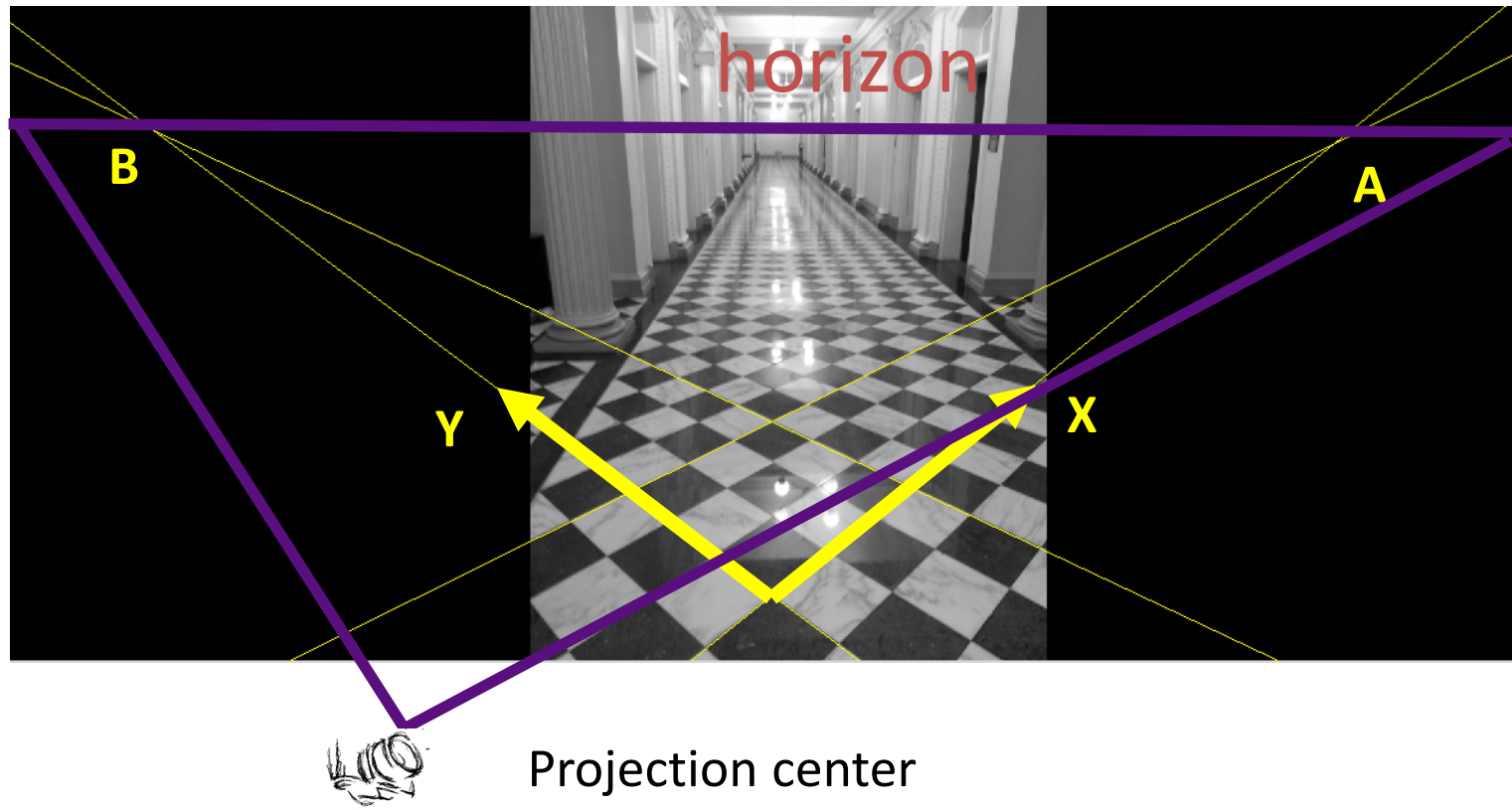
Plane through camera center and horizon line on image plane = vanishing plane of the world plane



# Summary

Vanishing rays/planes through the camera center are parallel to the world lines/planes

So, the horizon plane is parallel to the ground plane  
and hence  $h_1 \times h_2$  is the normal to the ground plane!



Horizon plane =  
Vanishing plane =  
Viewing plane

World plane //  
vanishing plane

World plane = Ground  
plane in this case

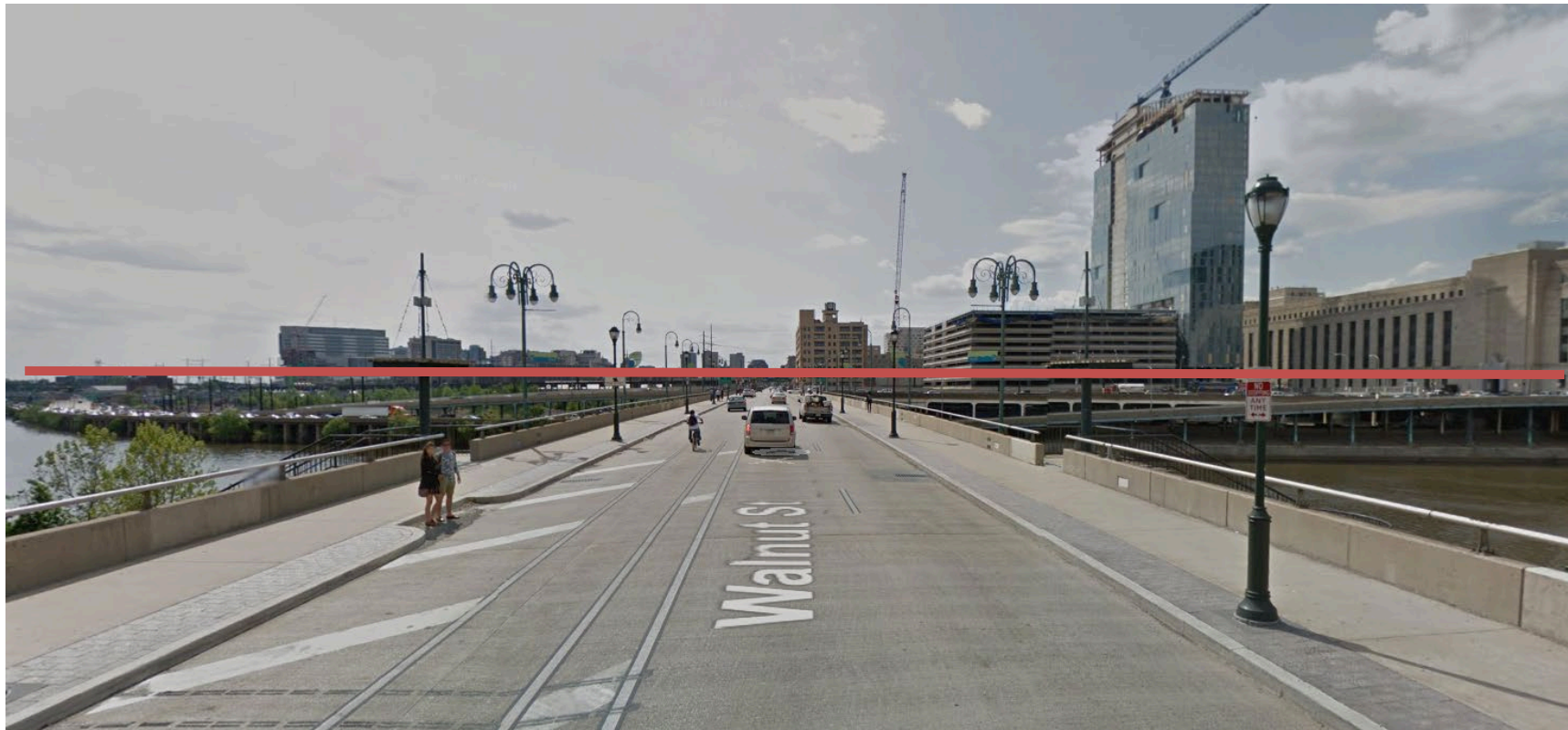




Using the horizon to orient the  
camera

Horizon gives complete info about how ground plane is oriented\*!

Thumb rule: “If horizon is horizontal & central, camera is correctly vertical & principal axis is parallel to world plane\*\*!”

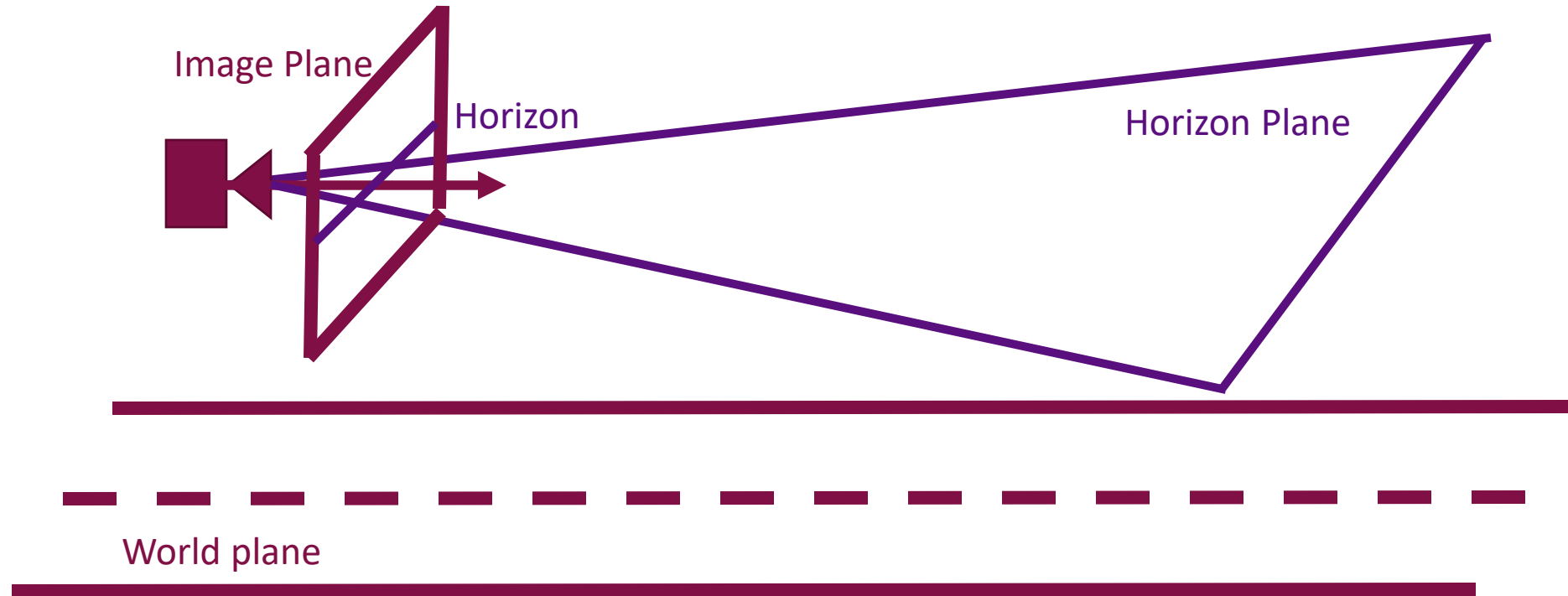


\*caveat: assuming known  $K$

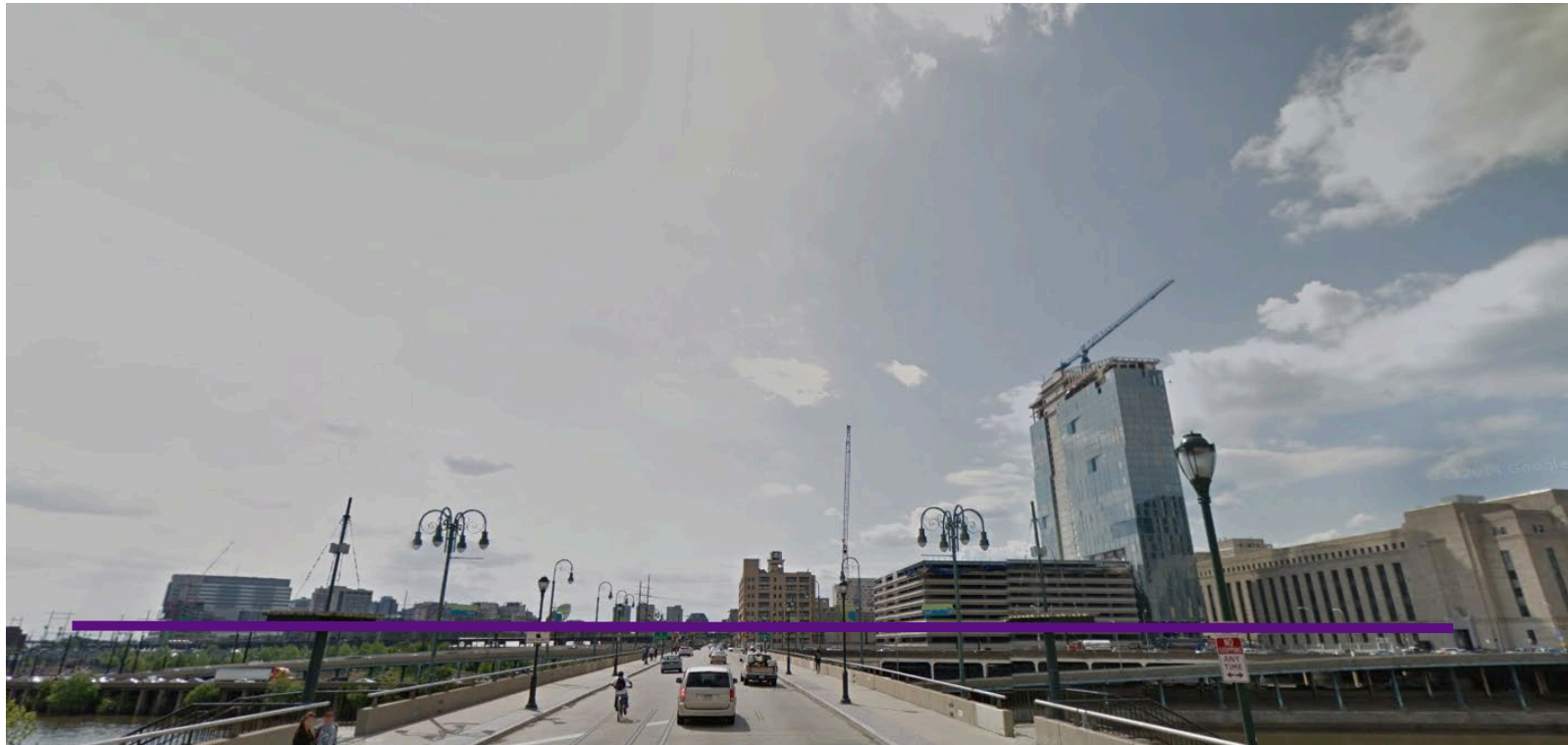
\*\*caveat: assuming that principal axis passes through image center, and sensor axes are horizontal. (usually approximately true)

Horizon gives complete info about how ground plane is oriented\*!

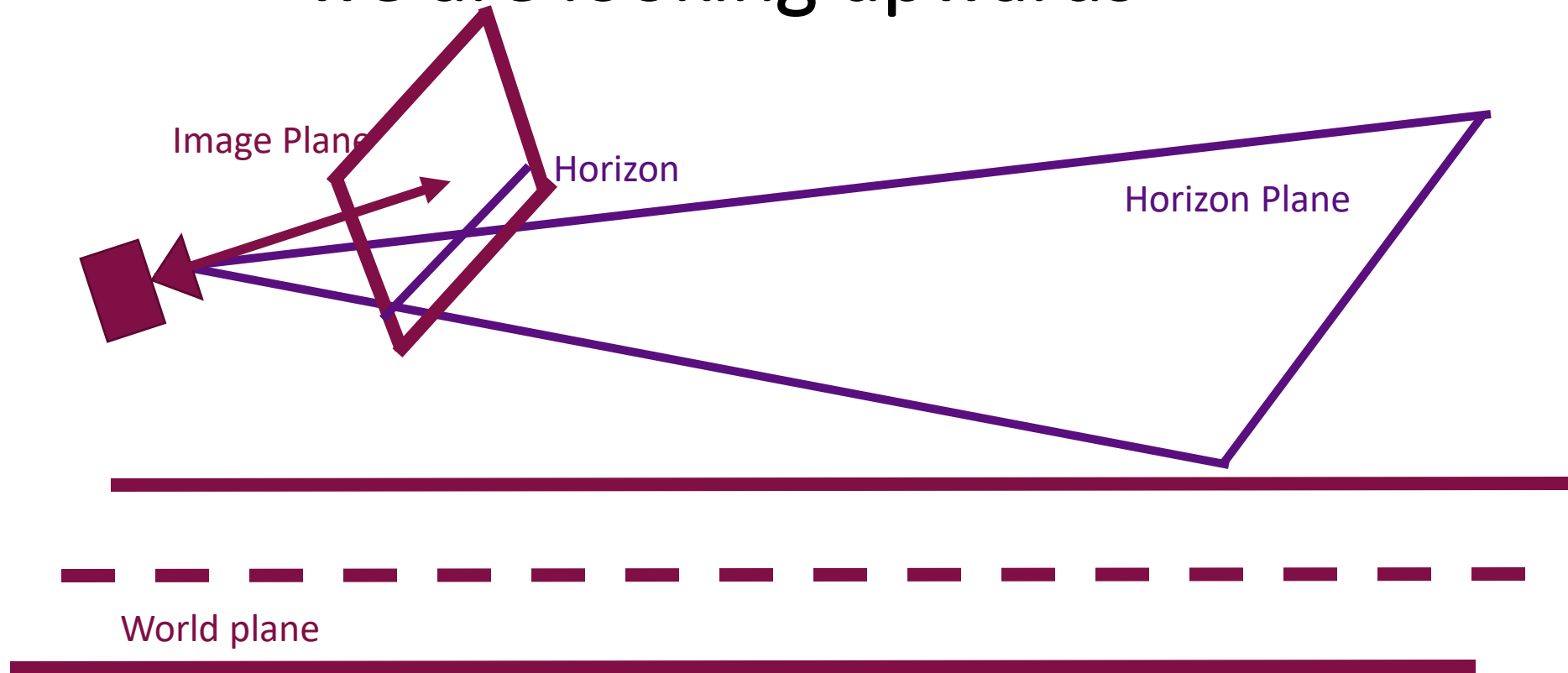
Thumb rule: “If horizon is horizontal & central, camera is correctly vertical & principal axis is parallel to world plane\*\*!”



Horizon below middle of image =>  
we are looking upwards



Horizon below middle of image =>  
we are looking upwards

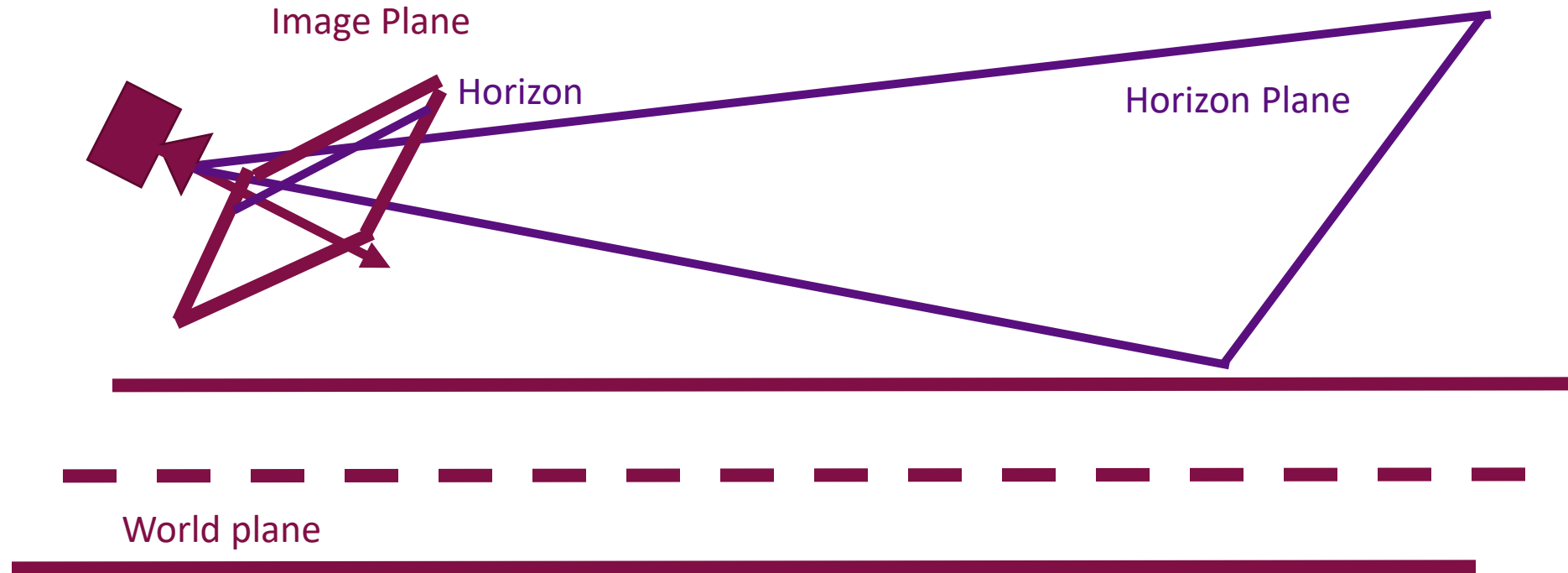




Horizon above middle of image =>  
we are looking downwards

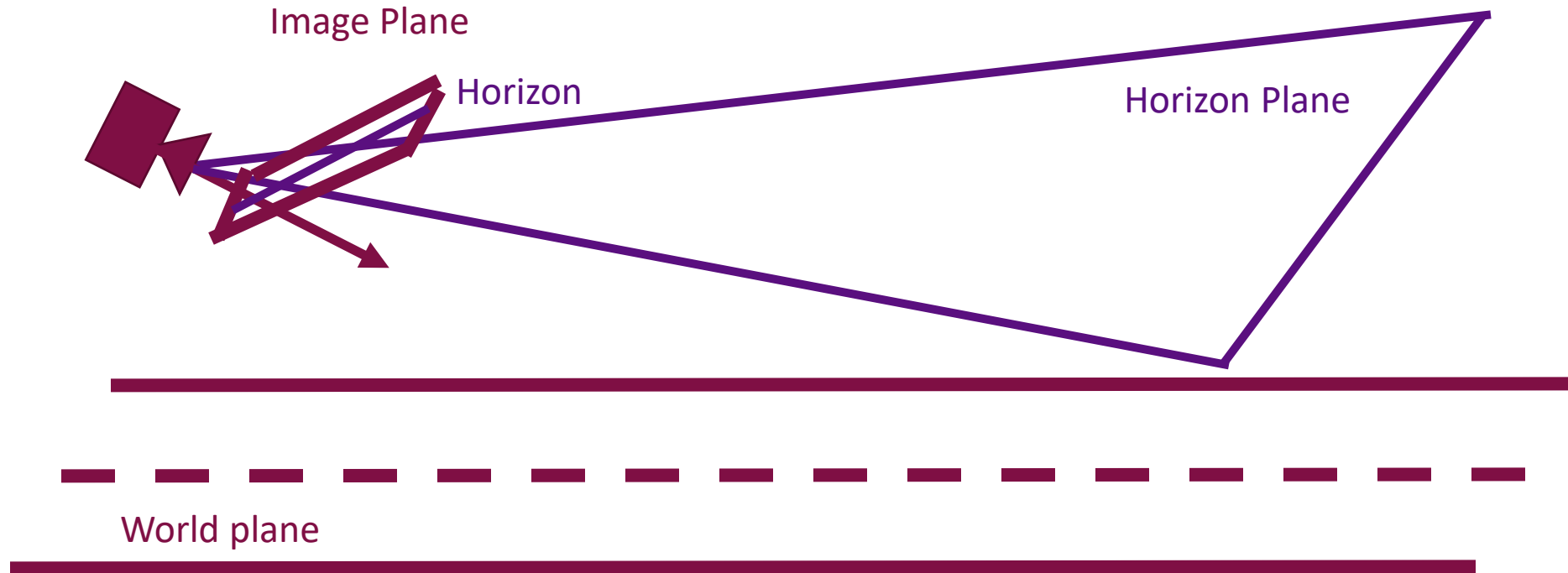


Horizon above middle of image =>  
we are looking downwards





Horizon above middle of image =>  
we are looking downwards

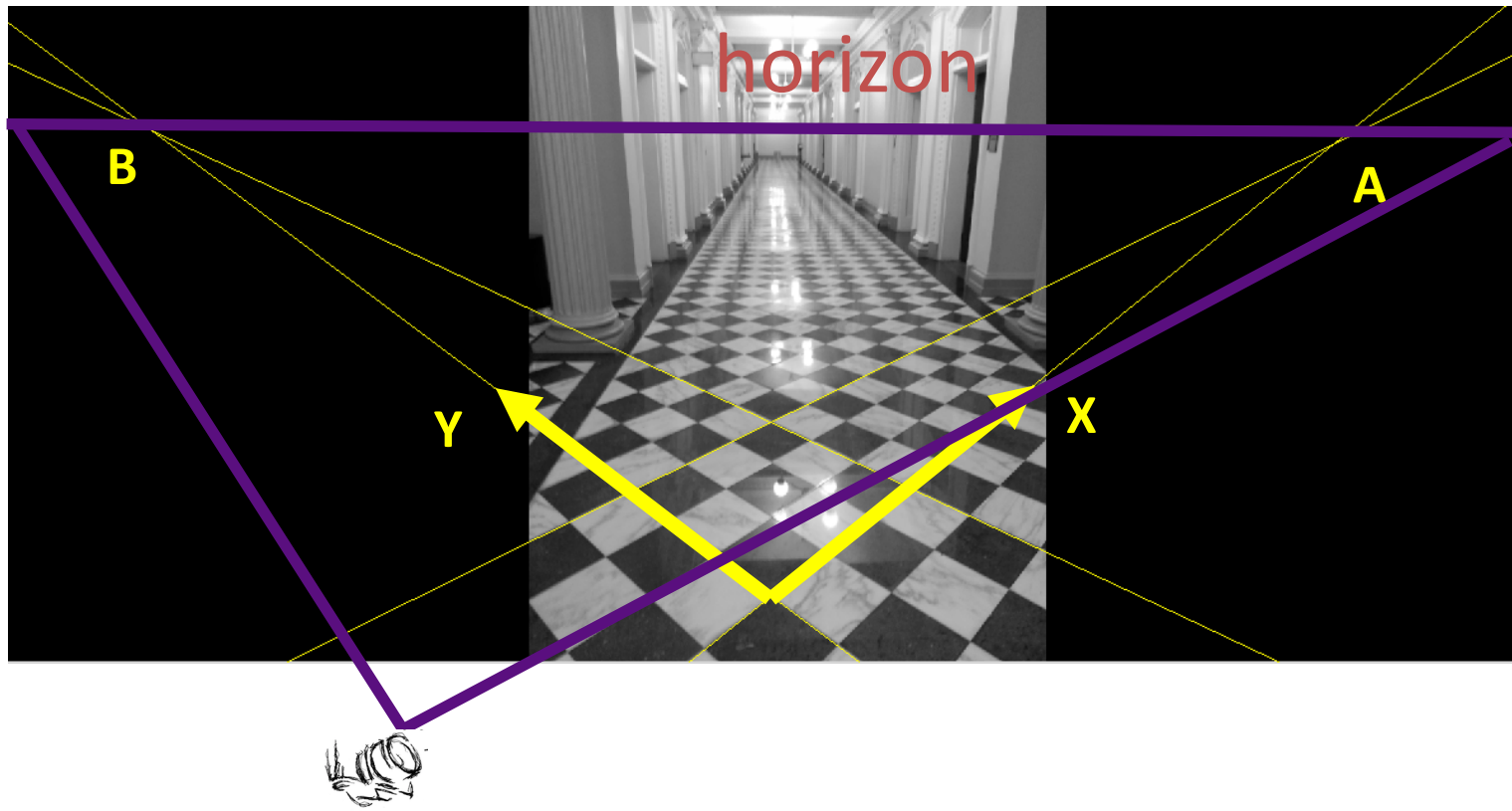


Q: What if you cropped the image? Would these rules still hold?

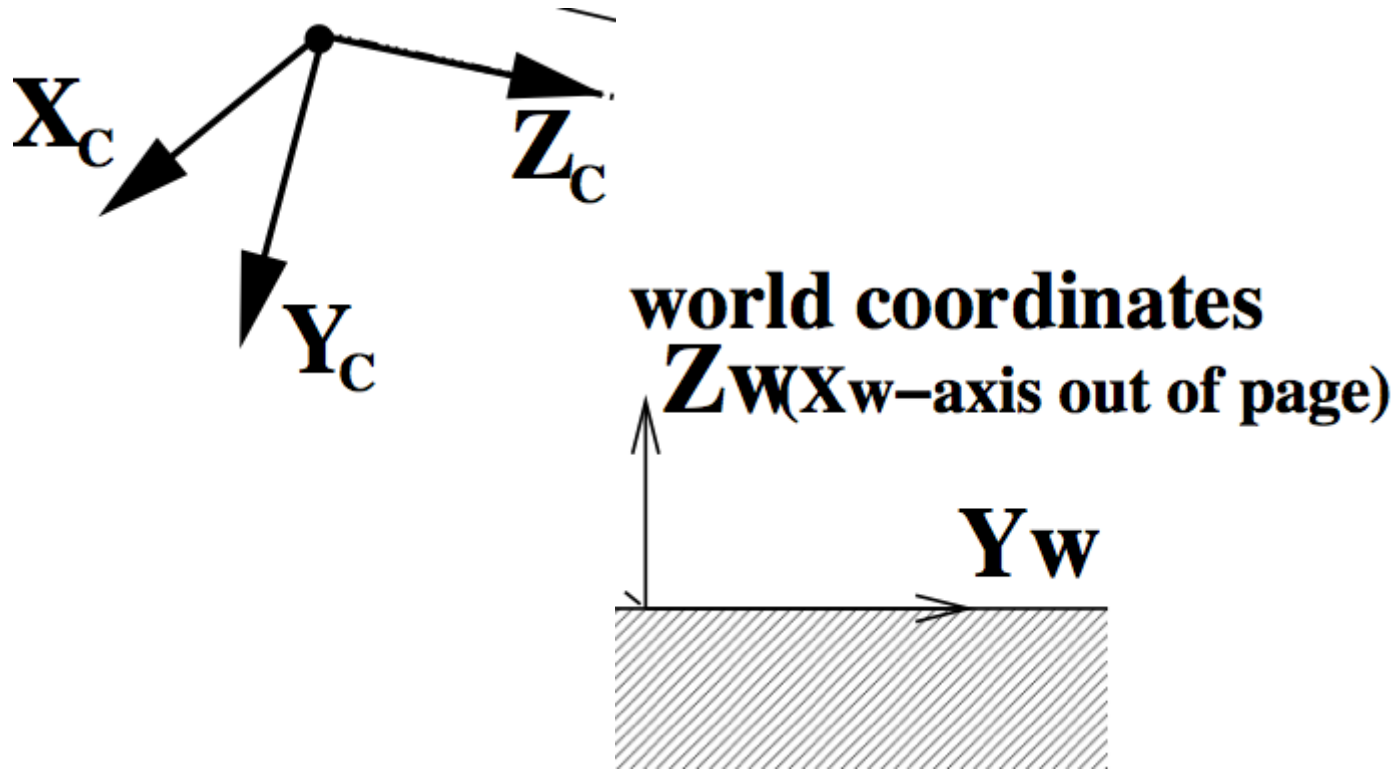
A: No. The effective intrinsics  $K$  would change and become “non-standard”, so these thumb rules wouldn’t hold.

# Summing up

Horizon tells us how camera is oriented.  
Constrains homography.



Recap: Assume world plane  $Z_w = 0$



# Recap: Pose From Homography

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane  $Z = 0$ .

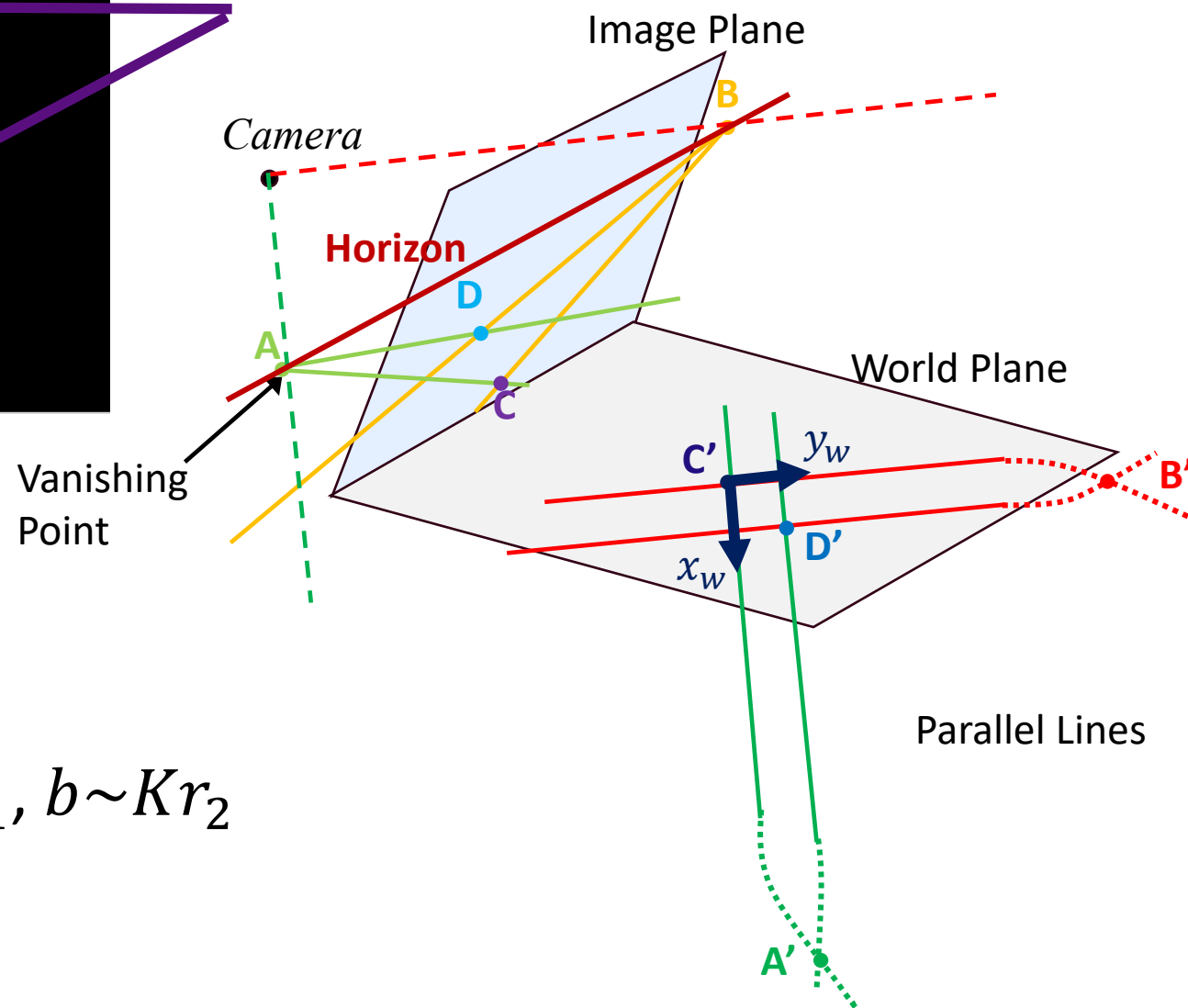
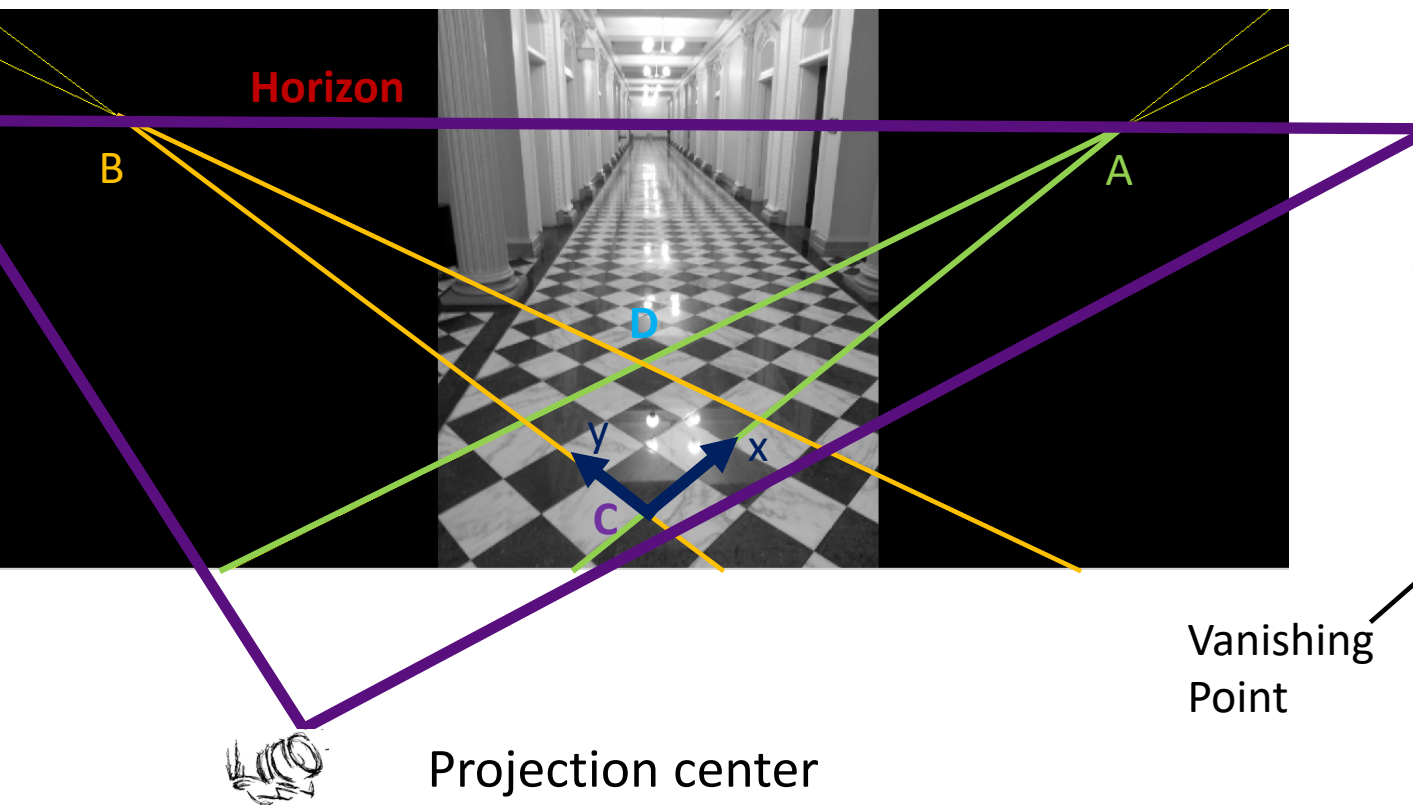
Then the transformation reads

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim \underbrace{K \begin{pmatrix} r_1 & r_2 & T \end{pmatrix}}_{\text{Homography}} \begin{pmatrix} X \\ Y \\ W \end{pmatrix}$$

The planar homography  
 $H: \mathbb{P}^2 \rightarrow \mathbb{P}^2$

Computing the homography can tell us how the camera (and therefore, e.g. a robot attached to the camera) is oriented w.r.t. to a world plane! (assuming known K)

Q: Where do you get  $r_3$  from though? **A:  $r_3 = r_1 \times r_2$**



$$H \sim K(r_1, r_2, T)$$

$$H \sim (a, b, c)$$

$$a \sim Kr_1, b \sim Kr_2$$

- We can get the rotation of camera from 2 orthogonal vanishing points on a plane, **assuming known intrinsics matrix  $K$** .
- Turns out, you can also get *intrinsics* from vanishing points.

$$\begin{array}{l} H \sim K(r_1, r_2, T) \\ H \sim (a, b, c) \end{array} \begin{array}{l} \nearrow \\ \nearrow \end{array} a \sim Kr_1, b \sim Kr_2 \longrightarrow r_1 \sim K^{-1}a, r_2 \sim K^{-1}b$$



CIS 5800

# Machine Perception

Instructor: Lingjie Liu

Lec 6: Feb 10, 2026



## Recap:

# Application Example: Using homographies to measure lengths!

- Homographies (i.e. World Plane  $\rightarrow$  Image homographies) allow us to measure lengths in the world plane of interest.



- We will discuss this in detail in today's class.

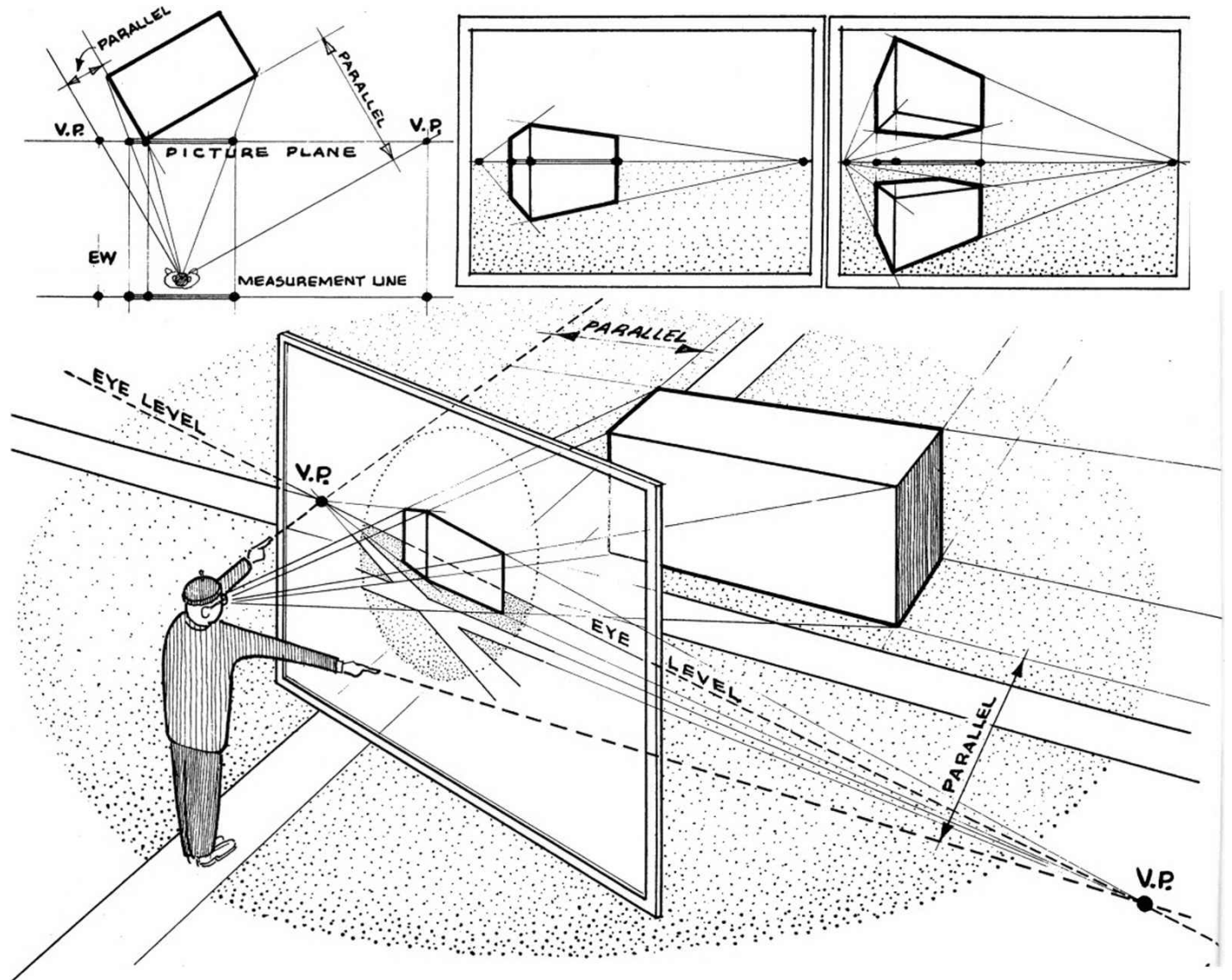
## Recap:

# How Artists Find Vanishing Points

Find VP of a world line by:

- Standing at “camera center”.
- Holding arm out parallel to the world line.
- Noting its intersection with the “canvas” or image plane. i.e. the arm represents the light ray.

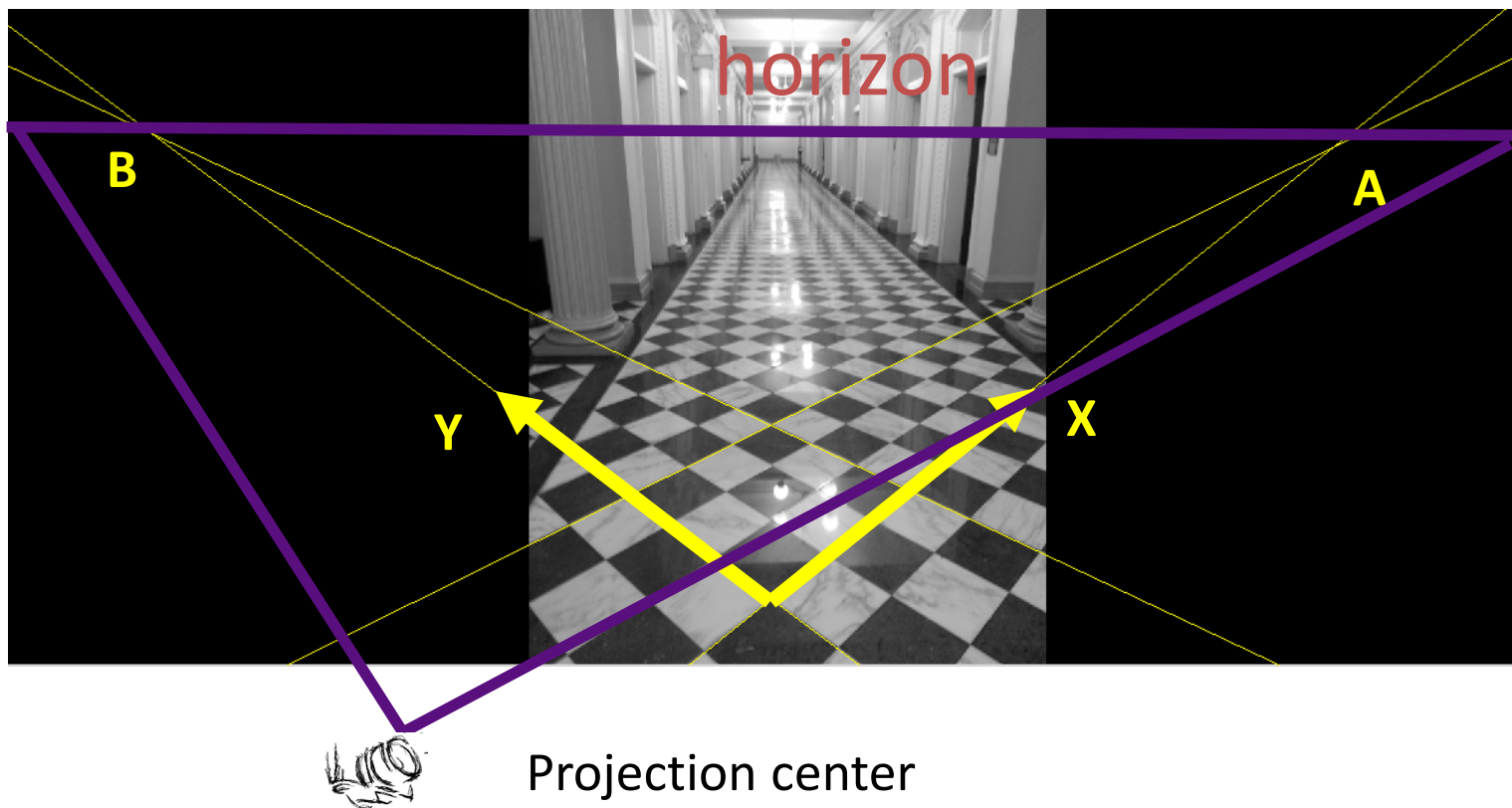
“Vanishing rays of a world line” (camera rays through the VP) are just rays parallel to that line, passing through the camera center.



## Recap:

Vanishing rays/planes through the camera center are parallel to the world lines/planes

So, the horizon plane is parallel to the ground plane  
and hence  $h_1 \times h_2$  is the normal to the ground plane!



Horizon plane =  
Vanishing plane =  
Viewing plane

World plane //  
vanishing plane

World plane = Ground  
plane in this case



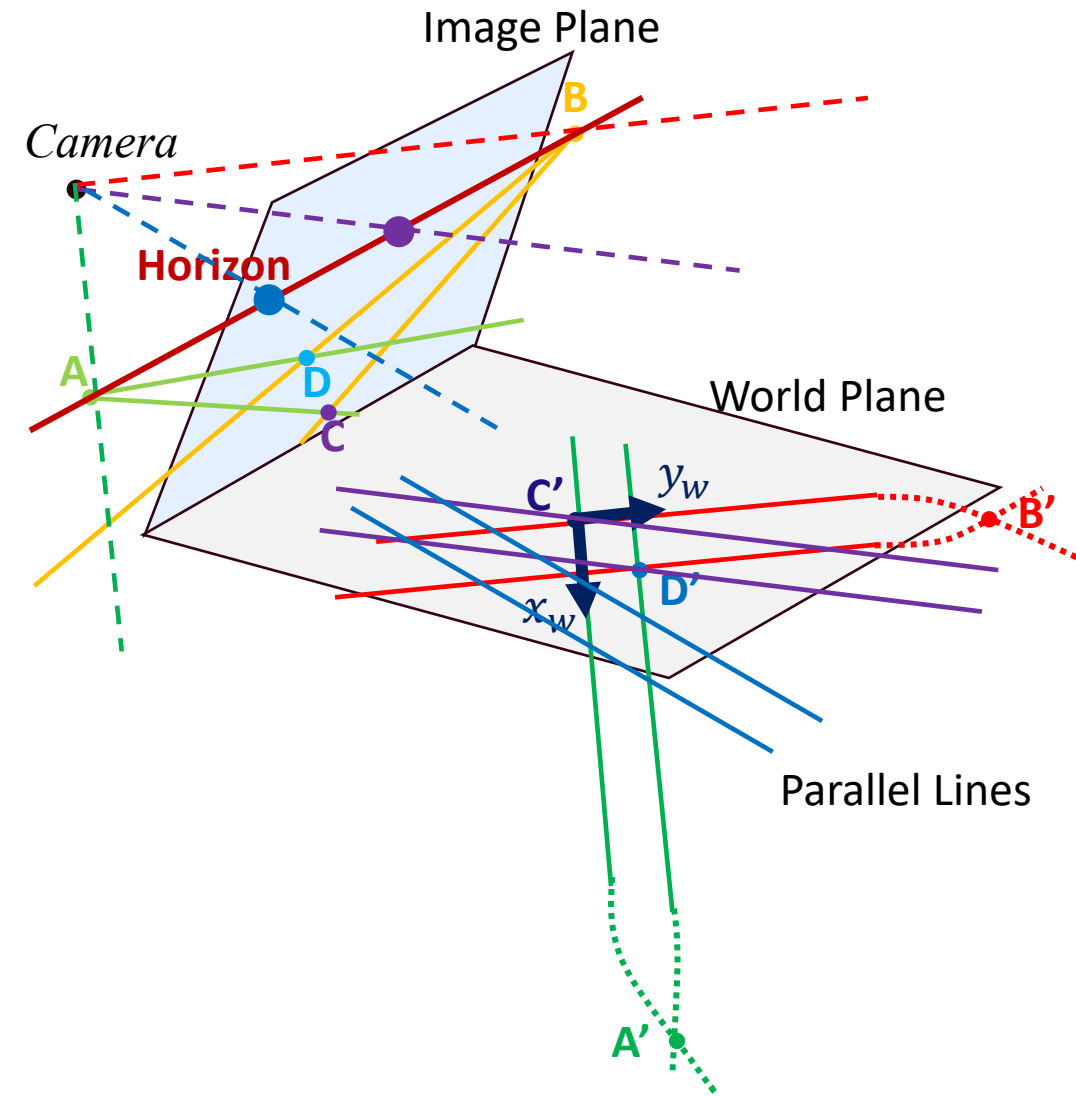
Recap:

# Projecting the line at infinity to compute the horizon

Points at infinity in the world plane look like  $(X, Y, W = 0)^T$

The “line” connecting them is  $W = 0$ , the “line at infinity”. The image of this line is the horizon, which contains all vanishing points. Expressed in world plane  $\mathbb{P}^2$ , this line’s coefficients are  $(0,0,1)^T$ .

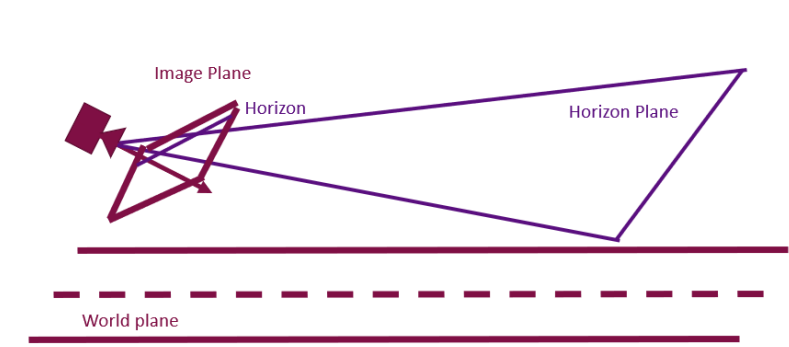
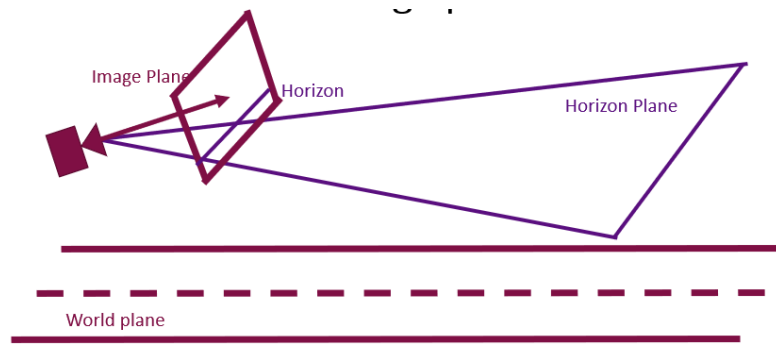
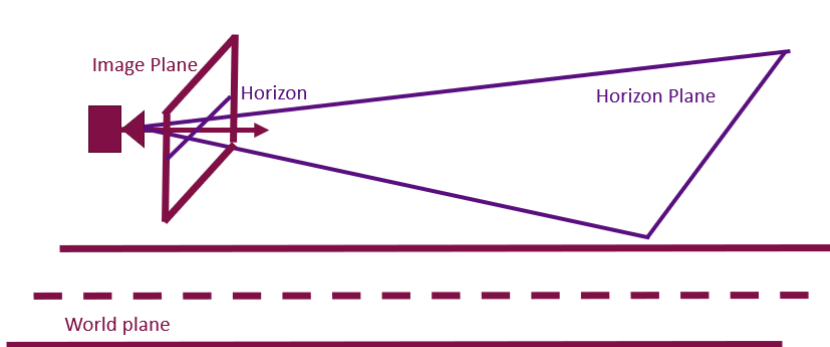
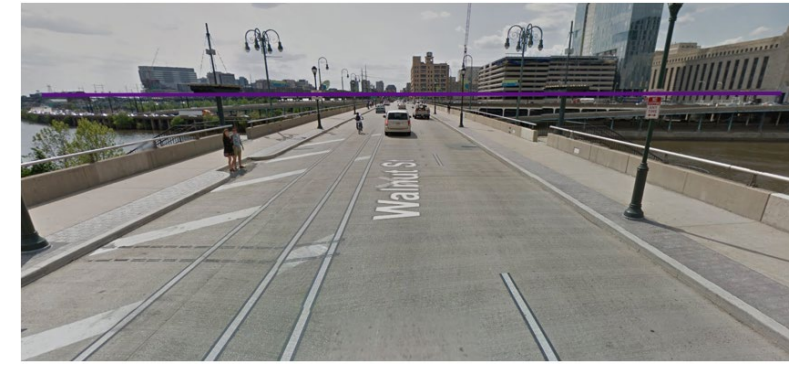
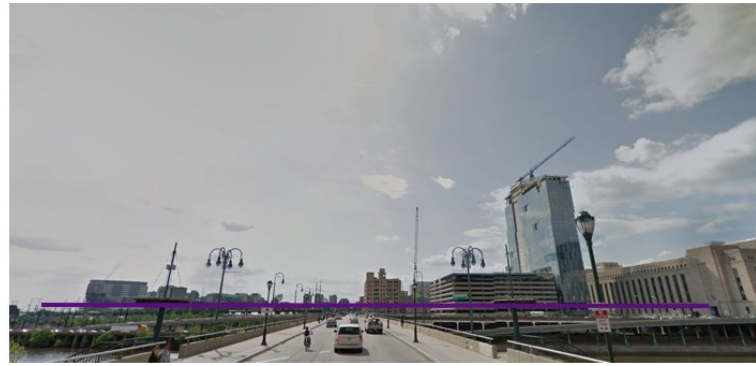
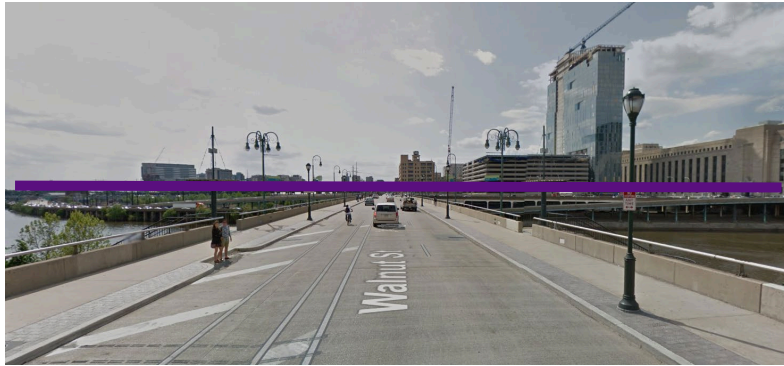
So if we could find the projection of this line, we could find the horizon



# Recap:

Horizon gives complete info about how camera is oriented w.r.t. world plane\*!

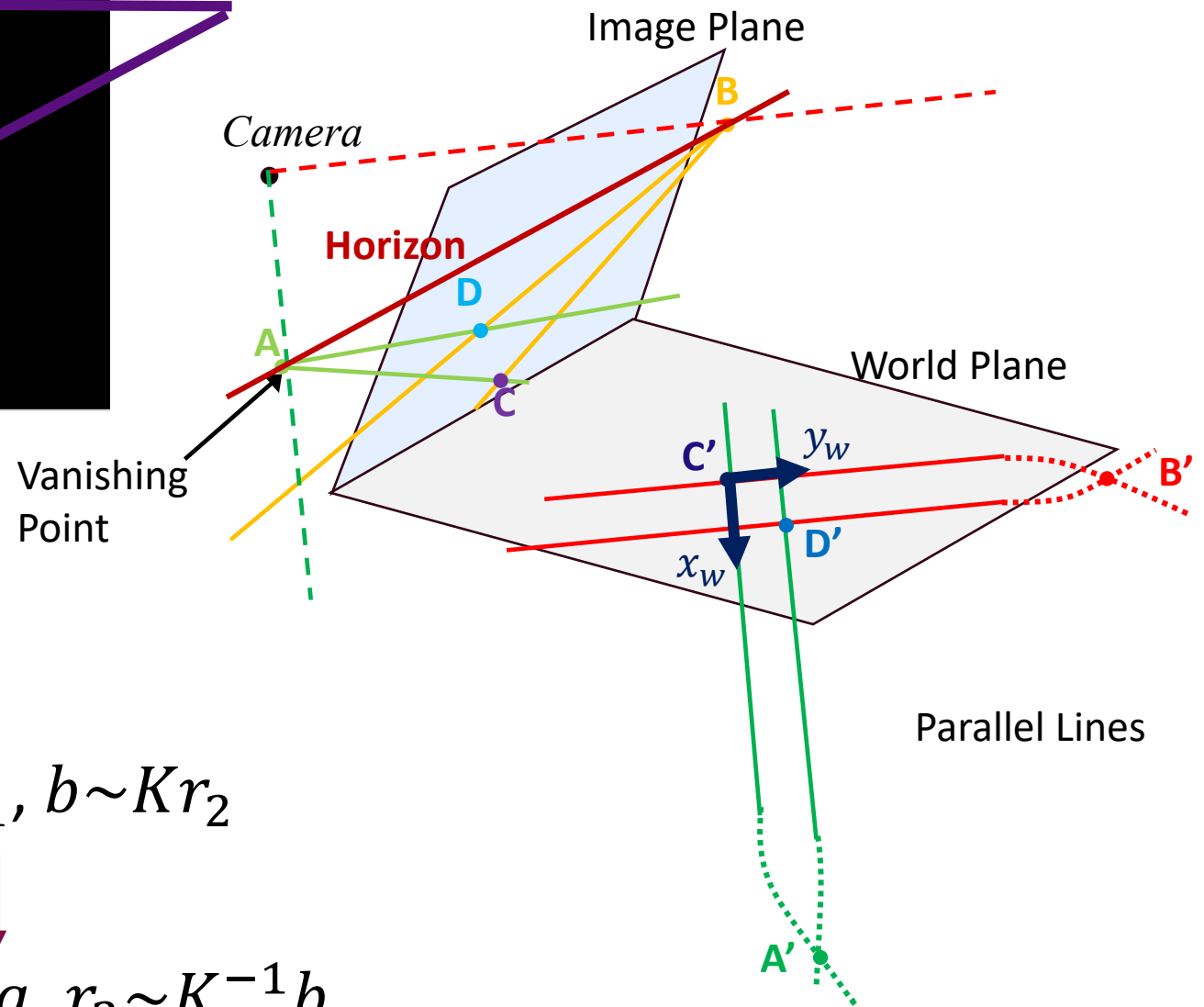
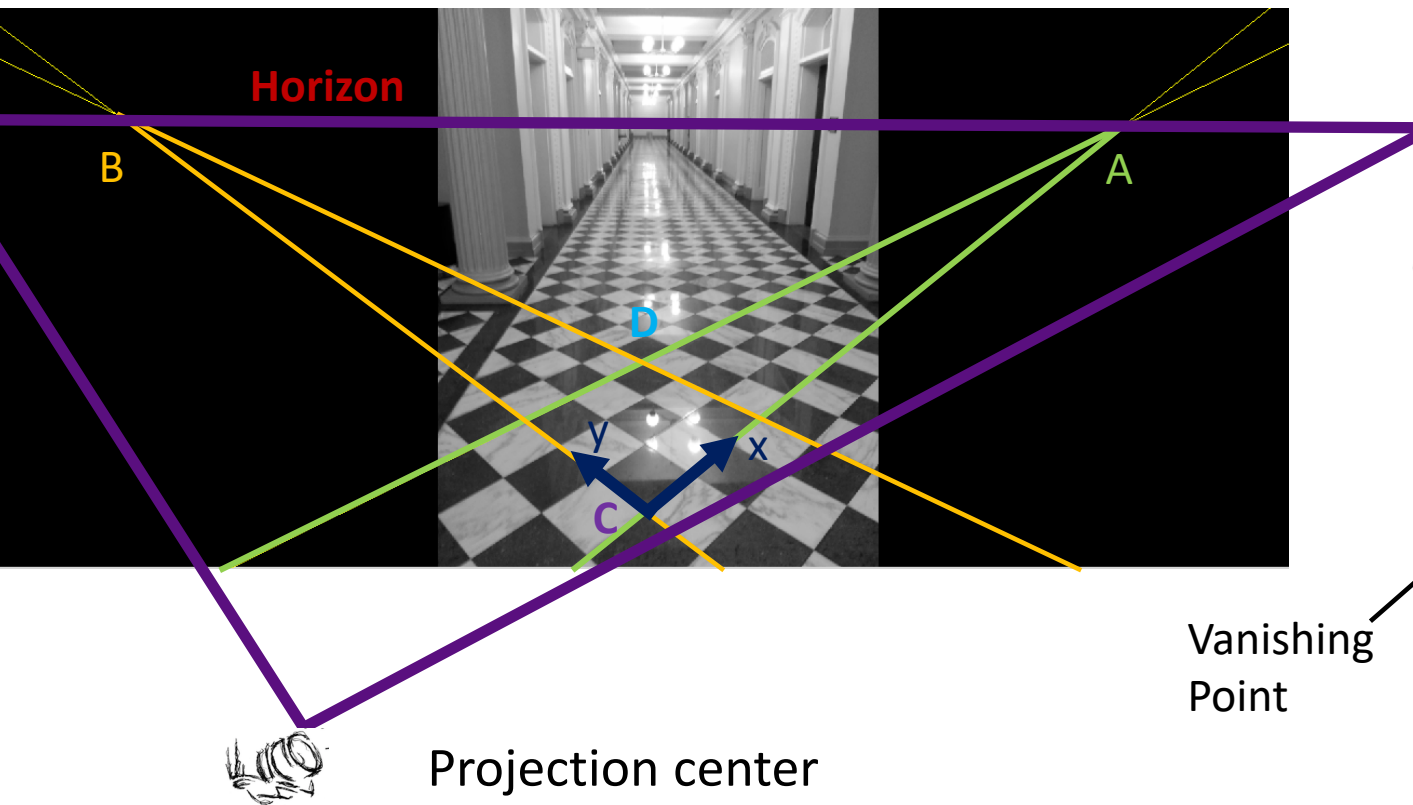
Thumb rule: “If horizon is horizontal & central, camera is correctly vertical & principal axis is parallel to world plane\*\*!”



\*caveat: assuming known  $K$

\*\*caveat: assuming that principal axis passes through image center, and camera axes are horizontal. (usually approximately true)

# Recap:



$$H \sim K(r_1, r_2, T)$$

$$H \sim (a, b, c)$$

$$a \sim Kr_1, b \sim Kr_2$$

$$r_1 \sim K^{-1}a, r_2 \sim K^{-1}b$$

## Recap:

- We can get the rotation of camera from 2 orthogonal vanishing points on a plane, **assuming known intrinsics matrix  $K$** .
- Turns out, you can also get *intrinsics* from vanishing points.

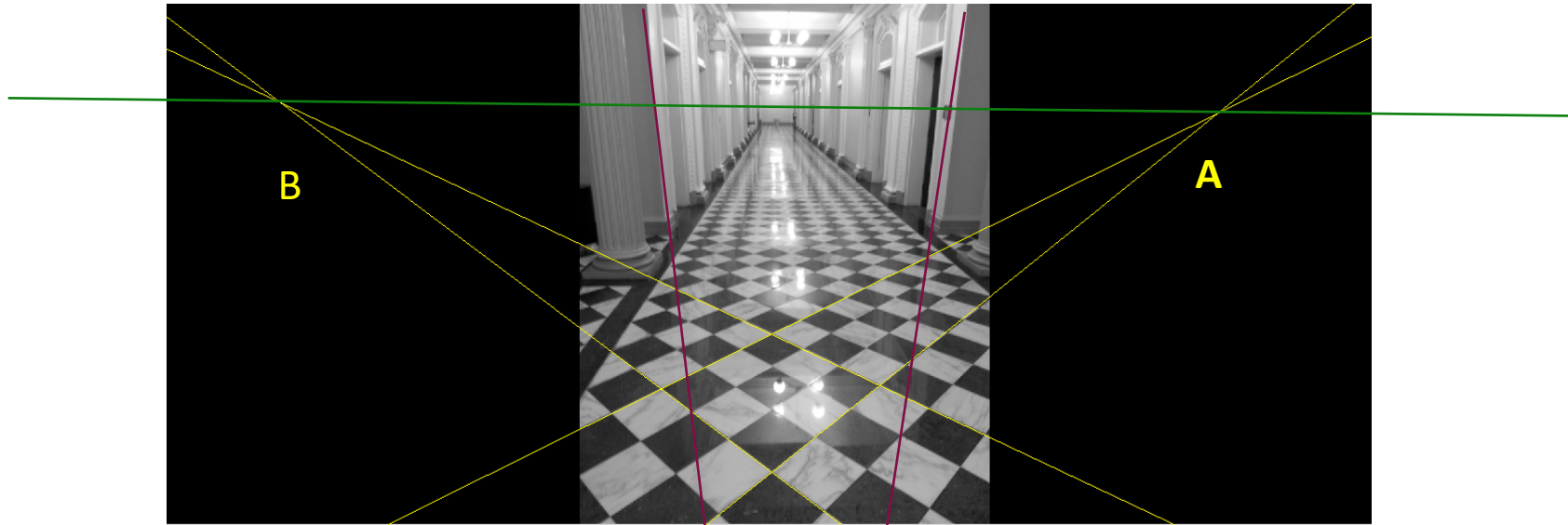
$$\begin{array}{l} H \sim K(r_1, r_2, T) \\ H \sim (a, b, c) \end{array} \begin{array}{l} \nearrow \\ \nearrow \end{array} a \sim Kr_1, b \sim Kr_2 \longrightarrow r_1 \sim K^{-1}a, r_2 \sim K^{-1}b$$

How to compute intrinsics  $K$  from vanishing points





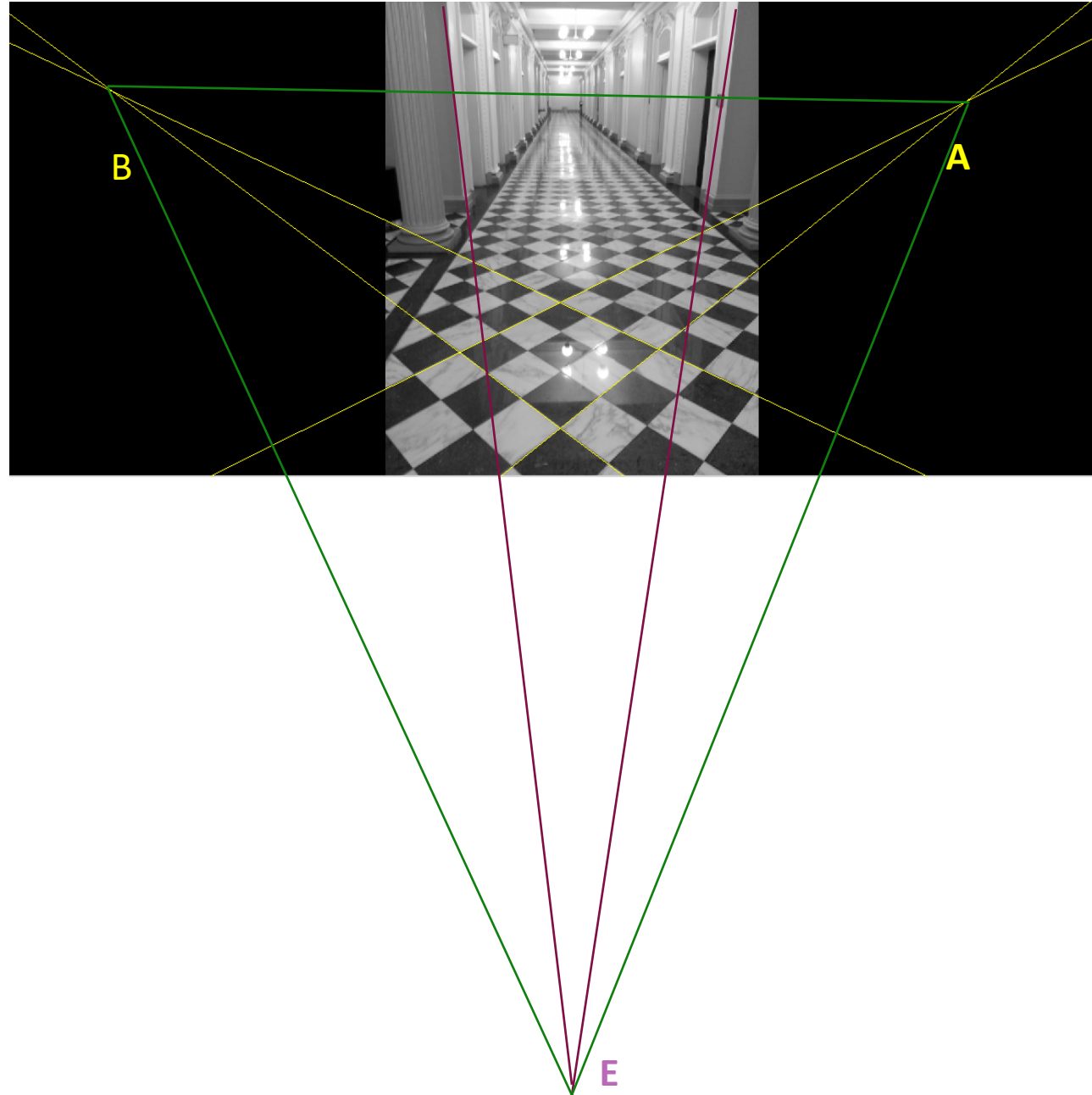
Line connecting AB is the horizon!



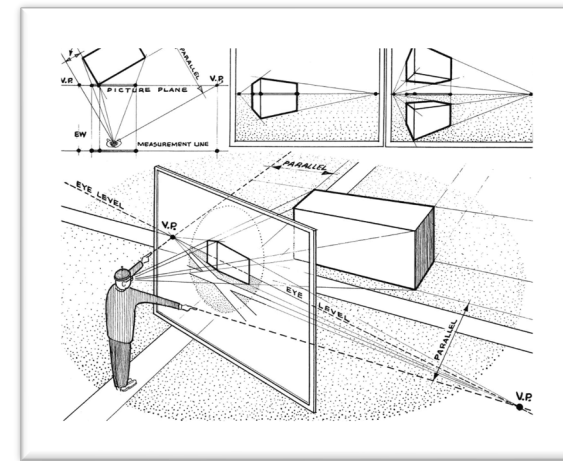
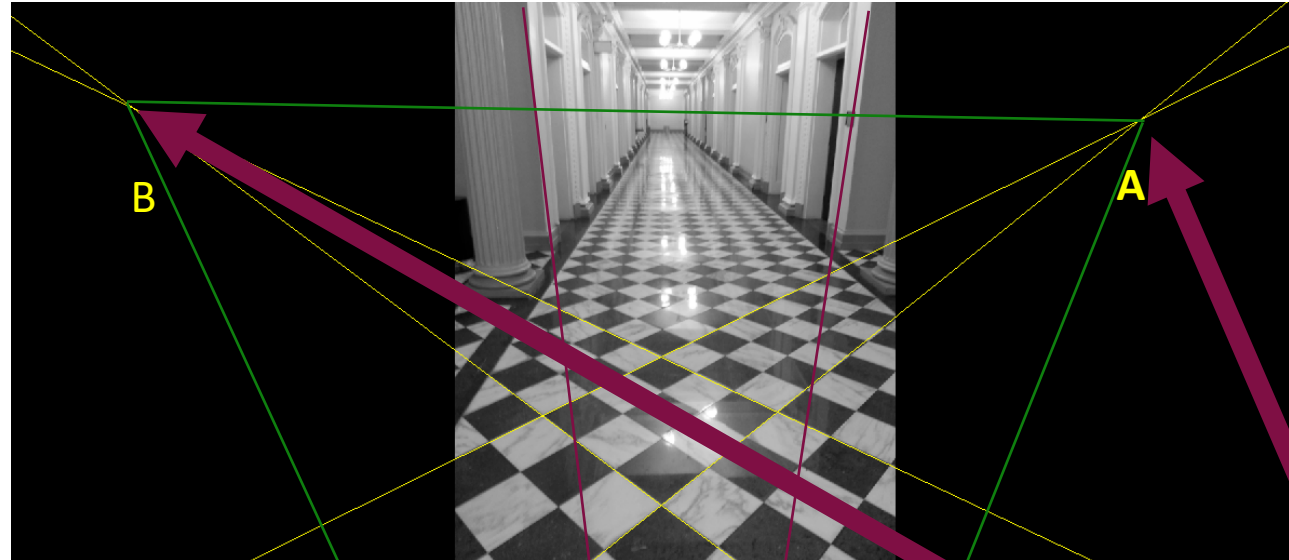
Remember that the horizon gives us the orientation of the ground plane with respect to the camera!

E

E is the vertical vanishing point!



Let's look at ABE as a tetrahedron OABE including the projection center



90°

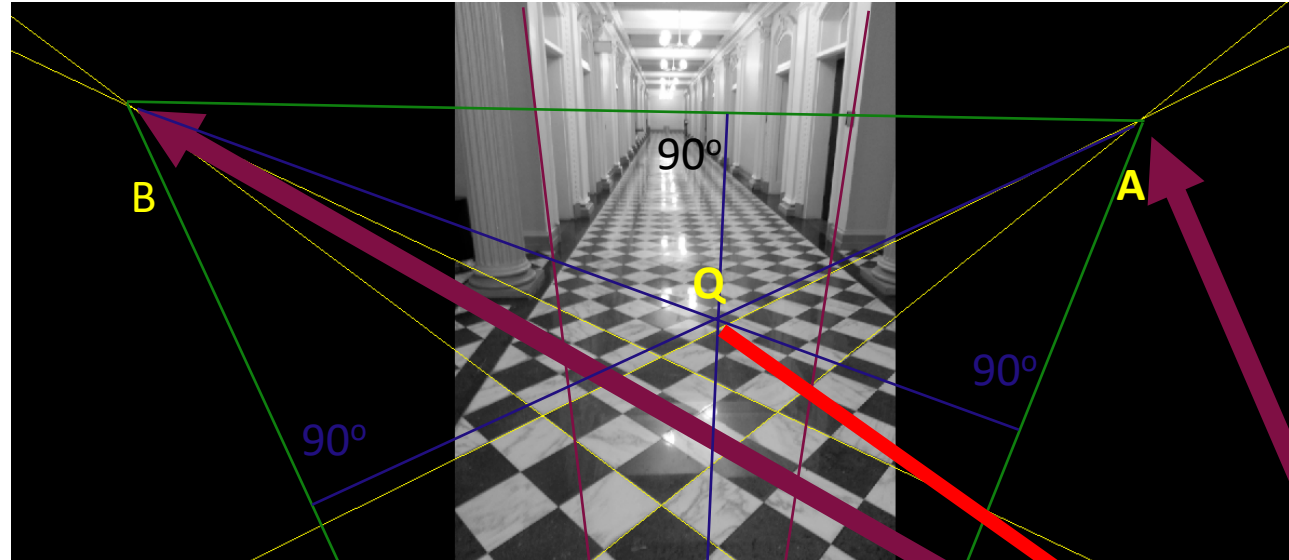
90°

O is Projection  
center

Tetrahedron vertex angles are 90 degrees

E

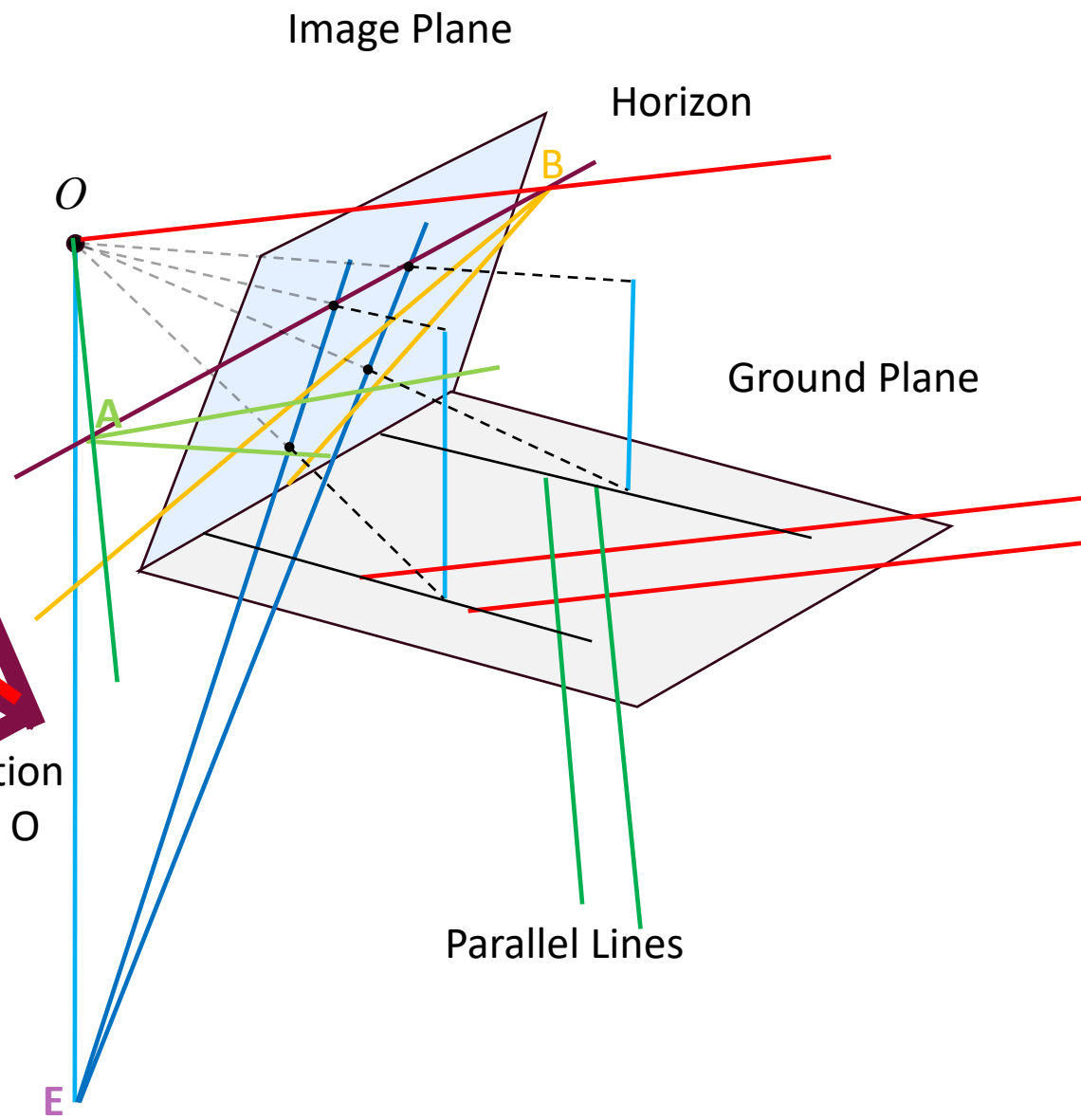
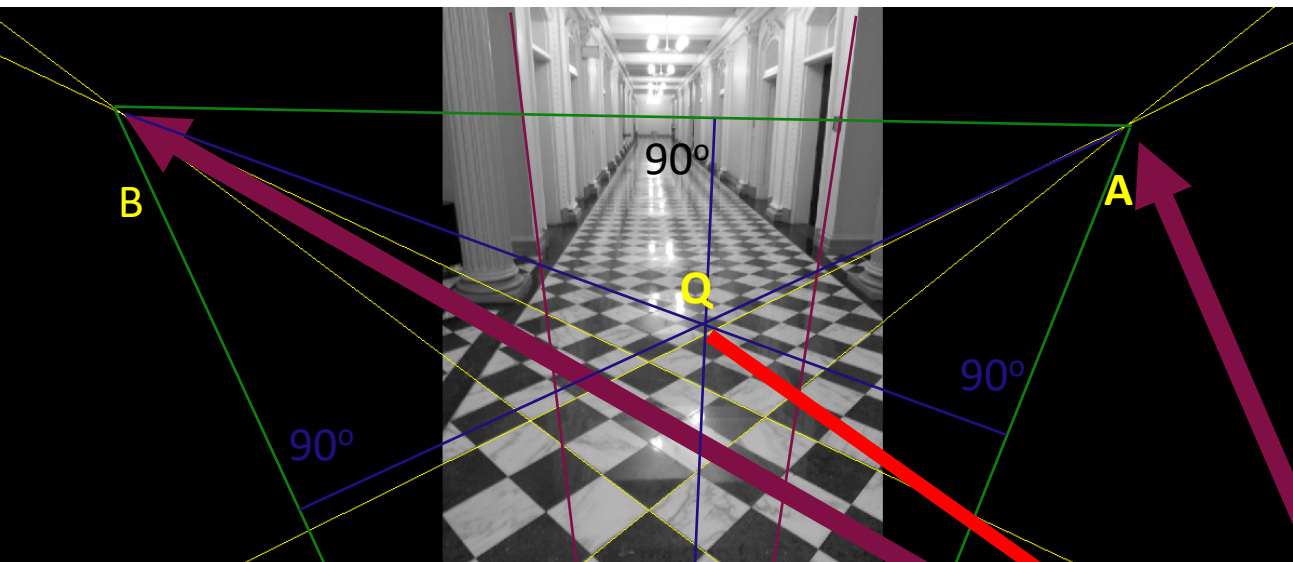
Let  $Q$  be the orthocenter of the triangle  $ABE$



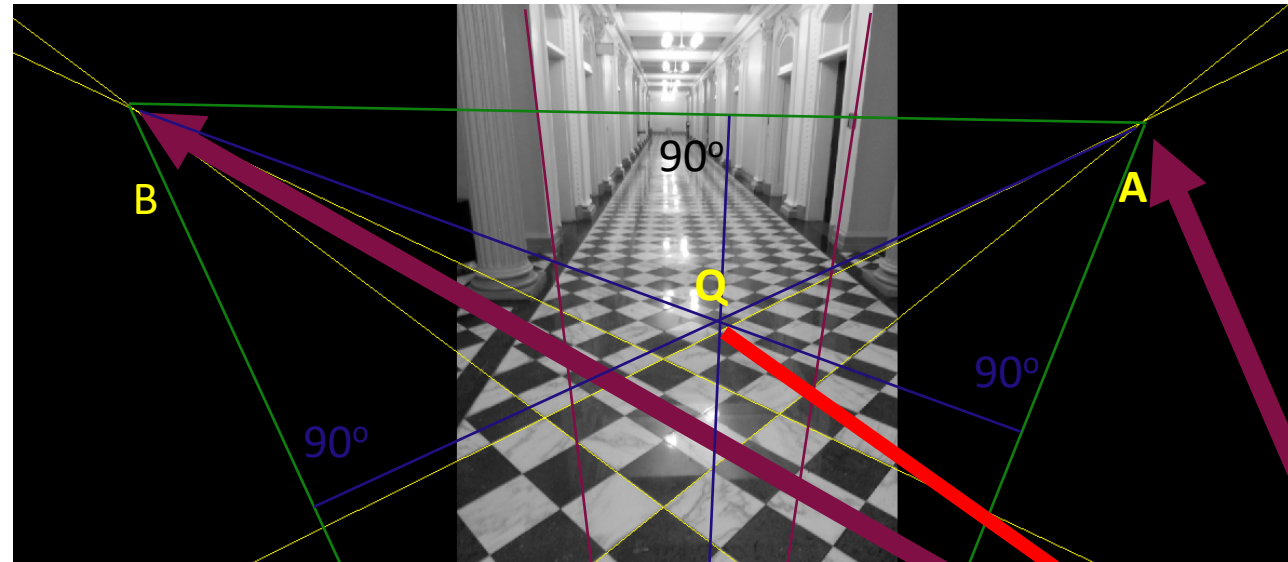
**Theorem from Euclidean Geometry:**

If  $Q$  is the orthocenter of  $ABE$  and all three angles  $AOB$ ,  $BOE$ , and  $EOA$  are right angles, the  $OQ$  is perpendicular to  $ABE$  plane!

$E$



Let Q be the orthocenter of the triangle ABC



Projection  
Center O

### Theorem from Euclidean Geometry:

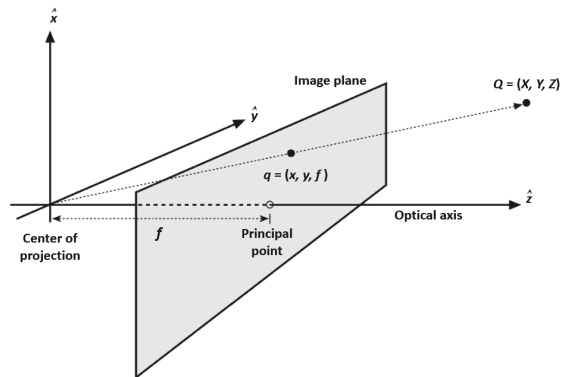
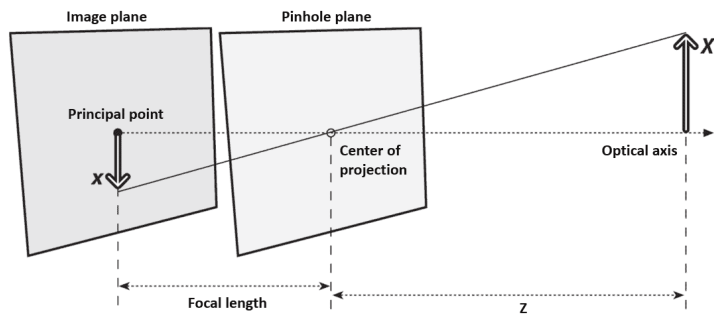
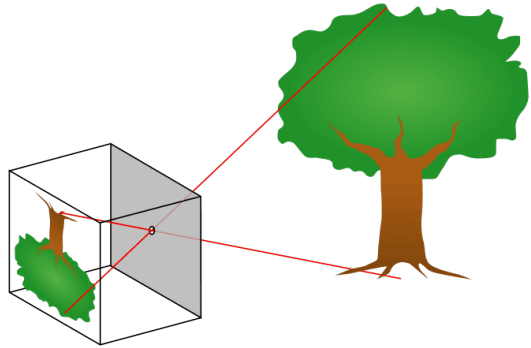
If Q is the orthocenter of ABE and all three angles AOB, BOE, and EOA are right angles, the OQ is perpendicular to ABE plane!

OQ is the principal axis and ABE is the image plane, hence, Q is the principal point

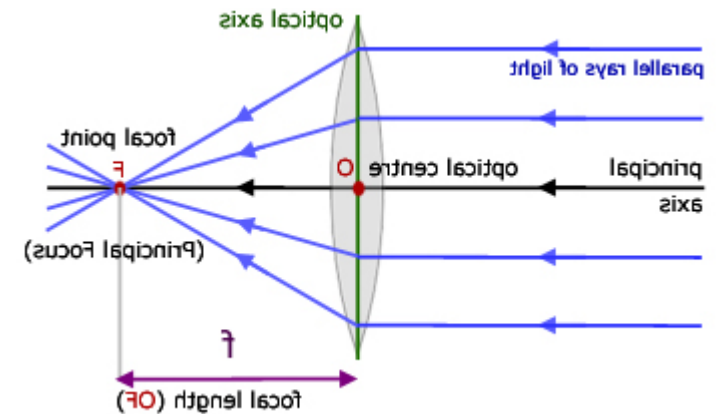
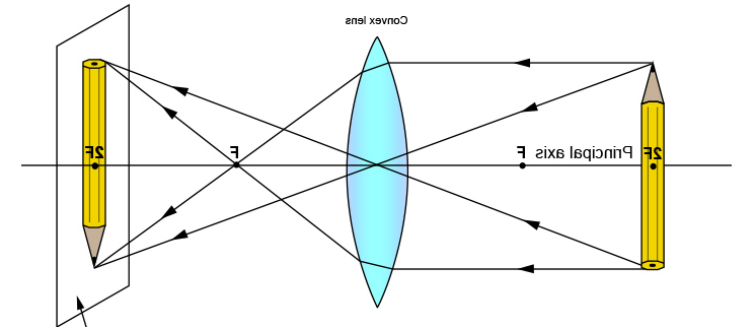
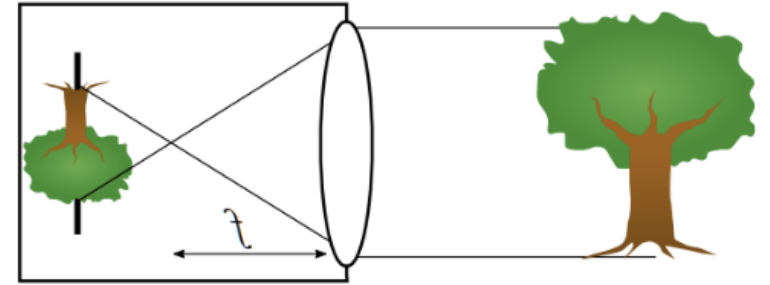
E

# Side Note: Principal Axis/Optical Axis, Principal Point

Pinhole Camera:



Lens Camera:





# Bonus proof: Perpendicular from camera center is the orthocenter of VPs

**THEOREM:** The image center  $(u_0, v_0)$  is the orthocenter of the triangle formed by the projections of three orthogonal vanishing points.

**PROOF:** See figure 2. Let  $C = (u_0, v_0, 1)$  denote the homogeneous coordinates of the image center: it is defined as the intersection of image plane  $V_1V_2V_3$  with the optical axis, which is the line through  $O$  and perpendicular to  $V_1V_2V_3$ .

$$OC \perp V_1V_2V_3 \Rightarrow OCV_1 \perp V_1V_2V_3$$

$$\Rightarrow OCV_1 \perp \text{any line contained in } V_1V_2V_3$$

By B: every plane containing  $OC$  is  $\perp$  to  $V_1V_2V_3$

By C:  $V_2V_3 \perp CV_1$ , so  $OCV_1 \perp V_2V_3$

In particular  $OCV_1 \perp V_2V_3$ . Moreover,  $OV_1 \perp OV_2$ , therefore  $OCV_1 \perp OV_2V_3$ .

When two planes are perpendicular, their intersections with a third plane are also perpendicular. Therefore, the intersection of  $OCV_1$  with  $V_1V_2V_3$  is perpendicular to intersection of  $OV_2V_3$  with  $V_1V_2V_3$ .

In other words  $V_1C \perp V_2V_3$ . A similar reasoning leads to  $V_2C \perp V_3V_1$  and  $V_3C \perp V_1V_2$ , therefore  $C$  is the orthocenter of  $V_1V_2V_3$ .

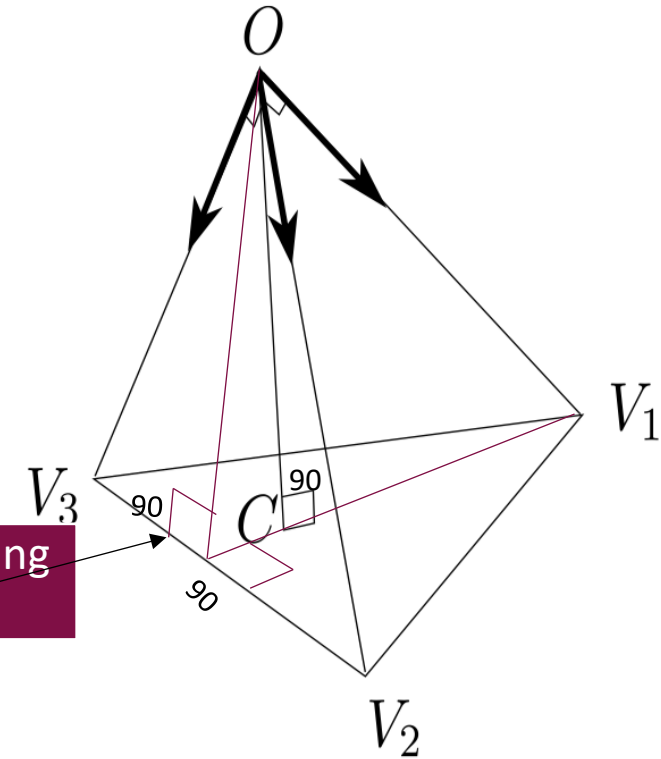
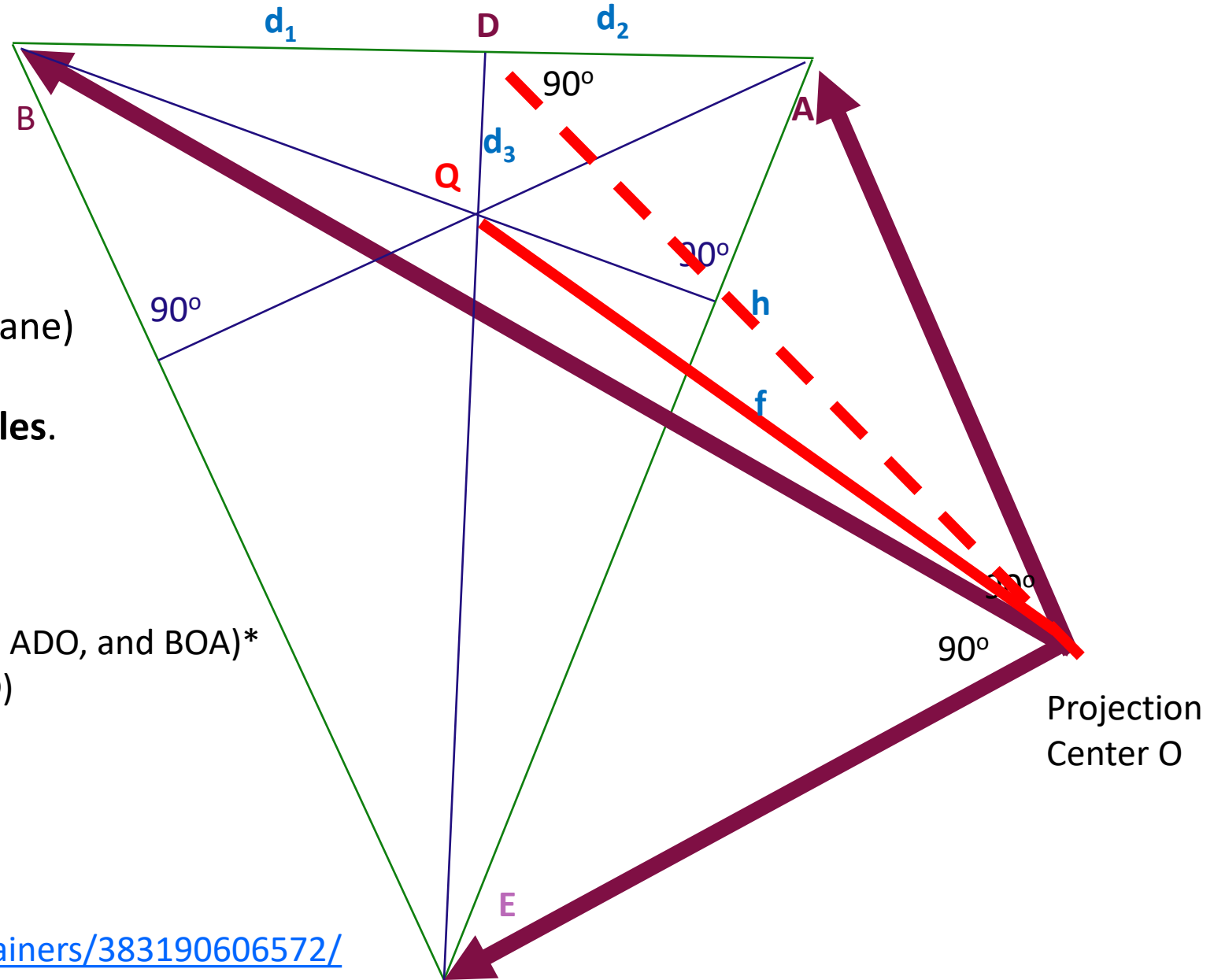


Figure 2: Three orthogonal vanishing points and image center.

We used the fact that the two non-parallel lines  $OV_2$  and  $V_2V_3$  contained in the plane  $OV_2V_3$  are perpendicular to  $OCV_1$ , therefore  $OV_2V_3 \perp OCV_1$ .

We found the principal point! What about the focal length ( $f=OQ$ )? Can it be computed from A,B, and E ?



OQ  $\perp$  image plane.  
So plane **OQD**  $\perp$  **image plane**

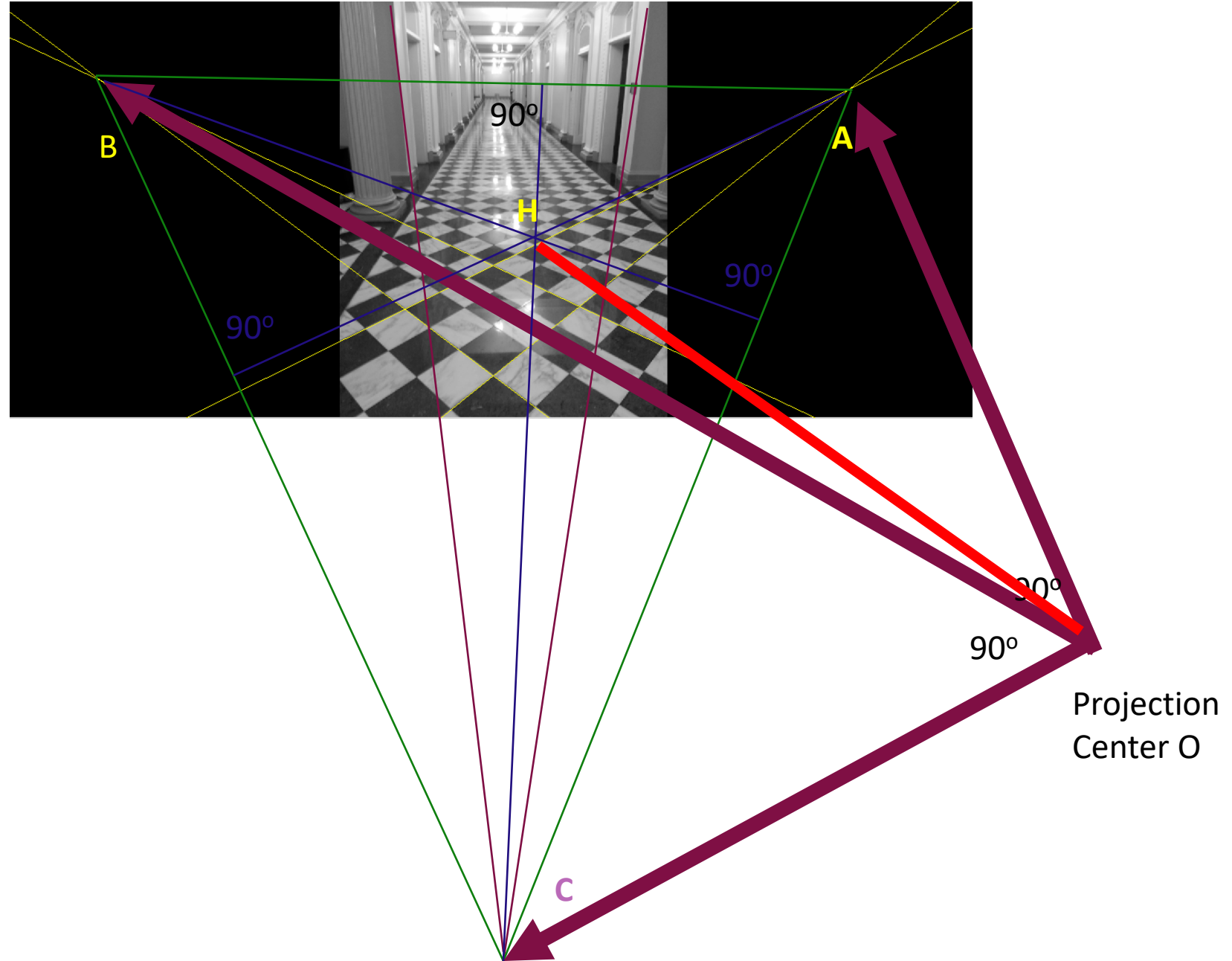
AD  $\perp$  ED (int. of OQD & img plane)  
So, AD  $\perp$  OD  
So, **BDO, ADO** are right triangles.

$h^2 = d_1 d_2$  (apply Pythagoras to BDO, ADO, and BOA)\*  
 $f^2 + d_3^2 = h^2$  (Pythagoras again to OHD)

Hence  $f^2 = d_1 d_2 - d_3^2$

\* <https://www.nagwa.com/en/explainers/383190606572/>

Three orthogonal vanishing points (if none is at infty) allow computation of full intrinsics K (focal length and image center)!



# Summary

- If we have 3 orthogonal VPs, we can get full intrinsics  $K$  (focal length and image center), and also extrinsics  $R$ .
  - What's missing? Just the translation  $t$ .
    - And that information is not contained in VPs, because camera translations don't affect the VPs!
    - Which is why, when we found homographies (that do contain full information about translation), we used more than just VPs.



# Cross Ratios & Length Measurements from Single Images (“Single View Metrology”)

# Are lengths preserved under homography?



Obviously not.

What about length ratios?

# Projection of a circle

From other perspective views, would the center of this circular clock face remain at center?



1:1



$\approx 1:2$

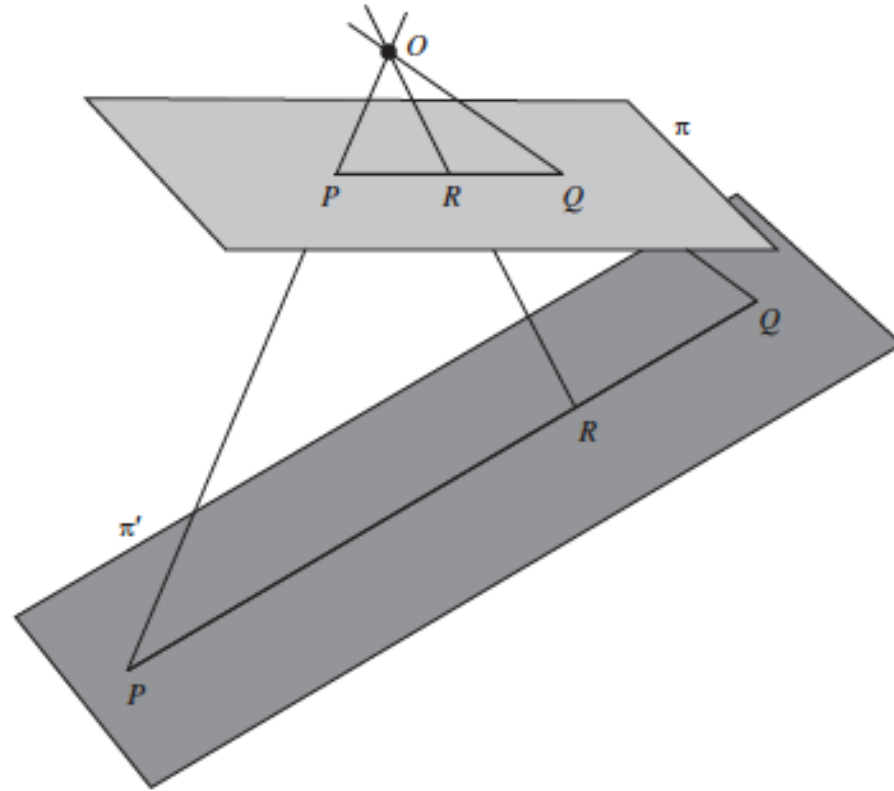


$\approx 1:2.5$

Clearly, length ratios are not preserved under homographies!



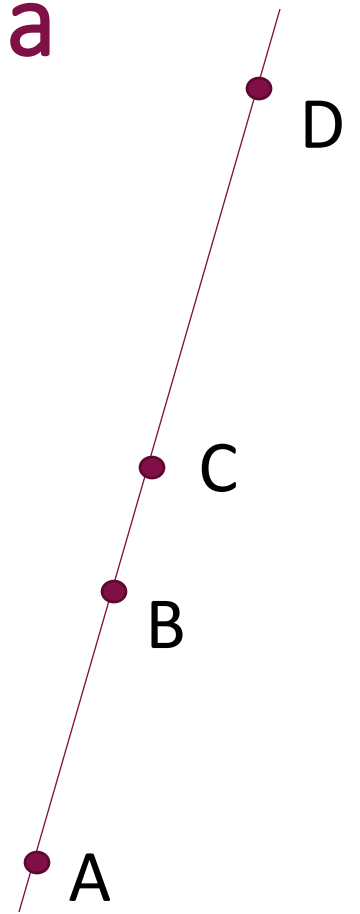
# Length ratios under homography



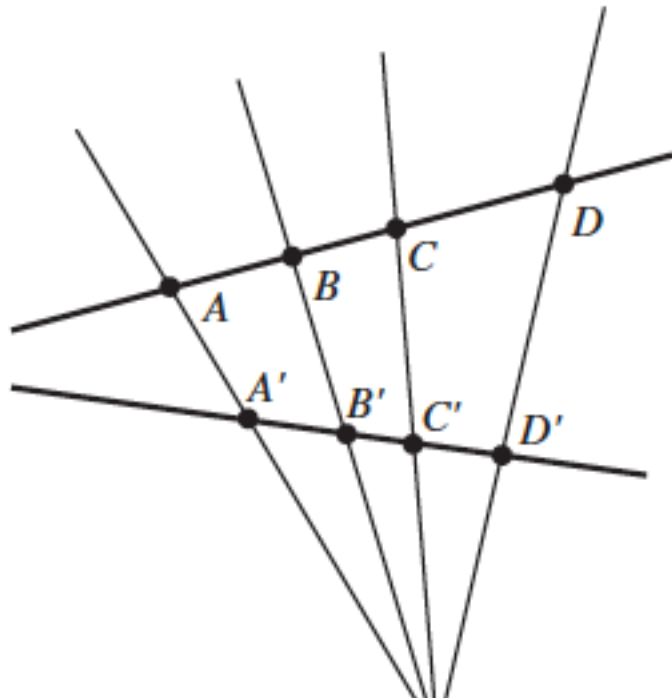
Clearly, length ratios are not preserved under homographies!

# What metric property along a line *is preserved* under a projective transformation?

- Not lengths AB i.e. distance of two pts from each other.
- Not length ratios i.e. distance of two pts from a third collinear pt. AC: BC
- Instead, what is preserved is:
  - **Ratio of ratios** of distances of two pts from two other collinear pts.
    - $\frac{AC}{AD} : \frac{BC}{BD} = \text{Cross ratio of A, B, C, D}$
- CR can also be written as  $\frac{AC \cdot BD}{AD \cdot BC}$



# Cross Ratios of Collinear Points



Given four points  $A, B, C, D$ , we define the cross-ratio of their distances as

$$CR(A, B, C, D) = \frac{AC}{AD} : \frac{BC}{BD}.$$

$CR(A, B, C, D)$  remains invariant under projective transformations

Denote  $\mathbf{x}'_i = (x'_i, 1)^T$  and  $\mathbf{x}_i = (x_i, 1)^T$ . Suppose  $\mathbf{x}'_i = H\mathbf{x}_i$ . Note  $|H| = \det(H)$ . Then

$$x'_i - x'_j = |\mathbf{x}'_i \ \mathbf{x}'_j| = |\lambda_i H\mathbf{x}_i \ \lambda_j H\mathbf{x}_j| = \lambda_i \lambda_j |H(\mathbf{x}_i \ \mathbf{x}_j)|$$

$$= \lambda_i \lambda_j |H| |\mathbf{x}_i \ \mathbf{x}_j| = \lambda_i \lambda_j \det(H) (x_i - x_j)$$

Note: C  
but it is  
e.g., CR



Pappus (290-350 AD)



Girard Desargues (1591-1661)



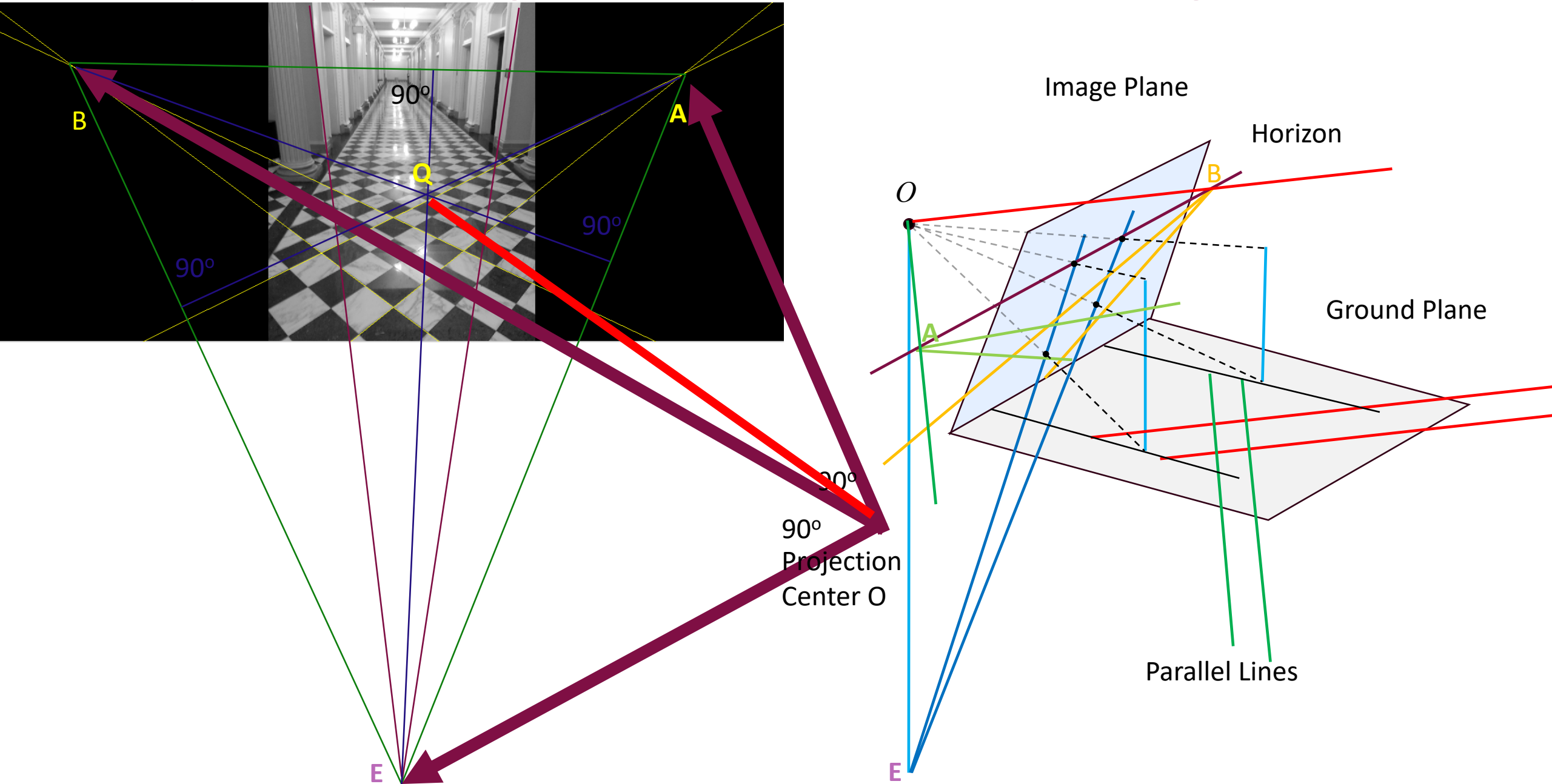
CIS 5800

# Machine Perception

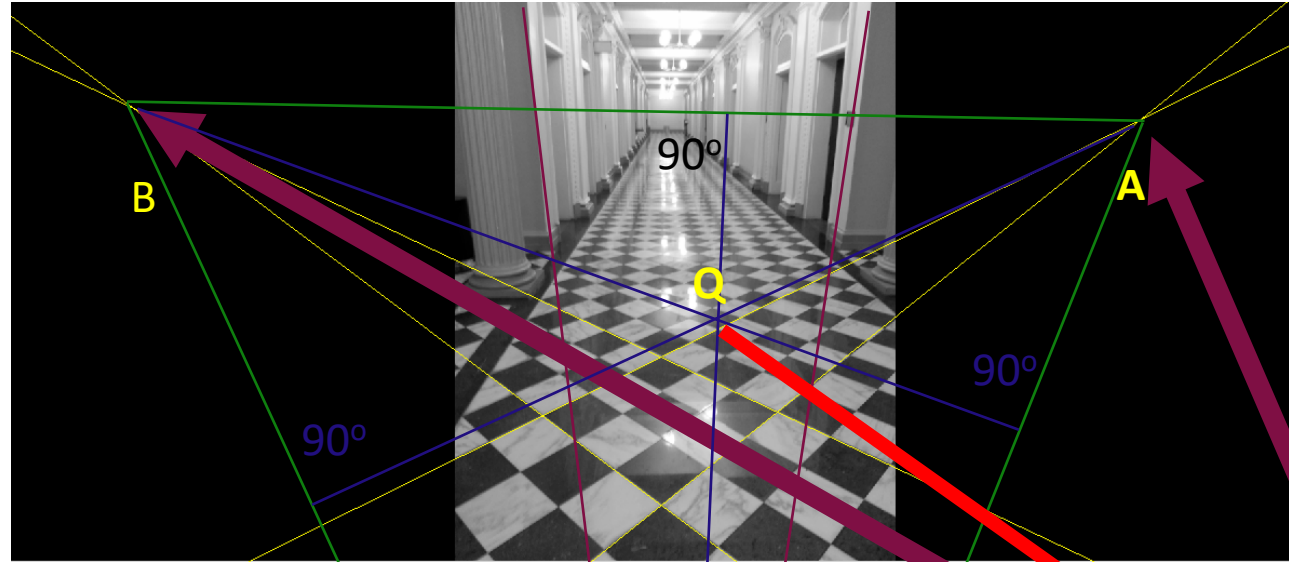
Instructor: Lingjie Liu

Lec 7: Feb 12, 2026

# Recap: Computing Intrinsic K from Vanishing Points



**Recap:** Let  $Q$  be the orthocenter of the triangle  $ABC$



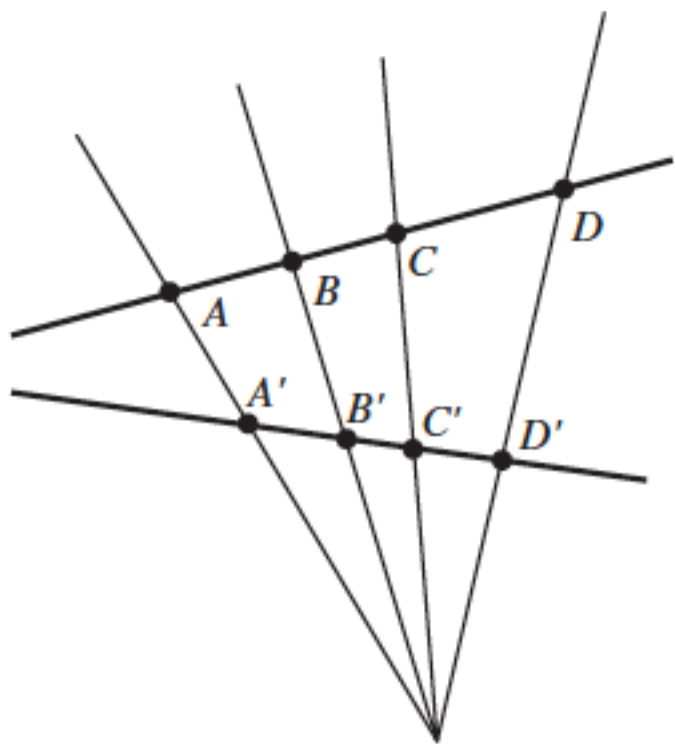
**Theorem from Euclidean Geometry:**

If  $Q$  is the orthocenter of  $ABE$  and all three angles  $AOB$ ,  $BOE$ , and  $EOA$  are right angles, the  $OQ$  is perpendicular to  $ABE$  plane!

$OQ$  is the principal axis and  $ABE$  is the image plane, hence,  $Q$  is the principal point



# Recap: Cross Ratios of Collinear Points



Given four points  $A, B, C, D$ , we define the cross-ratio of their distances as

$$CR(A, B, C, D) = \frac{AC}{AD} : \frac{BC}{BD}.$$

$CR(A, B, C, D)$  remains invariant under projective transformations

$$\frac{AC}{AD} : \frac{BC}{BD} = \frac{A'C'}{A'D'} : \frac{B'C'}{B'D'}$$

Denote  $\mathbf{x}'_i = (x'_i, 1)^T$  and  $\mathbf{x}_i = (x_i, 1)^T$ . Suppose  $\mathbf{x}'_i = H\mathbf{x}_i$ . Note  $|H| = \det(H)$ . Then

$$x'_i - x'_j = |\mathbf{x}'_i \ \mathbf{x}'_j| = |\lambda_i H\mathbf{x}_i \ \lambda_j H\mathbf{x}_j| = \lambda_i \lambda_j |H(\mathbf{x}_i \ \mathbf{x}_j)|$$

$$= \lambda_i \lambda_j |H| |\mathbf{x}_i \ \mathbf{x}_j| = \lambda_i \lambda_j \det(H) (x_i - x_j)$$



Pappus (290-350 AD)

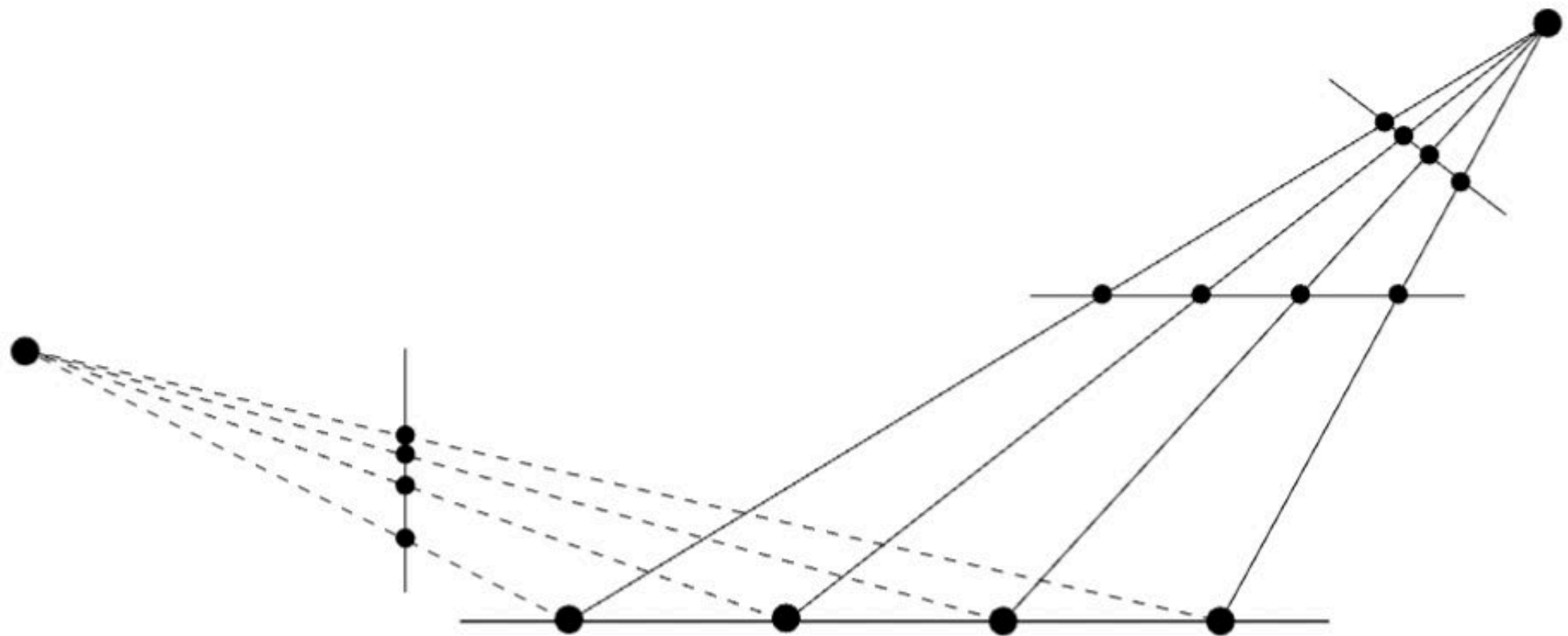


Girard Desargues (1591-1661)

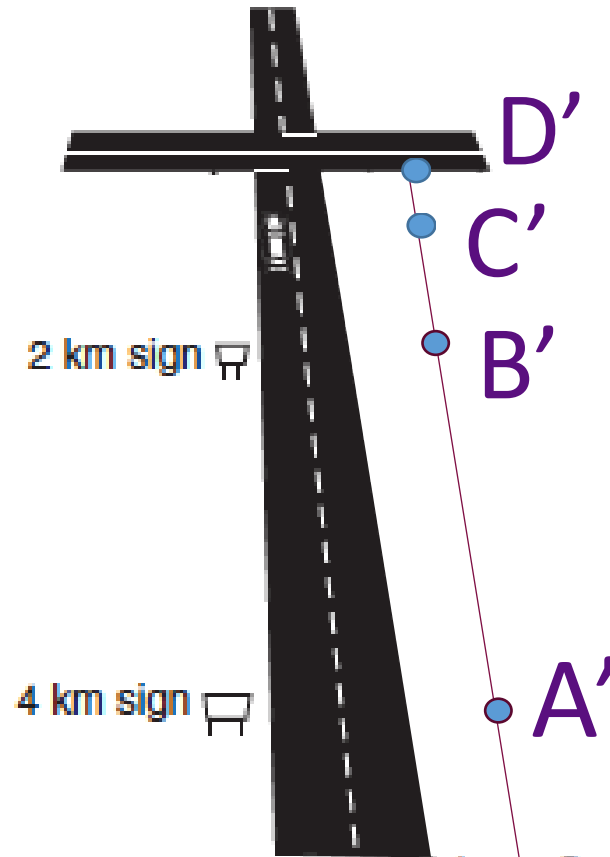
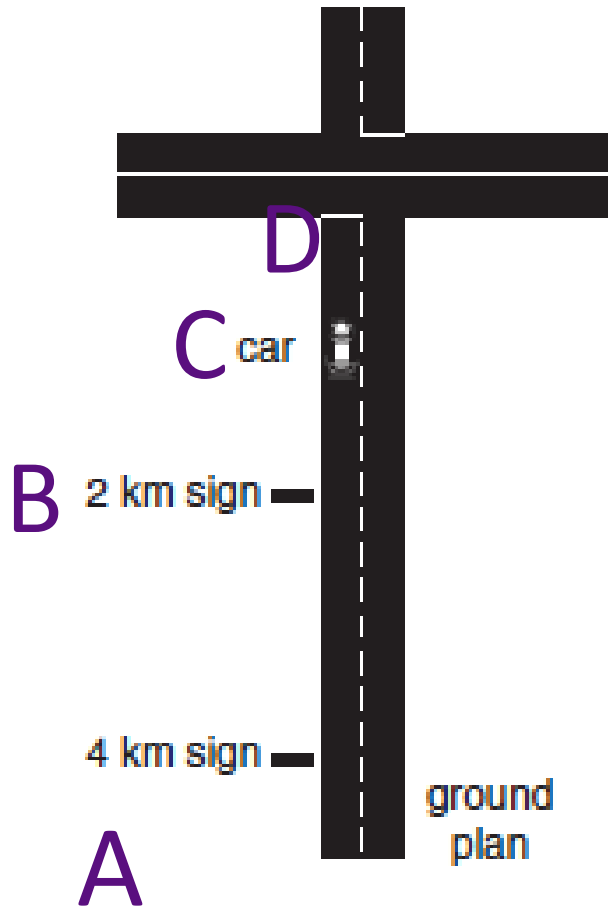
Note: C  
but it is  
e.g., CR



Same cross ratio for all these 4 point sets!



# Example: Cross ratios for metrology



How far is car from intersection:

CD=?

A'D'=300 pixels

A'C'=275 pixels

B'C'=50 pixels

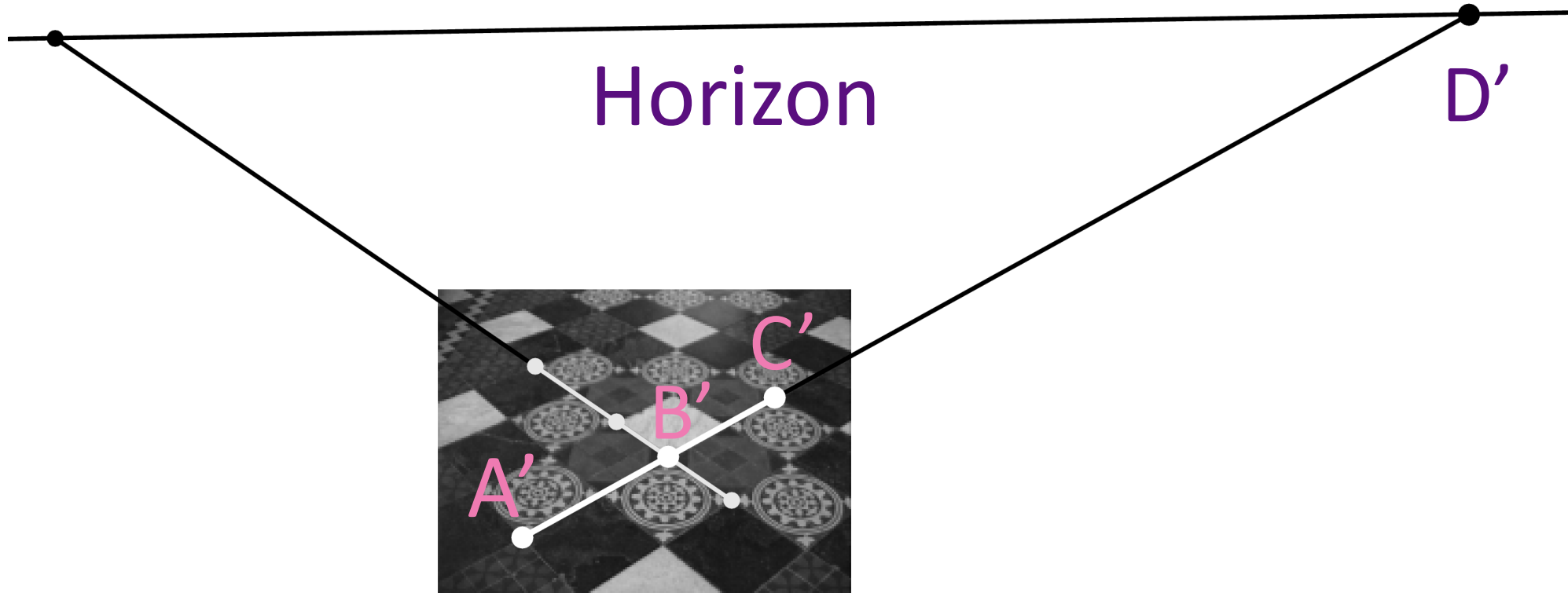
$$\frac{A'C'}{A'D'} : \frac{B'C'}{B'D'} = \frac{275}{300} : \frac{50}{75} = 1.375$$

$$\frac{AC}{AD} : \frac{BC}{BD} = \frac{4-CD}{4} : \frac{2-CD}{2} = 1.375$$

$$CD = 0.857km$$

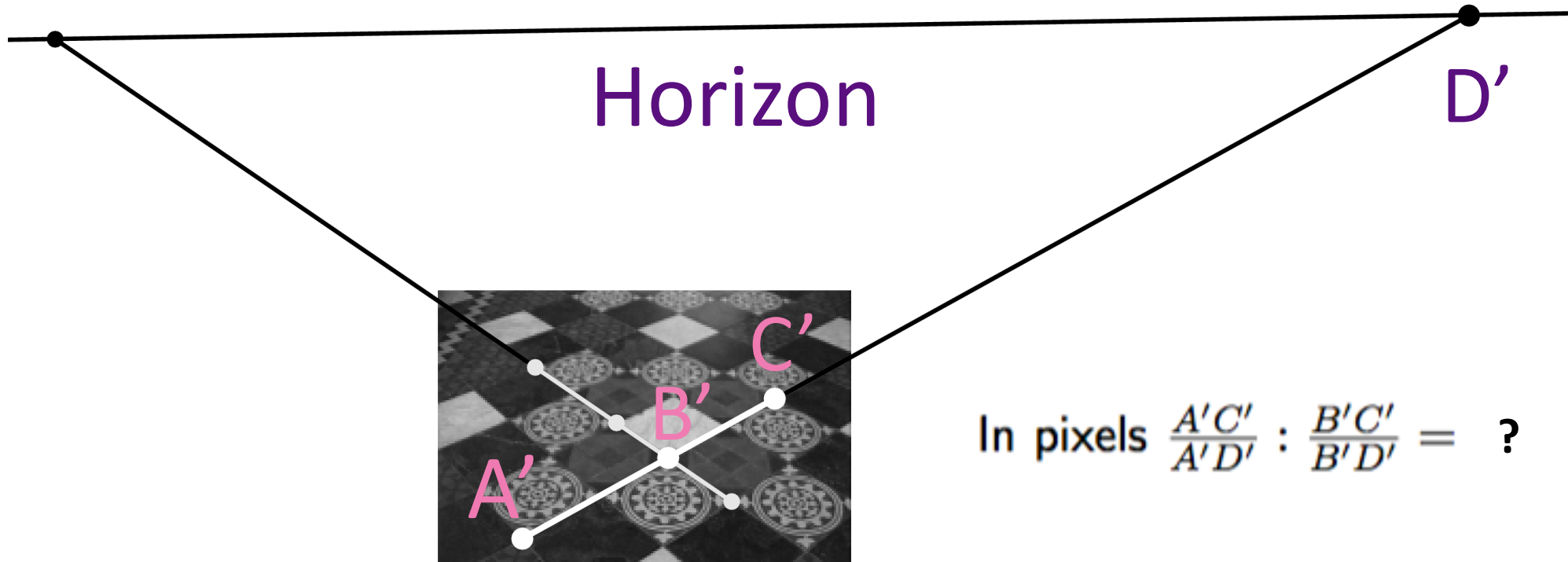
Note: all distances in image are measurable. And 2 distances in world are given.

What happens when one of the points is at infinity?



While  $D'$  is a finite point,  $D$  on the original plane is at infinity !

# What happens when one of the points is at infinity?

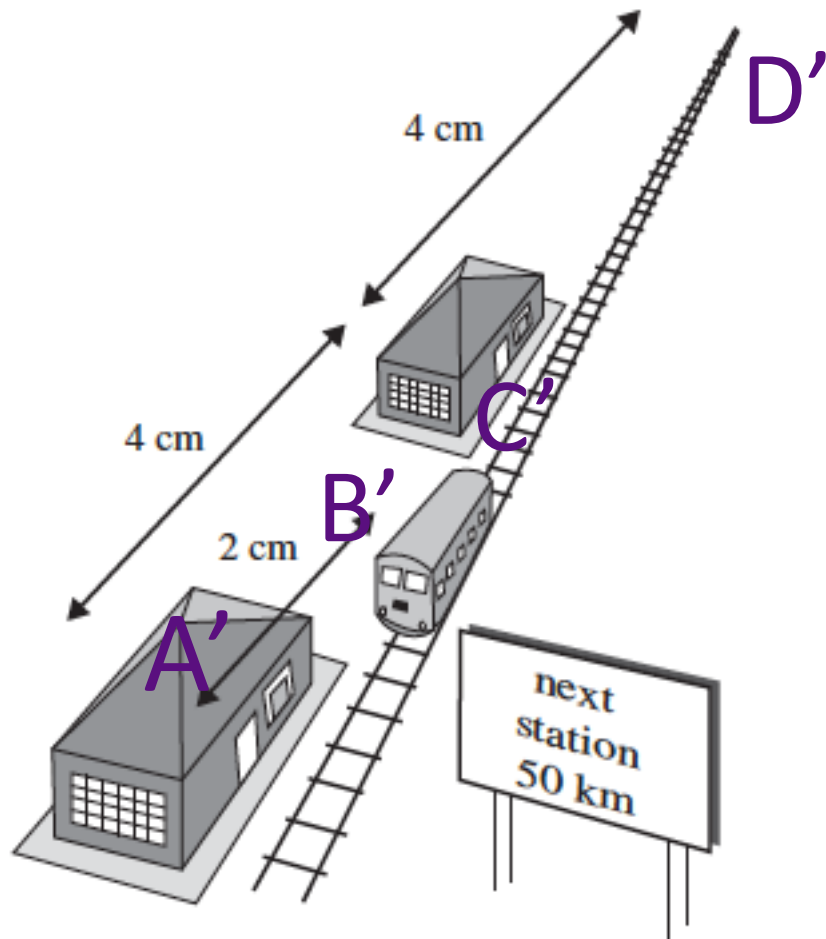


In pixels  $\frac{A'C'}{A'D'} : \frac{B'C'}{B'D'} = ?$

When a point  $D$  is at infinity, the cross-ratio becomes a ratio !

$$\frac{AC}{AD} : \frac{BC}{BD} = \frac{AC}{BC} \quad \left( \text{Think } \frac{AC}{\infty} : \frac{BC}{\infty} = \frac{AC}{BC} \times \frac{\infty}{\infty} = \frac{AC}{BC} \right)$$

# Vanishing points allow us to measure length ratios!



How far away is the train from the next station?  
i.e. Length of  $BC$ ?

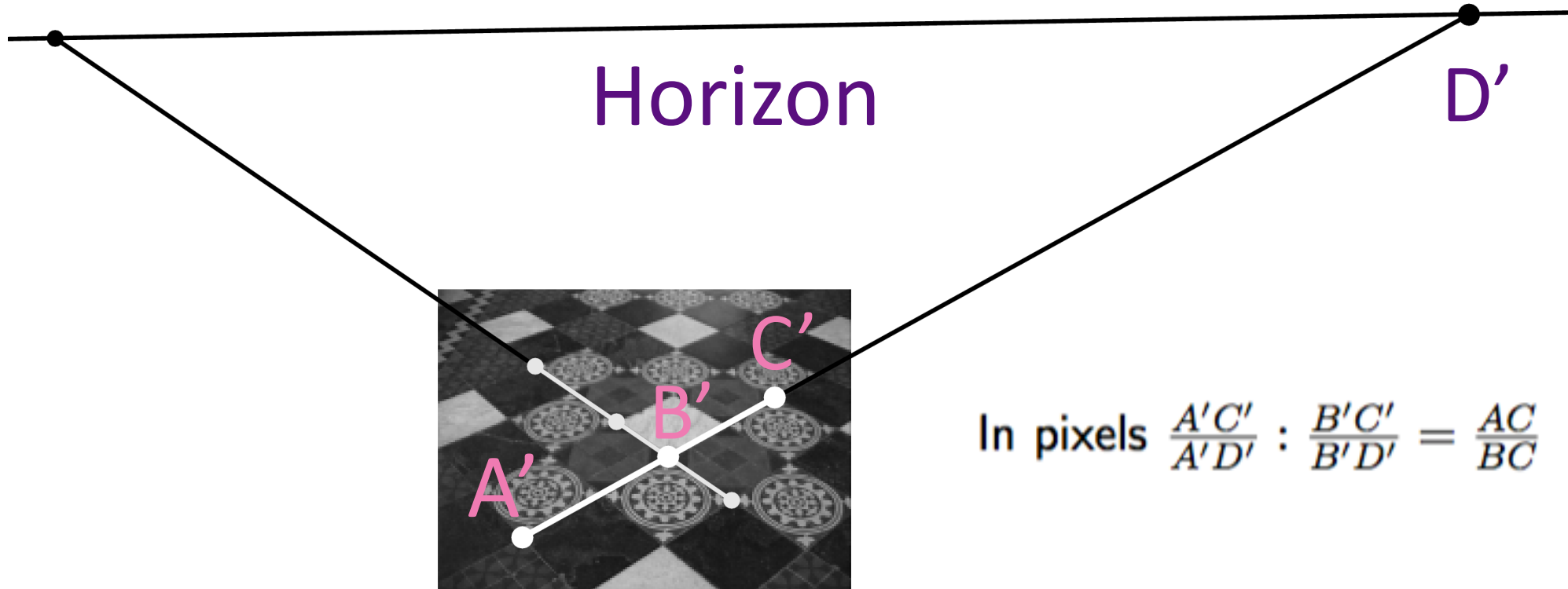
$$\frac{A'C'}{A'D'} : \frac{B'C'}{B'D'} = \frac{AC}{BC}$$

$$\frac{4}{8} : \frac{2}{6} = \frac{50}{BC}$$

$$\frac{4}{8} : \frac{2}{6} = \frac{50}{BC} \text{ and } BC = 33.33 \text{ km.}$$

Note: We only needed measurement of 1 distance in the real world.

Also, world plane length ratios determine vanishing points!



$$\frac{A'C'}{A'D'} : \frac{B'C'}{B'D'} = \frac{AC}{BC} = 2$$

Given:  $AB=BC$

If we know  $A'$ ,  $B'$ ,  $C'$  in pixels, we can find  $D'$

In this way, we can find vanishing points and the horizon without even needing parallel lines!



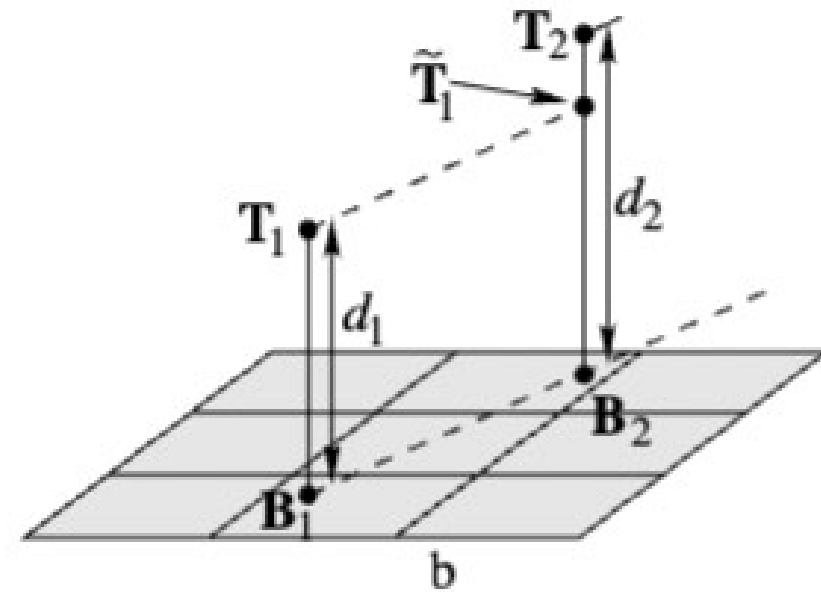
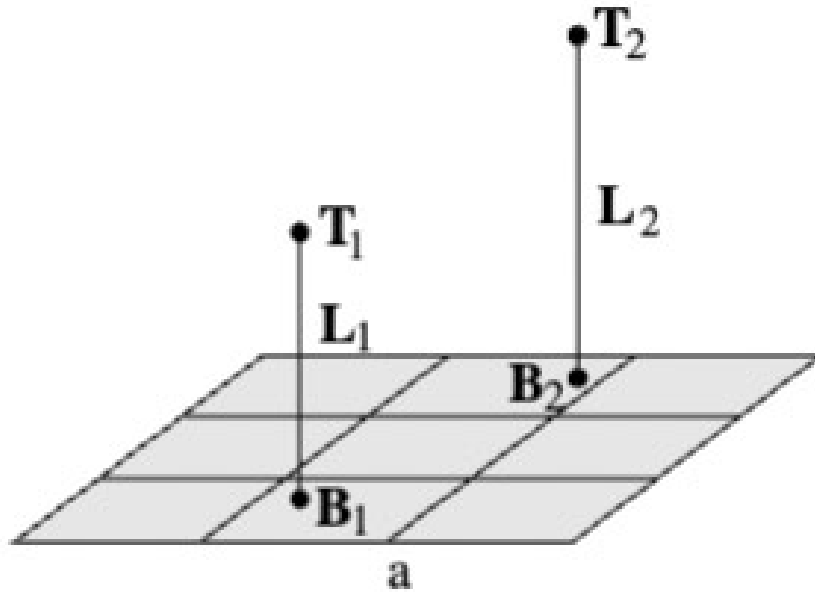
# Measuring heights, i.e. distances *from* a world plane

- So far, we've been looking at distances *on* a world plane.
- Next, distances *off it*?



# Length transfer in 3D

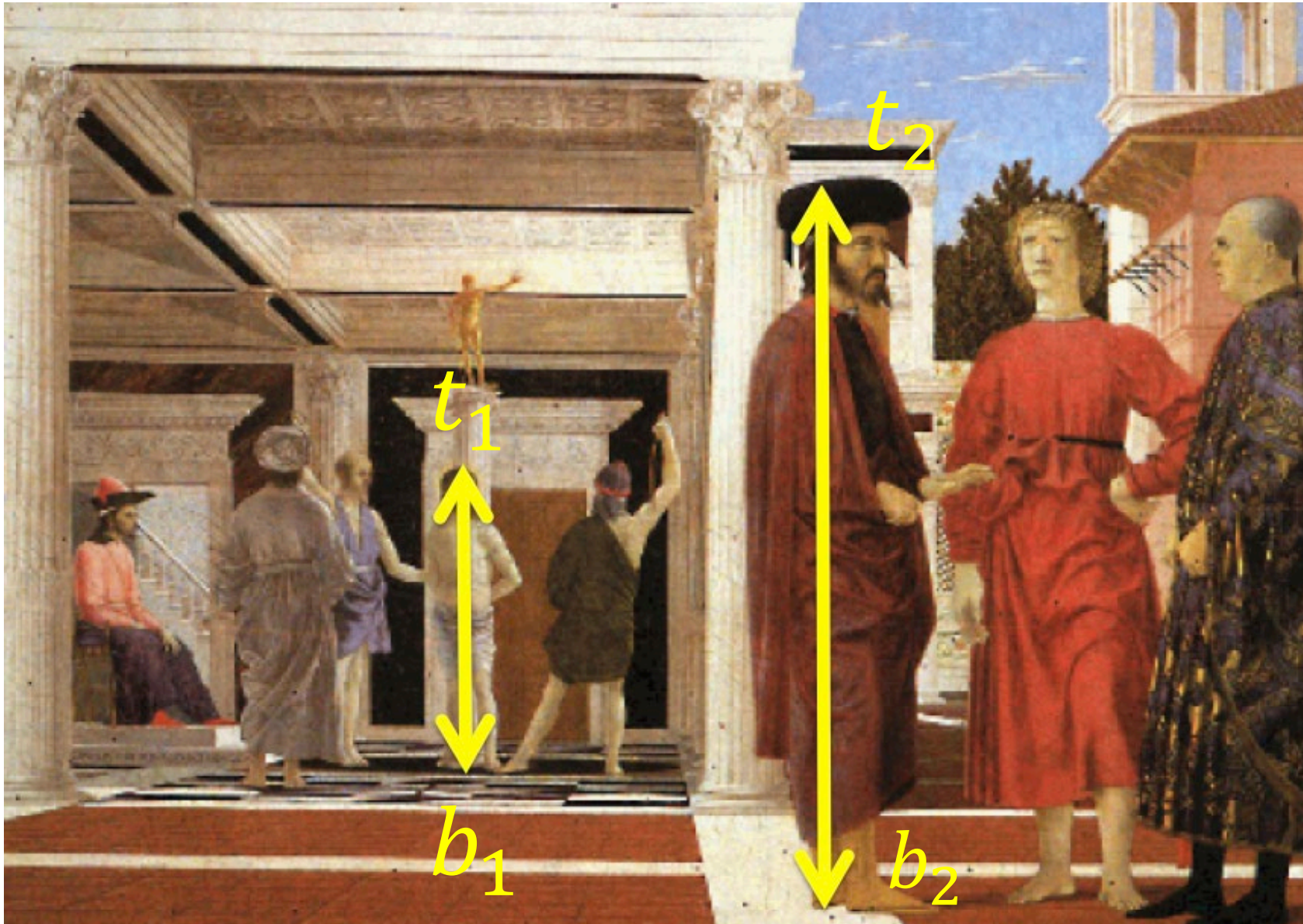
- In the real 3-D world, you can compare one object with known length to another to “transfer” its length.



How to do this in an image?

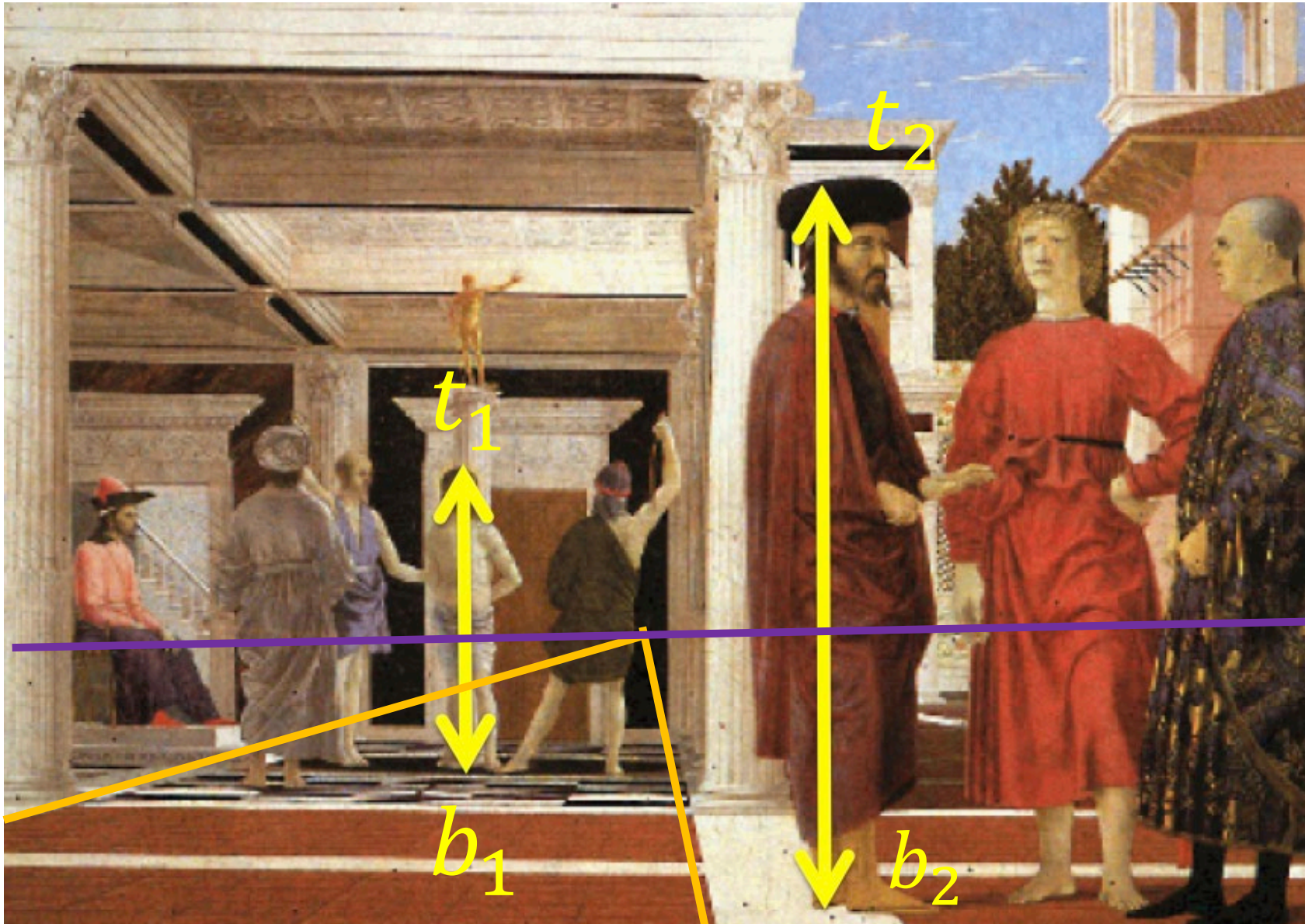
## Distance Transfer: How tall is the man if the statue is 180cm (in 3D world space)?

**Note:** we will be going back and forth between world points (denoted as capital letters) and their projections (denoted as small letters).



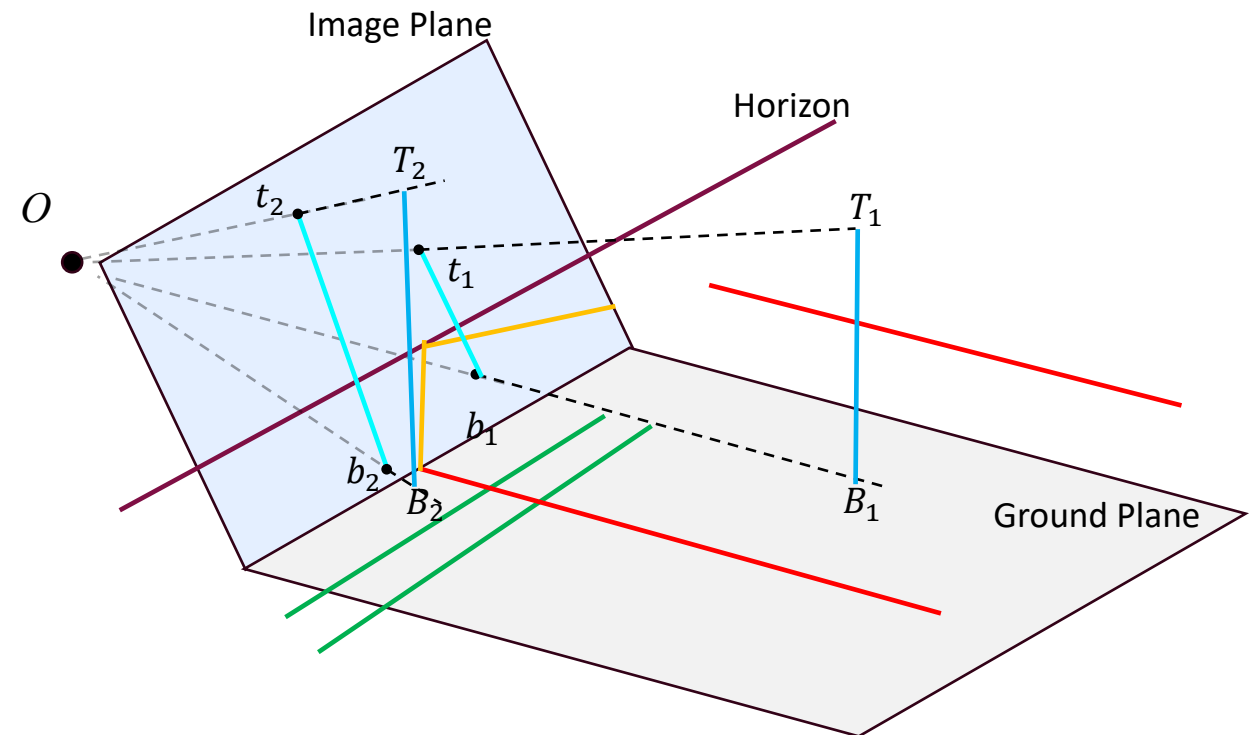
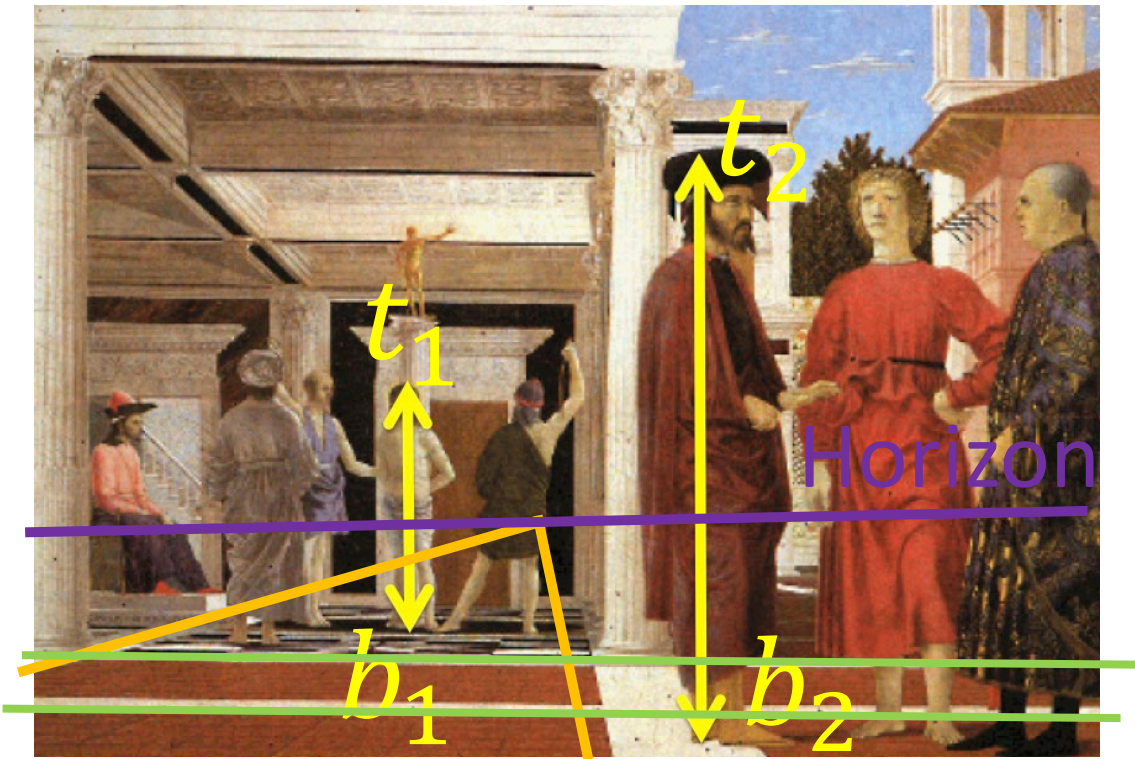


# 1. Find horizon (from homography, or by finding VPs)



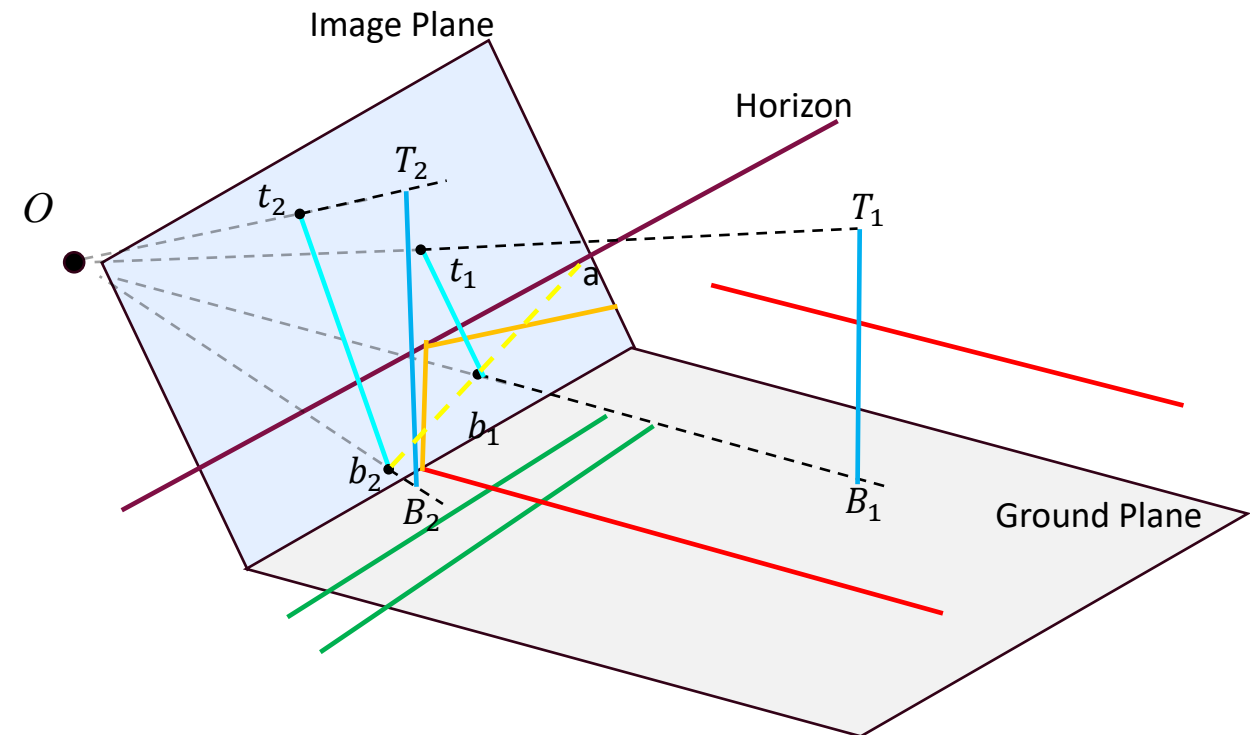
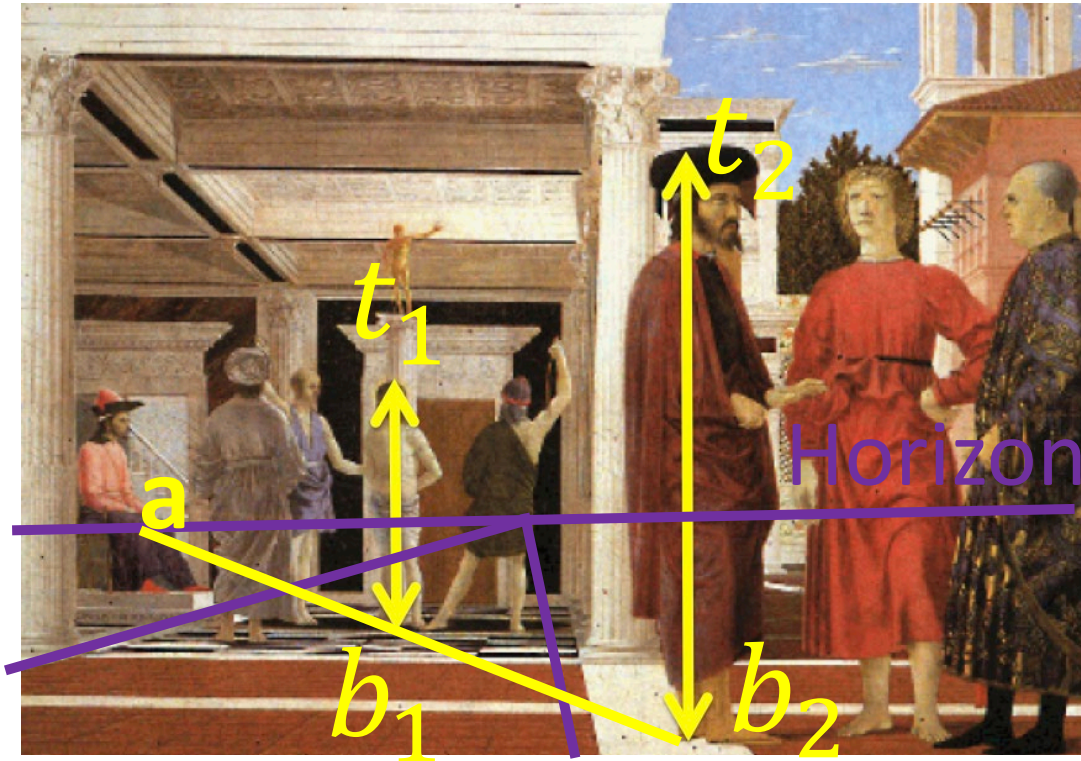
Horizon

# 1. Find horizon (from homography, or by finding VPs)

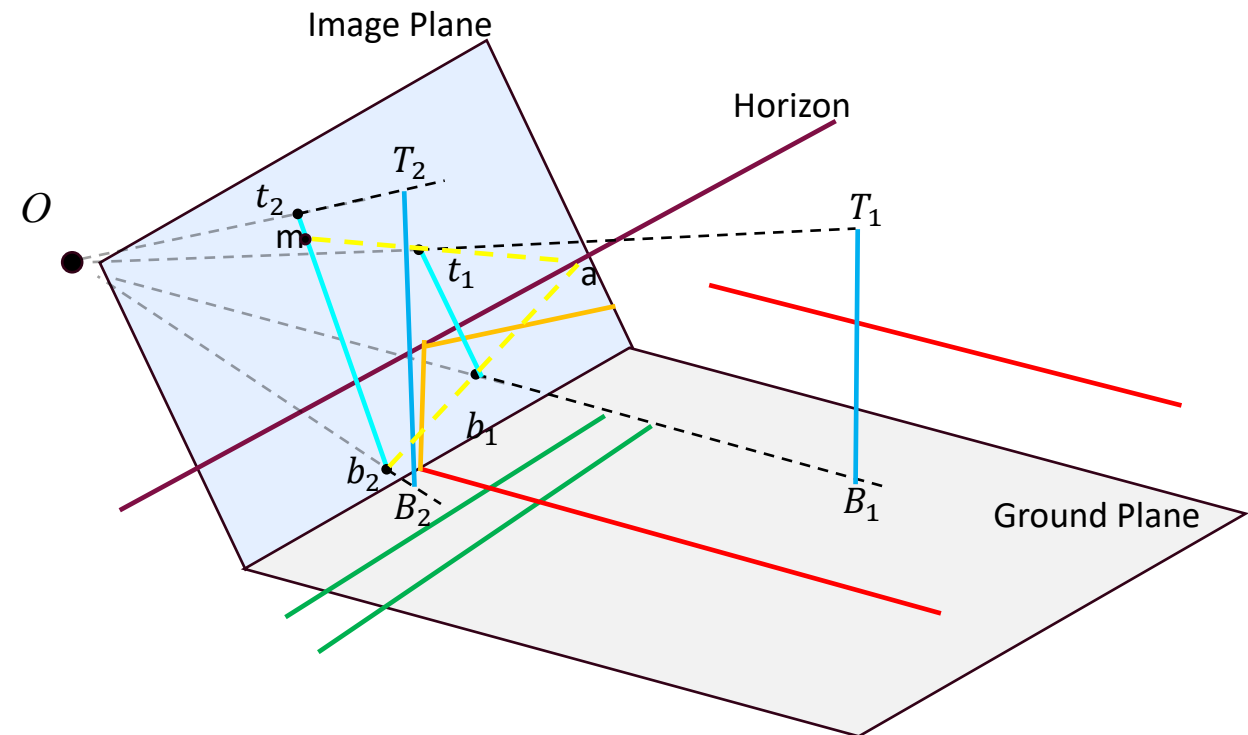
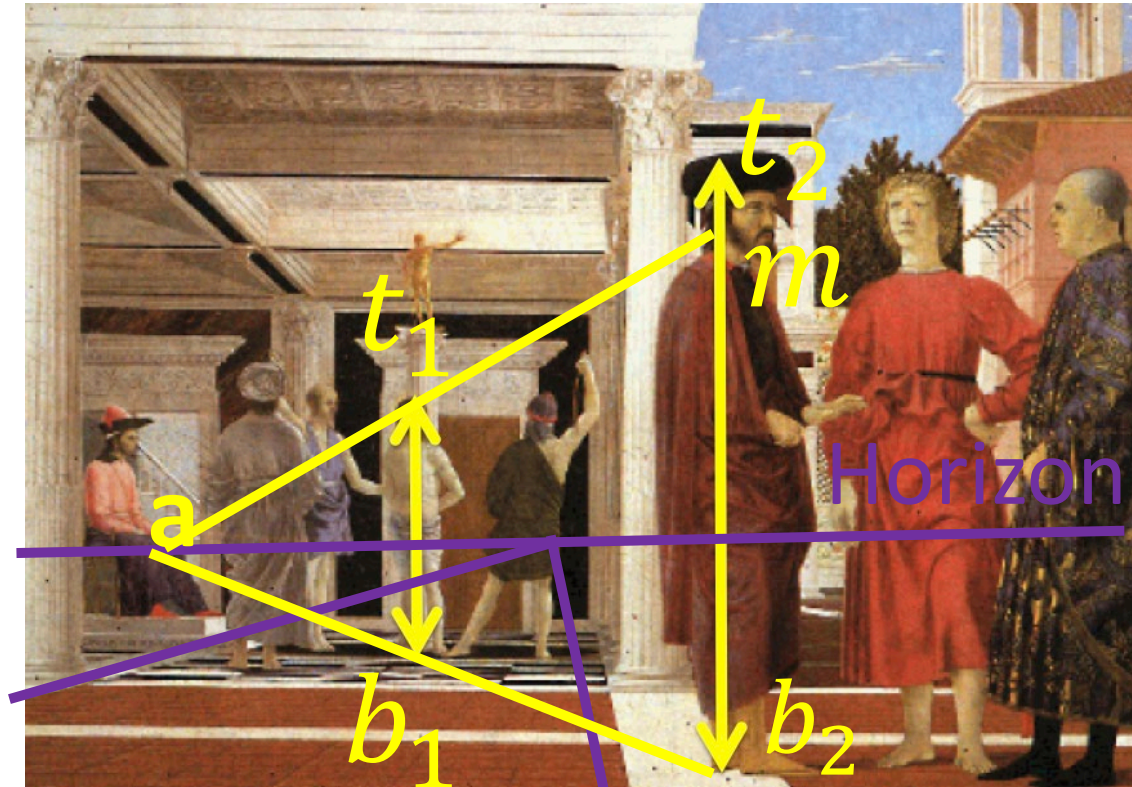




2. Connect the feet of the man and the statue and find intersection  $a'$  with horizon!

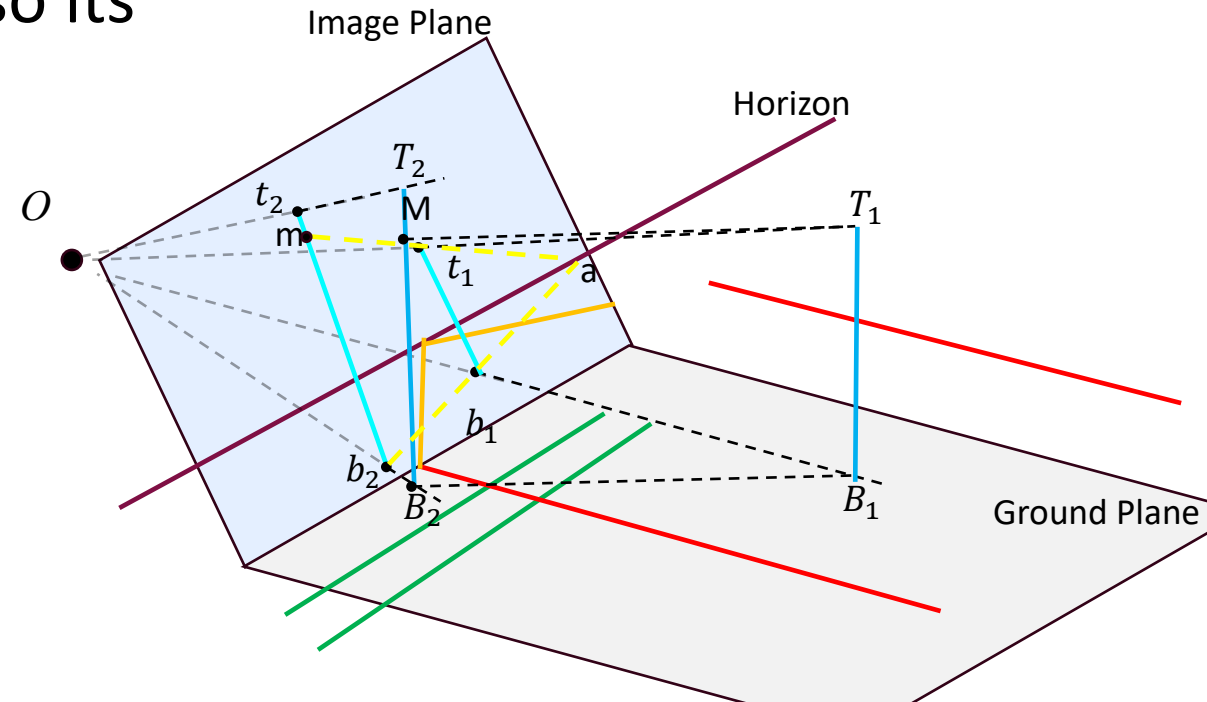


### 3. Connect $a$ with top of statue $t_1$



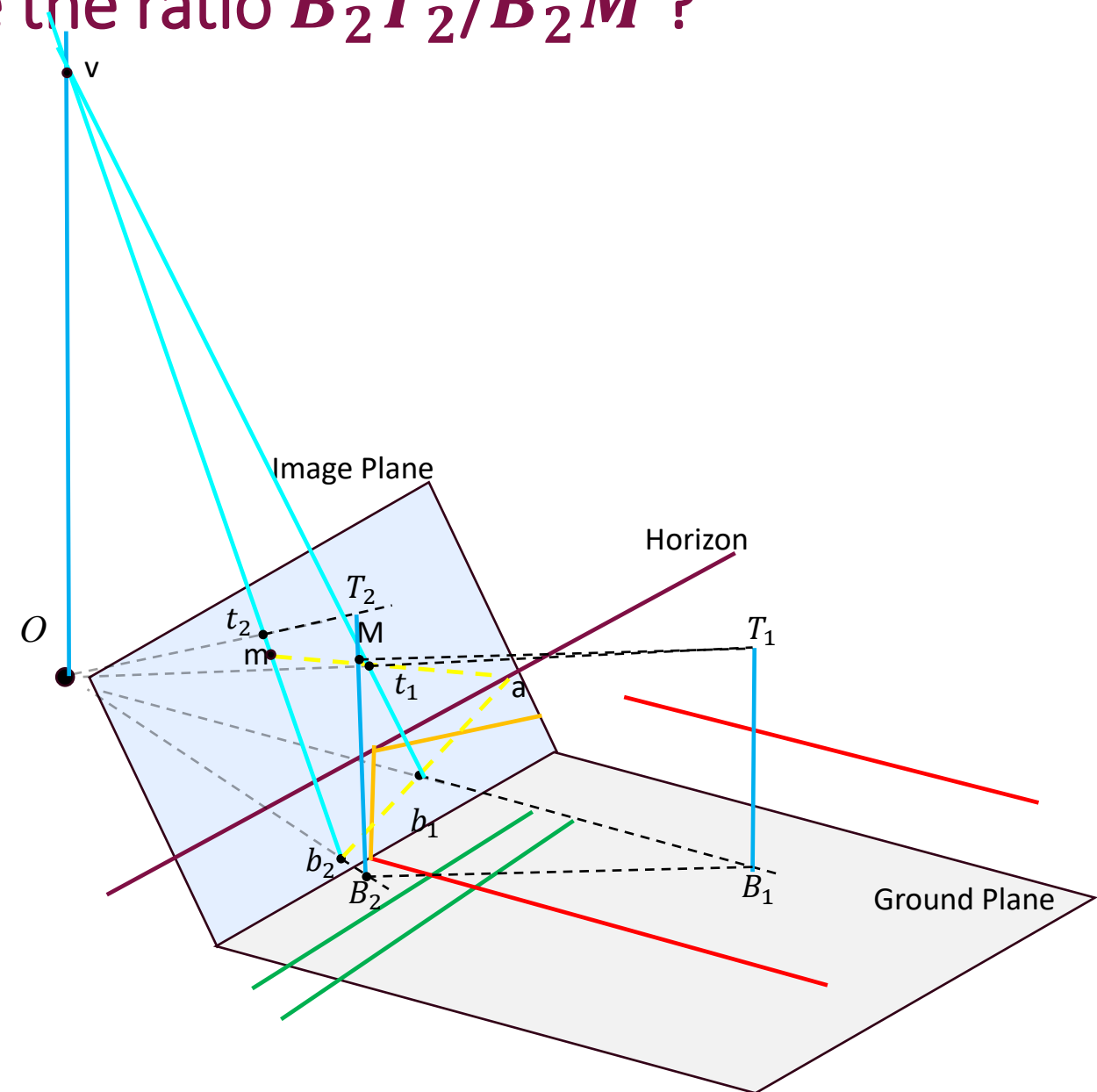
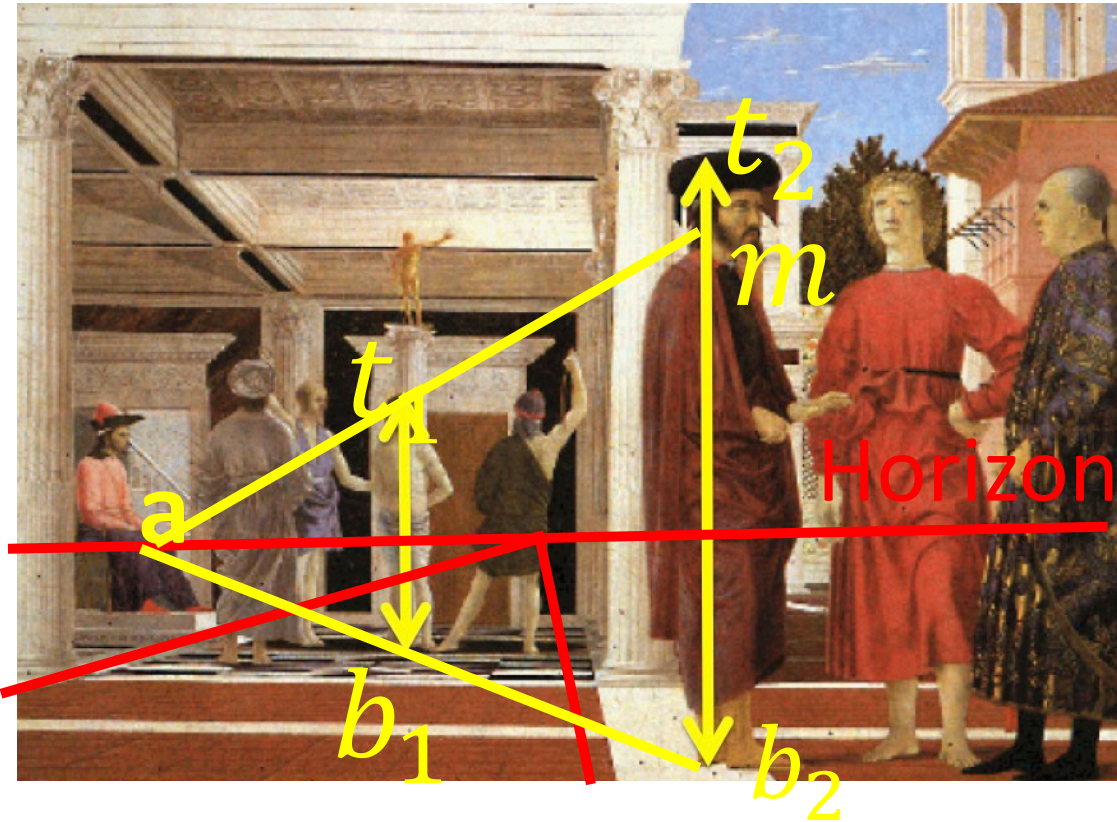
**$MT_1$  is parallel to the ground  $\rightarrow B_2M = B_1T_1$**

- Consider a world line parallel to  $B_2B_1$  and passing through top of statue  $T_1$ . It intersects the person at a point  $M$  that is at the same height as  $T_1$ . (for practical purposes, we will treat the person and the statue as “vertical lines”.)
- The world line  $MT_1$  is parallel to  $B_2B_1$ , so its image  $mt_1$  must meet  $b_2b_1$  at its VP= $a$ .
- So  $B_2M = B_1T_1$



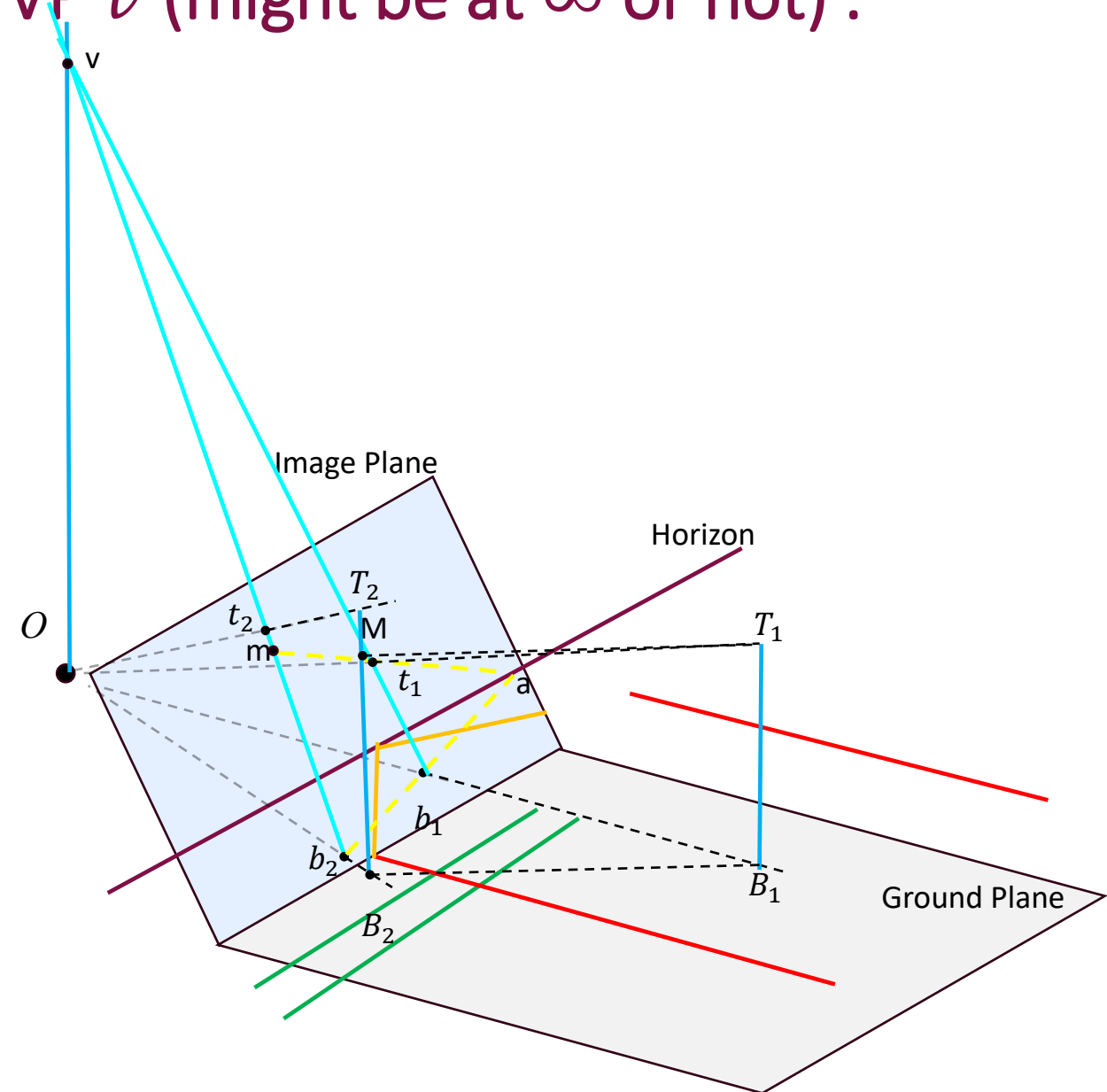
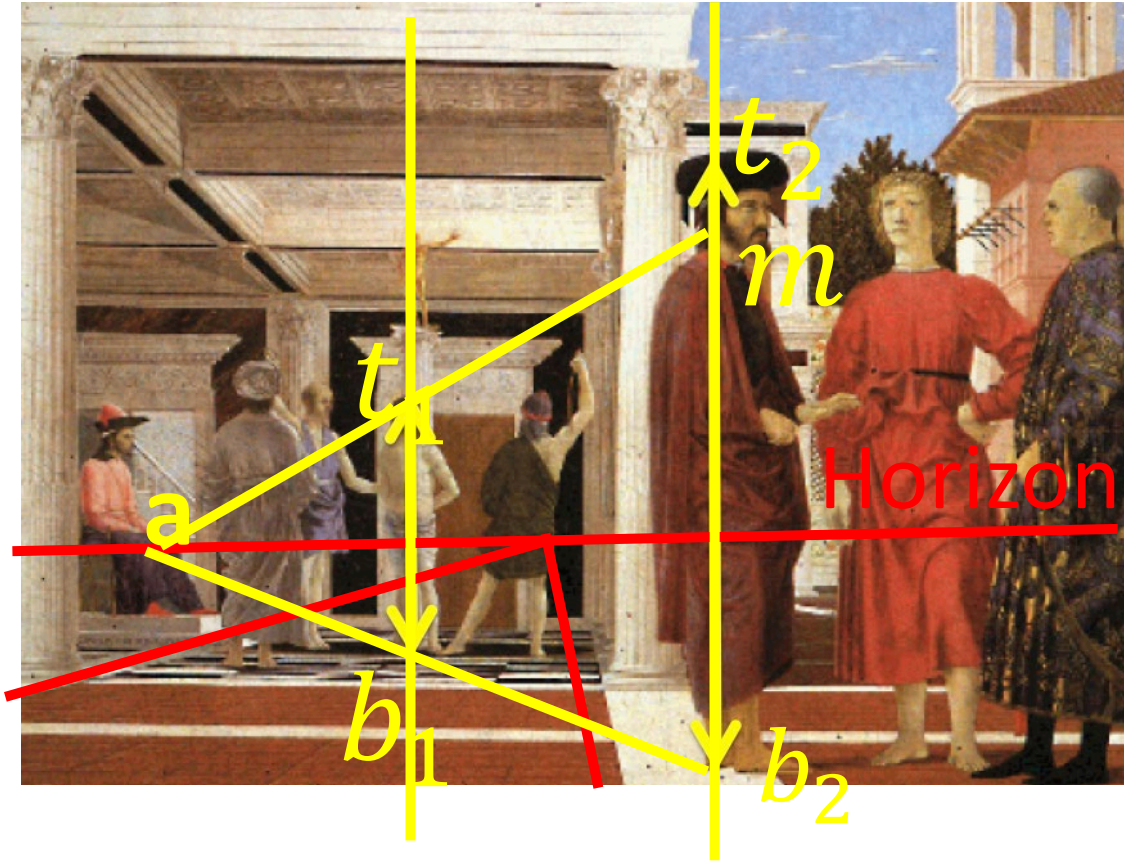


4. But we want  $B_2T_2$  in the world.  
How can we compute the ratio  $B_2T_2/B_2M$  ?

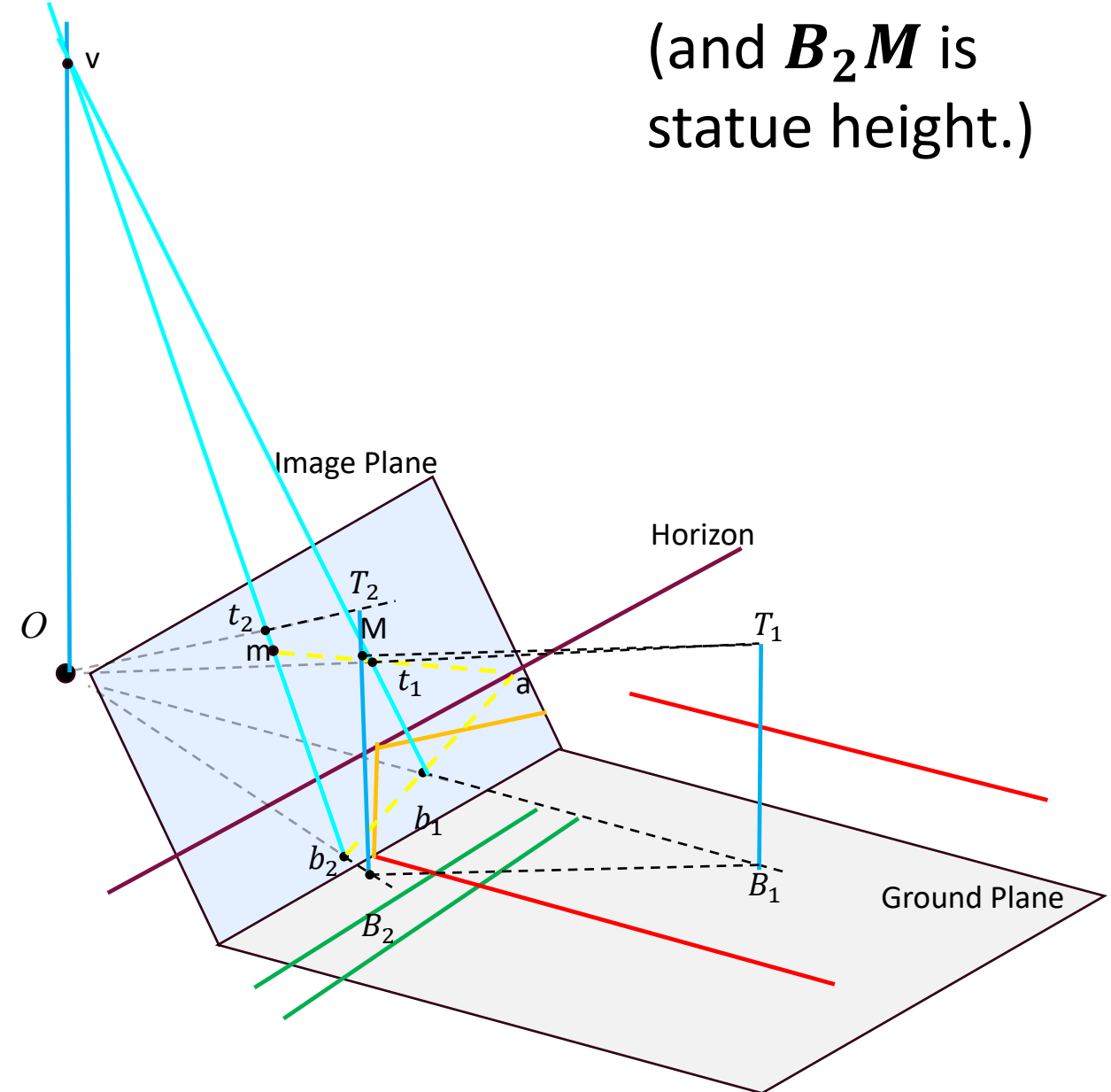
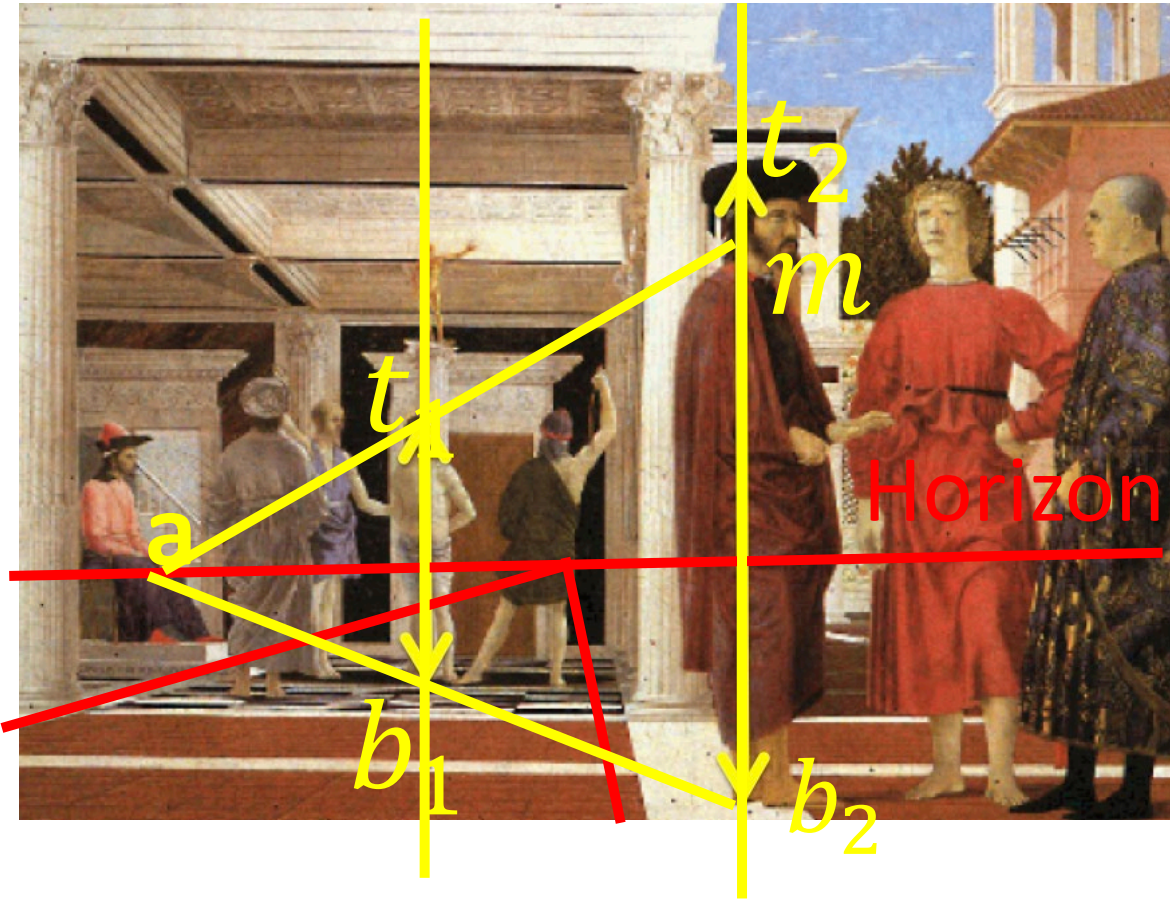




4. Only if we know a vanishing point in the vertical direction. Let  $b_1t_1$  and  $b_2m$  intersect at a vertical VP  $v$  (might be at  $\infty$  or not) .



5. Then cross ratio of  $\{v, t_2, m, b_2\}$  = ratio  $B_2T_2 : B_2M$   
 (and  $B_2M$  is statue height.)



# Single View Metrology via Cross Ratios

- If we know the vanishing point for a direction, we can compute any ratio along this direction!
- We can transfer lengths among parallel line segments in the world using knowledge of the vanishing point for their direction.
- All of this without explicitly computing any focal length, intrinsics, homographies etc.!

We can do “image forensics” on paintings or old photos!

See also: ZH Sec 8.7,  
ZH example 8.25

## How to Detect Faked Photos

*Techniques that analyze the consistency of elements within an image can help to determine whether it is real or manipulated.*



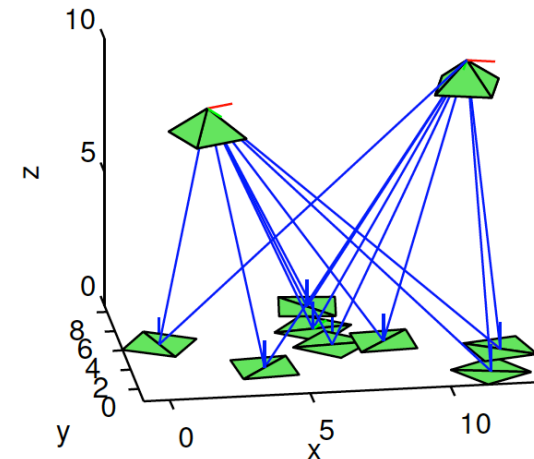
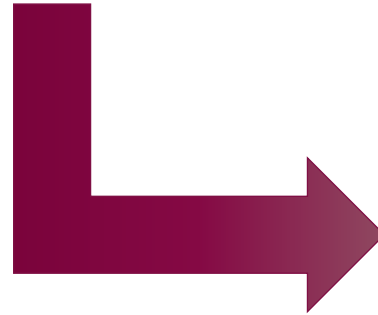
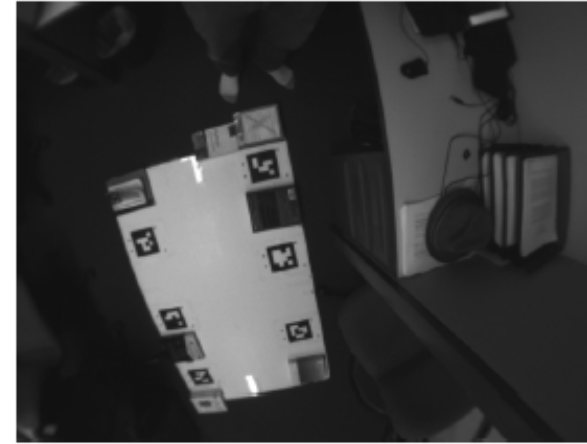
# Fixing Camera 6DoF Pose Estimation w.r.t world plane

(given homography  $H$  & camera intrinsics  $K$ )

# Camera 6DOF pose

- A camera's rotation (3DOF) and translation (3DOF) jointly is called its 6-DOF pose.
- “camera pose” estimation = finding the “extrinsics matrix”

Recall: we know how to find homography w.r.t. a planar pattern in the world.





# Recall: homography gives pose (given intrinsics K)

## Pose from Projective Transformation

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane  $Z = 0$ .

Then the transformation reads

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ W \end{pmatrix}$$

$$\text{And } r_3 = r_1 \times r_2$$



# But actually, not quite!

- According to the previous slide  $K(r_1 \ r_2 \ T) = H$ , or in other words,

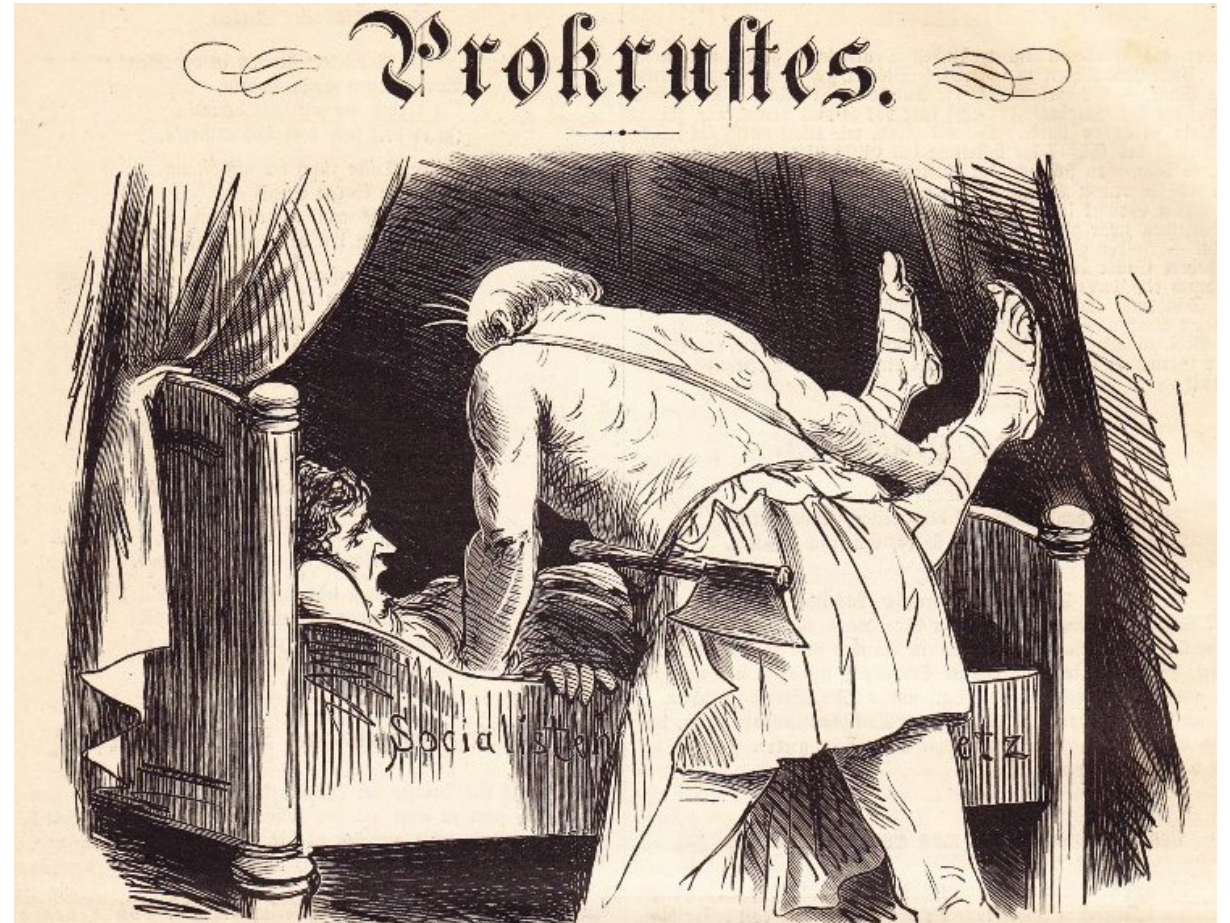
$$K^{-1}H = (r_1, r_2, T) \text{ and } r_3 = r_1 \times r_2$$

If only life were so simple!

- Problem: when we ***estimate*** homographies (e.g. through solving linear systems with  $2n$  equations from  $n \geq 4$  point correspondences), and then compute  $K^{-1}H$ , we aren't guaranteed to find a *valid*  $r_1$  and  $r_2$  pair. i.e. an orthonormal pair.
  - So, we need to find a way to first “correct”  $(K^{-1}H)_{3 \times 3}$  to get orthonormal  $r_1$  and  $r_2$ . Often called the “Procrustes”, or “special orthogonal (SO) Procrustes” problem.
  - And we must solve this in real-time for robotics applications, so preferably an inexpensive approach.

# The macabre Greek legend of Procrustes

We are trying to get every  $(K^{-1}H)$  “traveler” to fit the “bed” of valid rotation matrices by stretching it or chopping it off.



Let us name the columns of  $K^{-1}H$ :

$$K^{-1}H = (h'_1 \quad h'_2 \quad h'_3)$$

We seek orthogonal  $r_1$  and  $r_2$  that are the closest to  $h'_1$  and  $h'_2$ . The solution to this problem is given by the Singular Value Decomposition.

We find the orthogonal matrix  $R$  that is the closest to  $(h'_1 \quad h'_2 \quad h'_1 \times h'_2)$ :

$$\arg \min_{R \in SO(3)} \|R - (h'_1 \quad h'_2 \quad h'_1 \times h'_2)\|_F^2$$

# Kabsch algorithm for Procrustes

$$\arg \min_{R \in SO(3)} \|R - \begin{pmatrix} h'_1 & h'_2 & h'_1 \times h'_2 \end{pmatrix}\|_F^2$$

If the SVD of

$$\begin{pmatrix} h'_1 & h'_2 & h'_1 \times h'_2 \end{pmatrix} = USV^T$$

then the solution is

$$R = U \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(UV^T) \end{pmatrix} V^T$$

The diagonal matrix is inserted to guarantee that  $\det(R) = 1$ .

To find the translation :  $T = h'_3 / \|h'_1\|$

(In case original columns were not even unit norm)

proof in supp readings- Kabsch-Algorithm-RT-from-H-proof.pdf. We will also prove it in the next class.



# Full Kabsch algorithm for finding pose via homography

1. Find  $H$  up to a scale factor from the point correspondences

2. Compute  $H' = K^{-1}H$ . Let  $H'$ 's columns be  $(a \ b \ c)$

3. Minimize

$$\|(a \ b \ c) - \lambda(r_1 \ r_2 \ T)\|_F$$

w.r.t.  $\lambda \in \mathbb{R}, r_1, r_2, T \in \mathbb{R}^3$

s.t.  $r_1^T r_2 = 0$  and  $\|r_1\| = \|r_2\| = 1$

Let

$$(a \ b) = U_{3 \times 2} \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix} V_{2 \times 2}^T.$$

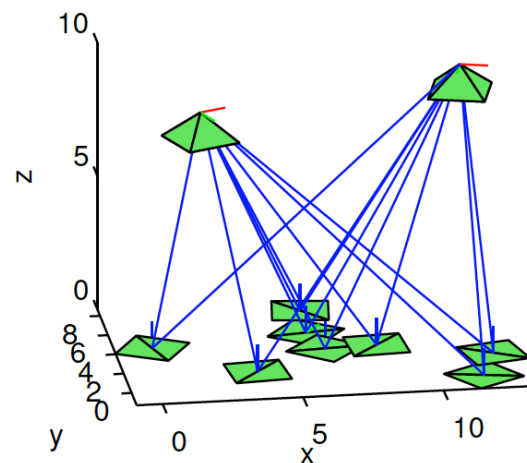
Then

$$(r_1 \ r_2) = U_{3 \times 2} V_{2 \times 2}^T \quad \text{and} \quad \lambda = \frac{s_1 + s_2}{2}$$

Alternative to running Kabsch including the 3<sup>rd</sup> column  $c = h'_1 \times h'_2$  as on last slide

4.  $T = c/\lambda$  and  $R = (r_1 \ r_2 \ r_1 \times r_2)$ . Scale  $R$  to have determinant 1 if needed.

So now, camera pose (actually) known w.r.t world plane!



CIS 5800

# Machine Perception

Instructor: Lingjie Liu

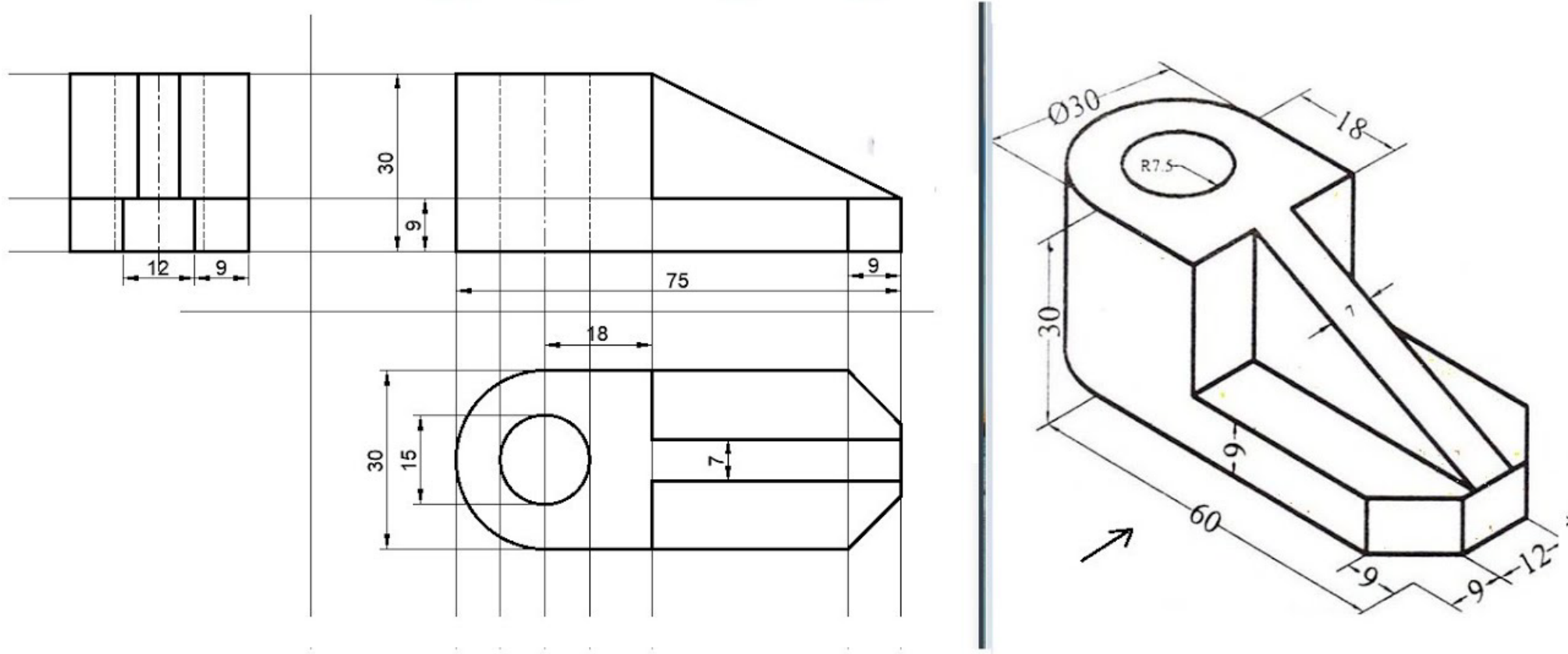
Lec 8: Feb 17, 2026

# Administrivia

- HW1, due this Friday.
- HW2, will release next Monday.

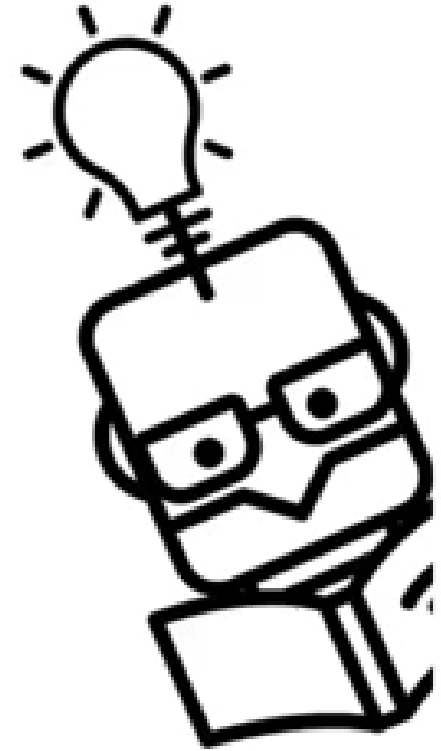


# Aside: Engineers Use “Orthographic” Projections



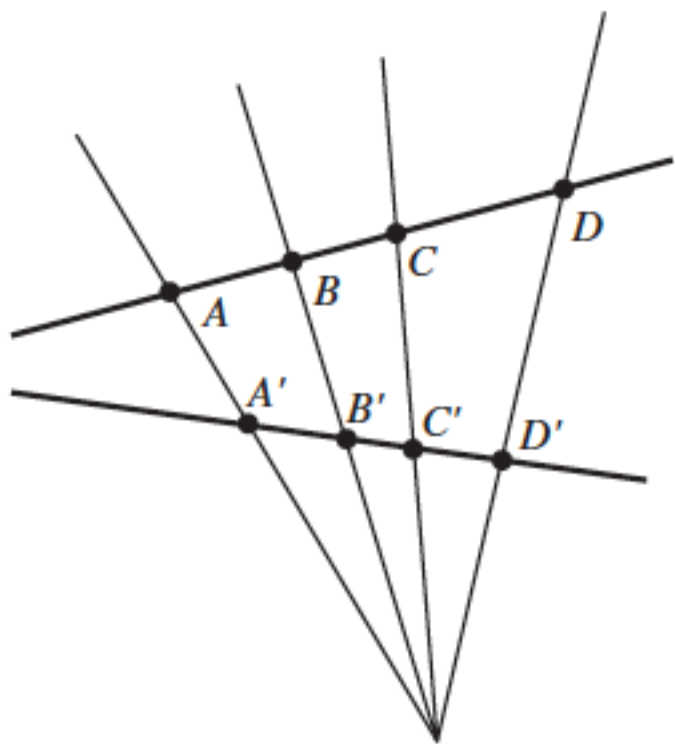
- Image formed by rays orthogonal to the image plane, hence “orthographic”. Field of view limited by size of image plane.
- In perspective projection which happens in eyes and cameras, rays pass through a camera center. Much larger field of view.

# Review of SVD (Singular Value Decomposition)





# Recap: Cross Ratios of Collinear Points



Given four points  $A, B, C, D$ , we define the cross-ratio of their distances as

$$CR(A, B, C, D) = \frac{AC}{AD} : \frac{BC}{BD}.$$

$CR(A, B, C, D)$  remains invariant under projective transformations

$$\frac{AC}{AD} : \frac{BC}{BD} = \frac{A'C'}{A'D'} : \frac{B'C'}{B'D'}$$

Denote  $\mathbf{x}'_i = (x'_i, 1)^T$  and  $\mathbf{x}_i = (x_i, 1)^T$ . Suppose  $\mathbf{x}'_i = H\mathbf{x}_i$ . Note  $|H| = \det(H)$ . Then

$$x'_i - x'_j = |\mathbf{x}'_i \ \mathbf{x}'_j| = |\lambda_i H\mathbf{x}_i \ \lambda_j H\mathbf{x}_j| = \lambda_i \lambda_j |H(\mathbf{x}_i \ \mathbf{x}_j)|$$

$$= \lambda_i \lambda_j |H| |\mathbf{x}_i \ \mathbf{x}_j| = \lambda_i \lambda_j \det(H) (x_i - x_j)$$

Note: C  
but it is  
e.g., CR

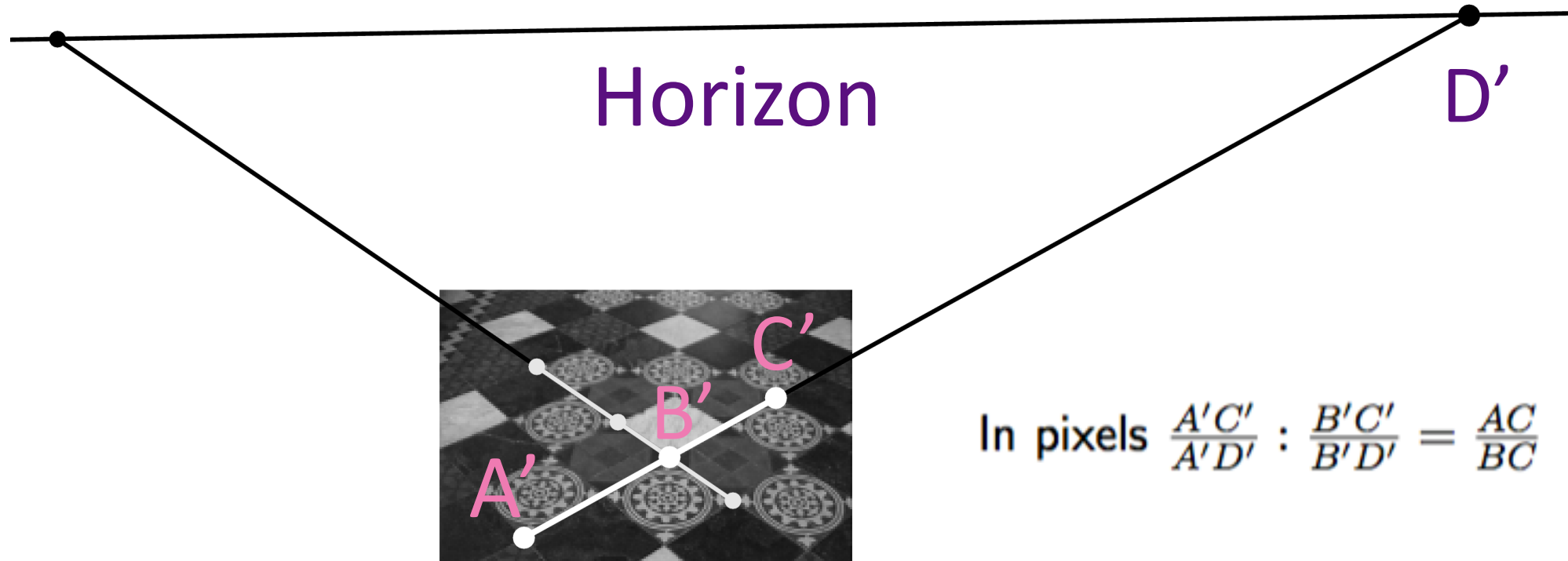


Pappus (290-350 AD)



Girard Desargues (1591-1661)

# Recap: Cross ratios for metrology

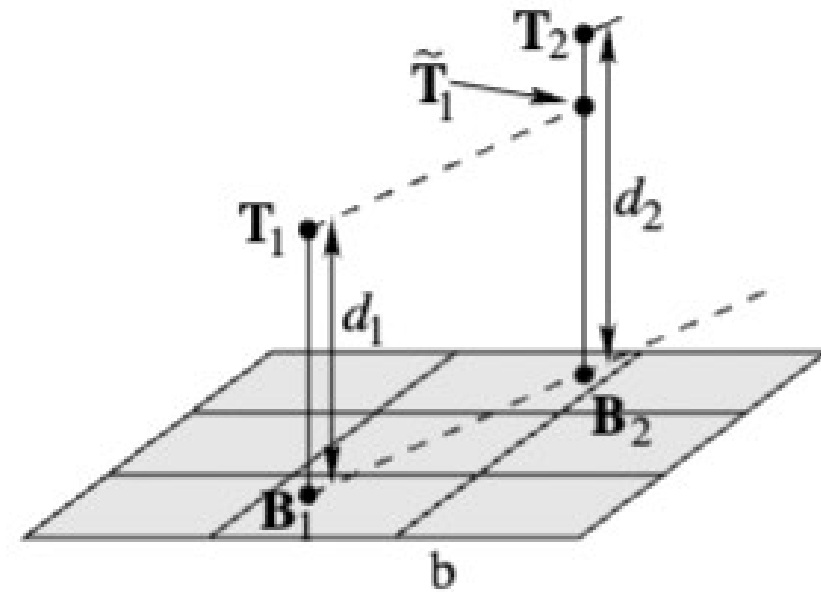
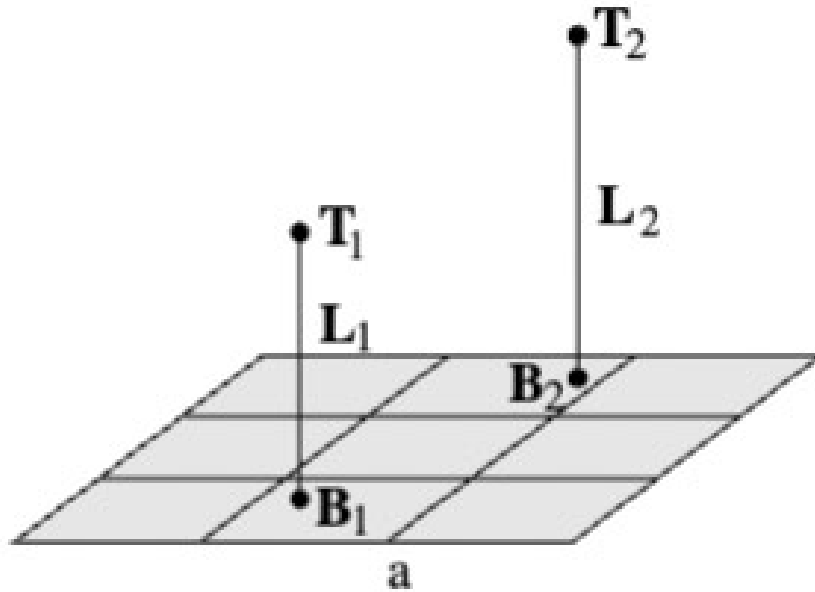


When a point  $D$  is at infinity, the cross-ratio becomes a ratio !

$$\frac{AC}{AD} : \frac{BC}{BD} = \frac{AC}{BC} \quad \left( \text{Think } \frac{AC}{\infty} : \frac{BC}{\infty} = \frac{AC}{BC} \times \frac{\infty}{\infty} = \frac{AC}{BC} \right)$$

# Recap: Length transfer in 3D

- In the real 3-D world, you can compare one object with known length to another to “transfer” its length. This is what you do with a ruler, for example.



How to do this in an image?

# Recap: Pose From Homography

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane  $Z = 0$ .

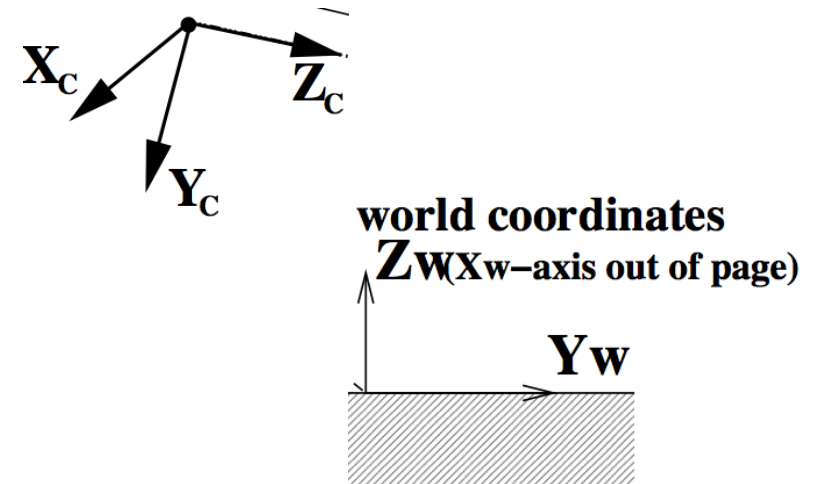
Then the transformation reads

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \underbrace{\begin{pmatrix} r_1 & r_2 & T \end{pmatrix}} \begin{pmatrix} X \\ Y \\ W \end{pmatrix}$$

The planar homography  
 $H: \mathbb{P}^2 \rightarrow \mathbb{P}^2$

Computing the homography can tell us how the camera (and therefore, e.g. a robot attached to the camera) is oriented w.r.t. to a world plane! (assuming known  $K$ )

Q: Where do you get  $r_3$  from though? **A:  $r_3 = r_1 \times r_2$**



# Recap: “Fixing pose from homography”

- $K^{-1}H$  is supposed to be our estimate of  $[r_1, r_2, T]$ , but there are no guarantees that  $r_1$  and  $r_2$  are in fact orthonormal, as they should be.
- So we construct the “Procrustes” problem of finding the “closest valid rotation matrix  $R$ ” to  $K^{-1}H$

We find the orthogonal matrix  $R$  that is the closest to  $(h'_1 \ h'_2 \ h'_1 \times h'_2)$ :

$$\arg \min_{R \in SO(3)} \|R - (h'_1 \ h'_2 \ h'_1 \times h'_2)\|_F^2$$



# Recap: Kabsch algorithm for Procrustes

$$\arg \min_{R \in SO(3)} \|R - \begin{pmatrix} h'_1 & h'_2 & h'_1 \times h'_2 \end{pmatrix}\|_F^2$$

If the SVD of

$$\begin{pmatrix} h'_1 & h'_2 & h'_1 \times h'_2 \end{pmatrix} = USV^T$$

then the solution is

$$R = U \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(UV^T) \end{pmatrix} V^T$$

The diagonal matrix is inserted to guarantee that  $\det(R) = 1$ .

To find the translation :  $T = h'_3 / \|h'_1\|$

(In case original columns were not even unit norm)

proof in supp readings- Kabsch-Algorithm-RT-from-H-proof.pdf. We will also prove it in the next class.

# References: The Kabsch papers

*Acta Cryst.* (1976). **A32**, 922

**A solution for the best rotation to relate two sets of vectors.** By WOLFGANG KABSCH, *Max-Planck-Institut für Medizinische Forschung*, 6900 Heidelberg, Jahnstrasse 29, Germany (BRD)

(Received 23 February 1976; accepted 12 April 1976)

A simple procedure is derived which determines a best rotation of a given vector set into a second vector set by minimizing the weighted sum of squared deviations. The method is generalized for any given metric constraint on the transformation.

In various crystallographic situations the problem arises of finding a best rotation to fit a given atomic arrangement to approximately measured coordinates. Examples have been given by McLachlan (1972) and Diamond (1976). Diamond determines the best unconstrained transformation between the two sets of coordinates and factorizes it into a symmetric and an orthogonal matrix. McLachlan finds a best rotation between the two sets of coordinates by an iterative process. The analysis below shows that a direct solution is also possible, despite the non-linear character of the problem.

Let  $\mathbf{x}_n$  and  $\mathbf{y}_n$  ( $n = 1, 2, \dots, N$ ) be two given vector sets and  $w_n$  the weight corresponding to each pair  $\mathbf{x}_n, \mathbf{y}_n$ . The problem is then to find an orthogonal matrix  $\mathbf{U} = (u_{ij})$  which minimizes the function

$$E = \frac{1}{2} \sum_n w_n (\mathbf{U} \mathbf{x}_n - \mathbf{y}_n)^2 \quad (1)$$

subject to the constraints

$$\sum_k u_{ki} u_{kj} - \delta_{ij} = 0 \quad (2)$$

where the  $\delta_{ij}$  are the elements of the unit matrix. A translation, if admitted, can always be removed from the problem by shifting the centroids of the vector sets to the origin.

Introducing a symmetric matrix  $\mathbf{L} = (l_{ij})$  of Lagrange multipliers an auxiliary function (see, for example, Brand, 1958)

$$F = \frac{1}{2} \sum_{ij} l_{ij} \left( \sum_k u_{ki} u_{kj} - \delta_{ij} \right) \quad (3)$$

is constructed and added to  $E$  to form the Lagrangian function

$$G = E + F. \quad (4)$$

Since for each different condition (2) an independent number  $l_{ij}$  is available, the constrained minimum of  $E$  is now included among the free minima of  $G$ . A free minimum of  $G$  can only occur where

$$\frac{\partial G}{\partial u_{ij}} = \sum_k u_{ik} \left( \sum_n w_n x_{nk} x_{nj} + l_{ij} \right) - \sum_n w_n y_{ni} x_{nj} = 0 \quad (5)$$

and

$$\frac{\partial^2 G}{\partial u_{im} \partial u_{ij}} = -\delta_{im} \left( \sum_n w_n x_{ni} x_{nj} + l_{ij} \right) \quad (6)$$

are the elements of a positive definite matrix.  $x_{ni}$  and  $y_{ni}$  are the  $i$ th components of the vectors  $\mathbf{x}_n$  and  $\mathbf{y}_n$ .

Let

$$r_{ij} = \sum_n w_n y_{ni} x_{nj} \quad (7)$$

and

$$s_{ij} = \sum_n w_n x_{ni} x_{nj} \quad (8)$$

be the elements of a matrix  $\mathbf{R} = (r_{ij})$  and a symmetric matrix  $\mathbf{S} = (s_{ij})$ , respectively. For  $i = m = 1$  from equation (6), a minimum of the Lagrangian function  $G$  requires that  $\mathbf{S} + \mathbf{L}$  is positive definite, and – by rewriting equation (5) – that

$$\mathbf{U} \cdot (\mathbf{S} + \mathbf{L}) = \mathbf{R}. \quad (9)$$

## SHORT COMMUNICATIONS

923

The problem is now to find a symmetric matrix  $\mathbf{L}$  of Lagrange multipliers such that  $\mathbf{U}$  is orthogonal. If both sides of (9) are multiplied by their transposed matrices, the unknown orthogonal matrix  $\mathbf{U}$  can be eliminated:

$$\mathbf{U}(\tilde{\mathbf{S}} + \mathbf{L})\mathbf{U}(\mathbf{S} + \mathbf{L}) = (\tilde{\mathbf{S}} + \mathbf{L})\tilde{\mathbf{U}}\mathbf{U}(\mathbf{S} + \mathbf{L}) = (\mathbf{S} + \mathbf{L})(\mathbf{S} + \mathbf{L}) = \tilde{\mathbf{R}}\mathbf{R}. \quad (10)$$

Since  $\tilde{\mathbf{R}}\mathbf{R}$  is a symmetric positive definite matrix the positive eigenvalues  $\mu_k$  and the corresponding eigenvectors  $\mathbf{a}_k$  can be found by well established procedures. Since  $\mathbf{S} + \mathbf{L}$  is symmetric and positive definite also, it is evident from (10) that it must have the same normalized eigenvectors  $\mathbf{a}_k$  and the positive eigenvalues  $1/\mu_k$ .

It can be easily verified that the Lagrange multipliers are then

$$l_{ij} = \sum_k \frac{1}{\mu_k} a_{ki} a_{kj} - s_{ij} \quad (11)$$

where  $a_{ki}$  denotes the  $i$ th component of  $\mathbf{a}_k$ . The effect of the orthogonal matrix  $\mathbf{U}$  on these eigenvectors  $\mathbf{a}_k$  is determined from (9) and defines unit vectors  $\mathbf{b}_k$  as

$$\mathbf{b}_k = \frac{1}{\sqrt{\mu_k}} \mathbf{U}(\mathbf{S} + \mathbf{L})\mathbf{a}_k = \frac{1}{\sqrt{\mu_k}} \tilde{\mathbf{R}}\mathbf{a}_k. \quad (12)$$

The orthogonal matrix  $\mathbf{U}$  is finally constructed as

$$u_{ij} = \sum_k b_{ki} a_{kj} \quad (13)$$

and the problem to find the constraint minimum of the function  $E$  is solved.

Sometimes it may happen that all of the vectors  $\mathbf{x}_n$  or  $\mathbf{y}_n$  lie in a plane. Then one of the eigenvalues of  $\tilde{\mathbf{R}}\mathbf{R}$ , e.g.  $\mu_3$ , will be zero. In this case a complete set of vectors  $\mathbf{a}_k, \mathbf{b}_k$  is constructed by setting

$$\mathbf{a}_3 = \mathbf{a}_1 \times \mathbf{a}_2 \quad \mathbf{b}_3 = \mathbf{b}_1 \times \mathbf{b}_2. \quad (14)$$

Note that the procedure described in this article can be easily extended to vector spaces of higher dimensions.

It is possible also to replace the constraints of equation (2) by the more general constraints

$$\tilde{\mathbf{U}}\mathbf{U} = \mathbf{M}, \quad (15)$$

where  $\mathbf{M}$  is a symmetric and positive definite matrix. If  $\mathbf{B}$  is any specific solution of (15), it is easy to prove that all possible other solutions  $\mathbf{U}$  of that equation can be written as

$$\mathbf{U} = \mathbf{V} \cdot \mathbf{B} \quad (16)$$

with an orthogonal matrix  $\mathbf{V}$ . If the initial vector set  $\mathbf{x}_n$  is transformed into  $\mathbf{x}_n' = \mathbf{B}\mathbf{x}_n$ , then this problem is reduced to minimizing  $E' = \frac{1}{2} \sum_n w_n (\mathbf{V}\mathbf{x}_n' - \mathbf{y}_n)^2$  with the constraint  $\tilde{\mathbf{V}}\mathbf{V} = \mathbf{1}$ .

I would like to thank Dr K. C. Holmes for reading the manuscript.

## References

DIAMOND, R. (1976). *Acta Cryst.* **A32**, 1–10.  
McLACHLAN, A. D. (1972). *Acta Cryst.* **A28**, 656–657.  
BRAND, L. (1958). *Advanced Calculus*, pp. 194–198. New York: John Wiley.

*Acta Cryst.* (1978). **A34**, 827–828

**A discussion of the solution for the best rotation to relate two sets of vectors.** By W. KABSCH, *Max-Planck-Institut für Medizinische Forschung, Abteilung Biophysik, Jahnstrasse 29, 6900 Heidelberg, Federal Republic of Germany*

(Received 3 April 1978; accepted 13 April 1978)

A method is discussed for obtaining the best proper rotation to relate two sets of vectors.

The simple procedure for obtaining the best rotation to relate two sets of vectors described in an earlier paper (Kabsch, 1976) has been used in processing oscillation films (Kabsch, 1977), for the determination of non-crystallographic symmetry elements (Kabsch, Gast, Schulz & Leberman, 1977), and for a comparison of macromolecules. In the last application an improper rotation was sometimes obtained from the procedure (Nyburg & Yuen, 1977). The purpose of this communication is to show how a best proper rotation can always be obtained from the procedure.

Let  $\mathbf{x}_n$  and  $\mathbf{y}_n$  ( $n = 1, \dots, N$ ) be two given vector sets and  $w_n$  the weights corresponding to each pair  $\mathbf{x}_n, \mathbf{y}_n$ . All possible orthogonal matrices  $\mathbf{U}$  for which the function

$$E = \frac{1}{2} \sum_n w_n (\mathbf{U} \mathbf{x}_n - \mathbf{y}_n)^2 \quad (1)$$

has an extremal point must obey [see equation (9) of Kabsch, 1976]

$$\mathbf{U}(\mathbf{S} + \mathbf{L}) = \mathbf{R}. \quad (2)$$

Writing  $x_{ni}$  and  $y_{ni}$  for the  $i$ th components of the vectors  $\mathbf{x}_n$  and  $\mathbf{y}_n$  the matrices  $\mathbf{R}$  and  $\mathbf{S}$  are defined as

$$\mathbf{R} = (r_{ij}) = \left( \sum_n w_n y_{ni} x_{nj} \right) \quad (3)$$

$$\mathbf{S} = (s_{ij}) = \left( \sum_n w_n x_{ni} x_{nj} \right). \quad (4)$$

$\mathbf{L} = (l_{ij})$  is a symmetric matrix of Lagrange multipliers which is determined from the equation

$$(\mathbf{S} + \mathbf{L})(\mathbf{S} + \mathbf{L}) = \tilde{\mathbf{R}}\mathbf{R}. \quad (5)$$

$\tilde{\mathbf{R}}\mathbf{R}$  is a known symmetric positive definite matrix whose positive eigenvalues  $\mu_k$  and eigenvectors  $\mathbf{a}_k$  can be determined by standard procedures. The general solution of (5) is of the form

$$(\mathbf{S} + \mathbf{L}) = (s_{ij} + l_{ij}) = \left( \sum_k a_{ki} a_{kj} \cdot \sigma_k \sqrt{\mu_k} \right), \quad (6)$$

where  $a_{ki}$  denotes the  $i$ th component of  $\mathbf{a}_k$  and the arbitrary quantities  $\sigma_k$  can only assume the values  $\pm 1$ . If an eigenvalue  $\mu_k$  is degenerate the eigenvector  $\mathbf{a}_k$  of  $\tilde{\mathbf{R}}\mathbf{R}$  cannot be determined uniquely. However,  $\mathbf{S} + \mathbf{L}$  will not be affected by this ambiguity if all its eigenvalues of the magnitude  $\sqrt{\mu_k}$  have identical signs. The final construction of all orthogonal matrices  $\mathbf{U} = (u_{ij})$  for which  $E$  assumes an extremal point is given by

$$u_{ij} = \sum_k b_{ki} a_{kj}, \quad (7)$$

where  $b_{ki}$  is the  $i$ th component of the vector

$$\mathbf{b}_k = \mathbf{U} \mathbf{a}_k = \mathbf{U}(\mathbf{S} + \mathbf{L})\mathbf{a}_k / (\sigma_k \sqrt{\mu_k}) = \mathbf{R} \mathbf{a}_k / (\sigma_k \sqrt{\mu_k}). \quad (8)$$

The residual  $E$  at each extremal point is

$$\begin{aligned} E &= \frac{1}{2} \sum_n w_n (\mathbf{U} \mathbf{x}_n - \mathbf{y}_n)^2 = \frac{1}{2} \sum_n w_n (\mathbf{x}_n^2 + \mathbf{y}_n^2) \\ &\quad - \sum_n w_n \mathbf{y}_n \cdot (\mathbf{U} \mathbf{x}_n) \\ &= \frac{1}{2} \sum_n w_n (\mathbf{x}_n^2 + \mathbf{y}_n^2) - \sum_n w_n \sum_k \left( \mathbf{b}_k \cdot \mathbf{y}_n \right) (\mathbf{x}_n \cdot \mathbf{a}_k) \\ &= \frac{1}{2} \sum_n w_n (\mathbf{x}_n^2 + \mathbf{y}_n^2) - \sum_k \mathbf{b}_k \cdot (\mathbf{R} \mathbf{a}_k) \\ &= \frac{1}{2} \sum_n w_n (\mathbf{x}_n^2 + \mathbf{y}_n^2) - \sum_k \sigma_k \sqrt{\mu_k}. \end{aligned} \quad (9)$$

828

## SHORT COMMUNICATIONS

The maximum of  $E$  is obtained if all  $\sigma_k$  are  $-1$ . The minimum of  $E$  is obtained if all  $\sigma_k$  are  $+1$ , which agrees with the result of Kabsch (1976).

It has also been shown in Kabsch (1976) that  $\mathbf{S} + \mathbf{L}$  must be positive definite at the minimum of  $E$ . Hence, from (2) the determinants of the two matrices,  $\mathbf{U}$  and  $\mathbf{R}$ , must have the same signs.

In the case that  $\det(\mathbf{R}) > 0$ , the orthogonal matrix  $\mathbf{U}$  corresponding to the minimum of  $E$  will be a proper rotation. In the case that  $\det(\mathbf{R}) < 0$ , an improper rotation will be obtained at the minimum of  $E$  (Nyburg & Yuen, 1977). From (9), the smallest residual  $E$  corresponding to a best true rotation is then obtained if  $\sigma_1 = \sigma_2 = +1$  and  $\sigma_3 = -1$  assuming that  $\mu_3$  is the smallest eigenvalue of  $\tilde{\mathbf{R}}\mathbf{R}$  (three-dimensional vector space). Note that if the smallest eigenvalue is degenerate a best rotation cannot be determined uniquely in the case  $\det(\mathbf{R}) < 0$ .

Finally, it might be worth mentioning that this procedure can be generalized to find a best unitary matrix to relate two sets of vectors in the complex finite-dimensional vector space.

Summarizing the above results, the following procedure

for obtaining a best proper rotation in a three-dimensional vector space is suggested.

(a) Remove any translation between the two given vector sets  $\mathbf{x}_n, \mathbf{y}_n$  and determine  $E_0 = \frac{1}{2} \sum_n w_n (\mathbf{x}_n^2 + \mathbf{y}_n^2)$  and  $\mathbf{R}$ .

(b) Form  $\tilde{\mathbf{R}}\mathbf{R}$ , determine eigenvalues  $\mu_k$  and the mutually orthogonal eigenvectors  $\mathbf{a}_k$  and sort so that  $\mu_1 \geq \mu_2 \geq \mu_3$ . Set  $\mathbf{a}_3 = \mathbf{a}_1 \times \mathbf{a}_2$  to be sure to have a right-handed system.

(c) Determine  $\mathbf{R} \mathbf{a}_k$  ( $k = 1, 2, 3$ ), normalize the first two vectors to obtain  $\mathbf{b}_2, \mathbf{b}_3$  and set  $\mathbf{b}_1 = \mathbf{b}_3 \times \mathbf{b}_2$ . This will also take care of the case  $\mu_3 = \mu_1 = 0$ .

(d) Form  $\mathbf{U}$  according to (7) to obtain the best rotation. Set  $\sigma_3 = -1$  if  $\mathbf{b}_3 \cdot (\mathbf{R} \mathbf{a}_3) < 0$ , otherwise  $\sigma_3 = +1$ . The residual error is then  $E = E_0 - \sqrt{\mu_1} - \sqrt{\mu_2} - \sigma_3 \sqrt{\mu_3}$ .

## References

KABSCH, W. (1976). *Acta Cryst.* **A32**, 922–923.  
KABSCH, W. (1977). *J. Appl. Cryst.* **10**, 426–429.  
KABSCH, W., GAST, W. H., SCHULZ, G. E. & LEBERMAN, R. (1977). *J. Mol. Biol.* **117**, 999–1012.  
NYBURG, S. C. & YUEN, P. S. (1977). Private communication.

[Kabsch '76: A solution for the best rotation to relate two sets of vectors](#)

Kabsch '78: A discussion of the solution for the best rotation to relate two sets of vectors

# Recap: Full Kabsch algorithm for finding pose via homography

1. Find  $H$  up to a scale factor from the point correspondences

2. Compute  $H' = K^{-1}H$ . Let  $H'$ 's columns be  $(h'_1 \ h'_2 \ h'_3)$

3. Minimize

$$\|(h'_1 \ h'_2 \ h'_3) - \lambda \begin{pmatrix} r_1 & r_2 & T \end{pmatrix}\|_F$$

w.r.t.  $\lambda \in \mathbb{R}, r_1, r_2, T \in \mathbb{R}^3$

s.t.  $r_1^T r_2 = 0$  and  $\|r_1\| = \|r_2\| = 1$

Let

$$(h'_1 \ h'_2) = U_{3 \times 2} \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix} V_{2 \times 2}^T.$$

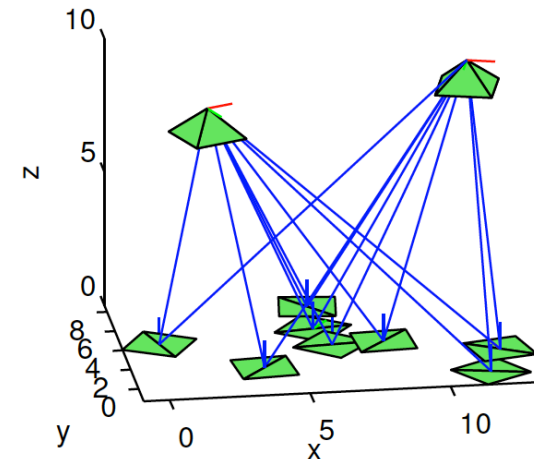
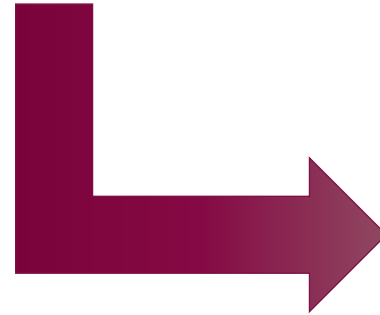
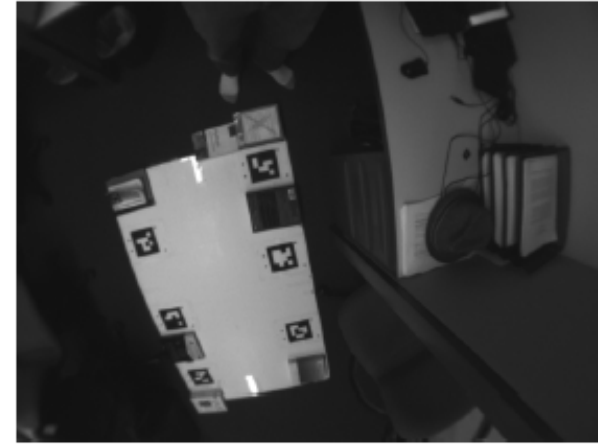
Then

$$\begin{pmatrix} r_1 & r_2 \end{pmatrix} = U_{3 \times 2} V_{2 \times 2}^T \quad \text{and} \quad \lambda = \frac{s_1 + s_2}{2}$$

Alternative to running Kabsch including the 3<sup>rd</sup> column  $c = h'_1 \times h'_2$  as on last slide

4.  $T = c/\lambda$  and  $R = \begin{pmatrix} r_1 & r_2 & r_1 \times r_2 \end{pmatrix}$ . Scale  $R$  to have determinant 1 if needed.

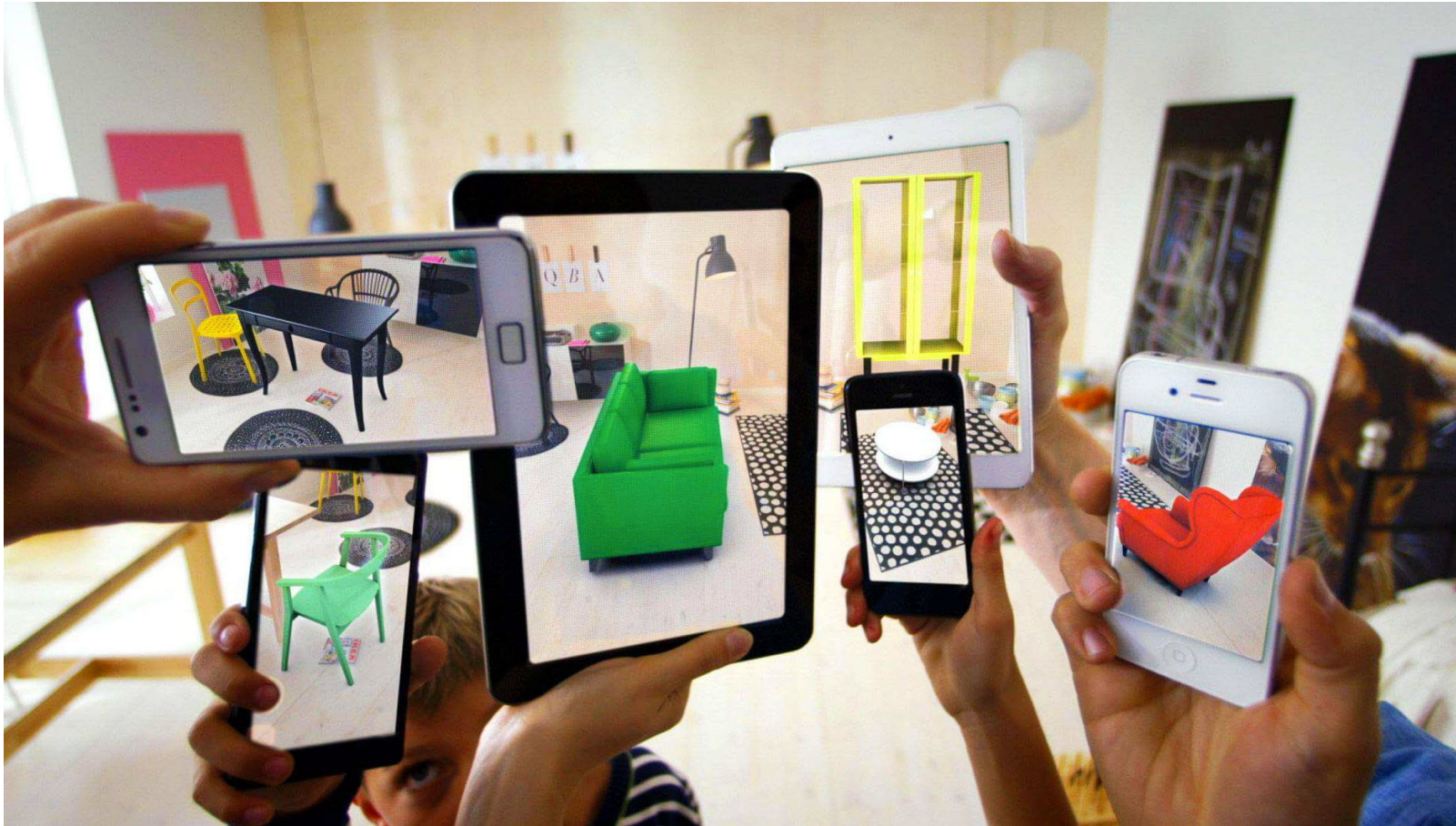
# So now, we finally have valid camera pose/extrinsics!





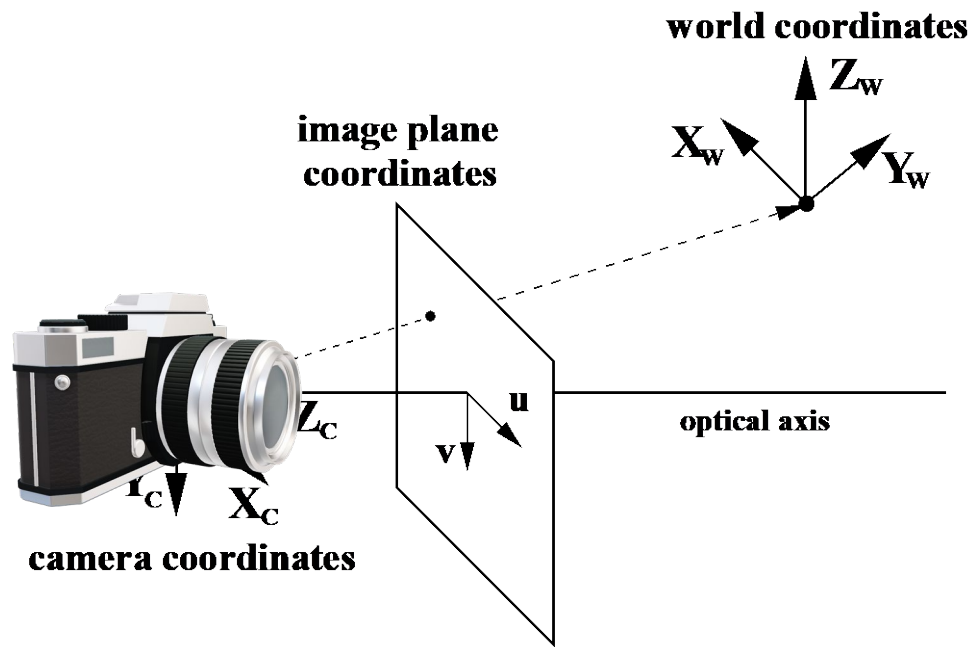
# Applying extrinsics (and intrinsics) for 3D shape projection

Can do AR-style projection of a 3D object onto the world plane once the full extrinsics and intrinsics are known! **You will do this in HW2.**



# Application of pose: projecting a solid shape into the world

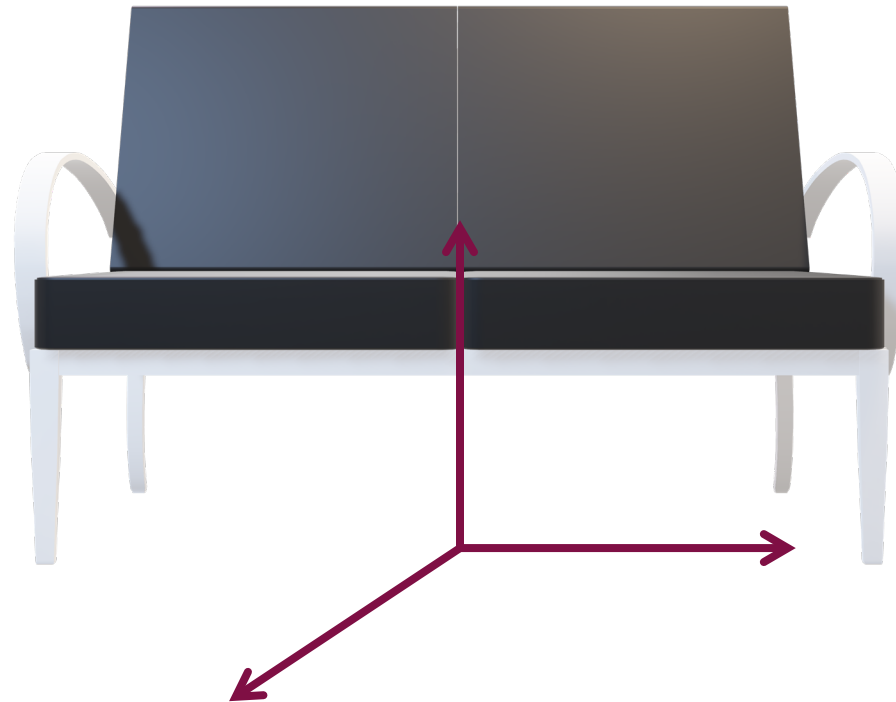
- Our normal projection equations tell us how world points in world coordinates project onto a camera, given camera pose  $(R, T)$  and intrinsics  $K$



$$x \sim K[R|t]X_w$$

# Application of pose: projecting a solid shape into the world

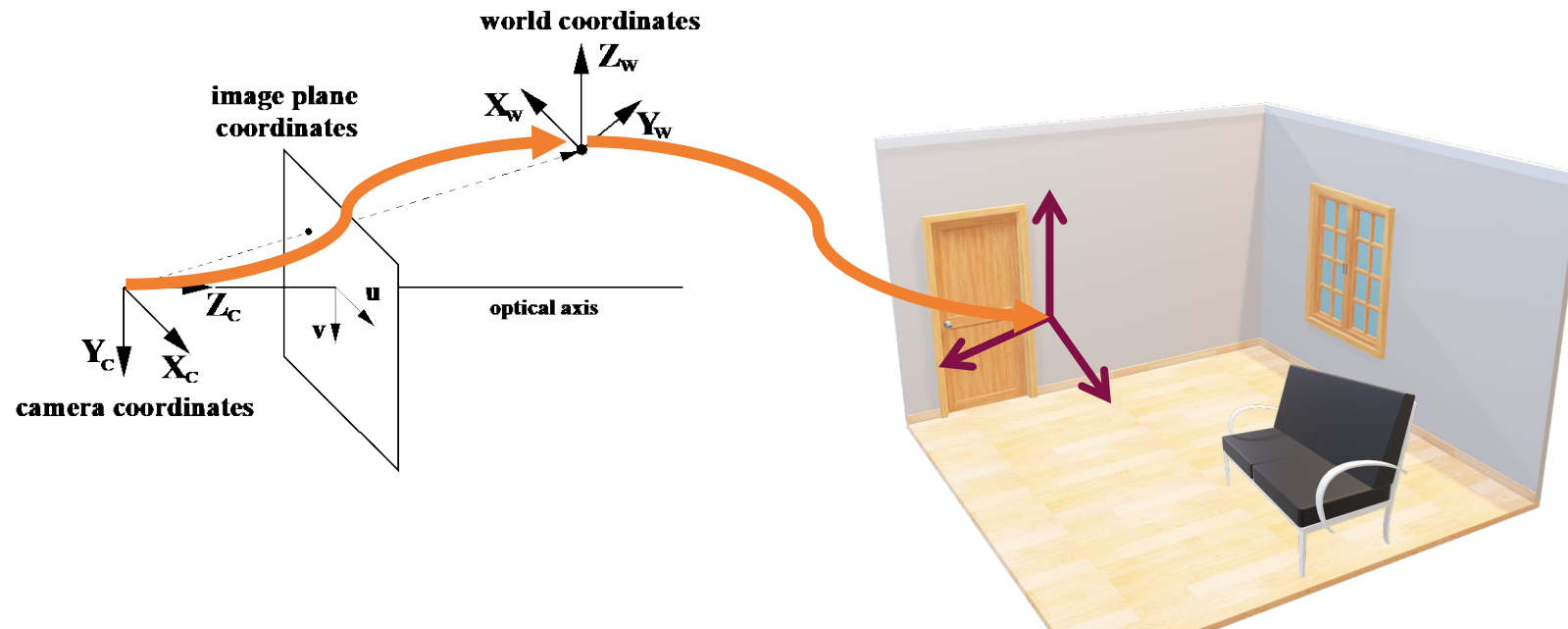
- Suppose the shape is expressed by the positions of points  $X_s$  in a "shape-coordinate system"



Coordinate system attached to the object

# Application of pose: projecting a solid shape into the world

- First find  $R_{sw}, t_{sw}$  that convert object-centric coordinates  $X_s$  into world-centric coordinates  $X_w = R_{sw}X_s + t_{sw}$  to place the object at the right place in the world. (Think: what do  $R_{sw}$  and  $t_{sw}$  mean exactly?)
- Then just render the object points at  $K[R|t]X_w$







# Recap: Projective Transformation = Homography = Collineation=Projectivity

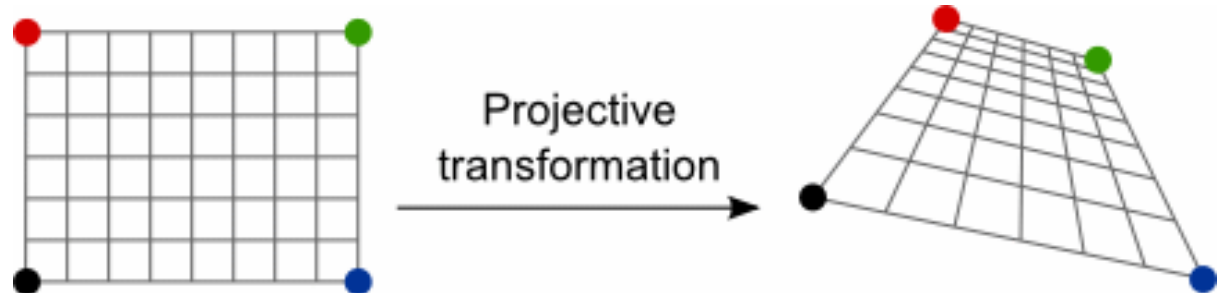
## Definition

A **projective transformation** is any invertible matrix transformation  $\mathbb{P}^2 \rightarrow \mathbb{P}^2$ .

A projective transformation  $H$  maps  $p$  to  $p' \sim Hp$

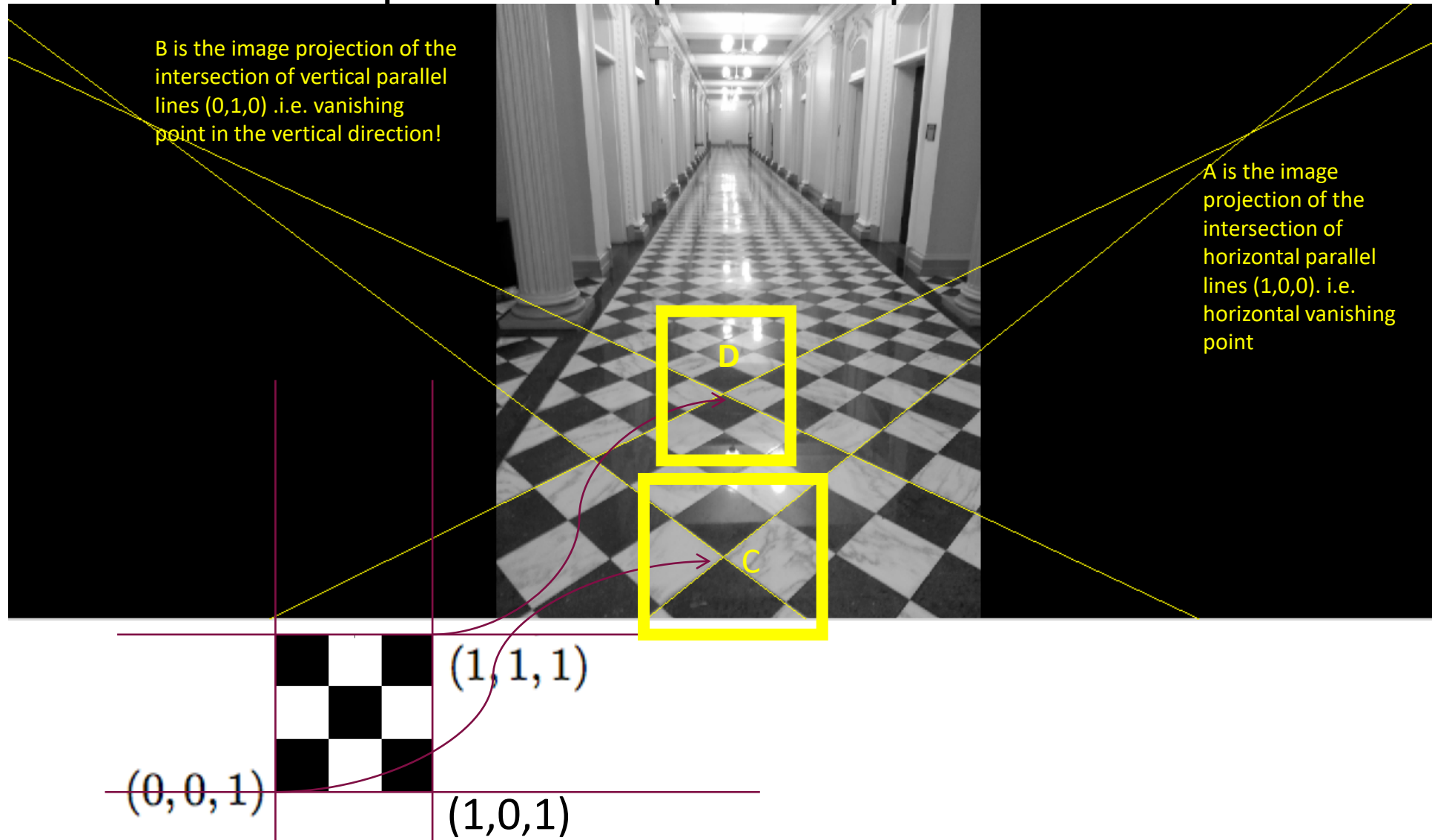
Invertibility means that  $\det(H) \neq 0$  and that there exists  $\lambda \neq 0$  such that  $\lambda p' = Hp$

Observe that we will write either  $p' \sim Hp$  or  $\lambda p' = Hp$

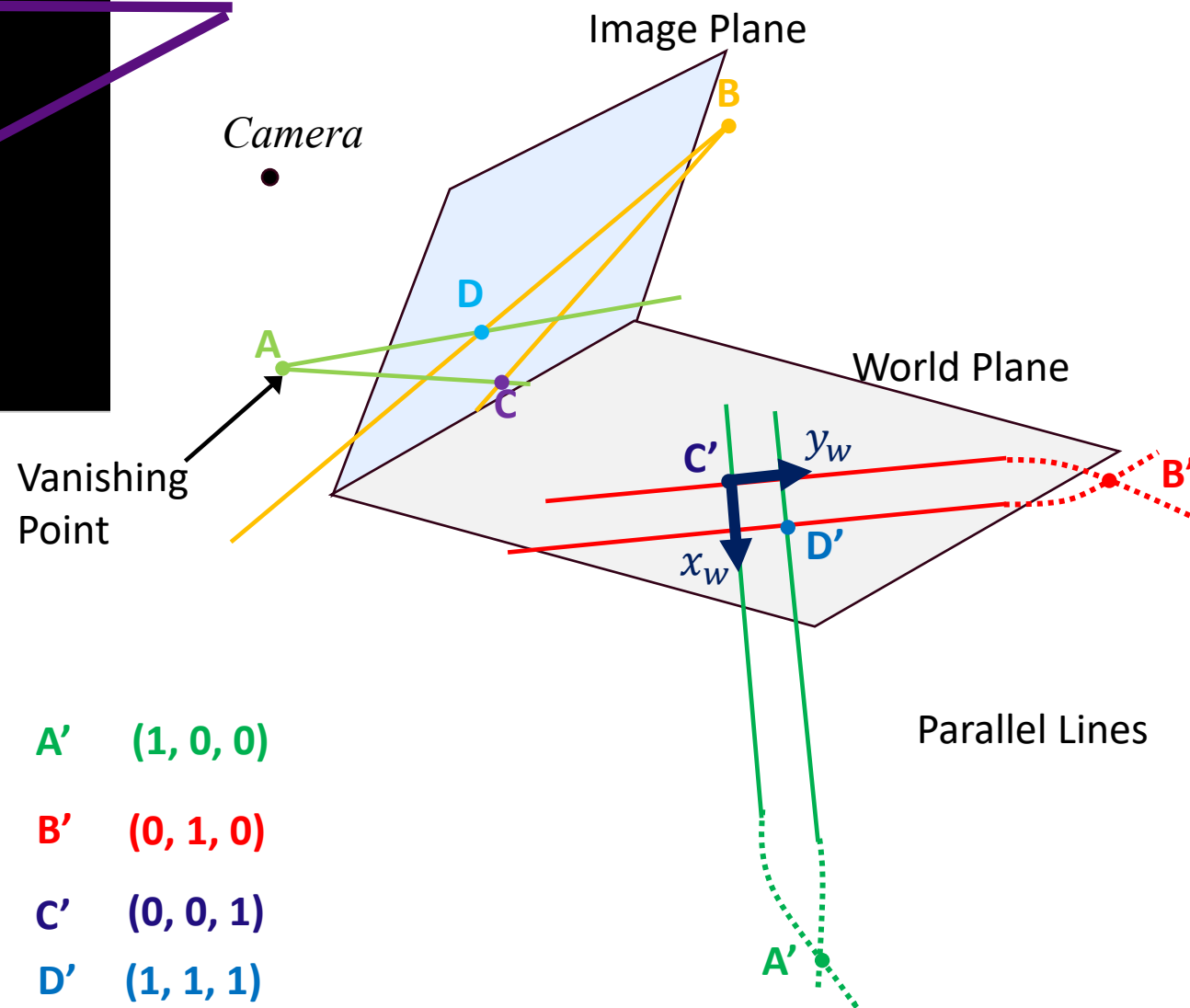
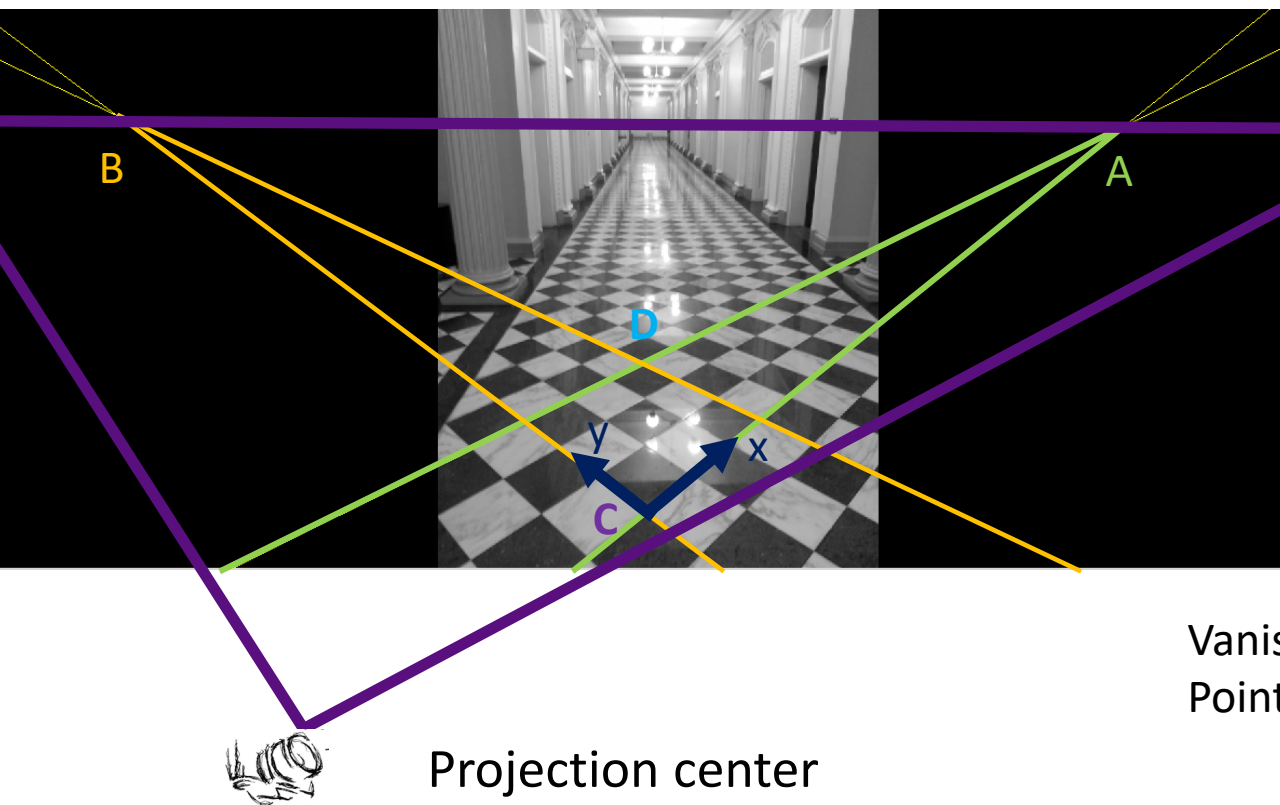


# Recap: Computing Homography from 4 Point Correspondences

Note: makes sense, because after all,  $H$  has 8 degrees of freedom, and each 2D point correspondence pins down 2DOF.



# Recap: Computing Homography from 4 Point Correspondences



Correspondences:  
( $P^2 \rightarrow P^2$ )

|   |   |                       |    |           |
|---|---|-----------------------|----|-----------|
| A | a | $\longleftrightarrow$ | A' | (1, 0, 0) |
| B | b | $\longleftrightarrow$ | B' | (0, 1, 0) |
| C | c | $\longleftrightarrow$ | C' | (0, 0, 1) |
| D | d | $\longleftrightarrow$ | D' | (1, 1, 1) |

Parallel Lines

# Pose from Point Correspondences, the Perspective N Point Problem (PnP)

# Localization by observing known 3D points from the world?



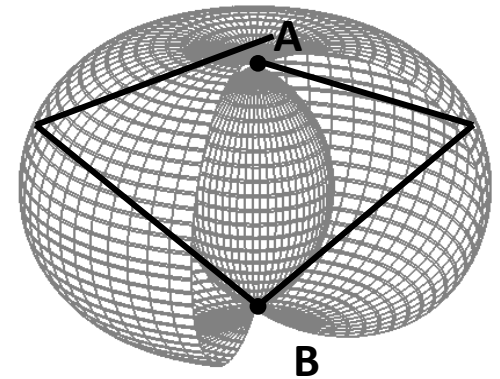
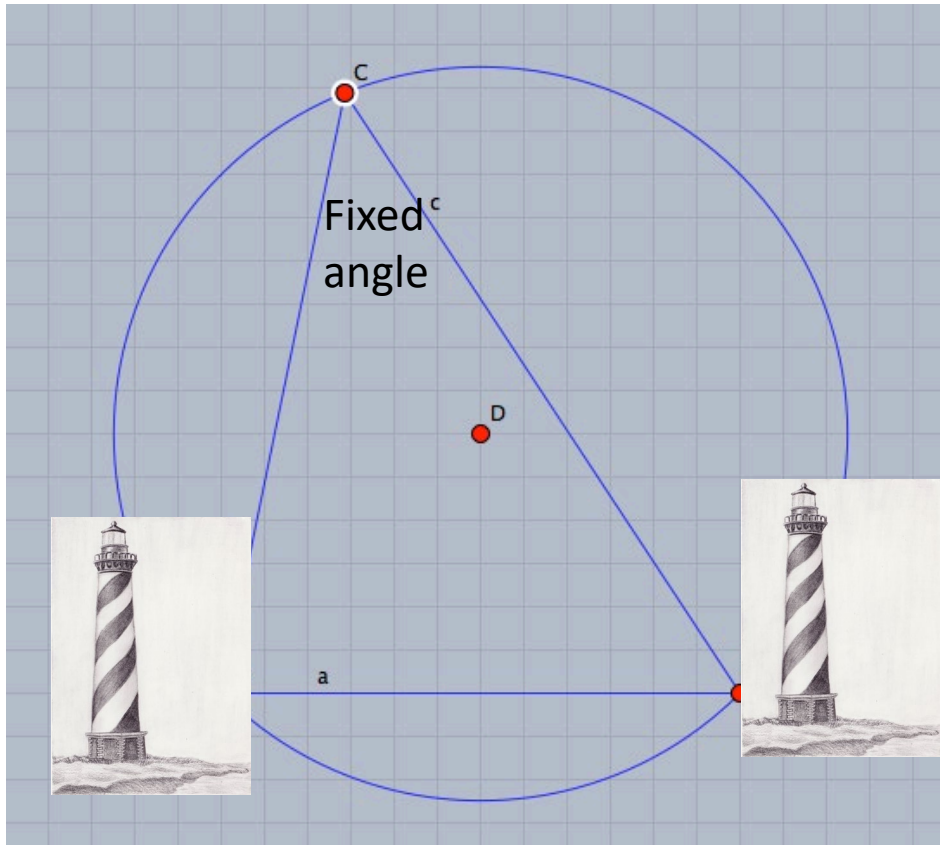
A real problem for autonomous cars, for example!  
GPS:  $\sim$  a few feet accuracy. Just not good enough.

**Instead, autonomous cars rely on 3D maps of the world to localize!**



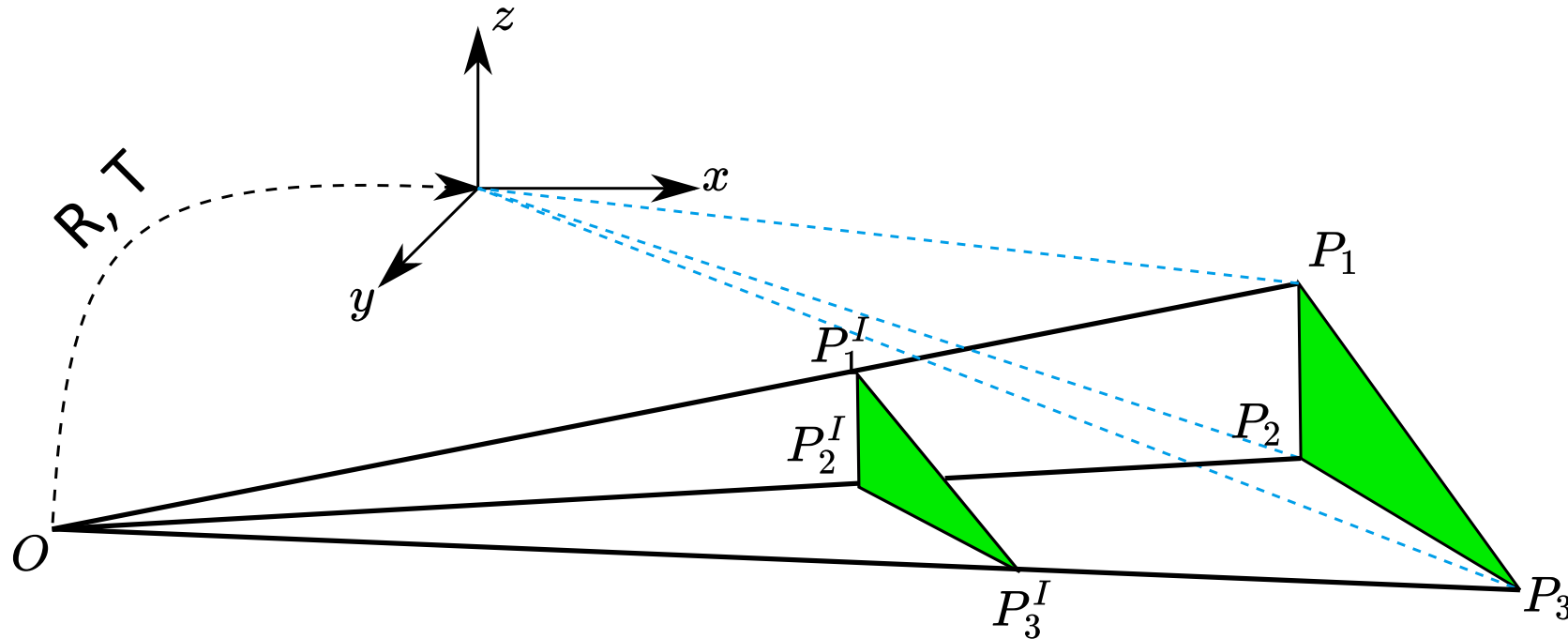
# Navigation with “bearings” from 2 points

If I observe two lighthouses being some fixed angle  $\theta$  apart, where am I?



Two points are not sufficient.  
Camera could be anywhere on an upper semicircle in 2D.  
Worse in 3D: anywhere on a toroid in 3D. Need more points!

# The *Perspective* 3-Point Problem



- Given the point correspondences, find camera pose  $R, T$

What are the differences from 4-Point Algorithm?



# Simplified 3-Point Problem w. 3D Camera Coordinates

A triangle's world 3D coordinates  $P_i \in \mathbb{R}^3$  are known, and its camera-centric 3D coordinates  $P_i^c \in \mathbb{R}^3$  are known

**The 3D->3D 3-Point Problem:** Find camera pose  $R, T$  such that

$$P_1^c = RP_1 + T$$

$$P_2^c = RP_2 + T$$

$$P_3^c = RP_3 + T$$

Full camera coordinates may come from depth cameras, but otherwise, we only have pixel coordinates.

**Plan:** starting from only pixel coordinates, first reduce the problem to 3D->3D.



CIS 5800

# Machine Perception

Instructor: Lingjie Liu

Lec 9: Feb 19, 2026

# Recap: Localization by observing known 3D points from the world?

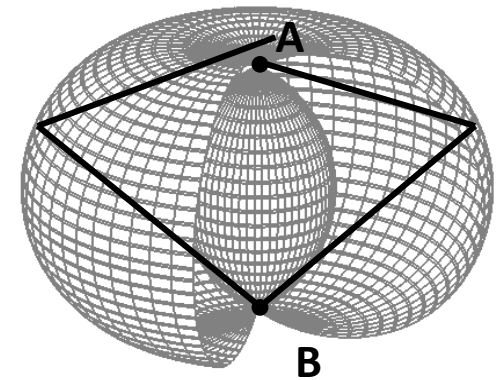
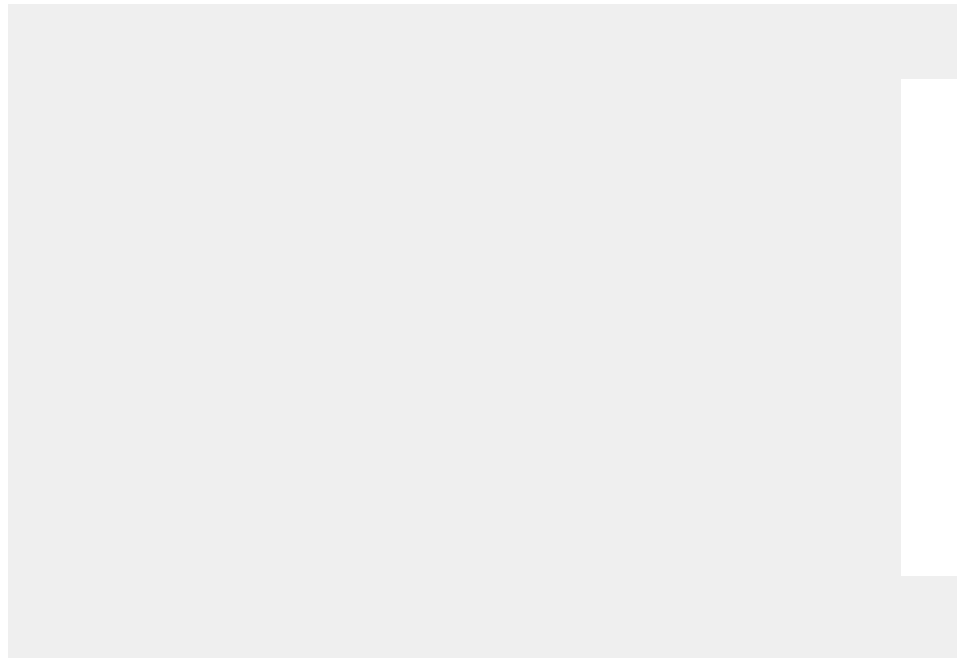
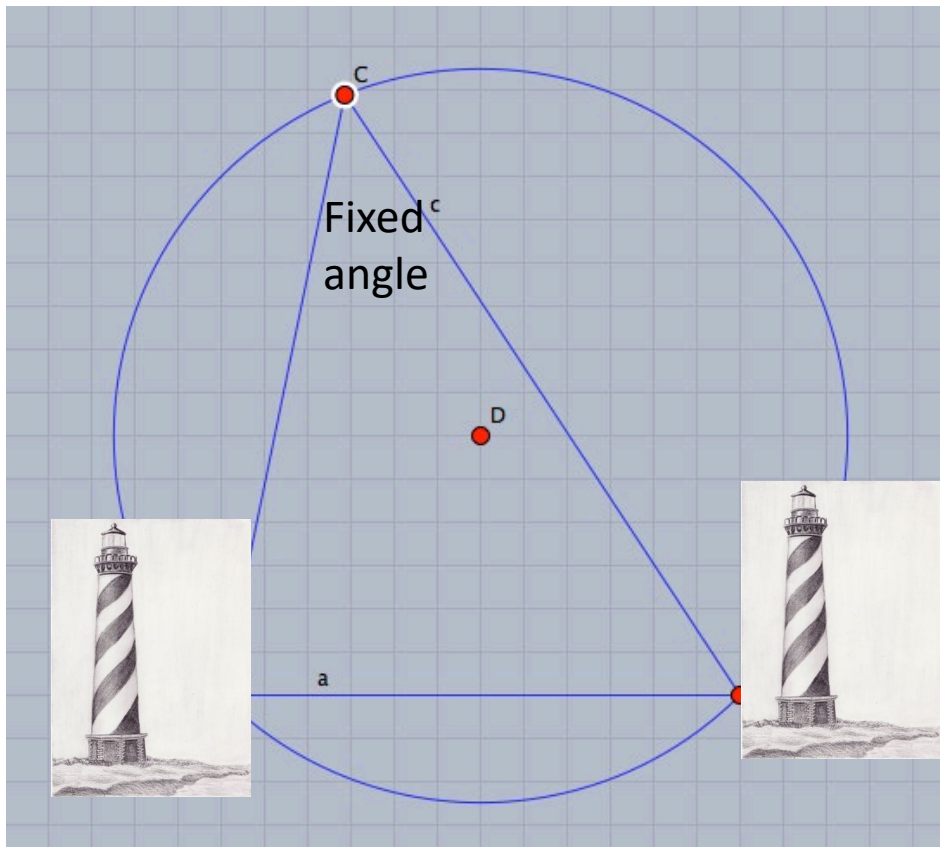


A real problem for autonomous cars, for example!  
GPS:  $\sim$  a few feet accuracy. Just not good enough.

**Instead, autonomous cars rely on 3D maps of the world to localize!**

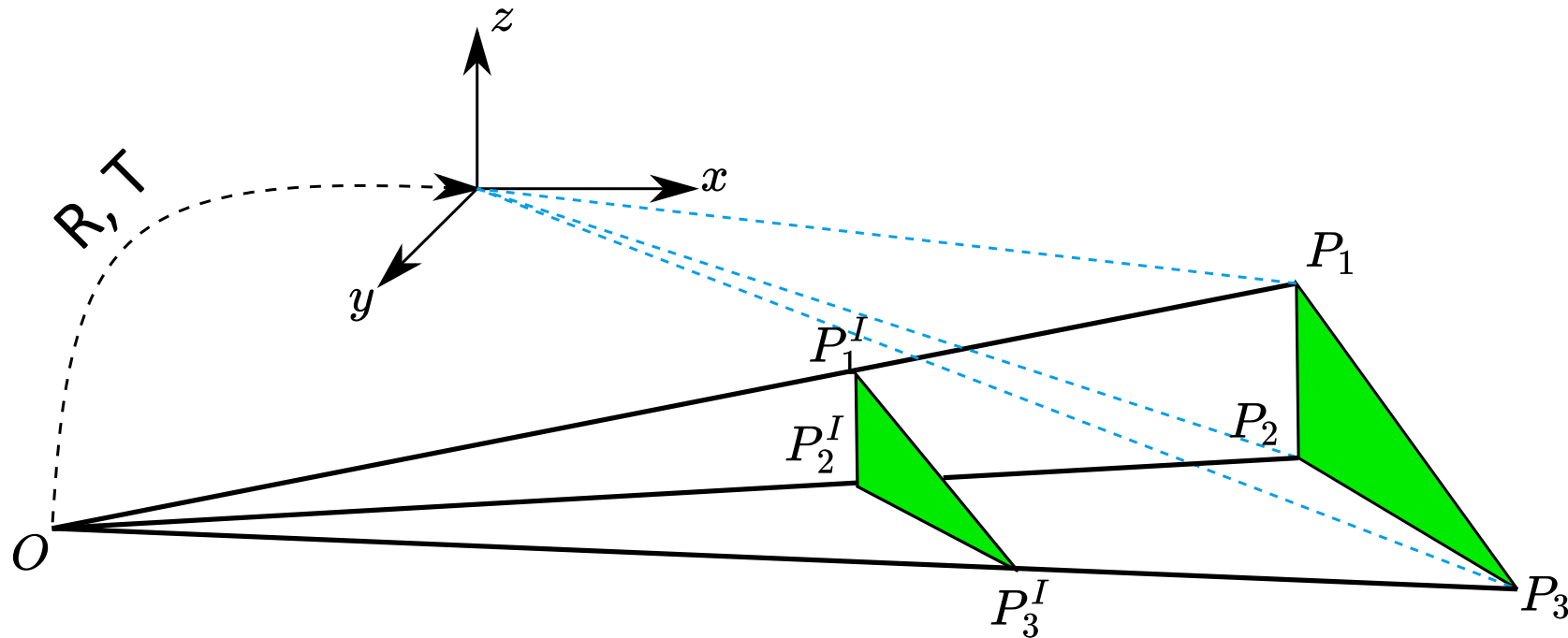
# Recap: Navigation with “bearings” from 2 points

If I observe two lighthouses being some fixed angle  $\theta$  apart, where am I?



Two points are not sufficient.  
Camera could be anywhere on an upper semicircle in 2D.  
Worse in 3D: anywhere on a toroid in 3D. Need more points!

# Recap: The *Perspective* 3-Point Problem



- Given the point correspondences, find camera pose  $R, T$

What are the differences from 4-Point Algorithm?

# Recap: Simplified 3-Point Problem w. 3D Camera Coordinates

A triangle's world 3D coordinates  $P_i \in \mathbb{R}^3$  are known, and its camera-centric 3D coordinates  $P_i^c \in \mathbb{R}^3$  are known

**The 3D->3D 3-Point Problem:** Find camera pose  $R, T$  such that

$$P_1^c = RP_1 + T$$

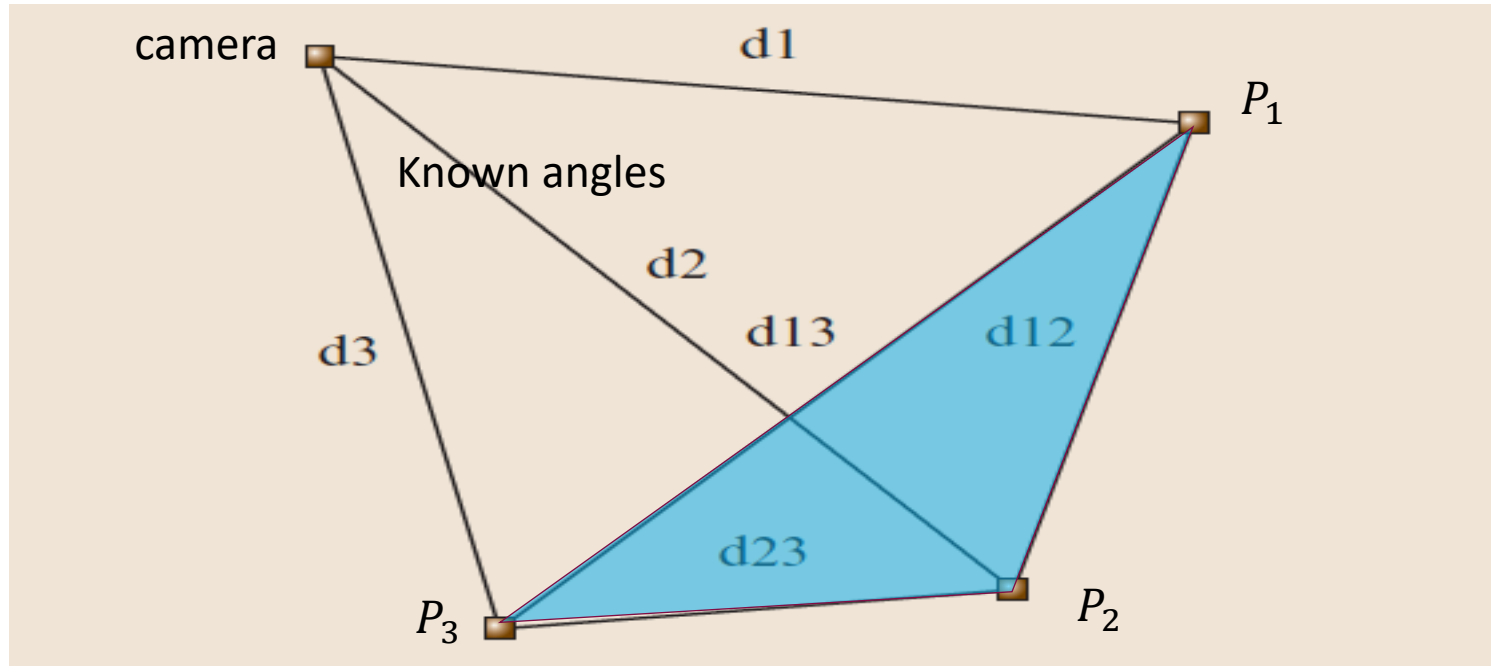
$$P_2^c = RP_2 + T$$

$$P_3^c = RP_3 + T$$

Full camera coordinates may come from depth cameras, but otherwise, we only have pixel coordinates.

**Plan:** starting from only pixel coordinates, first reduce the problem to 3D->3D.

# P3P from Pixels



**A triangle's world coordinates  $P_i$  are known, and its pixel coordinates are known**



# Pixels → “Calibrated coordinates”

RGB images only provide pixel coordinates  $u, v$  for each vertex, not camera-centric 3D coordinates. Convert these to:

**“Calibrated coordinates”:**  $\mathbf{p}_i \sim K^{-1}(u_i \quad v_i \quad 1)^T$

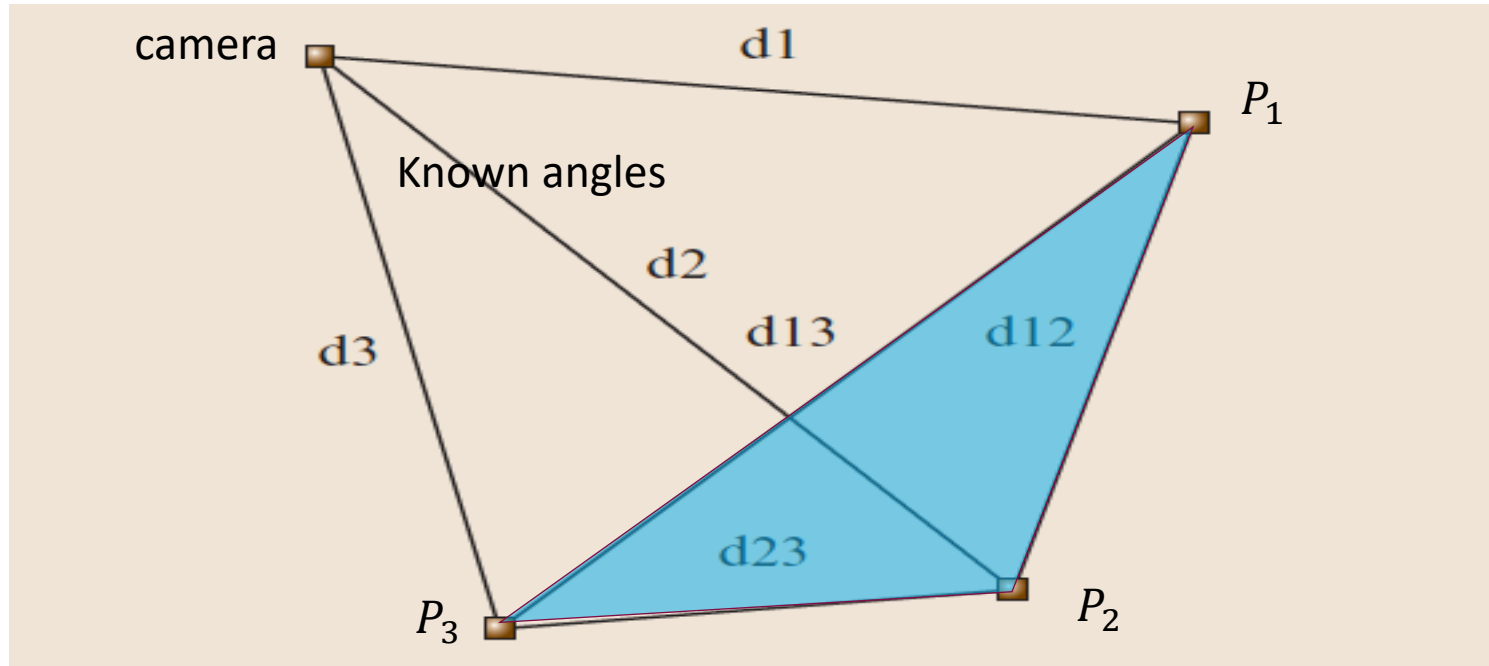
These are essentially image plane coordinates with principal point as origin, and focal length set to 1.

**Euclidean interpretation:**  $\mathbf{p}_i$  is a vector in camera coordinates, originating from the camera center and pointing toward the 3D point corresponding to pixel coordinates  $(u_i, v_i)$ .

# Calibrated coordinates + distance = Camera-centric 3D

- After writing as calibrated coordinates, camera-centric 3D coordinates is just one step away: finding the distance of the 3D point along that direction, i.e. “depth”
- To reduce P3P to 3D->3D as we had planned, we will need to find those depths. Indeed, that is the first step of P3P.

# P3P from ~~Pixels~~ Calibrated Coordinates



A triangle's world coordinates  $P_i$  are known, and its camera-centric calibrated coordinates  $p_i$  are known

**The P3P problem:** Find  $\lambda_i, R, T$  such that

$$\lambda_1 p_1 = R P_1 + T$$

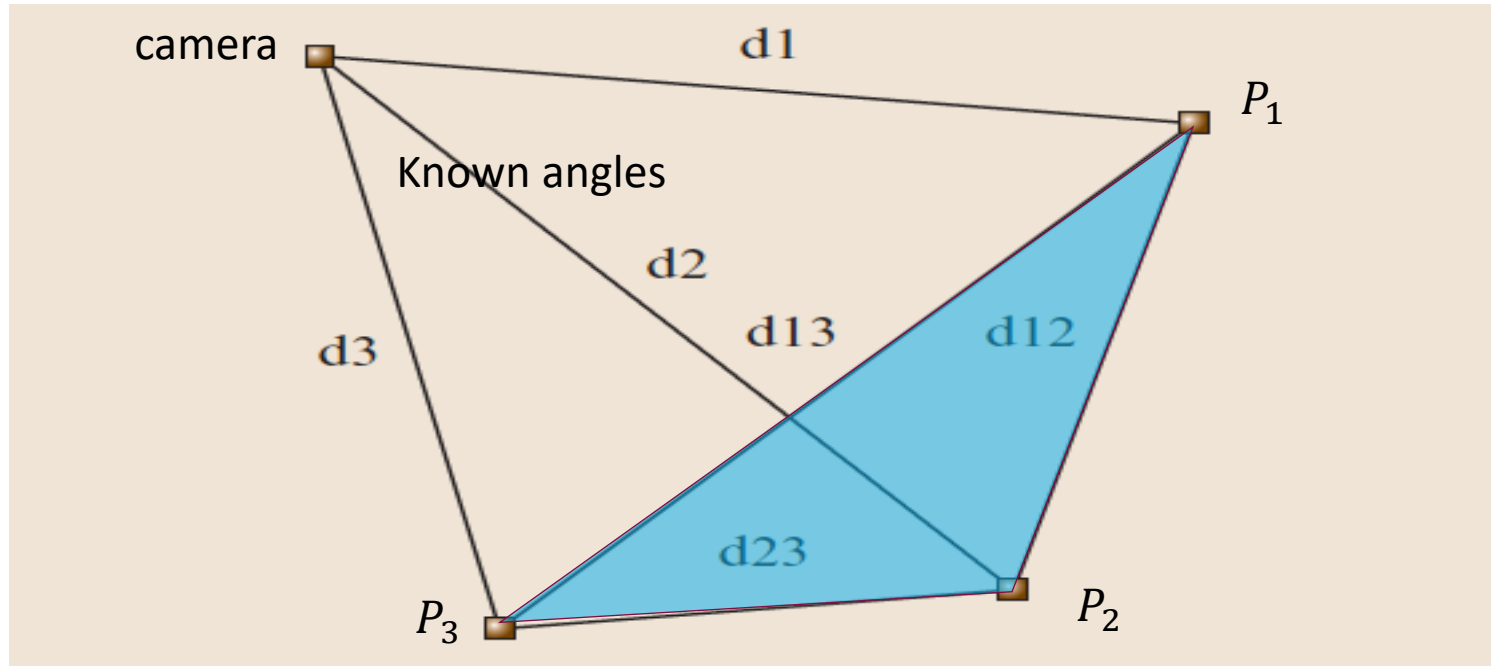
$$\lambda_2 p_2 = R P_2 + T$$

$$\lambda_3 p_3 = R P_3 + T$$

Q: Are  $\lambda_i$  the same as “depths”  $d_i$ ?

A: No, because  $p_i$  are not unit vectors.

# P3P from Calibrated Coordinates



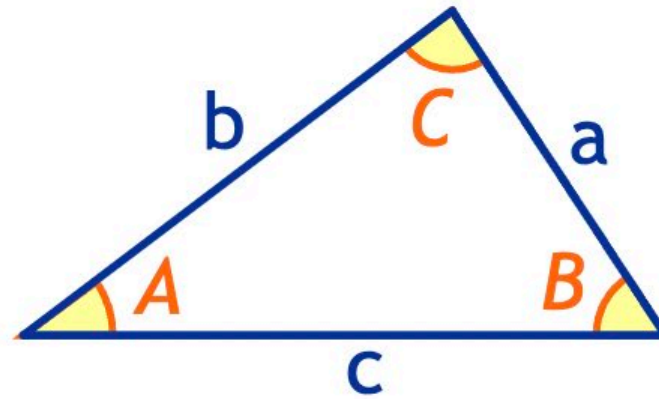
A triangle's world coordinates  $P_i$  are known, and its camera-centric calibrated coordinates  $p_i$  are known

**The P3P problem:** Find  $d_i, R, T$  such that

$$\frac{d_i}{\|p_i\|_2} p_i = R P_i + T, \quad \forall i = 1, 2, 3$$

# High School Flashback: Triangle Cosine Law

For any triangle:



**a**, **b** and **c** are sides.

**C** is the angle opposite side c

**The Law of Cosines** (also called the **Cosine Rule**) says:

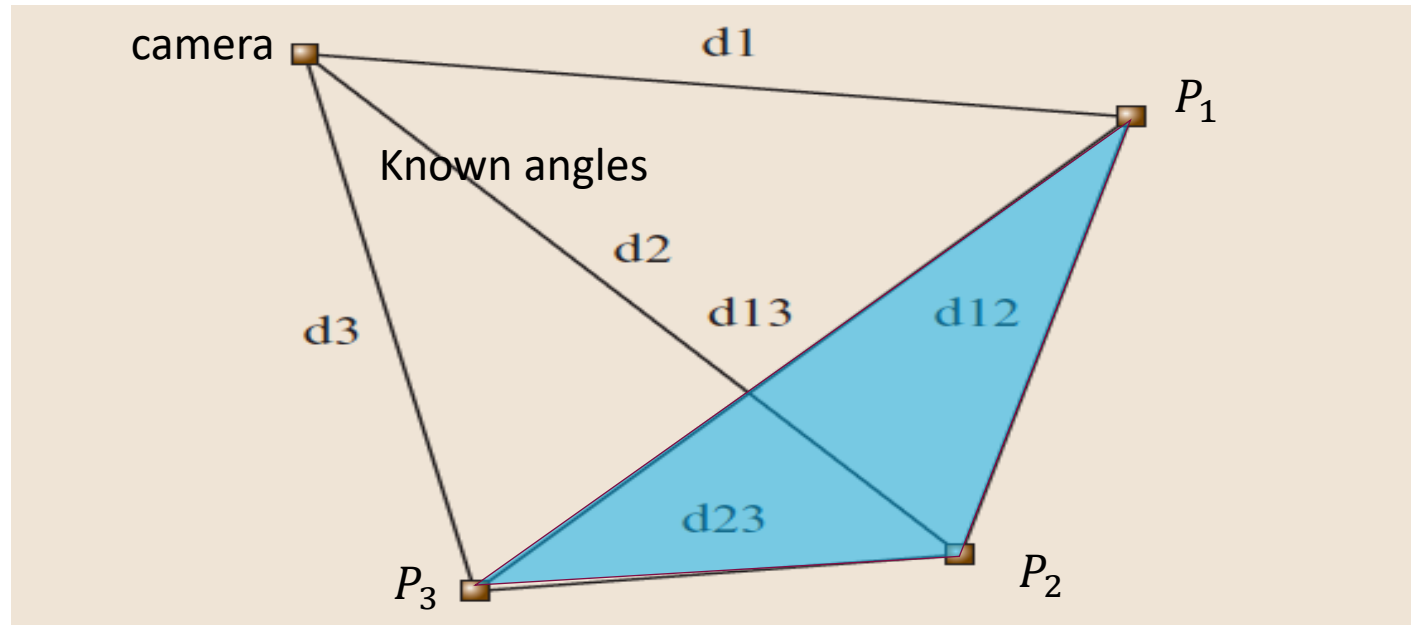
$$c^2 = a^2 + b^2 - 2ab \cos(C)$$

# P3P Step 1: Finding depths $d_i$ of triangle vertices

Let  $\delta_{ij}$  denote the observed angle between the calibrated coordinates  $p_i$  and  $p_j$

Then cosine law reads

$$d_i^2 + d_j^2 - 2d_i d_j \cos \delta_{ij} = d_{ij}^2$$



The cosine law

$$d_i^2 + d_j^2 - 2d_i d_j \cos \delta_{ij} = d_{ij}^2$$

applies for each point pair. With 3 points we could solve 3 quadratic equations for  $d_{i=1\dots 3}$ .

Set  $d_2 = ud_1$  and  $d_3 = vd_1$  and solve all three equations for  $d_1$ :

$$\begin{aligned} d_1^2 &= \frac{d_{23}^2}{u^2 + v^2 - 2uv \cos \delta_{23}} \\ d_1^2 &= \frac{d_{13}^2}{1 + v^2 - 2v \cos \delta_{13}} \\ d_1^2 &= \frac{d_{12}^2}{u^2 + 1 - 2u \cos \delta_{12}} \end{aligned}$$


# P3P Step 1: The algebraic drudgery

Reduces to two quadratic equations in  $u$  and  $v$ .

$$d_{13}^2(u^2 + v^2 - 2uv \cos \delta_{23}) = d_{23}^2(1 + v^2 - 2v \cos \delta_{13}) \quad (1)$$

$$d_{12}^2(1 + v^2 - 2v \cos \delta_{13}) = d_{13}^2(u^2 + 1 - 2u \cos \delta_{12}) \quad (2)$$

- Solve Eqn (1) for  $u^2$  in terms of  $u, v, v^2$  (and constants).
- Plug this solution into Eqn (2), so that it has no  $u^2$  term. Solve for  $u$  in terms of  $v, v^2$ , and constants.
- Plug this solution for  $u$  back into Eqn (1), so that it has no more  $u$  or  $u^2$ . Instead, it is a 4<sup>th</sup> degree equation in  $v$ . *Get the 4 real solutions analytically.*
- Then plug back into the solution found in step b) above, to get  $u$ .
- Then get  $d_1$  from the quadratic equations on the last page.
- Then plug back into  $d_2 = ud_1$  and  $d_3 = vd_1$  from the last page to get  $d_2, d_3$ .



# Reference for full version of P3P algebraic drudgery

- Grunert 1841, summarized in Haralick et al, 1993 which is linked in supplementary readings.

## **Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem**

ROBERT M. HARALICK

*Intelligent Systems Laboratory, Department of Electrical Engineering  
FT-10, University of Washington, Seattle, WA 98195, USA*

CHUNG-NAN LEE

*Institute of Information Engineering, National Sun Yat-Sen University, Kaohsiung, Taiwan 80424, ROC*

KARSTEN OTTENBERG

*Philips-Forschungslabor, Vogt-Kölln Straße 30, D-2000 Hamburg 54, Germany*

MICHAEL NÖLLE

*Technische Universität Hamburg-Harburg, Technische Informatik I, Harburger Schloßstraße 20, D-2100 Hamburg 90, Germany*

*Received October 30, 1990; revised March 27, 1992 and October 29, 1993*

# End result of algebraic drudgery

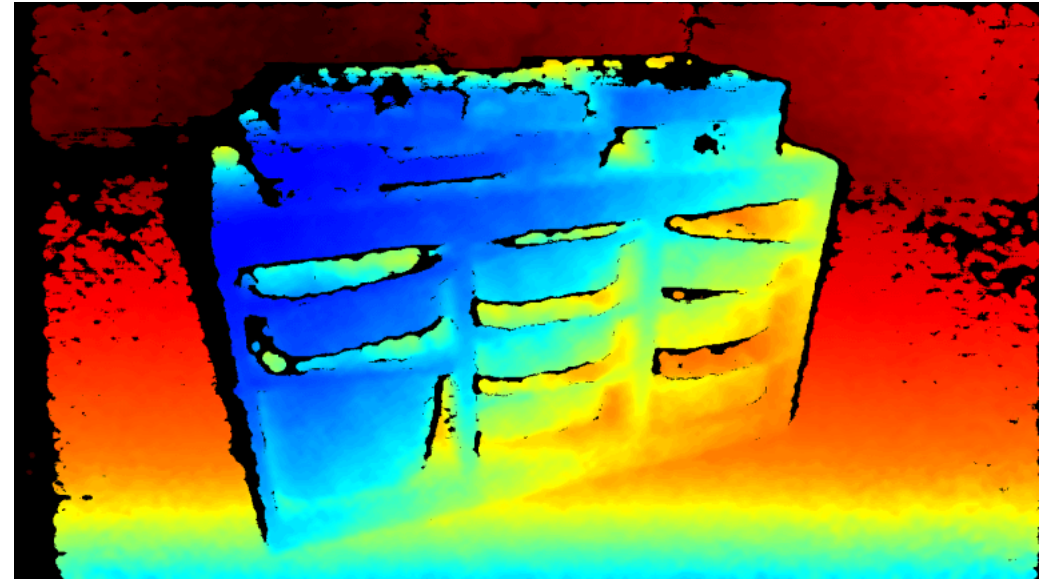
- We know the distances from camera to all three points! This means that we have effectively recovered “depth”, which combined with the calibrated coordinates we had before, means that we have:

**Camera-centric 3D coordinates for triangle vertices!**

**So, we can now do what we planned all along, and just solve the 3D- $\rightarrow$ 3D 3-Point Problem.**

# Aside: Depth cameras

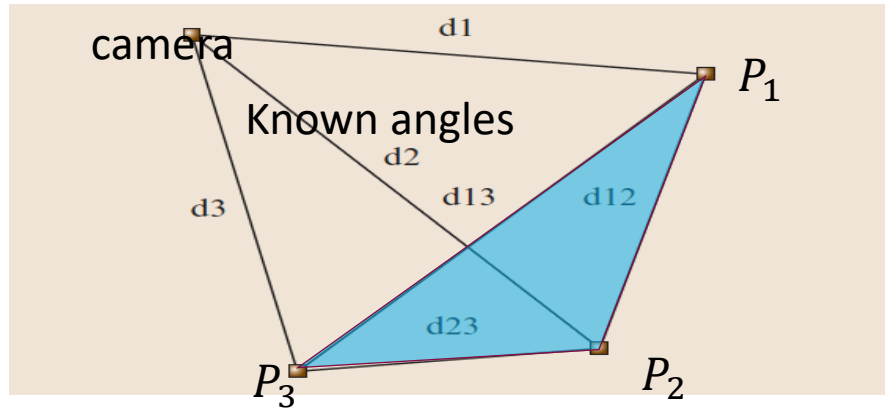
If we had started with depth images, we could have directly skipped to this point!



Note: Depth cameras commonly work from “stereo”: 2-view geometry!



# P3P Step 2: 3D->3D Pose/ 3D Registration. Find R&T!



**The P3P problem has reduced to:**

Find  $R, T$  such that

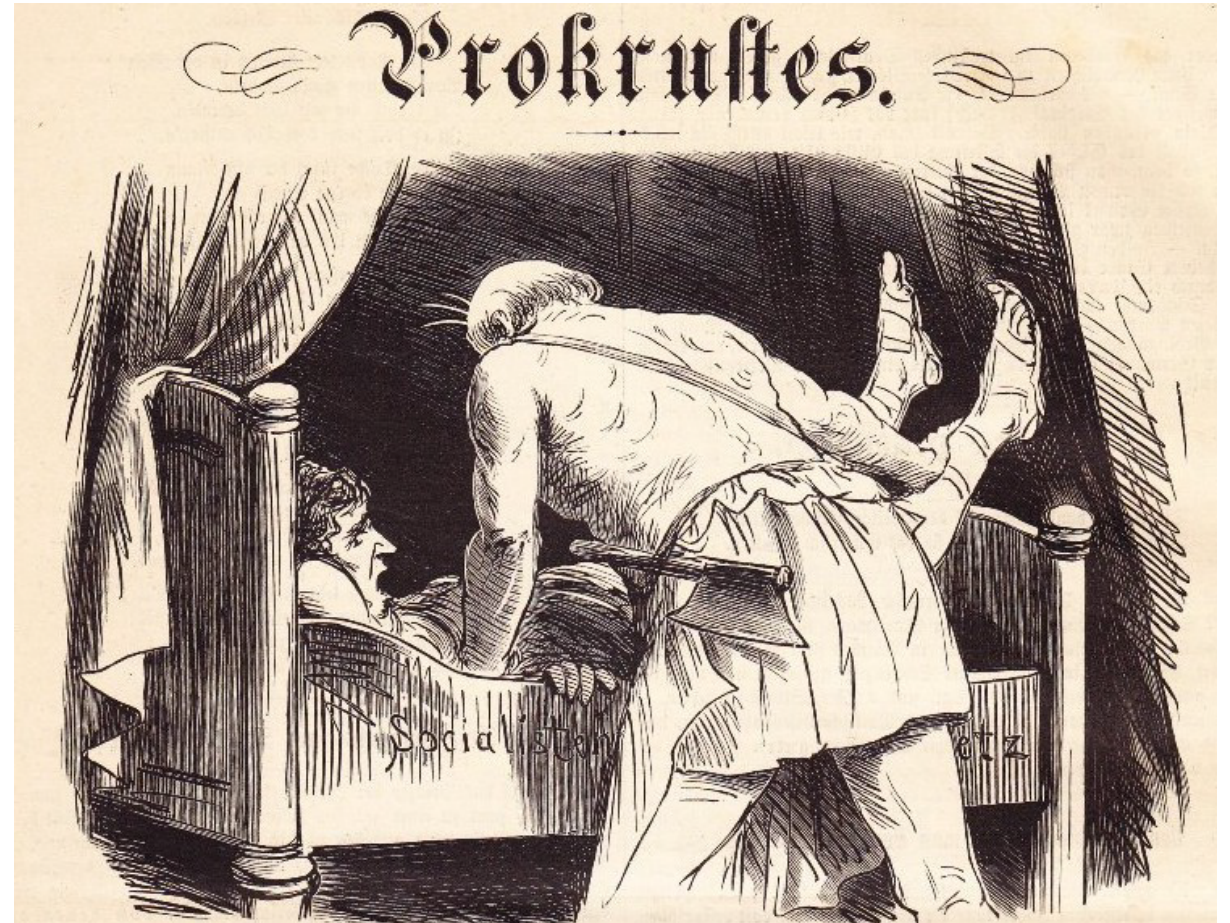
$$\frac{d_i}{\|p_i\|_2} p_i = R P_i + T, \quad \forall i = 1, 2, 3$$

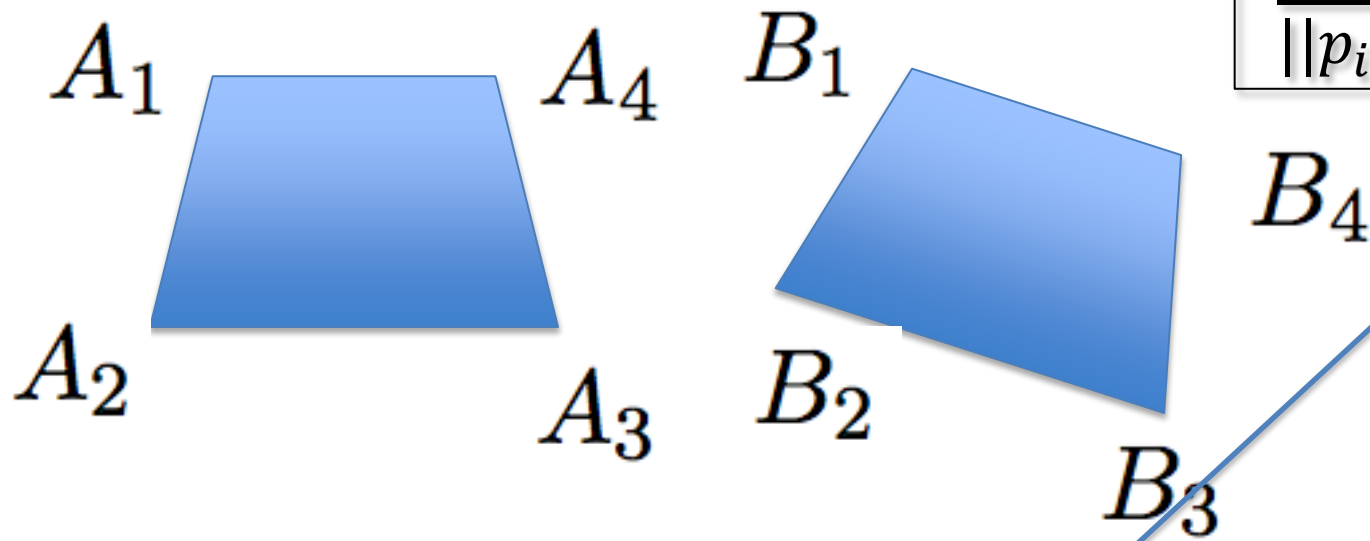
But naïve direct solution of the linear system is perilous, because rotation matrix  $R$  might not be valid.

**(Does this remind you of something?)**



# Procrustes Problem and Its Kabsch Algorithm





**The P3P problem:** Find  $R, T$  such that  
$$\frac{d_i}{||p_i||_2} p_i = \textcolor{red}{R} P_i + \textcolor{red}{T}, \quad \forall i = 1, 2, 3$$

How do we solve for  $R, T$  from  $n$  point correspondences?

$$A_i = RB_i + T$$

(Note: the method we discuss for P3P step 2 is the same for  $>3$  points too. So the rest of this discussion applies to more points too.)

(Nothing new on this slide except change of notation)

We solve a minimization problem for  $N > 3$  point correspondences:

$$\min_{R,T} \sum_i^N \|A_i - RB_i - T\|^2$$

After differentiating with respect to  $T$  we observe that the translation is the difference between the centroids:

$$T = \frac{1}{N} \sum_i^N A_i - R \frac{1}{N} \sum_i^N B_i = \bar{A} - R\bar{B}$$

Notes:

1. If we find the right rotation matrix  $R$ , we will be able to find  $T$  afterwards.
2. If the centroids  $\bar{A}$  and  $\bar{B}$  had both been zero, then we could have set  $T = 0$ .



3. Rotation matrix between point sets is unaffected by translating the point sets, so can centroid-subtract the point sets before finding rotation.

4. And after this centroid-subtraction, we know  $T = 0$  (by note#2 above), so:

$$\min_R \|A - RB\|_F^2$$

where

$$A = (A_1 - \bar{A} \quad \dots \quad A_N - \bar{A})$$

and

$$B = (B_1 - \bar{B} \quad \dots \quad B_N - \bar{B})$$

# Reducing to Procrustes problem

This problem can be shown to be equivalent to the Procrustes problem we have seen last class for finding 2D homographies!

$$\operatorname{argmin}_{R \in SO(3)} \|A - RB\|_F^2 = \operatorname{argmin}_{R \in SO(3)} \|R - AB^T\|_F^2$$



## Kabsch algorithm for Procrustes (proof in supp readings)

$$\operatorname{argmin}_{R \in SO(3)} \|R - (h'_1 \ h'_2 \ h'_1 \times h'_2)\|_F^2$$

If the SVD of

$$(h'_1 \ h'_2 \ h'_1 \times h'_2) = USV^T$$

then the solution is

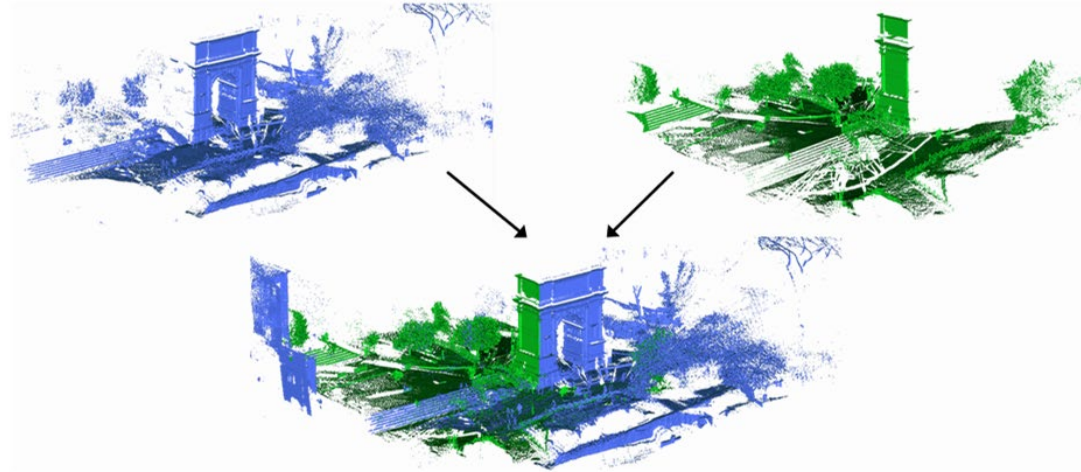
$$R = U \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(UV^T) \end{pmatrix} V^T$$

# Kabsch Algorithm for 3D->3D

- Compute centroids  $\bar{A}$  and  $\bar{B}$  of the two sets of 3D points.
- Create matrices  $A_{3 \times n}$ ,  $B_{3 \times n}$  after subtracting  $\bar{A}$  and  $\bar{B}$  from all points in the two sets.
- To find  $R$ , we must solve:  $\operatorname{argmin}_{R \in SO(3)} ||A - RB||_F = \operatorname{argmin}_{R \in SO(3)} ||R - AB^T||_F^2$ 
  - First set  $\hat{R} = (AB^T)_{3 \times 3}$
  - Then decompose  $\hat{R} = U\Sigma V^T$
  - Set  $R = U \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(VU^T) \end{pmatrix} V^T$
- Set  $T = \bar{A} - R\bar{B}$

## Sidenote: Extensions of 3D Point Set registration: Iterative Closest Point (ICP)

- Widely used for 3D point set registration ***without prior knowledge of correspondences***. Just needs 2 unordered point sets A and B. **Jointly finds correspondences and transformation.**



- Basic pseudocode:
  - **Step 1:** First, find “closest points” in set A for each point in set B. Treat these as pseudocorrespondences.
  - **Step 2:** Then solve for  $R$ ,  $T$  treating these correspondences as given. E.g. using Kabsch, but several other alternatives.
  - **Step 3:** Transform set B using  $R$ ,  $T$ , and go back to step 1.

## Sidenote: Extensions of 3D Point Set registration: Iterative Closest Point (ICP)

- Widely used for 3D point set registration ***without prior knowledge of correspondences***. Just needs 2 unordered point sets A and B. **Jointly finds correspondences and transformation.**
- Basic pseudocode:
  - **Step 1:** First, find “closest points” in set A for each point in set B. Treat these as pseudocorrespondences.
  - **Step 2:** Then solve for R, T treating these correspondences as given. E.g. using Kabsch, but several other alternatives.
  - **Step 3:** Transform set B using R, T, and go back to step 1.
- **Disadvantages:**
  - Much slower because of having to iterate between correspondences and transformation.
  - Requires a good initialization of R and T.

# PnP produces non-unique solutions ( $n > 3$ )?

- Recall that P3P Step 1 produced non-unique solutions for the distances  $d_i$ .
- But if we have  $n > 3$  point correspondences, we only need Step 2. This is the **“Perspective-N-Point” problem, or simply PnP.**

# Direct solution for PnP ( $n > 3$ ): Steps

Again, first switch to calibrated coordinates:

Pose from  $N$  points in space given intrinsic parameters  $K$  and correspondences  $(X_i, Y_i, Z_i, x_i, y_i)_{i=1\dots N}$  where

$$\begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} \sim K^{-1} \begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix}$$

where  $u_i, v_i$  are pixel coordinates.

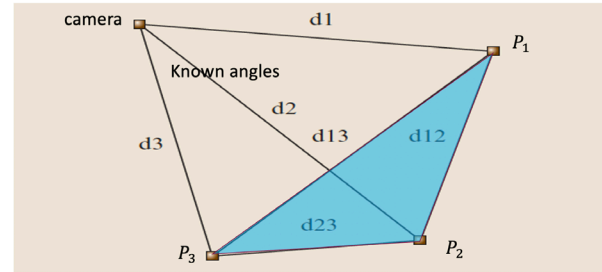
# Direct solution for PnP: Steps

Given  $(X_i, Y_i, Z_i, x_i, y_i)_{i=1\dots N}$

find  $R, T$  such that

$$\lambda_i \begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} = R \begin{pmatrix} X_i \\ Y_i \\ Z_i \end{pmatrix} + T$$

## P3P from Pixels



A triangle's world coordinates  $P_i$  are known, and its camera-centric calibrated coordinates  $p_i$  are known

**The P3P problem:** Find  $\lambda_i, R, T$  such that

$$\lambda_1 p_1 = R P_1 + T$$

$$\lambda_2 p_2 = R P_2 + T$$

$$\lambda_3 p_3 = R P_3 + T$$

Identical to the stage we reached with P3P, shown above.

But now, we are after a *direct* solution.

**No need to solve explicitly for depths as we did in P3P, so need to find unit vectors etc.**



# Direct solution for PnP: Steps

Instead substitute  $\lambda$  and get 2 linear equations per point correspondence

$$\lambda_i \begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} = R \begin{pmatrix} X_i \\ Y_i \\ Z_i \end{pmatrix} + T \quad \longrightarrow \quad \begin{aligned} x_i &= \frac{r_{11}X_i + r_{12}Y_i + r_{13}Z_i + T_1}{r_{31}X_i + r_{32}Y_i + r_{33}Z_i + T_3} \\ y_i &= \frac{r_{21}X_i + r_{22}Y_i + r_{23}Z_i + T_2}{r_{31}X_i + r_{32}Y_i + r_{33}Z_i + T_3} \end{aligned}$$

Get linear equations by cross-multiplying ...

$A_{2n \times 12} \mathbf{x}_{12} = 0,$   
where  $\mathbf{x}$  is a vector of all the unknowns including  $R$  and  $T$ .

Need at least 11 equations = at least 6 point correspondences to solve.

(just like for homography  $A_{2n \times 9} \mathbf{h}_{9 \times 1} = 0$ , we needed at least 4 point correspondences = 8 equations. )

Q: Why do we need more points for the direct method, when P3P was able to work with 3 points?

# We meet Procrustes and Kabsch yet again

Again, just solving the linear system won't give good rotations. So full steps:

1. Solve  $A_{2n \times 12} \mathbf{x}_{12} = 0$  from  $n \geq 6$  correspondences.
2. Then, assemble the rotation matrix  $\hat{R}$  from the solution for  $\mathbf{x}$ .
3. Then, find the closest valid rotation matrix by Kabsch!

Decompose  $\hat{R} = U\Sigma V^T$

Then, set  $R = U \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(VU^T) \end{pmatrix} V^T$

4. May now need to adjust translation  $T$  to be consistent with new  $R$ , e.g., by solving  $A_{2n \times 12} \mathbf{x}_{12} = 0$ , for only  $t_1, t_2, t_3$  (elements of  $\mathbf{x}$ ), holding  $R$  fixed.



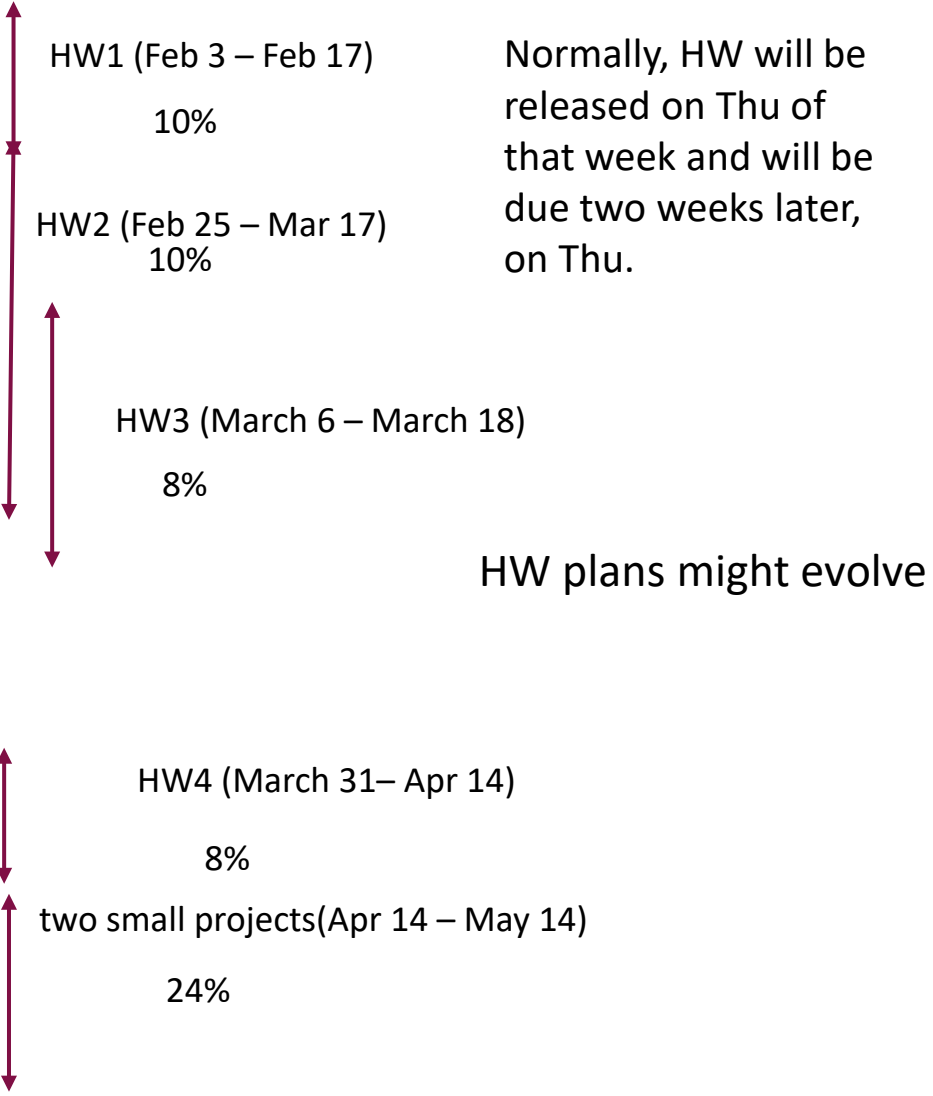
CIS 5800

# Machine Perception

Instructor: Lingjie Liu

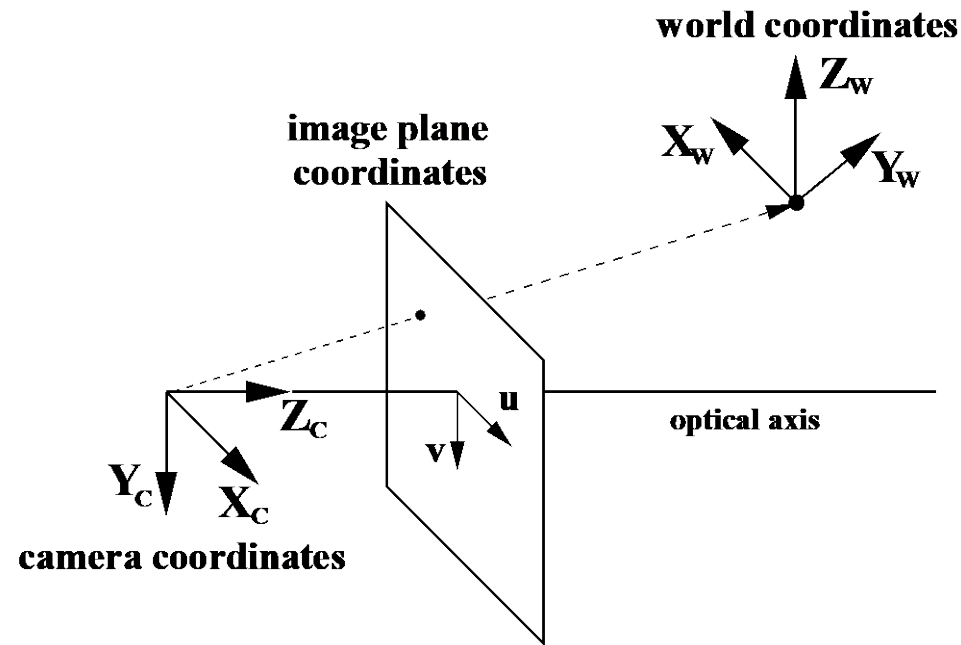
Lec 10 (Part 1): Feb 24, 2026

| Week | Date*                       | Topics                                                                                                    |
|------|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| 1    | Jan 20                      |                                                                                                           |
| 2    | Jan 27                      |                                                                                                           |
| 3    | Feb 3                       |                                                                                                           |
| 4    | Feb 10                      |                                                                                                           |
| 5    | Feb 17                      |                                                                                                           |
| 6    | Feb 24                      |                                                                                                           |
| 7    | Mar 3                       |                                                                                                           |
| 8    | Mar 10<br>(Spring<br>Break) |                                                                                                           |
| 9    | Mar 17                      |                                                                                                           |
| 10   | Mar 19                      | <b>TA class (March 19);<br/>March 24, mid term exam review class.<br/>Midterm (20%, <b>March 26</b>);</b> |
| 11   | Mar 31                      |                                                                                                           |
| 12   | Apr 7                       |                                                                                                           |
| 13   | Apr 14                      |                                                                                                           |
| 14   | Apr 21                      |                                                                                                           |
| 15   | Apr 28                      |                                                                                                           |
| 16   | Btw May 4-12                | <b>Final Exam (20%)</b>                                                                                   |



# Calibrating Camera Under More General Intrinsic

# Recap: Perspective Projection Matrix



camera 3D coords to pixels      Convert world to camera coordinates

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & & u_0 \\ & f & v_0 \\ & & 1 \end{bmatrix} \begin{bmatrix} R_{3 \times 3} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$

# Generalizing intrinsics (1/2)

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & & u_0 \\ & f & v_0 \\ & & 1 \end{bmatrix} [R_{3 \times 3} \quad \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$

1. If pixels are not square?

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & & u_0 \\ & s_y & v_0 \\ & & 1 \end{bmatrix} [R_{3 \times 3} \quad \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$



# Method: High-Level Overview

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} s_x & & u_0 \\ & s_y & v_0 \\ & & 1 \end{bmatrix} [R_{3 \times 3} \quad \mathbf{t}]}_{\text{Camera projection matrix } M_{3 \times 4}} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$

Camera projection matrix  $M_{3 \times 4}$   
(Note: we've also called this  $P$  before)

# Method: High-Level Overview $M = P = \text{projection matrix}$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} \approx \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

$$u_i = \frac{m_{00}X_i + m_{01}Y_i + m_{02}Z_i + m_{03}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + m_{23}}$$

$$v_i = \frac{m_{10}X_i + m_{11}Y_i + m_{12}Z_i + m_{13}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + m_{23}}$$

$$\begin{bmatrix} X_i & Y_i & Z_i & 1 & 0 & 0 & 0 & 0 & -u_iX_i & -u_iY_i & -u_iZ_i & -u_i \\ 0 & 0 & 0 & 0 & X_i & Y_i & Z_i & 1 & -v_iX_i & -v_iY_i & -v_iZ_i & -v_i \end{bmatrix} \begin{bmatrix} m_{00} \\ m_{01} \\ m_{02} \\ m_{03} \\ m_{10} \\ m_{11} \\ m_{12} \\ m_{13} \\ m_{20} \\ m_{21} \\ m_{22} \\ m_{23} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Estimation of the full projection matrix is linear  
 but  $M$  has to be decomposed into  $K, R, \mathbf{t}$  afterwards  
 (“QR decomposition” of first 3 columns of  $M$  to get  $K, R$ . Then solve for  $\mathbf{t}$ )

# Generalizing intrinsics (2/2)

1. If pixels are not square?

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & & u_0 \\ & s_y & v_0 \\ & & 1 \end{bmatrix} [R_{3 \times 3} \quad \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$

2. If “radial distortions”, then the intrinsics can no longer be represented as a linear operator any more

# Parametrizing radial distortion in large field-of-view cameras



Then, correct for radial distortion:

$$u_{\text{pre-distortion}}(r) = u_{\text{post-distortion}}(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots)$$

$$v_{\text{pre-distortion}}(r) = v_{\text{post-distortion}}(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots)$$

where  $r$  is the distance from a (usually unknown) image center location  $(u_0, v_0)$ .

Can choose the degree of the radial distortion to calibrate for. More => more accurate, but requires more images to fit well.

# What Does (Intrinsics) “Calibration” Do?

Estimates the *intrinsic parameters*

- $f$  focal length (or  $s_x, s_y$  to allow non-square pixels)
- $(u_o, v_o)$  Principle point
- $k_1, k_2, \dots$  radial distortion parameters

# Colab worksheet (Camera Calibration with OpenCV)!

- <https://colab.research.google.com/drive/1Vf6pBFo84OKR0E3j3t3HJjEN71o6k0sk?authuser=0>
- (Optional) Also see OpenCV Tutorial on Calibration.

## Camera Calibration Flowchart

Define real world coordinates of 3D points using checkerboard pattern of known size.



Capture the images of the checkerboard from different viewpoints.



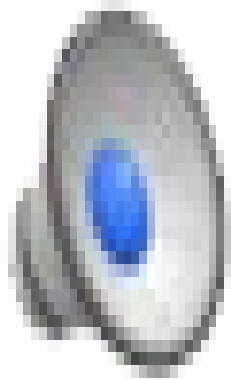
Use **findChessboardCorners** method in OpenCV to find the pixel coordinates  $(u, v)$  for each 3D point in different images



Find camera parameters using **calibrateCamera** method in OpenCV, the 3D points, and the pixel coordinates.

<https://learnopencv.com/camera-calibration-using-opencv/>

# Output: Undistorted images and video!





# Summary of Tools for Single-View Geometry (1/2)

- Finding vanishing points:
  - Using parallel lines
  - Using homographies
  - Using “cross ratios”
- Using VPs to compute horizon and determine camera orientation w.r.t. ground (assuming standard intrinsics  $K$ )
- Finding homographies from planes:
  - from 2 orthogonal VPs plus correspondences for the 2 points  $(0, 0)$ , and  $(1, 1)$ .
  - from 4 non-square points using inversion and composition of 2 homographies.
  - from  $\geq 4$  arbitrary non-collinear point correspondences.
- Using homographies for:
  - Virtual billboard display
  - Finding camera pose and refining with Kabsch (given intrinsics  $K$ )

# Summary of Tools for Single-View Geometry (2/2)

- Using cross ratios to measure real lengths:
  - Along a line, given 2 world lengths.
  - Along a line, given 1 world length and the VP.
  - Using VPs on ground plane, vertical VP, and vertical cross ratios for length transfer.
- Using pose and intrinsics (full projection matrix) to project AR objects onto world.
- Finding extrinsics from 3 2D->3D point correspondences (P3P problem)
  - What if we had depth images?
  - What if we only had standard RGB images?
- Finding extrinsics from  $n > 3$  2D->3D point correspondences not on a plane (PnP problem)
- Finding camera intrinsics (camera calibration):
  - Finding intrinsics  $K$  from 3 orthogonal VPs (camera calibration part 1)
  - Overview of more general camera intrinsics calibration under radial distortion

**Next, onwards to 2 Views!**